

**UNIVERSITY
OF OSLO**

Juliane Päßler

Formal Methods for Self-Adaptive Systems

Thesis submitted for the degree of Philosophiae Doctor

Department of Informatics
Faculty of Mathematics and Natural Sciences

2025



© **Juliane Päßler**, 2025

*Series of dissertations submitted to the
Faculty of Mathematics and Natural Sciences, University of Oslo
No. 2566*

ISSN 1501-7710

All rights reserved. No part of this publication may be
reproduced or transmitted, in any form or by any means, without permission.

Cover: UiO.

Print production: Graphic center, University of Oslo.

Preface

This thesis concludes the research conducted towards the degree of *Philosophiae Doctor* from 2021 to 2025 at the Department of Informatics, University of Oslo. The research presented here was carried out under the supervision of S. Lizeth Tapia Tarifa, Einar Broch Johnsen and Carlos Hernández Corbato. This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956 200).

The thesis is a collection of five papers. Two of them were published in peer-reviewed conference proceedings, two of them in peer-reviewed journals, and one was submitted to a peer-reviewed journal. The papers are preceded by four introductory chapters motivating the work, establishing the necessary background, stating the studied research questions, and providing information on how the research questions were addressed in each paper. All papers were written as joint work with several other authors; I am the first author of four of them.

This thesis has been written without the use of generative artificial intelligence. The English abstract was translated to Norwegian using DeepL.com and checked by a native Norwegian speaker afterwards. The implementation of the algorithms in Paper IV used ChatGPT-4 (OpenAI) as declared in the acknowledgements in Paper IV.

Acknowledgements

I would like to express my sincere gratitude to my three supervisors. To Lizeth and Einar for their support, their help in exploring a new research field, for connecting me with other researchers, and for giving me the freedom to pursue my own projects. To Carlos for his kindness, for introducing me to robotics, and for hosting me in Delft.

I am very grateful for having been privileged to work with so many smart and interesting people. Thank you to all of my co-authors: Clemens, Elvin, Esther, Ferruccio, Ilias, Jeroen, Logan, Marc, Marsha, Maurice, Mohammad, Sophie, and Verena. Working together with you has greatly enriched my PhD experience. A special thanks goes out to Gustavo, it has been a pleasure to work with you from the beginning. Our weekly meetings have often made my day and the research stays in Delft and Porto were a great experience.

Being part of REMARO has been invaluable. I would like to thank everyone involved in the project, especially my fellow ESRs. Thank you for our scientific discussions, sharing your experiences as PhD students, the cabin trips, and in general having a great time.

I would also like to thank my fellow PhD students at IFI for being part of this journey, fun lunches, balcony breaks, and boardgame nights. Thank you, Tobias, for providing me with valuable feedback for the thesis, for creating a much nicer picture of ARo, and for always being willing to spend the weekend outdoors. Furthermore, I would like to thank the numerous people who have provided me with guidance, help, mentorship, and compassion throughout these four years.

My PhD journey would not have been possible without the support of my friends and family. A special thanks goes to Anna for being a great listener, trying to understand what I am working on, reading my thesis, and explaining your research in return. Thank you to my family for your encouragement and belief in me. Last but not least, thank you, Lukas, for all your support, pushing me when I need it, and always being there for me.

Juliane Päßler

Oslo, September 2025

Abstract

Self-adaptation can be used to enable a system to adapt to internal and external changes, allowing it to be deployed in an uncertain environment. A self-adaptive system (SAS) can be implemented as a two-layered system consisting of a *managed subsystem* and a *managing subsystem*. The managed subsystem implements the application logic, while the managing subsystem implements the adaptation logic. The managing subsystem monitors the managed subsystem and the environment, uses this data to determine whether the managed subsystem needs to be adapted, and adapts it if necessary. The aim of an SAS is to perform its mission autonomously, ideally without human supervision. Especially when operating in safety and security-critical domains, it is crucial to ensure the safety and reliability of an SAS.

In this thesis, we show how *formal methods* can be used to analyse the safety and reliability of SASs. Formal methods are mathematically based techniques and tools for the specification and analysis of hardware and software systems. We focus on formal methods that are able to capture and analyse the two-layered structure of an SAS, including the interaction between the two layers and the influence of the environment that triggers self-adaptation of the SAS. Concretely, we show how rewriting logic and techniques from software product lines (SPLs) can be used to model an SAS and the environment it operates in. We exemplify the former using the tool Maude and the latter with the tool ProFeat. We use simulation in Maude to show that the inclusion of a managing subsystem, which implements self-adaptation, can increase the overall system performance. Family-based probabilistic model checking in ProFeat is utilised to illustrate how quantitative properties of an SAS can be analysed.

We further show how existing formal modelling techniques can be extended or modified to support reuse of SAS models, and show their potential to ease modelling and maintenance of SASs. By introducing *template nodes* in decision diagrams, common behaviour can be encapsulated and reused which also supports the specification of the two-layered structure of an SAS. Furthermore, we combine the modelling of the structure and the behaviour of SPLs into one compositional model, called *behavioural feature model*, which allows reuse of behaviour of single features and sets of features.

Sammendrag

Et *selvtilpassende system* (eng: self-adaptive system) kan tilpasse seg interne og eksterne endringer slik at det kan tas i bruk i et usikkert miljø. Det kan implementeres som et tolags system med et *administrert delsystem* og et *administrerende delsystem*. Det administrerte delsystemet implementerer applikasjonslogikken, mens det administrerende delsystemet implementerer tilpasningslogikken. Det administrerende delsystemet overvåker det administrerte delsystemet og miljøet, bruker disse dataene til å avgjøre om det administrerte delsystemet må tilpasses, og tilpasser det om nødvendig. Målet med et selvtilpassende system er at det skal utføre sitt oppdrag autonomt, ideelt sett uten menneskelig tilsyn. Spesielt når det opererer i sikkerhetskritiske domener, er det avgjørende å sikre sikkerheten og påliteligheten til systemet.

I avhandlingen viser vi hvordan formelle metoder kan brukes til å analysere sikkerheten og påliteligheten til selv-tilpassende systemer. Formelle metoder er matematisk baserte teknikker og verktøy for spesifikasjon og analyse av maskinvare- og programvaresystemer. Vi fokuserer på formelle metoder som er i stand til å modellere og analysere tolags-strukturen, inkludert samspillet mellom de to lagene og påvirkning fra omgivelsene som utløser selvtilpasningen. Konkret viser vi hvordan omskrivingslogikk og teknikker fra “software product lines” kan brukes til å modellere et selv-tilpassende system og omgivelsene det opererer i. Vi eksemplifiserer førstnevnte ved hjelp av verktøyet Maude og sistnevnte med verktøyet ProFeat. Videre bruker vi simulering i Maude for å vise at inkludering av et administrert delsystem, som implementerer selvtilpasning, kan øke den totale systemytelsen. Familiebasert probabilistisk modellsjekking i ProFeat brukes til å illustrere hvordan kvantitative egenskaper kan analyseres.

Vi viser videre hvordan eksisterende formelle modelleringsteknikker kan utvides eller modifiseres for å støtte gjenbruk av modeller, og viser deres potensial for å lette modellering og vedlikehold. Ved å introdusere malnoder i beslutningsdiagrammer kan felles atferd innkapsles og gjenbrukes, noe som også støtter spesifikasjonen av den todelte strukturen. Videre kombinerer vi modelleringen av strukturen og adferden i én komposisjonell modell, kalt “behavioural feature model”, som gjør det mulig å gjenbruke adferden til enkeltstående funksjoner og sett av funksjoner.

List of Papers

Paper I

A Formal Model of Metacontrol in Maude. Juliane Päßler, Esther Aguado, Gustavo Rezende Silva, S. Lizeth Tapia Tarifa, Carlos Hernández Corbato, Einar Broch Johnsen. In: *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles*. ISOLA 2022. Lecture Notes in Computer Science, vol. 13701, pp. 575–596. DOI: 10.1007/978-3-031-19849-6_32.

Paper II

Analysing Self-Adaptive Systems as Software Product Lines. Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, S. Lizeth Tapia Tarifa. In: *Journal of Systems and Software*. 2025, vol. 222, p. 112324. DOI: 10.1016/j.jss.2024.112324.

Paper III

Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System. Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Clemens Dubslaff, Einar Broch Johnsen, S. Lizeth Tapia Tarifa. In: *Formal Aspects of Computing*. Association for Computing Machinery, 2025. DOI: 10.1145/3709159.

Paper IV

Behavioral Feature Models. Juliane Päßler, Sophie Fortz, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, Mohammad Reza Mousavi, S. Lizeth Tapia Tarifa. Submitted for journal publication (short version).

Paper V

Template Decision Diagrams for Meta Control and Explainability. Clemens Dubslaff, Verena Klös, Juliane Päßler. In: *Explainable Artificial Intelligence*. xAI 2024. Communications in Computer and Information Science, vol. 2154, pp. 219–242. Springer, Cham. DOI: 10.1007/978-3-031-63797-1_12.

Contents

Preface	i
Abstract	iii
Sammendrag	v
List of Papers	vii
Contents	ix
1 Introduction	1
1.1 Research Questions	2
1.2 REMARO	3
1.3 Thesis Outline	4
References	4
2 Background	7
2.1 Self-Adaptive Systems	7
2.2 (Dynamic) Software Product Lines	10
2.3 Formal Modelling and Analysis Techniques	13
2.3.1 Rewriting Logic	13
2.3.2 Probabilistic Systems	15
2.3.3 Probabilistic Model Checking	17
2.3.4 Featured Transition Systems	20
2.3.5 Bundle Event Structures	25
2.3.6 Decision Trees	26
References	28
3 Overview of the Research Papers	35
4 Conclusion and Future Directions	43
4.1 Conclusion	43
4.1.1 Towards RQ 1 (Modelling)	45
4.1.2 Towards RQ 2 (Analysis)	47
4.1.3 Towards RQ 3 (Reuse)	48
4.2 Future Directions	49
References	51

Papers	54
I A Formal Model of Metacontrol in Maude	55
I.1 Introduction	55
I.2 Background	57
I.2.1 Metacontrol	57
I.2.2 Maude	58
I.3 The Smart House Use Case	59
I.3.1 The Controller Layer	60
I.4 The Metacontrol Layer for the Smart House Use Case . . .	62
I.4.1 The TOMASys Metamodel	63
I.4.2 Metacontrol Operation	64
I.5 Modelling the Smart House Use Case in Maude	66
I.5.1 Model Dynamics	66
I.5.2 The Smart House Model	67
I.5.3 The Controller Layer	69
I.5.4 The Metacontrol Layer	69
I.6 Evaluating the Executable Model of the Smart House . . .	71
I.7 Related Work	73
I.8 Conclusion and Future Work	74
References	75
II Analysing Self-Adaptive Systems as Software Product Lines	79
II.1 Introduction	79
II.2 Family-Based Modelling and Analysis of SASs	83
II.2.1 SASs and Uncertainties that Trigger Reconfiguration	83
II.2.2 Towards Managing Variability for a Family of Products	85
II.2.3 Family-Based Modelling and Analysis of Two-Layered SASs	87
II.3 Small-Scale Evaluation: Pipeline Inspection by AUV . . .	89
II.3.1 An Overview of the Small-Scale Evaluation . . .	89
II.3.2 Two-layered View of the AUV as a Family-based Model	91
II.3.3 Separation of Concerns between Managed and Managing Subsystem	92
II.4 Modelling the AUV Small-Scale Evaluation with ProFeat	93
II.4.1 The Feature Model	95
II.4.2 The Managed Subsystem	97
II.4.3 The Environment	103
II.4.4 The Hardware Failures	104
II.4.5 The Managing Subsystem	105
II.5 Analysis	110
II.5.1 Sensor Failures	112
II.5.2 Reward Properties	113
II.5.3 Unsafe States	114

II.5.4	Correctness Issues of the Adaptation Logic	115
II.6	Evaluation	116
II.6.1	RQ1: How Does the Proposed Methodology of Modelling an SAS As a DSPL Support Extensibility?	117
II.6.2	RQ2: To What Extent Does the Proposed Method- ology Enable the Analysis of SASs?	119
II.6.3	RQ3: How Can One Analyse Whether the Adap- tation Logic Has Been Realised According to Its Specification?	120
II.6.4	Threats to Validity	120
II.7	Related Work	121
II.8	Discussion and Future Work	123
	References	126
III	Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System	141
III.1	Introduction	142
III.2	Case Study: Pipeline Inspection by AUV	146
III.3	Software Product Lines and Family-Based Model Checking	147
III.4	Feature-oriented Modelling of the AUV Case Study	149
III.4.1	The Feature Model	150
III.4.2	The Managing Subsystem	153
III.4.3	The Managed Subsystem	158
III.4.4	The Environment	163
III.5	Analysis	164
III.5.1	Parameter Setup for the Scenarios	165
III.5.2	Reward Properties	168
III.5.3	Unsafe States	171
III.5.4	Different Environments	173
III.5.5	Reward Properties with Varying Probability of Currents	175
III.5.6	Multi-Objective Queries	177
III.6	Related Work	180
III.6.1	SPLs for Self-Adaptive and Robotic Systems . .	180
III.6.2	Family-Based Model Checking	180
III.6.3	Feature-Oriented Analysis of SASs	181
III.7	Discussion and Future Work	181
III.7.1	Using SPL Techniques for Modelling and Analysing SASs	182
III.7.2	Future Work	183
	References	184
IV	Behavioral Feature Models	195
IV.1	Introduction	195
IV.2	Background	198
IV.2.1	Feature Models	199

	IV.2.2	Transition Systems	202
	IV.2.3	Featured Transition Systems	204
	IV.2.4	Bundle Event Structures	207
IV.3		Behavioral Feature Models: Motivation and Overview . . .	210
IV.4		Behavioral Feature Models: A Formal Account	215
	IV.4.1	Behavioral Feature Models	215
	IV.4.2	Featured Event Structures: an Isomorphic Representation of BFM	218
		s	
	IV.4.3	Products and Behavioral Products of a BFM/FES	226
	IV.4.4	Configurations and Traces of a BFM/FES	227
	IV.4.5	Composing and Decomposing BFM	230
		s and FESs	
	IV.4.6	Trace Equivalence for BFM	233
		s, FESs and FTSs	
IV.5		From BFM	233
		s to FTSs and Back by Preserving Trace	
		Equivalence	
	IV.5.1	From Linear TSs to BESs by Preserving Traces .	234
	IV.5.2	From BESs to TSs by Preserving Traces	237
	IV.5.3	From Linear FTSs to FESs by Preserving Traces	240
	IV.5.4	From FESs to FTSs by Preserving Traces	243
	IV.5.5	From Linear FTSs to BFM	246
		s by Preserving Traces	
	IV.5.6	From BFM	247
		s to FTSs by Preserving Traces	
IV.6		Evaluation	249
	IV.6.1	Methodology	250
	IV.6.2	RQ1: In Theory, How do the Expressiveness and	256
		Succinctness of BFM	
		s and FTSs Compare?	
	IV.6.3	RQ2: In Practice, How Does the Size of a BFM	257
		Compare to the Size of an FTS?	
	IV.6.4	Threats to Validity	260
IV.7		Related Work	261
	IV.7.1	Compositionality of Structural Models of SPLs .	261
	IV.7.2	Compositionality of Behavioral Models of SPLs .	261
	IV.7.3	Unifying Structural and Behavioral Modeling of	262
		SPLs	
	IV.7.4	Analysis of Behavioral Models of SPLs	263
IV.8		Discussion	263
	IV.8.1	On the Complexity of Translating FTSs	264
	IV.8.2	On Composing and Decomposing FTSs	264
	IV.8.3	On Extending BFM	266
		s/FESs to Model Traces Where	
		Events May Occur More Than Once	
	IV.8.4	On the Location of Events in BFM	267
		s	
IV.9		Conclusion and Future Work	267
	IV.9.1	Summary and Reflections	267
	IV.9.2	Future Work	268
	References		269

V	Template Decision Diagrams for Meta Control and Explainability	283
V.1	Introduction	283
V.2	Related Work	287
V.3	Foundations	288
V.4	Template Decision Diagrams	289
	V.4.1 Hierarchical Decision Diagrams by Templates . .	290
	V.4.2 Standard Template Boxes	291
V.5	Templates for Self-Adaptive Systems and Meta Control . .	293
	V.5.1 An Overview of the Case Study	294
	V.5.2 Modeling the Case Study with Template DDs . .	294
V.6	Improving Control Strategy Explanations	296
	V.6.1 Explainability Metrics for Template DDs	297
	V.6.2 Decision Diagram Refactoring	298
	V.6.3 Implementation and Evaluation	299
V.7	Conclusion	301
	References	302

Chapter 1

Introduction

Modern software systems are increasing in complexity, and there is a growing interest in deploying them in uncertain environments. This includes environments that are hostile to humans or difficult to reach, such as a nuclear power facility, space or the deep sea. Manually dealing with uncertainties in the environment, i.e., letting a human decide what to do if an unexpected situation arises, is not only costly, error-prone and time-intensive [Wey20], it might also not be feasible at all. In some environments, especially in space or under water, communication with a human operator is limited or even impossible. Therefore, there is a need for designing systems that can deal with uncertainties themselves.

Self-adaptive systems (SASs) [Wey20] provide a solution to the above stated problem. The aim of an SAS is to perform its mission autonomously or with limited human supervision while handling uncertainties itself. These uncertainties can stem from the environment, such as a suddenly appearing obstacle, the system itself, such as a broken sensor, or requirements from the user, such as a changed goal. An SAS collects information during runtime, reasons about it, and adapts itself if necessary. An adaptation can, e.g., be a change of the system's architecture such as changing components.

SASs are deployed in safety and security-critical domains such as robotics, critical infrastructure and transportation [Pek+23]. Especially in these domains, unexpected behaviour of the SAS can lead to catastrophic consequences for humans or the environment. For example, in a nuclear power facility or under water, unexpected behaviour of an SAS can lead to humans being exposed to radiation or the damage of an oil pipeline, respectively. Therefore, before SASs are deployed autonomously, it is important to ensure their safety and reliability. Different from a non-self-adaptive system, this includes the additional challenge of assuring that the adaptation works reliably. As a conclusion, ensuring the safety and reliability of SASs is a key challenge [Cal+20].

One way to gain confidence in the safety and reliability of a system is to use *formal methods* [BH95; Hal+99], i.e., techniques and tools based on mathematics to model and analyse a system. A formal method is based on a specification language with a formal syntax and semantics. By forcing the developer to create a formal specification, formal methods can help to determine poor design choices, errors, ambiguities and misconceptions early on in the design phase of a system [CW96]. Formal methods allow different manual and (partly) automated system analyses including *formal verification*. Formal verification is the process of analysing whether a (model of) a system satisfies its specification.

In this thesis, we show how formal methods, including formal verification, can be used to analyse the safety and reliability of SASs. Thereby, we address the following challenges related to using formal methods for SASs:

- (1) The formal method has to enable *modelling self-adaptation and different configurations* of the SAS explicitly. Since the adaptation logic is an integral part of an SAS, the formal method should allow modelling it separately from the regular system behaviour. Furthermore, the formal method should allow capturing how the system can be changed by the adaptation logic. This includes modelling the different configurations of the SAS that the adaptation logic can select.
- (2) The formal method has to enable *modelling the environment* an SAS operates in, ideally including the uncertainties of the environment. Since the environment triggers self-adaptation, the formal method should be able to capture the environment an SAS operates in. Furthermore, it should allow modelling how the environment influences the operation of the SAS and vice versa.
- (3) The formal method has to enable *analysing (quantitative) properties related to self-adaptation*. The formal method should allow analysing the SAS including the adaptations it can perform. This should also facilitate an analysis of the adaptation logic and of how different adaptation strategies influence the system operation.
- (4) The formal method has to enable *reuse of formal models* of SASs. Since SASs are complex systems, consisting of various components, building a formal model of the system is a complicated and time-consuming task. However, if the formal method allows reuse of (parts of) the formal model of the SAS, this can help in developing formal models of SASs for other applications, or of SASs deployed in different environments.

1.1 Research Questions

As motivated above, ensuring the safety and reliability of SASs is a crucial step towards building systems that can operate fully autonomously in uncertain environments. The main aim of this thesis is to

*use formal methods at design time to analyse
the safety and reliability of self-adaptive systems.*

This thesis focusses on design-time analysis of SASs to detect poor design choices, errors or ambiguities in the SAS, including its adaptation logic, before the system is deployed. In the context of this thesis, *safety* refers the absence of the risk of harming humans, the environment, or the SAS itself. For example, a self-adaptive underwater robot should not harm divers, animals, or coral reefs, and it should avoid damaging itself. We refer to *reliability* as the ability of the SAS to perform as intended.

To achieve the above stated research goal, this thesis addresses the following three research questions which we shortly explain.

RQ1 (Modelling): *How can formal modelling techniques be used to capture the behaviour of a self-adaptive system that is influenced by an environment?*

This research question is related to Challenges 1–2 from above. We investigate how formal modelling techniques can be used to explicitly model the adaptation logic and the configurations of an SAS as well as the environment the SAS operates in. These three elements influence each other: The SAS monitors the (uncertain) environment, and changes in the environment trigger adaptations of the SAS. The executed adaptation then changes the configuration of the SAS, which in turn might influence how the SAS operates in the environment. Thus, all three elements and their interactions should be captured in our formal model. We consider how this can be done in a modular way such that our formal model reflects the separation of concerns which is present in the SAS.

RQ2 (Analysis): *How can formal analysis techniques be used at design time to analyse the behaviour of a self-adaptive system that is influenced by an environment?*

This research question is related to Challenge 3 from above. We investigate how formal methods can be used to analyse (quantitative) properties of an SAS. A particular focus of the analysis is on the adaptation logic and how an adaptation influences the operation of an SAS. Furthermore, we explore how different behaviours of the environment influence the behaviour of the SAS. Analysing these aspects is facilitated by using formal modelling techniques addressing Challenges 1–2 for modelling the SAS that should be analysed.

RQ3 (Reuse): *How can formal modelling techniques be extended or modified to support reuse of formal models of self-adaptive systems?*

This research question is related to Challenge 4 from above. We address the research question focussing on two ways of reuse of formal models of SASs: encapsulating common behaviour within an SAS and compositional modelling of an SAS. While the former focusses on reuse within the formal model of an SAS, the latter addresses reuse of (parts of) the formal model across applications and SASs. Both of them have the potential to ease the modelling and maintenance of SASs because (parts of) models of SASs can be reused and maintained individually.

1.2 REMARO

The research presented in this thesis was carried out in the context of the Marie Skłodowska-Curie training network REMARO, focused on developing REliable AI for MARine RObotics. The vision of this training network is to realise reliable autonomy for underwater robotic applications, including designing safety and reliability simultaneously with the deliberative architecture of the robot. My research within this network is focused on the reliability of the control system of an underwater robot and its autonomous manager which implements self-adaptation. Since an underwater robot has to operate in a constantly changing

environment where communication with human operators is limited, there is both a need for self-adaptation and a need for ensuring the safety and reliability of the self-adaptive underwater robot. The interdisciplinary consortium of REMARO, including experts in robotics, provided me with an angle on the problem of ensuring the safety and reliability of SASs. It furthermore helped me to understand the challenges faced in the robotics domain and to find suitable case studies. In particular, many PhD projects within REMARO, including mine, share the case study of an autonomous underwater robot which is used for inspecting a pipeline located on a seabed.

1.3 Thesis Outline

This thesis is structured as follows.

- Chapter 2 establishes background needed to discuss the contributions of this thesis. This includes an introduction to self-adaptive systems and the used formal modelling and analysis techniques.
- Chapter 3 provides a brief summary of the papers included in this thesis.
- Chapter 4 summarises how the research questions have been addressed and provides suggestions for future work.
- Finally, the papers that provide the main contribution of this thesis are included in Papers I to V.

References

- [BH95] J.P. Bowen and M.G. Hinchey. “Ten commandments of formal methods”. In: *Computer* vol. 28, no. 4 (1995), pp. 56–63. DOI: 10.1109/2.375178.
- [Cal+20] Radu Calinescu, Raffaella Mirandola, Diego Perez-Palacin, and Danny Weyns. “Understanding Uncertainty in Self-adaptive Systems”. In: *IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. 2020, pp. 242–251. DOI: 10.1109/ACSOS49614.2020.00047.
- [CW96] Edmund M. Clarke and Jeannette M. Wing. “Formal methods: state of the art and future directions”. In: vol. 28, no. 4 (1996), pp. 626–643. DOI: 10.1145/242223.242257.
- [Hal+99] Anthony Hall, Jonathan P. Bowen, Michael G. Hinchey, Jeannette M. Wing, and C. A. R. Hoare. “Formal Methods”. In: *High-Integrity System Specification and Design*. London: Springer, 1999, pp. 127–230. DOI: 10.1007/978-1-4471-3431-2_3.

- [Pek+23] Irdin Pekaric, Raffaella Groner, Thomas Witte, Jubril Gbolahan Adigun, Alexander Raschke, Michael Felderer, and Matthias Tichy. “A systematic review on security and safety of self-adaptive systems”. In: *Journal of Systems and Software* vol. 203 (2023), p. 111716. DOI: 10.1016/j.jss.2023.111716.
- [Wey20] Danny Weyns. *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons, 2020.

Chapter 2

Background

This chapter provides a brief introduction to the most important topics used for the research conducted in this thesis. The focus of this chapter is on the ideas and intuitions of the explained concepts rather than their formal definitions.

2.1 Self-Adaptive Systems

As systems become more autonomous, there is an increasing need for them to cope with changes in the environment and the system itself. By providing a system with the ability of *self-adaptation*, the system can monitor its environment and adapt itself accordingly. These systems are called *self-adaptive systems* (SASs) [Wey20]. SASs can be modelled as two-layered systems, as shown in Figure 2.1.

Managing Subsystem. The managing subsystem enables the SAS to adapt itself. It receives input from the managed subsystem and the environment, e.g., in the form of component statuses and sensor readings, respectively. It reasons about this information and adapts the managed subsystem if necessary.

A common reference model for the managing subsystem is a MAPE-K feedback loop [KC03]. The managing subsystem *monitors* the environment and the managed subsystem, *analyses* whether an adaptation of the managed subsystem is needed, *plans* how to adapt the managed subsystem, and *executes* this adaptation, where all steps refer to a common *knowledge base*.

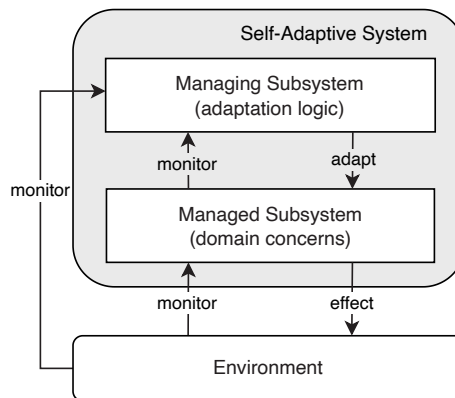


Figure 2.1: The two-layered structure of an SAS [Päß+23]

2. Background

An adaptation can concern different aspects of the system, including adapting parameters of the system and its structure [WWT22]. Structural adaptation concerns adding, removing, or changing components of the system while parametric adaptation involves changing parameters of the overall system.

Managed Subsystem. The managed subsystem of an SAS realises the functionality of the system. It is responsible for the operation of the system and implements the domain concerns. The managed subsystem contains sensors and actuators to sense and affect the environment, respectively. In this thesis, we assume that the managed subsystem is component-based, i.e., it is composed of a set of components that can form different configurations of the managed subsystem. These configurations represent the modes of operation of the managed subsystem that can be changed by the managing subsystem by changing components.

Environment. The environment is everything that is external to the SAS, i.e., everything that cannot be controlled by it. The SAS operates in an environment and both influence each other. The environment affects the operation of the SAS as unexpected events in the environment trigger an adaptation of the SAS. In turn, the SAS affects the environment, e.g., if an autonomous robot moves an object in a room.

Using the two-layered structure of an SAS provides a separation of concerns that ensures that the application and adaptation logic can be changed independently from each other. This eases the system's deployment in different environments and with distinct requirements [Wey20]. Therefore, we consider two-layered SASs in this thesis and explicitly reflect their structure in our formal models. In particular, we model the managing subsystem, the managed subsystem and the environment separately and include how they interact.

Application domains for SASs are diverse, including web services, the internet of things and cyber-physical systems [WWT22]. However, self-adaptation is especially relevant when systems are deployed in environments that are hostile to humans or impossible for them to reach. In these hostile environments, in which communication with human operators might be limited, self-adaptation can be seen as a means to increase the safety of a system. For example, self-adaptation can ensure that the SAS goes into a recovery mode if no other action is possible or before harming itself or its environment.¹

The work described in this thesis was conducted in the context of REMARO, as described in Chapter 1, so we considered physical SASs, mostly robots, in this thesis. We assume that these SASs operate in uncontrolled, uncertain environments that influence the operation of the SAS and enforce the need for adaptation.

¹For a comprehensive introduction to SASs, we refer the interested reader to Weyns [Wey20].

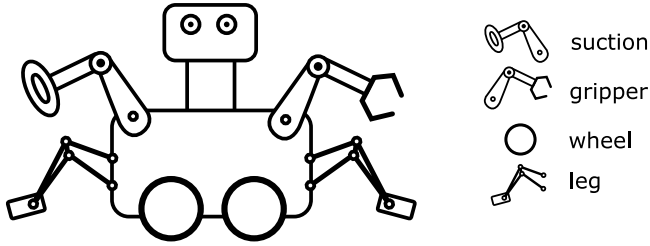


Figure 2.2: An illustration of ARo, the robot used as a running example

Example 2.1.1 (Self-adaptive robot running example). Let us consider an autonomous robot, which we will henceforth call ARo, that is used in a factory. ARo has two arms, one with a mechanical gripper and one with a suction mechanism. The gripper is better suited for opening and closing doors, while the suction arm is better suited for pick-and-place tasks. Each of these tasks can also be done with the other arm. However, opening and closing doors with the suction arm takes more time and energy, and executing pick-and-place tasks with the gripper has a greater risk of damaging objects by dropping them. The arms can have a failure, preventing them from being used. Furthermore, ARo moves using its wheels when operating in an open space, or its legs to pass obstacles such as stairs. For simplicity, we assume that the wheels and legs cannot fail in this example. An illustration of ARo is depicted in Figure 2.2.

To decide which means of grasping and movement to use in a given situation, ARo is equipped with a managing subsystem. The managing subsystem chooses between the arms depending on the task and the failure status of the arms, and between the legs and the wheels depending on the current environment of ARo.

Formal Methods for SASs. Even though there is a growing body of work on formal methods for SASs, assurance of the safety and reliability of SASs is still considered to be a challenge by the community [Cal+20].

Examples of modelling techniques that have been proposed for SASs include extensions of Petri nets [CC21; CGS15; FK18; TKB17], variants of abstract state machines [ARS17; Tar+12], interactive state machines [Yan+14], networks of timed automata [WI23] and rewriting logic [Bru+15]. However, these approaches do not explicitly take the two-layered structure of the SAS and the interaction between the two layers into account, or they do not explicitly model the environment, its uncertainty, and how it affects the operation of the SAS. In this thesis, we show how formal methods allow modelling these elements and their interaction explicitly.

SASs were analysed using different techniques such as model checking [ARS17; CC21; FDW13; WI23], probabilistic model checking [Tar+12], statistical model checking [TKB17], deductive verification [FDW13], and simulation [TKB17]. However, the analysis rarely concerns quantitative aspects, especially relating to the adaptation of the SAS, or the influence of the environment on the operation of the SAS. Indeed, according to a systematic review on testing,

validation and verification for autonomous robotic systems [AMV23] (which includes self-adaptive robotic systems), there is a “relative gap in interventions that perform a quantitative analysis of the system and provide quantitative metrics of quality”. In this thesis, we show how formal methods allow analysing (quantitative) properties of an SAS, especially related to the adaptation, while taking uncertainties of the SAS itself and of the environment into account. Furthermore, we show how different behaviours of the environment influence the operation of the SAS (Papers I to III).

2.2 (Dynamic) Software Product Lines

Software product line engineering [PBL05] is a software engineering paradigm for building a family of software systems that share the same core but differ in some functionalities. The resulting family of systems is called a *software product line* (SPL). The family members of an SPL, called *products*, are distinguished by their *features*, i.e., the functionalities that they only share with a part of the other family members. Definitions of what a feature is vary [CHS08; TP23]. In this thesis, we take the definition of Apel et al. [Ape+13]: “a feature is a characteristic or end-user-visible behaviour of a software system”.

A *feature model* [SHT06] can be used to specify which combinations of features form products of the SPL, i.e., which combinations of features are *valid*. These valid configurations are specified by describing relationships among features, i.e., how features can be combined in a valid way. Examples of the relationships in a features model are the following: feature f requires feature g , or feature h excludes feature l . A feature model can be represented graphically (as in Figure 2.3) or with a Boolean formula over the features of the feature model, called *feature expression*. Feature models reflect the *structural variability* of SPLs. The *behavioural variability* of an SPL can, e.g., be specified in a featured transition system, see Section 2.3.4.²

In this thesis, we show how an SPL can be used to model the structure and behaviour of the configurations of the managed subsystem of an SAS (Papers II and III). Furthermore, we combine the modelling of structural and behavioural variability of SPLs in one compositional model (Paper IV).

Example 2.2.1 (ARo SPL). Let us consider the robot ARo from Example 2.1.1. Some customers would like to buy ARo, but they do not need its self-adaptation capabilities because ARo will only be used in the same environment for the same task. However, different customers require different combinations of the movement and grasping capabilities of ARo. Therefore, the company decides to develop an SPL of ARos, called *ARo SPL*, to meet the customer’s needs.

The features of the ARo SPL are the *suction* arm, the *gripper* arm, the *legs* and the *wheels*, which we denote by s, g, l , and w , respectively. To specify how these features can be combined into products of the ARo SPL, we include the

²For a comprehensive introduction to the engineering of SPLs, we refer the interested reader to Pohl et al. [PBL05].

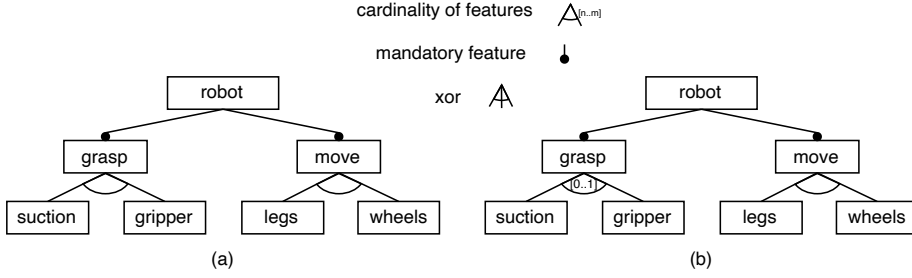


Figure 2.3: The feature model of the ARo SPL (a), and the ARo DSPL (b)

additional features *robot*, *grasp*, and *move*, denoted by r , gr , and m , respectively. This allows modelling the variability of the ARo SPL in a feature model as shown in Figure 2.3 (a).

Every robot in the ARo SPL contains a means to grasp and a means to move, indicated by the mandatory features *grasp* and *move*. For grasping, the customers can choose a gripper or a suction arm, indicated by the sub-features of *grasp* with an xor relationship. Furthermore, they can decide whether they want their robot to move using legs or wheels, indicated by the sub-features of *move* with an xor relationship. Thus, the products described by this feature model are $p_1, \dots, p_4$ with $p_1 = \{r, gr, m, s, l\}$, $p_2 = \{r, gr, m, s, w\}$, $p_3 = \{r, gr, m, g, l\}$, and $p_4 = \{r, gr, m, g, w\}$.

The feature expression ϕ_r corresponding to this feature model is $r \wedge gr \wedge m \wedge (s \vee g) \wedge \neg(s \wedge g) \wedge (l \vee w) \wedge \neg(l \wedge w)$. The feature expression ϕ_r represents the same set of products as the feature model: for each product p_i , $i = 1, \dots, 4$, when setting all feature variables in ϕ_r to true that are included in p_i , then ϕ_r evaluates to true, and these are the only sets of features with this property.

Dynamic Software Product Lines. SPLs are used for *static variability*, i.e., variability that can be resolved at design time. They have been extended to *Dynamic SPLs* (DSPLs) [Hal+13] to facilitate *dynamic variability*, i.e., variability at runtime. As in SPLs, variability in DSPLs can be modelled in a feature model. However, instead of choosing a product at design time that remains static throughout operation, products can be chosen and changed dynamically at runtime. Therefore, DSPLs provide a means to organise variability of a system and are a suitable formalism for SASs [AS22].

Note that there is a different semantics between a product of an SPL and a selected product of a DSPL: In contrast to a product of an SPL, a product of a DSPL describes the current mode of operation of a system. That is, the selected features of the feature model do not describe the system's functionalities, they only describe a subset. Namely, they describe the subset of features that are currently used. In this thesis, we show how this enables modelling (and analysing) SASs using techniques from DSPLs (Papers II and III).

2. Background

Example 2.2.2 (ARo DSPL). Let us consider the robot ARo from Example 2.1.1. In addition to the ARo SPL (Example 2.2.1), the company decides to produce a DSPL of ARos, called *ARo DSPL*. This ARo DSPL has all functionalities (features) described in Example 2.1.1 and can dynamically change between valid feature combinations (products) at runtime, i.e., it can adapt itself.

The variability of the ARo DSPL is modelled in a feature model. The feature model has to reflect that the robot cannot use an arm if both arms had a failure. (Remember that we assumed the legs and wheels cannot have a failure.) Therefore, the feature model of the ARo DSPL in Figure 2.3 (b) specifies that a product can include 0 to 1 sub-features of the feature *grasp*. Thereby, the chosen product reflects the current mode of operation of ARo.

The ARo DSPL contains the same product $p_1, \dots, p_4$ as the ARo SPL in Example 2.2.1. These products reflect the possible combinations of a subfeature of *grasp* and a subfeature of *move*. In addition, the ARo DSPL includes the products $p_5 = \{r, gr, m, l\}$ and $p_6 = \{r, gr, m, w\}$ that include neither a suction nor a gripper arm.

The products of the ARo DSPL can be changed at runtime, enabling a different behaviour of ARo depending on the conditions of the environment, ARo's tasks, and the failure status of its arms. In this way, customers can use the robot for different tasks and in different environments, e.g., for pick-and-place tasks in an open environment and for opening and closing doors in more challenging environments requiring the use of legs. Furthermore, the ARo DSPL can be used in uncertain environments because it is able to change between configurations, i.e., it is able to adapt itself.

Formal Methods for (D)SPLs. The number of products of an SPL can be exponential in the number of features, so managing all products of an SPL can quickly become infeasible [PBL05]. The same holds for modelling and analysing all products of an SPL in a one-by-one approach where every product is modelled and analysed separately. As an alternative, the behaviour of all products of an SPL can be modelled using an all-in-one approach, also called *family-based* approach. In a family-based modelling approach, the behaviour of all products of an SPL is modelled in one single model. This facilitates an all-in-one analysis, also called family-based analysis, where the behaviour of all products of the SPL is analysed at once [Cla+13]. Family-based techniques for modelling and analysing SPLs have been extended to DSPLs, see, for example, [Cor+13; DBK15].

In this thesis, we model SASs with family-based techniques from (D)SPLs and analyse all possible configurations and reconfigurations of an SAS at once using family-based probabilistic model checking (Papers II and III). We provide an overview of modelling and analysis approaches for DSPLs in Paper II and a discussion of using family-based model checking for SPLs in Paper III.

2.3 Formal Modelling and Analysis Techniques

Formal methods [BH95; Hal+99] are tools and techniques based on mathematics to model and analyse a system. Formal methods can be used at any stage of the system development, but are traditionally used to specify the desired properties of a system relating to its behaviour and its structure [Win90]. To specify the desired properties of a system, a specification language with a formal syntax and semantics has to be used.

Creating a formal specification can already help in determining poor design choices, errors, ambiguities, and misconceptions in the design of a system [CW96]. In addition, formal methods allow using different manual and (partly) automated system analyses, including *formal verification*. Formal verification is the process of determining whether a system satisfies its specification. Ideally, a formal verification is (i) automatic (requiring no input from humans), (ii) does not miss any errors (which is also called *complete*), and (iii) never falsely determines that there are errors (which is also called *sound*) [BP25].

However, as soon as the specification contains a notion of reachability, it is not possible to create a method that can take any system and any specification and ensure all of these three properties in a finite amount of time [BP25]. Therefore, every formal verification method has a trade-off between these properties. Brain et al. [BP25] characterise verification techniques using these three properties into three categories, each achieving two of the three required properties: *over-approximate* techniques are automatic and do not miss errors, *under-approximate* techniques are automatic and never falsely determine that there are errors, and *human-assisted* techniques do not miss any errors and never falsely determine that there are errors.

In this thesis, we show how the two-layered structure of an SAS and the environment the SAS operates in can be modelled and analysed using formal techniques. We show how an under-approximate formal verification technique, model checking (Section 2.3.3), can be used to verify properties of the SAS. In the following sections, we provide background on the formal modelling and analysis techniques used in the main contributions of this thesis.

2.3.1 Rewriting Logic

Modelling the two layers of an SAS, the environment it operates in, and the interaction between the SAS and the environment requires a flexible framework that is able to faithfully capture the behaviour of the SAS and the environment. *Rewriting logic* [Mes12] provides this flexible framework. It enables modelling a system with “ ε representational distance”, i.e., the represented system and its representation in rewriting logic are isomorphic structures. This is achieved by providing a flexible way of specifying the *static* and the *dynamic* aspects of a system. For example, the static aspects of an SAS are its states, while the dynamic aspects refer to the control actions taken by the managed subsystem and the adaptations of the managing subsystem. A *rewrite theory*, which models a system, consists of both the static and the dynamic aspects of the system.

2. Background

```
1 sort st .
2 op f : stlist -> st .
3 ceq t = t' if cond .
4 crl [label] :t -> t' if cond
```

Listing 2.1: Structure of a specification of a rewrite theory in Maude

The static aspects of a system are specified in an equational theory (Σ, E) and the dynamic aspects are specified as a set R of rewrite rules. Intuitively, the equational theory consists of a set of operators Σ with which the state of the system can be constructed, and a set E of equations that specify identities among states of the system. A rewrite rule in R intuitively describes how a (part of the) system can evolve. Rewriting logic allows different kinds of analysis including reachability analysis, simulation, and model checking.³

In this thesis, we model an SAS using rewriting logic and analyse it using simulation. We specify the state of an SAS, including the environment, using an equational theory and the behaviour of the two layers of an SAS using rewrite rules (Paper I).

Tools. There are several languages that implement rewriting logic; in this thesis, we use Maude [Cla+07]. Maude implements a general form of rewrite theories and can be used for different kinds of analysis, including reachability analysis through a breadth-first search and LTL model checking (see Section 2.3.3 for an explanation of model checking).

In Maude, an equational theory (Σ, E) can be specified as follows, where all line numbers reference to Listing 2.1. *Sorts* st and *functions* f from a (possibly empty) list of sorts $stlist$ to a sort st can be specified as in Lines 1 and 2. (Conditional) equations expressing the equality of patterns t and t' (if a condition $cond$ holds) can be expressed as in Line 3. Patterns are terms in which functions are replaced by variables. Conditional rewrite rules are specified as in Line 4, where $label$ is a label for the rule and a term matching pattern t can be rewritten to a term with pattern t' if the condition $cond$ holds.⁴

Example 2.3.1 (Rewrite rules for the adaptation logic of ARo). Let us consider the running example of the robot ARo as described in Example 2.1.1. The adaptation rules of ARo, written in Maude-like syntax, are displayed in Listing 2.2. The rule in Lines 1 to 3 specifies that the suction arm should be activated if ARo is supposed to do a pick-and-place task and the suction arm is working, or if ARo is supposed to do a different task than pick-and-place but the gripper is not working. Similarly, the rule in Lines 4 to 6 describes that the gripper arm should be activated if ARo is supposed to do a pick-and-place task and the suction arm is not working, or if ARo is supposed do a different task than pick-and-place and

³For a comprehensive introduction to rewriting logic, we refer the interested reader to Meseguer [Mes12].

⁴For a comprehensive introduction to modelling and analysing distributed systems using Maude, we refer the interested reader to Ölveczky [Ölv17].

```

1 crl [activateSuction] : suction=inactive -> suction=active
2   if (task==pick&place /\ suction_working==true)
3     \/(task!= pick&place /\ gripper_working==false) .
4 crl [activateGripper] : gripper=inactive -> gripper=active
5   if (task==pick&place /\ suction_working==false)
6     \/(task!= pick&place /\ gripper_working==true) .
7 crl [activateWheels] : wheels=inactive -> wheels=active
8   if position!=correct_position /\ space==open_space .
9 crl [activateLegs] : legs=inactive -> legs=active
10  if position!=correct_position /\ space!=open_space .
11 crl [getHelp] : get_help=false -> get_help=true
12  if suction_working==false /\ gripper_working==false .
    
```

Listing 2.2: Rewrite rules for the adaptation logic of ARo

the gripper is working. Lines 7 and 8 specify the adaptation rule for activating the wheels if ARo is not at the correct position and has to navigate in an open space. Furthermore, Lines 9 and 10 state the rule for activating ARo's legs if ARo is not at the correct position and has to navigate in a space that is not an open space (i.e., including obstacles such as stairs). Finally, the rule in Lines 11 and 12 describes an adaptation for getting help when both the suction arm and the gripper arm are not working.

2.3.2 Probabilistic Systems

SASs operate in an uncertain environment and their operation is subject to uncertainty too. For example, the sensor of a self-adaptive robot can fail during its mission and an unexpected gust of wind can push it off course. To include this uncertainty, we model SASs and their environments as probabilistic systems in this thesis (Papers II and III).

A *Markov Chain* (MC) models probabilistic behaviour. An MC is a transition system where the successor state of a state s is chosen according to a probability distribution, i.e., a probability is associated to every transition from s to another state s' . The sum of all probabilities associated to transitions exiting from a state s have to sum up to 1.

A *Markov Decision Process* (MDP) models systems that exhibit both probabilistic and non-deterministic behaviour. Thus, it can be seen as an MC containing non-deterministic transitions in addition to the probabilistic ones. Every transition of an MDP is labelled with an *action*, and the sum of all probabilities associated to transitions with action label a exiting from a state s have to sum up to 1. In an MDP, the probabilistic choices allow modelling uncertainty in the system or its environment, while the non-determinism allows modelling choices.

An MC and an MDP can additionally be enriched by including a cost function that associates a cost to every transition, e.g., a natural number. Costs can either be interpreted as an incentive that should be minimised (as the word “cost” implies) or as an incentive that should be maximised (which would be called “reward” in other contexts).

2. Background

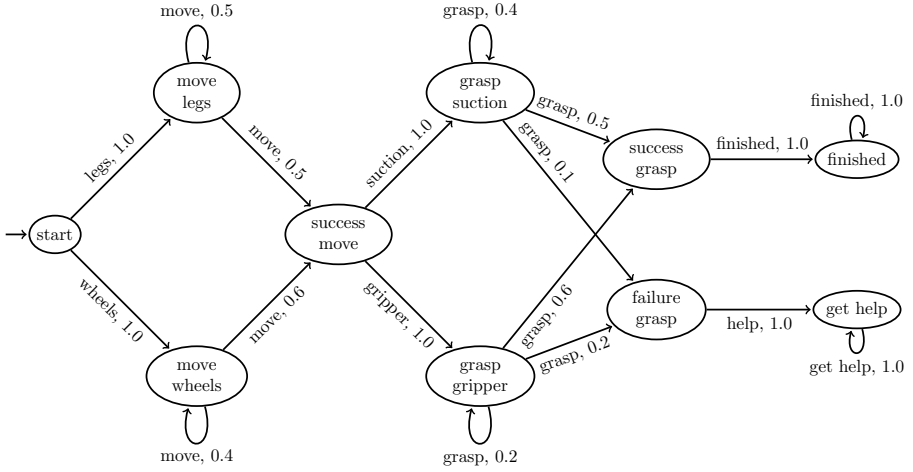


Figure 2.4: An MDP modelling the behaviour of ARO where the labels on the transitions contain the action and the probability of taking this transition, separated by a comma

Example 2.3.2 (MDP for ARO). Let us consider the running example of the robot ARO as described in Example 2.1.1. In this example, we consider the case where ARO cannot adapt itself but has to decide at the beginning of its mission how it wants to move and which form of grasping it wants to use. This corresponds to choosing a product of the ARO SPL in Example 2.2.1. An MDP modelling ARO's behaviour is shown in Figure 2.4.

Let us consider the case where the chosen method of movement are the wheels and the chosen method of grasping is the suction mechanism. At the beginning of the mission, the transition from the state *start* to the state *move wheels* is taken, indicating that ARO starts moving to its target location. From there, ARO stays in the same state, i.e. keeps moving to its target location using the wheels, with a probability of 0.4, or has successfully moved to its target location, indicated by transitioning to the state *success move*, with a probability of 0.6. From the state *success move*, the transition to the state *grasp suction* is taken because the chosen arm is the suction mechanism. In this state, there is a probability of 0.4 of staying in the same state and continuing to try to grasp the object, a probability of 0.5 of successfully grasping the object (and transitioning to the state *success grasp*), and a probability of 0.1 of failing to grasp the object (and transitioning to the state *failure grasp*). From the states *success grasp* and *failure grasp*, the transition to the state *finished* and *get help* is taken, respectively. For model checking (see Section 2.3.3) there are additional loops in the states *finished* and *get help* to avoid deadlock.

The probabilities of the transitions exiting from the state *move legs* are different from the ones exiting from the state *move wheels*. This reflects that moving with wheels is faster and more accurate than moving with legs. Similarly, the probabilities of the transitions existing the state *grasp suction* are different

from the probabilities of the transitions exiting the state *grasp gripper*. This reflects that grasping with the gripper is faster but also more error prone.

2.3.3 Probabilistic Model Checking

Model checking [CE81] is an automated technique to check if a model of a system satisfies the specification of the system. Following Brain et al. [BP25] model checking can be characterised as an under-approximate verification technique because it is automatic and never falsely determines that there are errors.

To use model checking for a probabilistic systems, the behaviour of the system can be modelled as an MC or an MDP with a finite set of states. The specification against which the model of the system is checked is stated in a logic. For probabilistic systems, this can be, e.g., a temporal probabilistic logic such as LTL with probabilities [Bai98] and probabilistic CTL (PCTL) [HJ94].

A *model checker*, i.e., a tool for model checking, explores the states of the system model and checks if the desired property holds. If the model checker finds that the property does not hold, it provides a counterexample. This counterexample can be used to refactor the model of the system, the property, or the design of the system. If no counterexample is found, the model satisfies the desired property. It can also happen that the tool runs out of memory, in which case there are different techniques to overcome this problem, e.g., finding suitable abstractions of the system model.⁵

When probabilities of transitions of an MC or MDP are unknown, *parametric model checking* [Daw04] can be used to analyse the system. Parametric model checking can be used to analyse which influence a parameter has on the behaviour of the system, e.g., how unpredictable winds influence the flight path of a drone. Furthermore, it can be used to determine for which values of a parameter the system satisfies a property, e.g., in which range the wind speed can be such that a drone can reach its goal.

In this thesis, we use probabilistic model checking to analyse the safety and reliability of SASs operating in an uncertain environment (Papers II and III). Furthermore, we explore how different behaviours of the environment influence the operation of an SAS using parametric probabilistic model checking (Paper III).

Properties. It is possible to analyse both *qualitative* and *quantitative* properties of MCs and MDPs. Qualitative properties usually express that an event will almost surely (not) happen, i.e., with probability 1 (or 0). Quantitative properties allow putting constraints on the probability that a certain event happens, or can ask for the probability of an event happening. A qualitative property could be, e.g., “Is the probability of reaching an unwanted state at most 0?”, while a quantitative property could be, e.g., “Is the (minimum) probability of reaching

⁵For a comprehensive introduction to (probabilistic) model checking, including different logics, MCs and MDPs, we refer the interested reader to Baier et al. [BK08].

2. Background

a wanted state above 0.99?” or “What is the (minimum) probability of reaching a wanted state?”.

Model checking an MDP involves resolving the non-deterministic choices. A *scheduler* resolves non-determinism in an MDP by choosing actions, e.g., based on the previous execution trace. For quantitative properties of MDPs, we can ask for the *minimal* or *maximal* probability. The scheduler leading to this minimal or maximal value can be returned.

Model checking allows analysing qualitative and quantitative properties that are important for analysing SASs.

Safety and Liveness. Safety properties allow analysing the safety of SASs, e.g., that the battery of an autonomous robot will never deplete. With liveness properties, we can analyse the reliability of SASs, e.g., that an autonomous robot can always reach its goal (and will not reach a deadlock).

Costs. If the analysed MC or MDP contains a cost function, this cost function can be used to analyse properties related to cost. Examples for costs important for analysing SASs are time and energy. Including a cost function that reflects the time allows analysing properties such as “What is the (minimal/maximal) expected time for completing the mission?”. When a cost function is included that reflects the energy usage of the SAS, properties such as “Is the (minimal/maximal) expected energy consumption for finishing the mission below x ?”.

Multi-objective. In an MDP, multi-objective queries allow analysing trade-offs between properties [FKP12].

Achievability query. It is possible to ask if there exists a scheduler for the MDP such that multiple properties are achieved at the same time. For example, we can analyse whether an autonomous robot can complete its mission with a time less than 10 and an energy less than 15 (when using cost functions).

Numerical query. Given a probability/cost threshold for a set of properties, a numerical query allows determining, how well another property can be achieved. For example, we want to ensure that an autonomous robot completes its mission with an energy of less than 15 and we want to find the optimal scheduler for minimising the time it takes to complete the mission.

Pareto query. Intuitively, a Pareto query returns the schedulers that give the best trade-off between a set of properties. For example, if we want to minimise both the energy and the time that an autonomous robot takes for completing its mission, a Pareto query can give schedulers that provide the best trade-off between minimising time and energy.

In this thesis, we analyse safety, liveness, reachability and cost properties of an SAS operating in a fixed, uncertain environment (Papers II and III). Furthermore, we use multi-objective queries to analyse trade-offs (Paper III).

Example 2.3.3 (Properties for ARo). Let us consider the MDP modelling the behaviour of the robot ARo as introduced in Example 2.3.2. We can, e.g., analyse the following properties.

- Is the probability of reaching the state *finished* above 0.9 for all possible schedulers?
- What is the maximum probability of reaching the state *get help*?

When analysing the second property, we can also ask the model checker to provide a scheduler that leads to this maximum probability.

Tools. In this thesis, we use the probabilistic model checkers PRISM [KNP11] (Papers II and III) and Storm [Hen+22] (Paper III), which are able to analyse MPDs. Storm supports (among others) model files specified in the PRISM input language, so we will focus on the PRISM input language here.

In the PRISM input language, a model can be composed of several *modules* that communicate and synchronise with one another. Each module can contain *variables* that, e.g., reflect the state of a component of the system. The state of the system at a given time is represented by the union of the values of all variables in all modules at this time. Transitions of the system are modelled by *commands* in the modules. A command has the following form

```
[action] guard -> prob_1: update_1 + ... + prob_n:update_n;
```

It is read as follows: If the guard *guard*, which is a Boolean formula over the model's variables (which might belong to a different module), is satisfied, then with probability *prob_i*, the variables are changed as specified in *update_i* for $i \in \{1, \dots, n\}$. The probabilities *prob_1* to *prob_n* have to sum up to 1. The command models the transitions from a state satisfying *guard* to *n* possibly different states where the variables have been updated as specified in *update_i*. The label *action* can be used for synchronisation of modules.

Example 2.3.4 (PRISM model fragment for the MDP of ARo). Let us consider the MDP of the robot ARo as introduced in Example 2.3.2. This MDP can be specified in PRISM; we provide a fragment of the PRISM model in Listing 2.3. The keyword **mdp** in Line 1 specifies that the model is an MDP. States are specified as constants in Lines 3 to 6, and the variable *s* ranges over all states and is initialised with the state *start*, see Line 10. The transitions between the states *start*, *moveLegs*, *moveWheels*, and *successMove* are specified in Lines 12 to 15 as shown in Figure 2.4. The actions on the transitions in Figure 2.4 correspond to the labels in the PRISM model.

The properties in Example 2.3.2 can be specified in the PRISM input language as shown in Listing 2.4. The model checker returns that the probability of eventually reaching the state *finished* is not above 0.9 (see Line 1) and the maximum probability of finally reaching the state *getHelp* (see Line 2) is approximately 0.25.

2. Background

```
1 mdp
2
3 const int start = 0;
4 const int moveLegs = 1;
5 const int moveWheels = 2;
6 const int successMove = 3;
7 // Declaration of the other states
8
9 module robot
10   s : [0..9] init start;
11
12   [legs] (s=start) -> 1: (s'=moveLegs);
13   [wheels] (s=start) -> 1: (s'=moveWheels);
14   [move] (s=moveLegs) -> 0.5: (s'=moveLegs) + 0.5: (s'=successMove);
15   [move] (s=moveWheels) -> 0.4: (s'=moveWheels) + 0.6: (s'=successMove);
16   // Declaration of the other transitions
17 endmodule
```

Listing 2.3: A fragment of the PRISM model for the MDP of ARo as specified in Example 2.3.2

```
1 P>0.9 [F s=finished];
2 Pmax=? [F s=getHelp];
```

Listing 2.4: Properties for ARo

```
1 start:legs
2 moveLegs:move
3 moveWheels:move
4 successMove:gripper
5 graspSuction:grasp
6 graspGripper:grasp
7 successGrasp:help
```

Listing 2.5: A scheduler for the property $P_{\max}=? [F \text{ s=getHelp}]$

When using PRISM or Storm to analyse the property in Line 2, a scheduler leading to the provided value can be obtained. When analysing the PRISM model (not only the fragment specified above) for the maximum probability of finally reaching the state `getHelp` (see Line 2), the strategy in Listing 2.5 is produced. It specifies for every state that is relevant for the scheduler, which action should be taken. For example, in the state `start`, the action `legs` should be taken.

2.3.4 Featured Transition Systems

Featured Transition Systems (FTSs) [Cla+13] are a family-based modelling approach for SPLs. They extend transition systems to allow the specification of the behaviour of all products of an SPL in one single model. This model makes common behaviour of products explicit. Given an SPL, an FTS is a transition system where each transition is additionally labelled with a *feature expression*, i.e., a Boolean expression over the features of the SPL. The feature expression

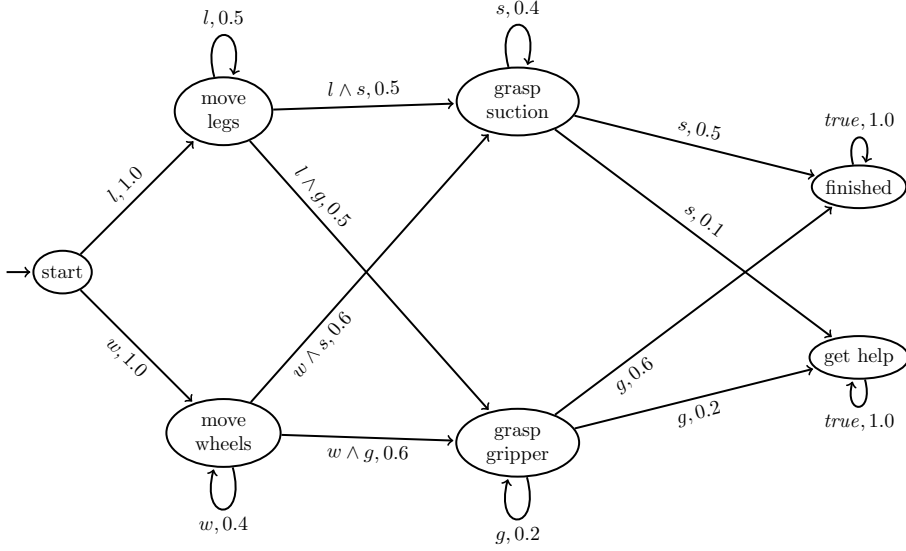


Figure 2.5: A featured MDP for the ARo SPL where the labels on the transitions contain a feature expression and the probability of taking this transition, separated by a comma

on a transition reflects which products can take this transition. The transition system modelling the behaviour of a product p is the *projection* of the FTS to p . That is, it is obtained from the FTS by removing all transitions that p does not satisfy and all states that are unreachable after this procedure.

Featured (discrete time) MCs [Rod+15; VK13] and featured MDPs [DBK15] are variants of FTSs that enable modelling probabilistic systems. The projection of a featured MC on a product yields an MC, while the projection of a featured MDP on a product yields an MDP.⁶

Example 2.3.5 (Featured MDP for the ARo SPL). Let us consider the ARo SPL as introduced in Example 2.2.1. Each product of the SPL has a different behaviour, but products also share parts of their behaviour. For example, the products $p_1 = \{r, gr, m, s, l\}$ and $p_2 = \{r, gr, m, s, w\}$ of Example 2.2.1 both contain the suction arm. Thus, their behaviour relating to grasping is the same.

We can model the behaviour of all products of the SPL in a featured MDP as depicted in Figure 2.5, where we omit the action labels on the transitions for better readability. Each transition has an associated probability and an associated feature expression that indicates for which products this transition can be taken.

Let us consider the product $p_1 = \{r, gr, m, s, l\}$ and create the projection of the featured MDP to p_1 . The product p_1 does not contain the features w and

⁶For a comprehensive introduction to FTSs, we refer the interested reader to Cordy et al. [Cor+19].

2. Background

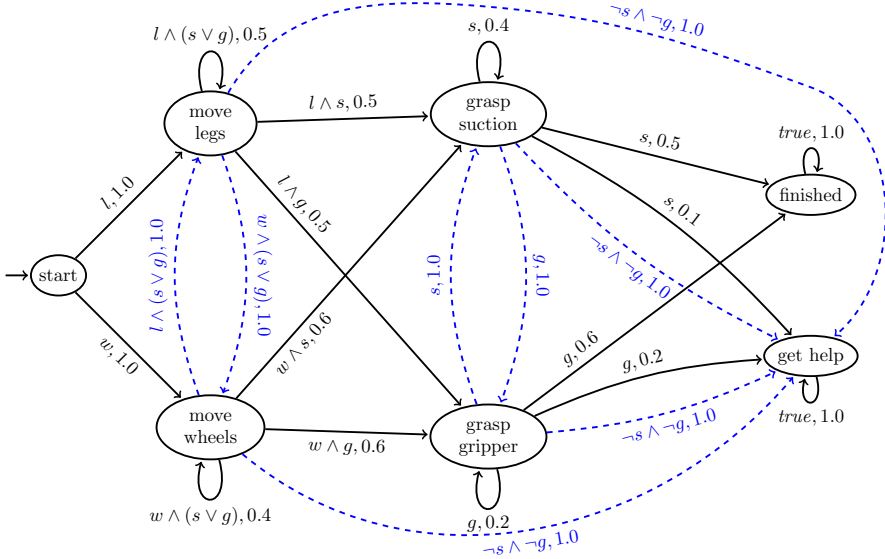


Figure 2.6: A featured MDP for the ARo DSPL; the labels on the transitions contain a feature guard and the probability of taking this transition, separated by a comma; the blue, dashed transitions are transitions between products, enabled by the feature controller

g . Therefore, the projection of the featured MDP to p_1 does not contain the transitions leading to or exiting from the states *move wheels* and *grasp gripper*, nor the states themselves. The projection of the featured MDP to p_1 contains all other states and transitions, and the feature expressions on the transitions can be omitted. The resulting MDP models the behaviour of p_1 .

Modelling the Behaviour of a DSPL as a Featured MDP/MC. Variants of FTSs have been introduced to model DSPLs [Cor+13; DBK15; Dub21]. In this thesis, we use the work of Dubslaff et al. [DBK15; Dub21]: The behaviour of all products of a DSPL can be modelled as a featured MDP. A *feature controller* models how the products of the DSPL are changed. Thus, this approach allows modelling the managed subsystem of an SAS as a featured MDP and the managing subsystem of an SAS as a feature controller. We elaborate more on how SASs can be modelled using this technique for DSPLs in Paper II.

Example 2.3.6 (Featured MDP for the ARo DSPL). Consider the ARo DSPL as described in Example 2.2.2. We assume that either the suction arm or the gripper arm does not have a failure when deploying the ARo DSPL. Furthermore, we do not include the behaviour of the feature controller in this example. The feature controller acts according to the adaptation logic of ARo, see also Examples 2.3.1 and 2.3.9.

The behaviour of all products of the ARo DSPL, modelled as a featured MDP, is displayed in Figure 2.6. The featured MDP also reflects the possible switches between the products, displayed as blue, dashed transitions. ARo is deployed with a configuration including one of the arms, i.e., using one of the products $p_1, \dots, p_4$. When moving to its target location (in the states *move wheels* and *move legs*) and encountering a different environment than before, the managing subsystem of ARo deactivates the currently active feature related to moving and activates the other one. This leads to ARo taking one of the blue, dashed transitions between *move legs* and *move wheels*. When ARo is trying to grasp something but the currently used arm fails, the managing subsystem deactivates this arm and activates the other one if possible. This leads to ARo taking one of the blue, dashed transitions between *grasp suction* and *grasp gripper*. If both the suction arm and the gripper arm have a failure, a blue, dashed transition to the state *get help* is taken immediately. This reflects the behaviour of the products p_5 and p_6 which include neither the suction arm nor the gripper.

Analysis of FTSs. In this thesis, we analyse featured MDPs, i.e., variants of FTSs, using family-based probabilistic model checking (Papers II and III). Model checking for SPLs whose behaviour is modelled as an FTS works as follows: To check whether a model of the behaviour of the SPL (an FTS) satisfies a logic formula ϕ , we have to determine for every product p of the SPL whether ϕ is satisfied in the projection of the FTS to p . This can be done using a one-by-one approach, where classical model checking is applied for every product separately, called *product-based model checking*. However, by modelling the behaviour of the whole SPL in one single model, FTSs enable analysing all products of an SPL at once, called *family-based model checking*. Since the product space of an SPL can be very large, family-based model checking can provide an advantage over product-based model checking [Cla+13]. There are also approaches for family-based model checking of featured MCs and featured MDPs [Bee+20; DBK15; Rod+15]. A discussion about the analysis of FTSs is included in Papers II and III.

Tools. In this thesis, we use ProFeat [Chr+18] for probabilistic family-based model checking. ProFeat allows specifying the behaviour of a DSPL with an explicit representation of the feature model, *feature modules* for specifying the behaviour of the features, and a *feature controller* for activating and deactivating features. That is, it allows us to specify a DSPL as a featured MDP with a feature controller, building on Dubslaff et al. [DBK15]. ProFeat extends the input language of PRISM such that features of the feature model can be used in the feature modules as guards in commands to determine for which product the specified transitions can be taken. The feature controller can activate and deactivate features of the feature model to enable and disable transitions of the other modules. PRISM can be used for the analysis of (quantitative) properties of the DSPL.

2. Background

```
1 mdp
2
3 root feature
4     all of robot;
5     modules robot;
6 endfeature
7 feature robot
8     all of grasp, move;
9 endfeature
10 // Declaration of the other features
11
12 const int start = 0;
13 const int moveLegs = 1;
14 // Declaration of the other states
15
16 module robot
17     s : [0..7] init start;
18
19     // From state start
20     [] (s=start & active(legs)) -> 1: (s'=moveLegs);
21     [] (s=start & active(wheels)) -> 1: (s'=moveWheels);
22
23     // From state moveLegs
24     [] (s=moveLegs & active(legs) & active(suction)) ->
25         0.5: (s'=moveLegs) + 0.5: (s'=graspSuction);
26     [] (s=moveLegs & active(legs) & active(gripper)) ->
27         0.5: (s'=moveLegs) + 0.5: (s'=graspGripper);
28     [] (s=moveLegs & active(wheels) & (active(suction) | active(gripper))) ->
29         1.0: (s'=moveWheels);
30     [] (s=moveLegs & !active(suction) & !active(gripper)) -> 1.0: (s'=getHelp);
31
32 // Declaration of the other transitions
33 endmodule
```

Listing 2.6: A fragment of the ProFeat model for the featured MDP of the ARO DSPL

Example 2.3.7 (ProFeat model fragment for the featured MDP of the ARO DSPL). Let us consider the featured MDP of the ARO DSPL as described in Example 2.3.6. This model can be implemented in ProFeat; we provide a fragment of the ProFeat model in Listing 2.6. The features of the feature model are specified in **feature ... endfeature** blocks (Lines 7 to 9), the root feature in a **root feature ... endfeature** block (Lines 3 to 6). Each **(root) feature ... endfeature** block contains the subfeatures of the specified feature and the relationship between these subfeatures. For example, in Line 8, we specify that the feature **robot** contains the features **grasp** and **move** as mandatory subfeatures. Furthermore, each block contains information on which modules implement the behaviour of the specified feature (Line 5) and cost functions can be included. In this example, the behaviour of all features (and thus of all products) is specified in a single module, similar to the module in Example 2.3.4. However, in this ProFeat module, transitions are also guarded by feature expressions. When analysing the model with ProFeat, the results are given for every product of the DSPL.

In addition to the featured MDP, we can also specify a feature controller for the ARO DSPL in ProFeat. The feature controller reflects the behaviour of

the managing subsystem of ARo. ProFeat allows analysing a run of the system including the feature controller. That is, it allows analysing the self-adaptive behaviour of ARo.

2.3.5 Bundle Event Structures

Event structures [Win86] have been introduced to model concurrent processes. A process is modelled as *events* where relations between events can be described in the form of *conflicts* and *causalities*. Event structures describe the behaviour of a process as a set of traces of events. A conflict between events e and e' intuitively means that e and e' cannot occur in the same trace, while causality describes enabling of events, i.e., a causality from e to e' describes that e' can only occur in a trace if e has already occurred in this trace. Thereby, causality describes a partial order on events.

In *bundle event structures* (BESs) [Lan92], used in this thesis, the causality relation can describe *conjunctive* and *disjunctive* causality. Conjunctive causality allows specifying that a set of events has to occur before an event e in a trace, while disjunctive causality means that exactly one element of a set of events has to occur before an event e in a trace.

In this thesis, we attach behaviour in the form of BESs to the features of a feature model (Paper IV). This allows us to obtain a compositional model of the structure and behaviour of an SPL. Thereby, we provide a basis for analysing SPLs using this compositional model.

Example 2.3.8 (BES for ARo). Let us consider the running example of the robot ARo as described in Example 2.1.1. In this example, we consider the case where ARo cannot adapt itself but has to decide at the beginning of its mission how it wants to move and which form of grasping it wants to do. This corresponds to choosing a product of the ARo SPL in Example 2.2.1.

The behaviour of ARo is described by the BES in Figure 2.7. The events of the BES are *moveLegs*, *moveWheels*, *graspSuction*, *graspGripper*, *finish*, and *getHelp*. The conflict relationships express that the conflicting events cannot both occur in a run of the system, e.g., it is not possible to move with both the legs and the wheels. The causality relationships express that exactly one of two events has to come before another event, e.g., that either *moveLegs* or *moveWheels* has to come before *graspSuction* (i.e., ARo first has to move to the correct position before it can do a pick-and-place task). Note that we use disjunctive causality here, meaning that exactly one of the causing events has to occur, not both of them.

The BES describes the behaviour of ARo for every choice of product of the SPL. For example, when moving with the wheels and using the suction arm, the BES describes the following traces (including the prefixes of these traces): $\langle \text{moveWheels} \cdot \text{graspSuction} \cdot \text{finish} \rangle$ and $\langle \text{moveWheels} \cdot \text{graspSuction} \cdot \text{getHelp} \rangle$.

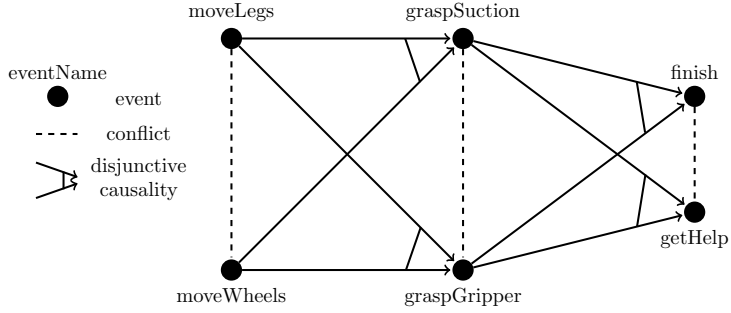


Figure 2.7: An illustration of the BES describing the behaviour of ARO (following [Lan92])

2.3.6 Decision Trees

A *decision tree* (DT) [Qui86] can be used to represent a control strategy. The strategy is described in the form of a tree whose inner nodes are *decision nodes* and whose leaf nodes are *action nodes*. Each decision node is labelled by a predicate containing system variables, e.g., sensor input, and has two outgoing edges. One edge is taken if the predicate is fulfilled and one if it is not fulfilled. Each action node represents a set of control actions. The DT can be evaluated by evaluating its predicates with the current state of the system (starting from the root) and taking the respective edges until one arrives at an action node. This action node includes a set of possible control actions. DTs are often synthesised such that the control actions in the action nodes guarantee safety properties (if the predicates leading to this action are satisfied). DTs are also used for explaining synthesised control strategies due to their simplistic nature [Ash+21].

A *decision diagram* (DD) is a DT that allows multiple incoming edges for every decision and action node. That is, it allows for a smaller representation of a control strategy than a DT by making common decisions and actions explicit.

In this thesis, we extend DDs by introducing *templates* that can encapsulate common sub-diagrams of a DD (Paper V). This allows modelling two-layered SASs in which the managing subsystem adapts the control strategy of the managed subsystem. Furthermore, it increases the explainability of the control strategies of the two layers of an SAS.

Example 2.3.9 (DT for the adaptation logic of ARO). Let us consider the running example of the robot ARO as described in Example 2.1.1. The adaptation logic of ARO is represented by a DT as shown in Figure 2.8. As an example, consider a situation where ARO should do a pick-and-place task, is at the correct position and the suction arm is working. Then the first predicate “task==pick&place” evaluates to true, so the left edge to “position==correct position” is taken. This predicate also evaluates to true, so the left edge to “suction working==true” is taken, which again evaluates to true, leading to the control action “activate suction”. This is the adaptation that ARO takes in the described situation.

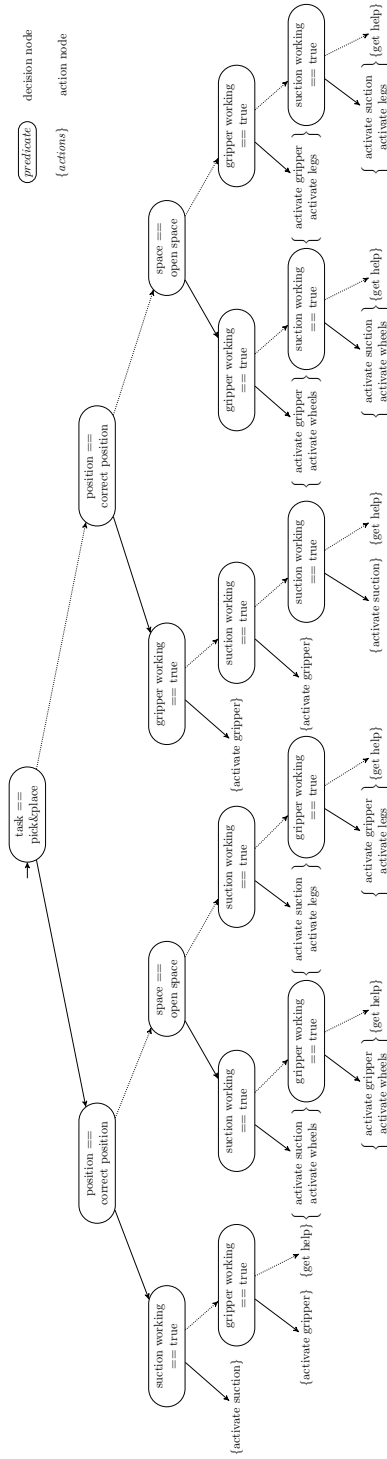


Figure 2.8: A decision tree for the adaptation logic of ARo where filled edges are taken if the predicate is satisfied and dotted ones otherwise.

References

- [AMV23] Hugo Araujo, Mohammad Reza Mousavi, and Mahsa Varshosaz. “Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review”. In: *ACM Transactions on Software Engineering and Methodology* vol. 32, no. 2 (2023), 51:1–51:61. DOI: [10.1145/3542945](https://doi.org/10.1145/3542945).
- [Ape+13] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Germany: Springer, 2013. DOI: [10.1007/978-3-642-37521-7](https://doi.org/10.1007/978-3-642-37521-7).
- [ARS17] Paolo Arcaini, Elvinia Riccobene, and Patrizia Scandurra. “Formal Design and Verification of Self-Adaptive Systems with Decentralized Control”. In: *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* vol. 11, no. 4 (2017), pp. 1–35. DOI: [10.1145/3019598](https://doi.org/10.1145/3019598).
- [AS22] Oscar Aguayo and Samuel Sepúlveda. “Variability Management in Dynamic Software Product Lines for Self-Adaptive Systems—A Systematic Mapping”. In: *Applied Sciences* vol. 12, no. 20 (2022). DOI: [10.3390/app122010240](https://doi.org/10.3390/app122010240).
- [Ash+21] Pranav Ashok, Mathias Jackermeier, Jan Kretínský, Christoph Weinhuber, Maximilian Weininger, and Mayank Yadav. “dtControl 2.0: Explainable Strategy Representation via Decision Tree Learning Steered by Experts”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12652. Lecture Notes in Computer Science. Springer, 2021, pp. 326–345. DOI: [10.1007/978-3-030-72013-1_17](https://doi.org/10.1007/978-3-030-72013-1_17).
- [Bai98] Christel Baier. *On Algorithmic Verification Methods for Probabilistic Systems*. Universität Mannheim. Habilitation thesis. 1998.
- [Bee+20] Maurice H. ter Beek, Axel Legay, Alberto Lluch Lafuente, and Andrea Vandin. “A Framework for Quantitative Modeling and Analysis of Highly (Re)configurable Systems”. In: *IEEE Transaction on Software Engineering* vol. 46, no. 3 (2020), pp. 321–345. DOI: [10.1109/TSE.2018.2853726](https://doi.org/10.1109/TSE.2018.2853726).
- [BH95] J.P. Bowen and M.G. Hinchey. “Ten commandments of formal methods”. In: *Computer* vol. 28, no. 4 (1995), pp. 56–63. DOI: [10.1109/2.375178](https://doi.org/10.1109/2.375178).
- [BK08] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. USA: MIT Press, 2008.
- [BP25] Martin Brain and Elizabeth Polgreen. “A Pyramid Of (Formal) Software Verification”. In: *Formal Methods*. Cham: Springer Nature Switzerland, 2025, pp. 393–419.

-
- [Bru+15] Roberto Bruni, Andrea Corradini, Fabio Gadducci, Alberto Lluch Lafuente, and Andrea Vandin. “Modelling and Analyzing Adaptive Self-Assembly Strategies with Maude”. In: *Science of Computer Programming* vol. 99 (2015), pp. 75–94. DOI: [10.1016/j.scico.2013.11.043](https://doi.org/10.1016/j.scico.2013.11.043).
- [Cal+20] Radu Calinescu, Raffaella Mirandola, Diego Perez-Palacin, and Danny Weyns. “Understanding Uncertainty in Self-adaptive Systems”. In: *IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. 2020, pp. 242–251. DOI: [10.1109/ACSOS49614.2020.00047](https://doi.org/10.1109/ACSOS49614.2020.00047).
- [CC21] Matteo Camilli and Lorenzo Capra. “Formal Specification and Verification of Decentralized Self-Adaptive Systems Using Symmetric Nets”. In: *Discrete Event Dynamic Systems* vol. 31, no. 4 (2021), pp. 609–657. DOI: [10.1007/s10626-021-00343-3](https://doi.org/10.1007/s10626-021-00343-3).
- [CE81] Edmund M. Clarke and E. Allen Emerson. “Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic”. In: *Logics of Programs*. Ed. by Dexter Kozen. Vol. 131. Lecture Notes in Computer Science. Springer, 1981, pp. 52–71. DOI: [10.1007/BFB0025774](https://doi.org/10.1007/BFB0025774).
- [CGS15] Matteo Camilli, Angelo Gargantini, and Patrizia Scandurra. “Specifying and verifying real-time self-adaptive systems”. In: *International Symposium on Software Reliability Engineering (ISSRE)*. 2015, pp. 303–313. DOI: [10.1109/ISSRE.2015.7381823](https://doi.org/10.1109/ISSRE.2015.7381823).
- [Chr+18] Philipp Chrszon, Clemens Dubsloff, Sascha Klüppelholz, and Christel Baier. “ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking”. In: *Formal Aspects of Computing* vol. 30, no. 1 (2018), pp. 45–75. DOI: [10.1007/s00165-017-0432-4](https://doi.org/10.1007/s00165-017-0432-4).
- [CHS08] Andreas Classen, Patrick Heymans, and Pierre-Yves Schobbens. “What’s in a Feature: A Requirements Engineering Perspective”. In: *International Conference on Fundamental Approaches to Software Engineering (FASE)*. Ed. by José Luiz Fiadeiro and Paola Inverardi. Vol. 4961. Lecture Notes in Computer Science. Germany: Springer, 2008, pp. 16–30. DOI: [10.1007/978-3-540-78743-3_2](https://doi.org/10.1007/978-3-540-78743-3_2).
- [Cla+07] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn L. Talcott. *All About Maude - A High-Performance Logical Framework, How to Specify, Program and Verify Systems in Rewriting Logic*. Vol. 4350. Lecture Notes in Computer Science. Springer, 2007. DOI: [10.1007/978-3-540-71999-1](https://doi.org/10.1007/978-3-540-71999-1).

2. Background

- [Cla+13] Andreas Classen, Maxime Cordy, Pierre-Yves Schobbens, Patrick Heymans, Axel Legay, and Jean-François Raskin. “Featured Transition Systems: Foundations for Verifying Variability-Intensive Systems and Their Application to LTL Model Checking”. In: *IEEE Transaction on Software Engineering* vol. 39, no. 8 (2013), pp. 1069–1089. DOI: [10.1109/TSE.2012.86](https://doi.org/10.1109/TSE.2012.86).
- [Cor+13] Maxime Cordy, Andreas Classen, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Model Checking Adaptive Software with Featured Transition Systems”. In: *Assurances for Self-Adaptive Systems: Principles, Models, and Techniques*. Ed. by Javier Cámara, Rogério de Lemos, Carlo Ghezzi, and Antónia Lopes. Vol. 7740. Lecture Notes in Computer Science. Germany: Springer, 2013, pp. 1–29. DOI: [10.1007/978-3-642-36249-1_1](https://doi.org/10.1007/978-3-642-36249-1_1).
- [Cor+19] Maxime Cordy, Xavier Devroey, Axel Legay, Gilles Perrouin, Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, and Jean-François Raskin. “A Decade of Featured Transition Systems”. In: *From Software Engineering to Formal Methods and Tools, and Back*. Ed. by Maurice H. ter Beek, Alessandro Fantechi, and Laura Semini. Vol. 11865. Lecture Notes in Computer Science. Germany: Springer, 2019, pp. 285–312. DOI: [10.1007/978-3-030-30985-5_18](https://doi.org/10.1007/978-3-030-30985-5_18).
- [CW96] Edmund M. Clarke and Jeannette M. Wing. “Formal methods: state of the art and future directions”. In: vol. 28, no. 4 (1996), pp. 626–643. DOI: [10.1145/242223.242257](https://doi.org/10.1145/242223.242257).
- [Daw04] Conrado Daws. “Symbolic and Parametric Model Checking of Discrete-Time Markov Chains”. In: *International Colloquium on Theoretical Aspects of Computing (ICTAC)*. Ed. by Zhiming Liu and Keijiro Araki. Vol. 3407. Lecture Notes in Computer Science. Germany: Springer, 2004, pp. 280–294. DOI: [10.1007/978-3-540-31862-0_21](https://doi.org/10.1007/978-3-540-31862-0_21).
- [DBK15] Clemens Dubslaff, Christel Baier, and Sascha Klüppelholz. “Probabilistic Model Checking for Feature-Oriented Systems”. In: *Transactions on Aspect-Oriented Software Development XII*. Ed. by Shigeru Chiba, Éric Tanter, Erik Ernst, and Robert Hirschfeld. Vol. 8989. Lecture Notes in Computer Science. Germany: Springer, 2015, pp. 180–220. DOI: [10.1007/978-3-662-46734-3_5](https://doi.org/10.1007/978-3-662-46734-3_5).
- [Dub21] Clemens Dubslaff. “Quantitative Analysis of Configurable and Reconfigurable Systems”. PhD thesis. TU Dresden, 2021.
- [FDW13] Michael Fisher, Louise Dennis, and Matt Webster. “Verifying autonomous systems”. In: *Communications of the ACM* vol. 56, no. 9 (2013), pp. 84–93. DOI: [10.1145/2494558](https://doi.org/10.1145/2494558).
- [FK18] Muhammad Ilyas Fakhir and Syed Asad Raza Kazmi. “Formal Specification and Verification of Self-Adaptive Concurrent Systems”. In: *IEEE Access* vol. 6 (2018), pp. 34790–34803. DOI: [10.1109/ACCESS.2018.2849821](https://doi.org/10.1109/ACCESS.2018.2849821).

-
- [FKP12] Vojtěch Forejt, Marta Kwiatkowska, and David Parker. “Pareto Curves for Probabilistic Model Checking”. In: *Automated Technology for Verification and Analysis*. Ed. by Supratik Chakraborty and Madhavan Mukund. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 317–332.
 - [Hal+13] Svein Hallsteinsen, Mike Hinchey, Sooyong Park, and Klaus Schmid. “Dynamic Software Product Lines”. In: *Systems and Software Variability Management: Concepts, Tools and Experiences*. Ed. by Rafael Capilla, Jan Bosch, and Kyo-Chul Kang. Germany: Springer, 2013, pp. 253–260. DOI: [10.1007/978-3-642-36583-6_16](https://doi.org/10.1007/978-3-642-36583-6_16).
 - [Hal+99] Anthony Hall, Jonathan P. Bowen, Michael G. Hinchey, Jeannette M. Wing, and C. A. R. Hoare. “Formal Methods”. In: *High-Integrity System Specification and Design*. London: Springer, 1999, pp. 127–230. DOI: [10.1007/978-1-4471-3431-2_3](https://doi.org/10.1007/978-1-4471-3431-2_3).
 - [Hen+22] Christian Hensel, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. “The Probabilistic Model Checker Storm”. In: *International Journal on Software Tools for Technology Transfer* vol. 24, no. 4 (2022), pp. 589–610. DOI: [10.1007/S10009-021-00633-Z](https://doi.org/10.1007/S10009-021-00633-Z).
 - [HJ94] Hans Hansson and Bengt Jonsson. “A Logic for Reasoning about Time and Reliability”. In: *Formal Aspects of Computing* vol. 6, no. 5 (1994), pp. 512–535. DOI: [10.1007/BF01211866](https://doi.org/10.1007/BF01211866).
 - [KC03] Jeffrey O. Kephart and David M. Chess. “The Vision of Autonomic Computing”. In: *IEEE Computer* vol. 36, no. 1 (2003), pp. 41–50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
 - [KNP11] Marta Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of Probabilistic Real-Time Systems”. In: *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV 2011)*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Germany: Springer, 2011, pp. 585–591. DOI: [10.1007/978-3-642-22110-1_47](https://doi.org/10.1007/978-3-642-22110-1_47).
 - [Lan92] Rom Langerak. “Bundle event structures: a non-interleaving semantics for LOTOS”. In: *Proceedings of the 5th IFIP TC6/WG6.1 International Conference on Formal Description Techniques for Distributed Systems and Communication Protocols (FORTE’92)*. Ed. by Michel Diaz and Roland Groz. Vol. C-10. IFIP Transactions. The Netherlands: North-Holland, 1992, pp. 331–346.
 - [Mes12] José Meseguer. “Twenty years of rewriting logic”. In: *The Journal of Logic and Algebraic Programming*. vol. 81, no. 7-8 (2012), pp. 721–781. DOI: [10.1016/j.jlap.2012.06.003](https://doi.org/10.1016/j.jlap.2012.06.003).
 - [PBL05] Klaus Pohl, Günter Böckle, and Frank J. van der Linden. *Software Product Line Engineering: Foundations, Principles, and Techniques*. Germany: Springer, 2005. DOI: [10.1007/3-540-28901-1](https://doi.org/10.1007/3-540-28901-1).

2. Background

- [Päß+23] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, S. Lizeth Tapia Tarifa, and Einar Broch Johnsen. “Formal Modelling and Analysis of a Self-Adaptive Robotic System”. In: *Proceedings of the 18th International Conference on integrated Formal Methods (iFM 2023)*. Ed. by Paula Herber and Anton Wijs. Vol. 14300. Lecture Notes in Computer Science. Springer, 2023, pp. 343–363. DOI: [10.1007/978-3-031-47705-8_18](https://doi.org/10.1007/978-3-031-47705-8_18).
- [Qui86] J. Ross Quinlan. “Induction of Decision Trees”. In: *Mach. Learn.* vol. 1, no. 1 (1986), pp. 81–106.
- [Rod+15] Genáina N. Rodrigues, Vander Alves, Vinicius Nunes, André Lanna, Maxime Cordy, Pierre-Yves Schobbens, Amir Molzam Sharifloo, and Axel Legay. “Modeling and Verification for Probabilistic Properties in Software Product Lines”. In: *2015 IEEE 16th International Symposium on High Assurance Systems Engineering*. 2015, pp. 173–180. DOI: [10.1109/HASE.2015.34](https://doi.org/10.1109/HASE.2015.34).
- [SHT06] Pierre-Yves Schobbens, Patrick Heymans, and Jean-Christophe Trigaux. “Feature Diagrams: A Survey and a Formal Semantics”. In: *Proceedings of the 14th IEEE International Conference on Requirements Engineering (RE’06)*. USA: IEEE, 2006, pp. 136–145. DOI: [10.1109/RE.2006.23](https://doi.org/10.1109/RE.2006.23).
- [Tar+12] Anton Tarasyuk, Inna Pereverzeva, Elena Troubitsyna, Timo Latvala, and Laura Nummila. “Formal Development and Assessment of a Reconfigurable On-board Satellite System”. In: *Computer Safety, Reliability, and Security*. Ed. by Frank Ortmeier and Peter Daniel. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 210–222.
- [TKB17] Samir Tigane, Laid Kahloul, and Samir Bouekkache. “Reconfigurable Stochastic Petri Nets for Reconfigurable Manufacturing Systems”. In: *Service Orientation in Holonic and Multi-Agent Manufacturing*. Ed. by Theodor Borangiu, Damien Trentesaux, André Thomas, Paulo Leitão, and José Barata Oliveira. Cham: Springer International Publishing, 2017, pp. 383–391.
- [TP23] Paul Temple and Gilles Perrouin. “Explicit or Implicit? On Feature Engineering for ML-based Variability-intensive Systems”. In: *Proceedings of the 17th International Working Conference on Variability Modelling of Software-Intensive Systems*. VaMoS ’23. Odense, Denmark: Association for Computing Machinery, 2023, pp. 91–93. DOI: [10.1145/3571788.3571804](https://doi.org/10.1145/3571788.3571804).
- [VK13] Mahsa Varshosaz and Ramtin Khosravi. “Discrete time Markov chain families: modeling and verification of probabilistic software product lines”. In: *Proceedings of the 17th International Software Product Line Conference Co-Located Workshops*. SPLC ’13 Workshops. Tokyo, Japan: Association for Computing Machinery, 2013, pp. 34–41. DOI: [10.1145/2499777.2500725](https://doi.org/10.1145/2499777.2500725).

- [Wey20] Danny Weyns. *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons, 2020.
- [WI23] Danny Weyns and Usman M. Iftikhar. “ActivFORMS: A Formally Founded Model-based Approach to Engineer Self-adaptive Systems”. In: *ACM Transactions on Software Engineering and Methodology* vol. 32, no. 1 (2023). DOI: 10.1145/3522585.
- [Win86] Glynn Winskel. “Event Structures”. In: *Proceedings of an Advanced Course on Petri Nets: Central Models and Their Properties (ACPN’86)*. Ed. by Wilfried Brauer, Wolfgang Reisig, and Grzegorz Rozenberg. Vol. 255. Lecture Notes in Computer Science. Germany: Springer, 1986, pp. 325–392. DOI: 10.1007/3-540-17906-2_31.
- [Win90] Jeannette M. Wing. “A Specifier’s Introduction to Formal Methods”. In: *Computer* vol. 23, no. 9 (1990), pp. 8–24. DOI: 10.1109/2.58215.
- [WWT22] Terence Wong, Markus Wagner, and Christoph Treude. “Self-adaptive systems: A systematic literature review across categories and domains”. In: *Information and Software Technology* vol. 148 (2022), p. 106934. DOI: 10.1016/j.infsof.2022.106934.
- [Yan+14] Wenhua Yang, Chang Xu, Yepang Liu, Chun Cao, Xiaoxing Ma, and Jian Lu. “Verifying self-adaptive applications suffering uncertainty”. In: *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering. ASE ’14*. Vasteras, Sweden: Association for Computing Machinery, 2014, pp. 199–210. DOI: 10.1145/2642937.2642999.
- [Ölv17] Peter Csaba Ölveczky. *Designing Reliable Distributed Systems - A Formal Methods Approach Based on Executable Modeling in Maude*. Springer, 2017. DOI: 10.1007/978-1-4471-6687-0.

Chapter 3

Overview of the Research Papers

Paper I: A Formal Model of Metacontrol in Maude

Authors: Juliane Päßler, Esther Aguado, Gustavo Rezende Silva, S. Lizeth Tapia Tarifa, Carlos Hernández Corbato, Einar Broch Johnsen

Published in: *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles*. ISOLA 2022. Lecture Notes in Computer Science, vol. 13701, pp. 575–596. DOI: 10.1007/978-3-031-19849-6_32.

Abstract: Nowadays smart applications appear in domains spanning from commodity household applications to advanced underwater robotics. These smart applications require adaptation to dynamic environments, changing requirements and internal system errors. Metacontrol takes a systems of systems view on autonomous control systems and self-adaptation, by means of an additional layer of control that manipulates and combines the regular controllers. This paper develops a formal model of a Metacontrol architecture. We formalise this Metacontrol architecture in the context of an autonomous house heating application, enabling different controllers to be dynamically combined in order to meet user requirements to a better extent than the individual controllers in isolation. The formal model is developed in the Maude rewriting system, where we show results comparing different scenarios.

Paper II: Analysing Self-Adaptive Systems as Software Product Lines

Authors: Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, S. Lizeth Tapia Tarifa

Published in: *Journal of Systems and Software*. 2025, vol. 222, p. 112324. DOI: 10.1016/j.jss.2024.112324.

Abstract: Self-adaptation is a crucial feature of autonomous systems that must cope with uncertainties in, e.g., their environment and their internal state. Self-adaptive systems (SASs) can be realised as two-layered systems, introducing a separation of concerns between the domain-specific functionalities of the system (the *managed* subsystem) and the adaptation logic (the *managing* subsystem), i.e., introducing an external feedback loop for managing adaptation in the system. We present an approach to model SASs as dynamic software product lines (SPLs) and leverage existing approaches to SPL-based analysis for the analysis of SASs. To do so, the functionalities of the SAS are modelled in a feature model, capturing the SAS’s variability. This allows us to model the managed subsystem of the SAS as a family of systems, where each family member corresponds to a valid feature configuration of the SAS. Thus, the managed subsystem of an SAS is modelled as an SPL model; more precisely, a probabilistic featured transition system. The managing subsystem of an SAS is modelled as a control layer capable of dynamically switching between these valid configurations, depending on both environmental and internal conditions. We demonstrate the approach on a small-scale evaluation of a self-adaptive autonomous underwater vehicle used for pipeline inspection, which we model and analyse with the feature-aware probabilistic model checker ProFeat. The approach allows us to analyse probabilistic reward and safety properties for the SAS, as well as the correctness of its adaptation logic.

Paper III: Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

Authors: Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Clemens Dubslaff, Einar Broch Johnsen, S. Lizeth Tapia Tarifa

Published in: *Formal Aspects of Computing*. Association for Computing Machinery, 2025. DOI: 10.1145/3709159.

Abstract: Improved autonomy in robotic systems is needed for innovation in, e.g., the marine sector. Autonomous robots that are let loose in hazardous environments, such as underwater, need to handle uncertainties that stem from both their environment and internal state. While self-adaptation is crucial to cope with these uncertainties, bad decisions may cause the robot to get lost or even to cause severe environmental damage. Autonomous, self-adaptive robots that operate in uncontrolled environments full of uncertainties need to be reliable! Since these uncertainties are hard to replicate in test deployments, we need methods to formally analyse self-adaptive robots operating in uncontrolled environments. In this paper, we show how feature-oriented techniques can be used to formally model and analyse self-adaptive robotic systems in the presence of such uncertainties. Self-adaptive systems can be organised as two-layered systems with a *managed* subsystem handling the domain concerns and a *managing* subsystem implementing the adaptation logic. We consider a case study of an autonomous underwater vehicle (AUV) for pipeline inspection, in which the managed subsystem of the AUV is modelled as a family of systems, where each family member corresponds to a valid configuration of the AUV which can be seen as an operating mode of the AUV's behaviour. The managing subsystem of the AUV is modelled as a control layer that is capable of dynamically switching between such valid configurations, depending on both environmental and internal uncertainties. These uncertainties are captured in a probabilistic and highly configurable model. Our modelling approach allows us to exploit powerful formal methods for feature-oriented systems, which we illustrate by analysing safety properties, energy consumption, and multi-objective properties, as well as performing parameter synthesis to analyse to what extent environmental conditions affect the AUV. The case study is realised in the probabilistic feature-oriented modelling language and verification tool ProFeat, and in particular exploits family-based probabilistic and parametric model checking.

Paper IV: Behavioral Feature Models

Authors: Juliane Päßler, Sophie Fortz, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, Mohammad Reza Mousavi, S. Lizeth Tapia Tarifa

Submitted for journal publication (short version).

Abstract: Modeling systems as software product lines (SPLs) involves addressing two challenges: representing the structure of the product line, called structural variability, and capturing its behavior, called behavioral variability. For structural variability, several established compositional modeling languages, such as feature models (FMs), have been proposed. For behavioral variability, there are no proposals that offer the same level of compositionality offered by structural models. Moreover, these two types of models are traditionally kept separate. This separation often necessitates the maintenance of different models for structural and behavioral aspects, requiring synchronized updates to prevent discrepancies.

In this paper, we propose a novel formalism called Behavioral Feature Models (BFMs) to integrate structural and behavioral aspects within a single unified model. A BFM is an FM in which each feature has an associated behavioral model, namely an event structure, that models the behavior of the feature; constraints between behaviors of different features can be modeled as constraints between their associated events; and the behavior of each product of the SPL consist of the combined behaviors of its features and the constraints between their behaviors. Our proposed formalism thus integrates both structural and behavioral variability in a compositional model while preserving the separation of concerns expressed in feature modeling. We demonstrate the concept of BFMs through the running example of a service robot SPL, and provide a theoretical and an empirical evaluation where we show that BFMs are as expressive and succinct as a certain class of featured transition systems, the state-of-the-art approach for behavioral modeling of SPLs.

Paper V: Template Decision Diagrams for Meta Control and Explainability

Authors: Clemens Dubslaff, Verena Klös, Juliane Päßler

Published in: *Explainable Artificial Intelligence*. xAI 2024. Communications in Computer and Information Science, vol. 2154, pp. 219–242. Springer, Cham. DOI: 10.1007/978-3-031-63797-1_12.

Abstract: *Decision tree classifiers (DTs)* provide an effective machine-learning model, well-known for its intuitive interpretability. However, they still miss opportunities well-established in software engineering that could further improve their explainability: separation of concerns, encapsulation, and reuse of behaviors. To enable these concepts, we introduce *templates* in *decision diagrams (DDs)* as an extension of multi-valued DDs. Templates allow to encapsulate and reuse common decision-making patterns. By a case study from the autonomous underwater robotics domain we illustrate the benefits of template DDs for modeling and explaining *meta controllers*, i.e., hierarchical control structures with underspecified entities. Further, we implement a template-generating refactoring method for DTs. Our evaluation on standard controller benchmarks shows that template DDs can improve explainability of controller DTs by reducing their sizes by more than one order of magnitude.

Additional Papers

In addition to the research presented in this thesis, I further contributed to other publications which are listed here. For each of them, a short summary and its relation to the research presented in this thesis is given.

SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles

Authors: Gustavo Rezende Silva, Juliane Päßler, Jeroen Zwanepol, Elvin Alberts, S. Lizeth Tapia Tarifa, Ilias Gerostathopoulos, Einar Broch Johnsen, Carlos Hernández Corbato

Published in: *IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2023, pp. 181–187. DOI: 10.1109/SEAMS59076.2023.00031.

Summary: We present an exemplar for two-layered self-adaptive systems with which different adaptation strategies can be tested and compared. The exemplar presents a mission of an underwater pipeline inspection by an autonomous underwater robot that has been used in Papers II, III and V.

Formal Modelling and Analysis of a Self-Adaptive Robotic System

Authors: Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, S. Lizeth Tapia Tarifa

Published in: *Integrated Formal Methods. iFM 2023*. Lecture Notes in Computer Science, vol. 14300. Springer, Cham. DOI: 10.1007/978-3-031-47705-8_18.

Remark: This paper was extended for journal publication in Paper II (focussing on RQ 1) and in Paper III (focussing on RQ 2).

Summary: We model the case study of an autonomous underwater vehicle, inspired by the above mentioned paper “SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles”, using probabilistic featured transition systems, and analyse the model with the feature-aware probabilistic model checker ProFeat.

A Configurable Software Model of a Self-Adaptive Robotic System

Authors: Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, S. Lizeth Tapia Tarifa

Published in: *Science of Computer Programming*, 2025, vol. 240. DOI: 10.1016/j.scico.2024.103221.

Summary: We present the software model used in the above mentioned paper “Formal Modelling and Analysis of a Self-Adaptive Robotic System” and show how to extend it.

ROSA: A Knowledge-based Solution for Robot Self-Adaptation

Authors: Gustavo Rezende Silva, Juliane Päßler, S. Lizeth Tapia Tarifa, Einar Broch Johnsen, Carlos Hernández Corbato

Published in: *Frontiers in Robotics and AI*, 2025, vol. 12. DOI: 10.3389/frobt.2025.1531743.

Summary: We present a novel knowledge-based solution for RObot Self-Adaptation (ROSA) that addresses limitations of previous methods and simplifies the design of task-and-architecture co-adaptation in robotic applications.

Towards Systematic Maintenance of Assurance for Evolving Self-Adaptive Systems

Authors: Logan Murphy, Marc Carwehl, Juliane Päßler, Marsha Chechik

Published in: *IEEE Engineering Reliable Autonomous Systems (ERAS)*, 2025, pp. 1–8, DOI: 10.1109/ERAS63351.2025.11135664.

Summary: Assurance cases (ACs) provide a structured way to demonstrate reliability of a system. Dynamic ACs have been used for assurance of SASs, reflecting that part of the data needed for assurance is only available at runtime. In this paper we present an AC framework that extends and refines existing approaches for dynamic AC development to integrate *evolutions* of an SAS, i.e., changes to the SAS that cannot be handled by self-adaptation.

Chapter 4

Conclusion and Future Directions

In this thesis, we investigate how formal methods can be used at design time to analyse the safety and reliability of SASs. We do so by addressing the three research questions outlined in Section 1.1 in a total of five papers. In this section, we summarise our findings and conclusions for each research question in Section 4.1 and provide ideas for future work in Section 4.2.

4.1 Conclusion

The main findings of this thesis can be summarised as follows:

- Rewriting logic can be used to model the two-layered structure of an SAS and the environment it operates in. The behaviour of the managed subsystem can be analysed on its own and the behaviour of the whole SAS, including the managing subsystem, can be analysed using simulation (Paper I).
- Techniques from DSPLs can be used to model and analyse a two-layered SAS and the environment it operates in. Probabilistic family-based model checking can be used to analyse quantitative properties of the SAS (Papers II and III).
- Compositional modelling of the structure and the behaviour of an SPL in one single model can be achieved by using bundle event structures to describe the behaviour of single features, leading to the notion of *Behavioural Feature Models* (BFMs). The compositional approach taken in BFMs promotes reuse of behaviour across SPLs (and thus SASs) and leads to a smaller representation of behaviour compared to FTSs (Paper IV).
- Decision diagrams (DDs) can be extended with templates to enable encapsulation of common behaviour of SASs, leading to *template DDs*. Template DDs allow modelling the two-layered structure of an SAS explicitly. Furthermore, template DDs increase the explainability of SASs and promote reuse of common behaviour (Paper V).

Table 4.1 provides an overview of the papers included in this thesis and the research questions they address. It shows that the papers can be divided into two parts: one part addressing RQs 1 and 2 and one part addressing RQ 3. This separation is also depicted in Figure 4.1, which provides an abstract illustration of the topics studied in this thesis, their relationships, and the papers addressing these topics. The modelling part is split into two parts: one part using existing modelling approaches (addressing RQ 1) and one part where existing modelling

4. Conclusion and Future Directions

Table 4.1: Papers included in this thesis and the research question(s) each of them addresses

Paper No.	Title	Research Question
Paper I	A Formal Model of Metacontrol in Maude [Päß+22]	RQ1 (Modelling) & RQ2 (Analysis)
Paper II	Analysing Self-Adaptive Systems as Software Product Lines [Päß+25b]	RQ1 (Modelling) & RQ2 (Analysis)
Paper III	Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System [Päß+25a]	RQ1 (Modelling) & RQ2 (Analysis)
Paper IV	Behavioral Feature Models	RQ3 (Reuse)
Paper V	Template Decision Diagrams for Meta Control and Explainability [DKP24]	RQ3 (Reuse)

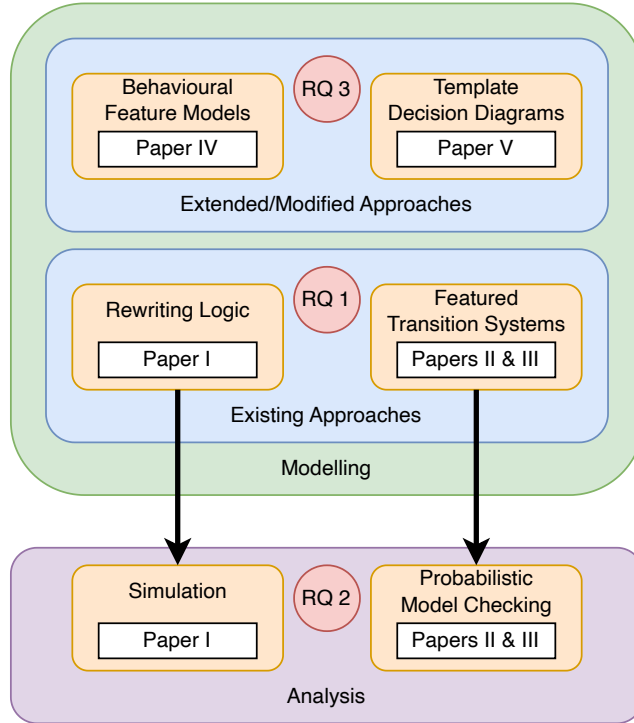


Figure 4.1: An abstract illustration of the topics studied in this thesis, their relationships, and the papers addressing these topics

approaches are extended or modified to provide support for reuse (addressing RQ 3). Only the existing modelling techniques are used for analysis (addressing RQ 2).

4.1.1 Towards RQ 1 (Modelling)

How can formal modelling techniques be used to capture the behaviour of a self-adaptive system that is influenced by an environment?

We address this research question using two modelling techniques, as shown in Figure 4.1: rewriting logic (Paper I) and a variant of featured transition systems (FTSs) (Papers II and III).

We structure the answer to this research question according to the challenges introduced in Chapter 1. First, we explain for every used formal method, how we model the two layers of an SAS (Challenge 1). Then we describe for every formal method how we modelled the environment (Challenge 2). Lastly, we describe how we modelled the interaction between the two layers and the environment (Challenges 1–2) and consider tool support.

Rewriting Logic. We use a case study of a smart house to show how rewriting logic can be used to model an SAS and the environment it operates in (Paper I).

Managed Subsystem. We show that the managed subsystem of an SAS can be modelled as a set of actuators and a set of domain controllers with rewriting logic (Paper I). The actuators are modelled as part of the static aspects of the SAS and the domain controllers as part of the dynamic aspects of the SAS (see Section 2.3.1). The domain controllers use input from the environment and the actuators to decide how the actuators should be changed, e.g., whether a window should be opened. The control policies of the domain controllers are modelled as rewrite rules.

Managing Subsystem. We show that the managing subsystem of an SAS (the smart house) can be modelled with rewriting logic (Paper I). The managing subsystem is modelled as part of the dynamic aspects of the SAS (see Section 2.3.1). It gets information from the managed subsystem and the environment and uses rewrite rules for analysing whether an adaptation is needed, planning how the managed subsystem should be adapted, and executing this adaptation. That is, it uses rewrite rules for all steps of the MAPE loop (see Section 2.1) apart from monitoring. The planning step selects the appropriate domain controller of the managed subsystem.

Environment. We show that the environment can be modelled in a parametric way as a static aspect of the system (Paper I). In our case study, the environment changes depending on the time of the day and on the status of the actuators. We model different versions of the environment that can be changed by setting the value of a variable.

4. Conclusion and Future Directions

Interaction Between Layers. We show that the managing subsystem can receive input from the managed subsystem and the environment via shared variables (Paper I). An adaptation of the managed subsystem can be modelled by passing a message from the managing subsystem to the managed subsystem that activates the suitable controller. Furthermore, we use rewrite rules to coordinate the operation of the managed subsystem and the managing subsystem.

Tool Support. We use the tool Maude [Cla+07] to model the case study of a smart house using rewriting logic (Paper I). As discussed in Section 2.3.1, Maude provides a flexible framework for specifying systems in rewriting logic and allows different forms of analysis.

Featured Transition Systems. We use case studies of an autonomous underwater vehicle (AUV) to show how a variant of FTSs can be used to model an SAS and the environment it operates in (Papers II and III).

Managed Subsystem. We show how techniques from DSPLs can be used to model the managed subsystem of an SAS (Papers II and III). The variability of the managed subsystem is modelled in a feature model. In particular, the components of the managed subsystem and the parameters that can be adjusted are modelled as features. The feature model reflects the valid configurations of the managed subsystem, i.e., every product of the feature model is a valid configuration of the managed subsystem. The behaviour of all products of the managed subsystem is modelled as a featured Markov Decision Process (MDP) that reflects both the behaviour of all configurations of the managed subsystem and the possible changes between these behaviours. Thereby, the managed subsystem of the SAS is modelled as an SPL.

However, our way of modelling the behaviour of the managed subsystem does not allow us to determine the behaviour of single features. That is, it is not possible to see which component exhibits which behaviour. This limits the reuse of models of behaviour across managed subsystems.

Managing Subsystem. We show that the managing subsystem of an SAS can be modelled as a feature controller (Papers II and III). The managing subsystem gets input from the environment and the managed subsystem which is modelled as an SPL. The managing subsystem activates or deactivates features of the feature model to change between (the behaviours of) the configurations of the managed subsystem.

Environment. We show that the environment can be modelled as a Markov Chain (MC) that synchronises with the behaviour of the managed subsystem and the managing subsystem (Papers II and III). By modelling the behaviour of the environment as an MC, we reflect the uncertainty of the environment in the form of probabilistic behaviour. In our case studies, we model different versions of the environment (Papers II and III), including a parametric

version where we do not specify the value of transitions included in the behaviour of the environment (Paper III).

Interaction Between Layers. We show that the managing subsystem can receive input from the managed subsystem and the environment via shared variables (Papers II and III). The managing subsystem can adapt the managed subsystem by activating and deactivating features, i.e., by choosing a different configuration of the managed subsystem.

Tool Support. We use the tool ProFeat [Chr+18] to model case studies of AUVs using techniques from DSPLs (Papers II and III). ProFeat allows specifying the behaviour of the managed subsystem of an SAS as a featured MDP. The managing subsystem of an SAS can be modelled as a feature controller, and the environment can be modelled as an MC or an MDP that synchronises with the managed subsystem and the managing subsystem.

4.1.2 Towards RQ 2 (Analysis)

How can formal analysis techniques be used at design time to analyse the behaviour of a self-adaptive system that is influenced by an environment?

We use two analysis techniques to address this research question as shown in Figure 4.1: simulation (Paper I) and family-based probabilistic model checking (Papers II and III).

Simulation. We use simulation to analyse the behaviour of a smart house case study modelled using rewriting logic (Paper I). The analysis was conducted with the tool Maude. We show that our modelling enables an analysis of every domain controller by itself. That is, different configurations of the managed subsystem can be analysed in isolation. Furthermore, we analyse the behaviour of the whole SAS, including a managing subsystem. Each of these analyses was conducted using different versions of behaviour for the environment.

The analysis shows that each domain controller is able to maintain a subset of requirements of the smart house. The whole SAS, including a managing subsystem, ensures a better satisfaction of the complete set of requirements than the domain controllers by themselves. Even though we are not able to maintain all requirements throughout the whole simulation, the number and intensity of requirement violations is significantly less when including the managing subsystem compared to only using a domain controller. Thus, we show that the addition of a managing subsystem can improve the overall system performance. Furthermore, we exemplified how rewriting logic and the tool Maude can be used to model and analyse two-layered SASs.

Family-based Probabilistic Model Checking. We use family-based probabilistic model checking to analyse case studies of AUVs in ProFeat (Papers II

and III). This enables us to analyse all possible configurations and reconfigurations of the AUV in one run. We analyse the case studies with two, fixed versions of behaviour of the environment: We analyse properties related to safe and unsafe states of the AUV and the cost of completing a mission in the form of time and energy (Papers II and III). Furthermore, we analyse sensor failures and the correctness of the adaptation logic (Paper II). We use different versions of managed subsystems to analyse how these variations influence the analysis results, and analyse trade-offs between the costs time and energy using multi-objective queries (Paper III).

In addition, we analyse a case study with a parametric version of the environment (Paper III). We use parametric model checking to analyse how the behaviour of the environment affects the behaviour of the AUV.

Using these different analyses, we show that the approach of modelling and analysing SASs with techniques from DSPLs (Papers II and III) enables a wide range of (quantitative) analyses of SASs.

4.1.3 Towards RQ 3 (Reuse)

How can formal modelling techniques be extended or modified to support reuse of formal models of self-adaptive systems?

We propose two new modelling techniques as shown in Figure 4.1: Behavioural Feature Models (Paper IV) and template Decision Diagrams (Paper V).

Behavioural Feature Models. We introduce *Behavioural Feature Models* (BFMs), a compositional model for the structure and behaviour of SPLs (Paper IV). A BFM has the structure of a feature model. In addition, behaviour is associated to every feature in the form of a bundle event structure. Furthermore, constraints among the behaviours associated to different features are modelled. The behaviour of a product of the SPL can be obtained by combining the behaviours of the features included in the product and the relevant constraints. This way of compositional modelling enables the reuse of the behaviour of single features by including a feature and its associated behaviour in another SPL. Furthermore, it enables the reuse of the behaviour of parts of the SPL by allowing to add and delete behaviour related to single features. This provides a basis for developing compositional models of SASs, see Section 4.2.

We provide translation algorithms between BFMs and (a class of) FTSs and apply our translation algorithm to FTSs from the literature. Our evaluation shows that the translation from FTS to BFM can reduce the size of the model by up to 98%. Therefore, BFMs have the potential to improve reuse among models of SPLs by providing a compositional structure and a smaller representation compared to FTSs.

Template Decision Diagrams. We introduce *template decision diagrams* (DDs) to allow reuse of behaviour in the form of control strategies within an SAS (Paper V). Template DDs extend DDs with *box nodes*. Box nodes allow

encapsulating common decision making within a DD. Thereby, common parts of the control strategy can be reused within the template DD. A box node is labelled by a template that can itself represent a template DD. The template can be instantiated by a (template) DD, but it can also be left unspecified. This underspecification allows us to specify families of controllers.

We show that template DDs can be used to specify the control strategy of SASs where the managing subsystem adapts the control strategy of the managed subsystem: The control strategy of the managed subsystem is modelled as a template DD with unspecified templates. That is, it is modelled as a family of controllers. The managing subsystem controls the control strategy of the managed subsystem by instantiating the templates. We use a case study of an AUV to exemplify how template DDs can be used for modelling the control strategies of a managed subsystem and a managing subsystem of an SAS including their interaction.

Template DDs also allow reuse of behaviour in the form of control strategies across SASs: The box nodes and template instantiations can be taken from one SAS and be reused for the control strategy of another SAS.

We implement an algorithm that transforms a decision tree (DT) into a template DD. We conduct experiments on the standard benchmark set of dtControl [Ash+21], a tool for generating DDs from DTs, to show the reduction of inner nodes of template DDs in comparison to DTs. These experiments show a clear reduction in size of up to more than one order of magnitude. Thus, template DDs have the potential to improve reuse of SAS models and the explainability of SASs.

4.2 Future Directions

There are many future directions to further investigate how to use formal methods to analyse the safety and reliability of SASs. Possible extensions to Papers I to V are presented in the respective related work sections. We present overall directions for future work here.

Analysing Families of SASs. In Papers II and III we modelled an SAS using techniques from DSPLs: We modelled the managed subsystem as an SPL with products corresponding to configurations, and the managing subsystem as a controller that activates and deactivates features to switch between the configurations of the managed subsystem. It would be interesting to consider a product line of SASs. That is, a product line whose products are SASs. This could, e.g., facilitate an analysis of which SAS is best suited for operating in a certain environment or which SAS is the most energy efficient one. One possible direction for creating an SPL of SASs is to consider (parts of) managed subsystems and managing subsystems to be features of a feature model. The feature model describes how these parts can be combined to form SASs. Every feature corresponding to (parts of) a managed subsystem can in turn be represented as

4. Conclusion and Future Directions

an SPL (see Papers II and III). Future work could investigate if this facilitates modelling and analysing a product line of SASs.

It would also be interesting to investigate if and how *reflection* in rewriting logic [Mes12] can be used to model families of SASs. Intuitively, reflection allows to represent a rewrite theory as an object in another rewrite theory, and thus provides a meta level of reasoning. It might be possible to use reflection to model an additional adaptation layer that changes between managing subsystems. That is, an adaptation layer that adapts the adaptation strategy of an SAS. Since Maude also supports reflection [Cla+98], this could facilitate an analysis of a family of SASs.

Combining Approaches. It would be interesting to combine the approaches studied in this thesis. One possibility would be to combine the results of Paper IV with the results of Papers II and III. It would be worth investigating if the compositional structure of a BFM can be used to model the managed subsystem of an SAS. This compositional model could allow to reuse parts of the behaviour of a managed subsystem for other SASs, overcoming the limitation of our approach mentioned in Section 4.1.1. The translation from BFMs to FTSs (Paper IV) could also facilitate an analysis of the SAS using the approach proposed in Papers II and III. Furthermore, an interesting question would be if the combination of these works would facilitate a compositional analysis of SASs, which might be helpful for analysing families of SASs.

In addition to a formal analysis of an SAS, it would be interesting to explore the validation of the design and application logic of an SAS by using template DDs (Paper V) to represent the control strategy of an SAS. The control strategy of the SAS could, e.g., be obtained using Papers II and III or as a family member of a SPL of SASs as mentioned before.

Uncertainty. As mentioned before, the aim of SASs is to operate in uncertain environments. In Papers II and III we include uncertainty of the environment and of the operation of the SAS in our formal model in the form of probabilities. However, for a realistic SAS, it is unclear how these probabilities can be obtained. One way of reflecting this uncertainty of the probabilities would be to attach intervals of probabilities to transitions instead of concrete values, i.e., representing the environment and the managed subsystem of an SAS as Interval Markov Decision Processes. It would be interesting to investigate if and how this could help to analyse SASs.

Another possibility would be to collect data with a real robot to calculate probabilities, e.g., of sensor failures and changes of the environment. It would be interesting to use the approach of modelling and analysing SASs with techniques from DSPLs (Papers II and III) not only at design time, but also at runtime. At runtime, the data collected by the robot could be used to update the probabilities of the model. The model could then be verified again (using Papers II and III), including the new values for the probabilities. Updating the probabilities at runtime would make the model more realistic, while verifying the model again

would ensure a safe operation even in case of a different behaviour than expected. However, verification at runtime leads to the issue of scalability, as addressed next.

Scalability. The case studies we used in Papers I to III to exemplify how to model and analyse an SAS using the proposed approaches were small compared to real SASs. It would be worth investigating how our approaches scale, and which measures can be taken to improve scalability. One step towards this could be the combination of BFMs (Paper IV) with modelling and analysing SASs with techniques from DSPLs (Papers II and III), as described above. By providing a compositional model that has the potential to lead to a smaller representation, BFMs could improve the scalability of SAS models.

References

- [Ash+21] Pranav Ashok, Mathias Jackermeier, Jan Kretínský, Christoph Weinhuber, Maximilian Weininger, and Mayank Yadav. “dtControl 2.0: Explainable Strategy Representation via Decision Tree Learning Steered by Experts”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12652. Lecture Notes in Computer Science. Springer, 2021, pp. 326–345. DOI: 10.1007/978-3-030-72013-1_17.
- [Chr+18] Philipp Chrszon, Clemens Dubslaff, Sascha Klüppelholz, and Christel Baier. “ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking”. In: *Formal Aspects of Computing* vol. 30, no. 1 (2018), pp. 45–75. DOI: 10.1007/s00165-017-0432-4.
- [Cla+07] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn L. Talcott. *All About Maude - A High-Performance Logical Framework, How to Specify, Program and Verify Systems in Rewriting Logic*. Vol. 4350. Lecture Notes in Computer Science. Springer, 2007. DOI: 10.1007/978-3-540-71999-1.
- [Cla+98] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, and José Meseguer. “Metalevel Computation in Maude”. In: *International Workshop on Rewriting Logic and its Applications, (WRLA)*. Ed. by Claude Kirchner and Hélène Kirchner. Vol. 15. Electronic Notes in Theoretical Computer Science. Elsevier, 1998, pp. 331–352. DOI: 10.1016/S1571-0661(05)80020-9.
- [DKP24] Clemens Dubslaff, Verena Klös, and Juliane Päßler. “Template Decision Diagrams for Meta Control and Explainability”. In: *Explainable Artificial Intelligence (xAI)*. Ed. by Luca Longo, Sebastian Lapuschkin, and Christin Seifert. Vol. 2154. Communications in

- Computer and Information Science. Springer, 2024, pp. 219–242. DOI: [10.1007/978-3-031-63797-1_12](https://doi.org/10.1007/978-3-031-63797-1_12).
- [Mes12] José Meseguer. “Twenty years of rewriting logic”. In: *The Journal of Logic and Algebraic Programming*. vol. 81, no. 7-8 (2012), pp. 721–781. DOI: [10.1016/j.jlap.2012.06.003](https://doi.org/10.1016/j.jlap.2012.06.003).
- [Päß+22] Juliane Päßler, Esther Aguado, Gustavo Rezende Silva, Silvia Lizeth Tapia Tarifa, Carlos Hernández Corbato, and Einar Broch Johnsen. “A Formal Model of Metacontrol in Maude”. In: *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles*. Ed. by Tiziana Margaria and Bernhard Steffen. Vol. 13701. Lecture Notes in Computer Science. Springer, 2022, pp. 575–596. DOI: [10.1007/978-3-031-19849-6_32](https://doi.org/10.1007/978-3-031-19849-6_32).
- [Päß+25a] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Clemens Dubslaff, Einar Broch Johnsen, and Silvia Lizeth Tapia Tarifa. “Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System”. In: *Formal Aspects of Computing* (2025). Just Accepted. DOI: [10.1145/3709159](https://doi.org/10.1145/3709159).
- [Päß+25b] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, and S. Lizeth Tapia Tarifa. “Analysing Self-Adaptive Systems as Software Product Lines”. In: *Journal of Systems and Software* vol. 222 (2025), p. 112324. DOI: [10.1016/j.jss.2024.112324](https://doi.org/10.1016/j.jss.2024.112324).

Papers

Paper I

A Formal Model of Metacontrol in Maude

**Juliane Päßler, Esther Aguado, Gustavo Rezende Silva,
S. Lizeth Tapia Tarifa, Carlos Hernández Corbato,
Einar Broch Johnsen**

Published in *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles*. ISOLA 2022. Lecture Notes in Computer Science, vol. 13701, pp. 575-596. DOI: 10.1007/978-3-031-19849-6_32. Reproduced with permission from Springer Nature.

Abstract

Nowadays smart applications appear in domains spanning from commodity household applications to advanced underwater robotics. These smart applications require adaptation to dynamic environments, changing requirements and internal system errors. Metacontrol takes a systems view on autonomous control systems and self-adaptation, by means of an additional layer of control that manipulates and combines the regular controllers. This paper develops a formal model of a Metacontrol architecture. We formalise this Metacontrol architecture in the context of an autonomous house heating application, enabling different controllers to be dynamically combined in order to meet user requirements to a better extent than the individual controllers in isolation. The formal model is developed in the Maude rewriting system, where we show results comparing different scenarios.

I.1 Introduction

There is an emerging development of “smart” applications in an increasing number of domains, ranging from commonplace IoT-based household applications to advanced underwater robotics. A key to this smartness is the applications’ ability to flexibly adapt to variability in their operational conditions. Control theory and recent AI-based methods enable the development of different domain controllers that adapt to variability in the physical environment by combining sensor data and a diversity of models to determine the output of the system actuators, achieving great performance and a degree of system-level autonomy for different requirements. This is called first-order self-adaptation [Gar21]. However, varying

user requirements and internal system variability due to faults or emergent behaviour in systems that comprise multiple controllers pose additional second-order adaptation challenges [Gar21]. Self-adaptive systems with multiple points of view and methodologies have been proposed from the software community to develop software systems that meet the previous adaptation requirements. These systems have been commonly conceptualised following the MAPE-K schema [Bru+09; KC03] that stands for Monitor, Analyse, Plan, and Execute based on Knowledge.

Metacontrol is a layered framework for achieving autonomy, independently of the application and the system; i.e., it takes a very structured system of systems view on the self-adaptation problem. It adds an additional layer of control that manipulates and combines the domain controllers to fulfil different combinations of requirements or to ensure resilience by enabling the overall application to adapt to changes in the underlying system such as, e.g., broken devices. Thus, Metacontrol enables extensible self-adaptation for autonomous systems that can accommodate new user requirements or new functionality to an application, by addressing only second-order adaptation based on application-independent reasoning. For this purpose, its MAPE-K loop is driven by knowledge that adheres to TOMASys [Cor13] (the *Teleological and Ontological Model of Autonomous Systems* metamodel), which aims to provide the necessary concepts for modelling the functional knowledge of autonomous systems [HBS18].

In this paper, we formalise a Metacontrol architecture for a smart house, detailing an autonomous house heating application. This use case is inspired by the work of Arcaini et al. [ARS16] on a model of concurrent MAPE-K controllers (but not of Metacontrol), using abstract state machines [BS03]. Our main focus here is on the logical structure of architectures based on Metacontrol; i.e., the model captures the input assumed to be available to the different layers of the control structure and the control decisions that the different layers make. We develop models of independent controllers for the house heating application and show that each of these is able to maintain some of the given user requirements, but not the overall combined requirements. These models are then extended with a Metacontrol layer that, by dynamically combining the different existing controllers, is able to meet the requirements to a greater extent.

The formalisation is realised in Maude [Cla+07], a tool to develop and analyse executable models in rewriting logic. Rewriting Logic is a flexible framework combining computation and logic in which systems can be modelled with low representational distance [Mes12]. In this paper, we use simulations in Maude to analyse the performance of the controllers with respect to the user requirements in fairly predictable environments; in future work, we plan to expand our model to analyse the additional responsiveness that seems achievable by exploiting a Metacontrol layer in less predictable environments by means of model checking techniques.

The main contributions in this paper are:

- an executable formal model of a system of systems Metacontrol architecture developed in the Maude rewrite tool;

- a use case of autonomous layered control for a smart house heating application, which we use to explain and illustrate the formal model; and
- we show how the formal model can be used to analyse the developed metacontrol designs with respect to user requirements.

I.2 Background

I.2.1 Metacontrol

Metacontrol [Cor13] gives systems the capability to perform self-adaptation in order to maintain their functionalities at an expected performance, in the presence of external disturbances, faults, and unexpected behaviour. Thus, Metacontrol enables an increased level of autonomy and reliability in a system.

Metacontrol is realised by adding an extra control layer that closes a feedback loop with the original system, monitoring, and adapting the original system through architectural reconfiguration when necessary. According to the MAPE-K model for self-adaptive systems, this results in the separation of the system into two distinct layers or subsystems: the *managing system*, whose main element is a reusable metacontroller, and the *managed system*, which is the original system.

For the metacontroller to be application independent, it exploits engineering knowledge at runtime. System engineers model relevant information about how the system works following the TOMASys metamodel [Cor13], and this model forms the basis for the knowledge base (KB), which drives the Metacontrol MAPE-K adaptation loop. All steps interact with the KB, storing additional information, and retrieving system knowledge to facilitate decision making (see Figure I.1).

The *monitor* step is responsible for measuring so-called *quality attributes* (discussed below) for the functionalities of the system. The *analyse* step is responsible for deciding whether the managed system needs to reconfigure. For this, it needs to decide whether the measured quality attributes meet the requirements of the current configuration of the managed system. When reconfiguration is needed, the *plan* step is responsible for selecting a new configuration for the managed system that potentially satisfies its requirements. The *execute* step is responsible for carrying out the reconfiguration of the managed system.

Figure I.2 shows a high-level representation of the TOMASys metamodel for the Metacontrol functional layer, which is our focus in this paper. A *function* represents an abstract functionality of the system, e.g., controlling the temperature. A *function design* is an engineering design solution that solves a specific function, e.g., a specific controller to control the temperature. An *objective* is a concrete instance of a function that the system desires to achieve at runtime, e.g., maintaining the room temperature in the range 18–22°C. A *function grounding* is the runtime instantiation of a function design that is selected to solve a specific objective. A quality attribute (QA) is a measurable

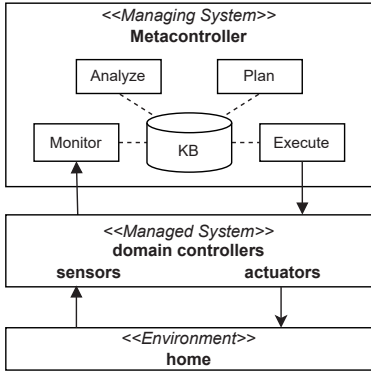


Figure I.1: The Metacontrol framework.

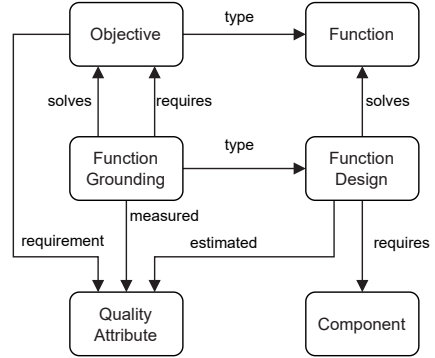


Figure I.2: Elements of TOMASys.

property of a system that is used to indicate how well the system satisfies its requirements, e.g., the amount of energy that the temperature control system consumes. In TOMASys there are *required*, *expected*, and *measured* QAs:

- *required* QAs capture quantitative requirements that a function should meet;
- *estimated* QAs are assumed for each function design as expected performance values with respect to the corresponding quantitative requirements; and
- *measured* QAs are associated to function groundings to capture the current performance with respect to the corresponding quantitative requirements.

I.2.2 Maude

Maude [Cla+07; Ölv17] is a specification and analysis system based on rewriting logic (RL) [Mes12]. While algebraic specification techniques [Wir90] can be used to specify the static structure of the system and the relations between the data, RL extends an algebraic specification with transitions rules which capture the dynamic behaviour of a system. In a rewrite theory (Σ, E, R) , Σ defines the (ground) terms tr as operators over sorts (which can be understood as types), E is a set of equations that define equivalences between terms in Σ , and R is a set of labelled rewrite rules.

Equational theories (Σ, E) are developed by first defining *sorts* st and *functions* from a (possibly empty) list $stlist$ of sorts to a given sort st :

```
1 sort st .
2 op f : stlist -> st .
```

Terms tr are then built from functions in a sort-correct manner, and *patterns* t are terms in which some functions are replaced by variables. *Conditional equations*

in E , which express the equality of any terms that match given patterns if a given condition holds, are specified by

```
1 ceq  $t = t'$  if  $cond$ 
```

where the matching substitution additionally applies to the condition. The condition can be a conjunction of equalities between patterns. Maude assumes that the equations form a terminating and confluent reduction system which is used to represent equivalence classes by canonical terms (i.e., all terms can be reduced to unique normal forms).

Rewrite theories (Σ, E, R) additionally specify *conditional rewrite rules* in R , which express that a term matching a given pattern can transition into another term if a condition holds, as follows:

```
1 cr1 [ $label$ ]:  $t \Rightarrow t'$  if  $cond$ .
```

Here, the condition is a conjunction of rewrites and equalities that must hold for the rule to apply to a given term which matches t . Rewrite rules apply to equivalence classes of terms; i.e., Maude uses the equations to reduce terms to normal form in between rule applications. Therefore, when auxiliary functions are needed, these can be defined in equational logic and thus evaluated in between the state transitions [Mes12].

When modelling executable systems, system components are typically modelled by terms of suitable sorts, organised hierarchically in modules, and the global state configuration is represented as a multiset of these terms [Ölv17]. An object in a given state can have the form $\langle \text{Oid} : \text{Class} \mid \mathbf{a}_1 : \mathbf{v}_1, \dots, \mathbf{a}_n : \mathbf{v}_n \rangle$, where **Class** is the class name, **Oid** the object identifier, and $\mathbf{a}_i$ are attributes with corresponding values $\mathbf{v}_i$. Given an initial configuration, the Maude tool supports simulation and breadth-first search through reachable states, and model checking of systems with a finite number of reachable states [Cla+07; Ölv17].

I.3 The Smart House Use Case

Let us consider a smart house with an automatic heating system. This use case was originally presented in [ARS16]. Here, we present a variation of the use case to motivate a self-adaptive heating system that includes various controllers that partially fulfil the requirements and a metacontroller that turns the different controllers on and off with the aim of improving the system behaviour. The house consists of a room with one window, one heater and one water heater. Its intended behaviour is specified through a set of requirements defined in Figure I.3. The smart house system has various controllers that control three actuators, the heater, the water heater, and the window, and it has a temperature sensor (a thermometer), an air quality sensor, and a global clock to keep track of time.

The heater can be set to very hot *VH*, fairly hot *FH* or *OFF*, the water heater to *ON* or *OFF*, and the window to open *O*, half open *HO*, or closed *C*. A thermometer measures the temperature as a float value, where a comfortable temperature ranges over values between (and including) 18 and 22 °C. An air quality sensor measures the air quality as a float value, where a good level is

Req.	Name	Description
R_1	<i>Morning water heating</i>	The water heater should be turned on in the morning.
R_2	<i>Minimise dispersion</i>	The system should avoid high dispersion states, i.e., window wide open and strong heating together.
R_3	<i>Comfortable temperature</i>	The smart house temperature shall always be between a range of comfortable temperatures.
R_4	<i>Air purity</i>	The heating system shall maintain a good air quality level.

Figure I.3: Requirements of the heating system. Here, the requirements R_3 and R_4 are quantitative system properties which are monitored by the quality attributes.

Act.	Stat.	Temp.	Air Qu.
Heater	<i>VH</i>	1.23	-0.9
	<i>FH</i>	0.615	-0.45
	<i>OFF</i>	0	0
Water heater	<i>ON</i>	0,3	-0,125
	<i>OFF</i>	0	0
Window	<i>O</i>	-2.0	2.0
	<i>HO</i>	-1.0	1.0
	<i>C</i>	0	-0.125

Figure I.4: Assumed actuator effects on temperature and air quality

represented by a value ≥ 0 . The clock ranges over integer values from 1 to 24, and changes in a round-robin manner to represent discrete time units as time advances during a 24-hour day. We define all time units between, and including, 6 and 12 to be *MORNING* and all other time units as *NOT MORNING*.

In Figure I.4 we model the impact that the state of the actuators has on the temperature and air quality. For every time step, the total impact on the temperature and air quality is calculated as the sum of the effects of each actuator. The temperature of the smart house is also affected by the environment, e.g., the outside temperature, which varies depending on the time of the day. We further consider additional variability of the environment (see the online material).

I.3.1 The Controller Layer

The operation of the smart house system is based on controllers that receive readings from the temperature and air quality sensors as well as the clock value as input. Depending on the sensor readings and whether it is morning or not, the active controller selects a status for each actuator. The controller's goal is to fulfil some of the requirements displayed in Figure I.3 by changing the state of the actuators. The smart house heating system has four controllers: a controller

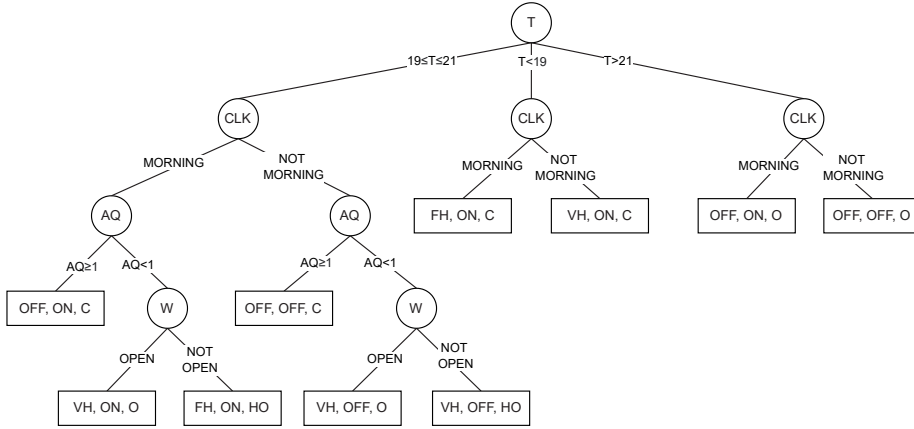


Figure I.5: Decision diagram for the comfort controller, where the leaf nodes indicate the controller's decision for the state of the heater, water heater and window.

that prioritises temperature (comfort controller), a controller that prioritises air quality (eco controller), and two degraded controllers, which only act upon one of the heaters, using only two of the three actuators in the smart house.

The controllers have policies based on rules to decide upon the state of the actuators, depending on the inputs (temperature, air quality, and clock), and assuming that the actuators' effect on the temperature and air quality in the room are as the presented in Figure I.4.

The comfort controller. uses all actuators and has a strong preference for *comfort temperature*. Its control policy is defined to meet to a certain degree the requirements in Figure I.3: (1) a policy stricter than R_3 : reach and maintain the desired comfort temperature measurements as a priority, even if that means slightly worse air purity measurements, and (2) R_1 : the water heater is always on in the morning.

The controller policy is depicted as a decision diagram in Figure I.5. The nodes represent the temperature T , clock CLK , air quality AQ , and window W . The edges represent the status of each node. The leaves represent the desired state of the actuators, as described in Figure I.4, in the order: heater, water heater, and window. Note that the controllers check whether the temperature is within 19.0°C and 21.0°C and whether the air quality is above 1, so they use different values than the ones defined in the system requirements. This was done to create some buffer during which the controllers can react before the temperature and air quality pass the thresholds of 18.0°C and 22.0°C, respectively 0. Based on the decision tree, we can already observe that some requirements of Figure I.3 are not met, as the controller prioritises a comfortable temperature. In particular, some rules violate R_2 since the controller uses the very hot heater state VH and the open window state O together.

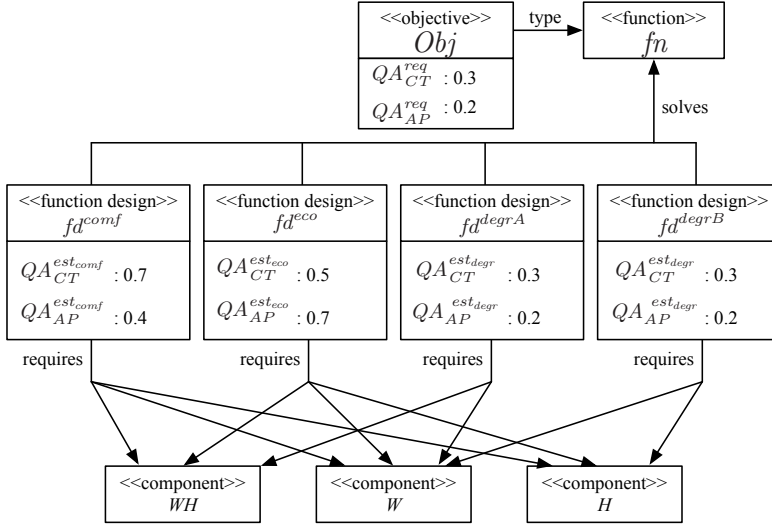


Figure I.6: Elements of the smart house TOMASys metamodel.

The eco controller. uses all actuators and has a strong preference for *air purity*. Its control policy is defined to meet to a certain degree the requirements in Figure I.3: (1) a policy stricter than R_4 : reach and maintain the desired air purity measurements as a priority, even if it means a slightly worse comfortable temperature measurements, (2) R_1 : the water heater is always on in the morning, (3) an extension of R_2 : if the temperature is high and it is morning, only reduce the temperature with the window half open to avoid losing heat too quickly.

Degraded controllers. are defined for each heater that may break, so the degraded controllers can be used when that happens. *Degraded controller A* assumes that the heater is broken and therefore always *OFF*. *Degraded controller B* assumes that the water heater is broken and therefore always *OFF*.

Decision diagrams, similar to the one showed in Figure I.5, are included for the remaining controllers in the online repository¹.

I.4 The Metacontrol Layer for the Smart House Use Case

In this section we present the Metacontrol layer for the smart house use case, modelling R_3 and R_4 in Figure I.3 as an abstract functionality of the system that the metacontroller aims to maintain. The overall system includes all the controllers presented in Section I.3.1. The comfort controller prioritises R_3 over R_4 , while the policy of the eco controller does the opposite. In case a heater breaks, the degraded controllers can be used. The adaptation problem that the

¹The full model of the smart house Metacontrol architecture, including all scenarios and results, is available at https://github.com/remaro-network/Maude_Metacontrol.

metacontroller solves is configuring the smart house application by selecting at each instant the best controller to perform both R_3 and R_4 .

I.4.1 The TOMASys Metamodel

The quantitative requirements R_3 and R_4 are captured in the TOMASys metamodel, as explained in Section I.2.1. Figure I.6 provides an overview of the TOMASys model for this use case.

Function. Requirements R_3 and R_4 can be represented as a *function* fn : use a controller to keep the smart house temperature and air quality at a desired level. Observe that in the general case, the metacontroller can operate based on different functions that depend on different quality attributes at the same time.

Quality attributes. To quantify the extent to which the system satisfies the requirements R_3 and R_4 , we use the *quality attributes* QA_{CT} (comfortable temperature) and QA_{AP} (air purity). These quality attributes occur as *required* quality attributes in the objective, as *estimated* quality attributes in the function designs, and as *measured* quality attributes in the function grounding, and will be further discussed in the respective paragraphs.

Objective. An objective is a concrete instance of the function fn . A desired range for the temperature and a good level for the air quality are given by the *objective* Obj : keep the smart house temperature between (and including) 18 and 22 °C and the air quality ≥ 0 . To evaluate whether an objective has been fulfilled, the metacontroller uses minimum thresholds for each quality attribute, such thresholds are denoted as *required quality attributes* QA_{CT}^{req} and QA_{AP}^{req} . These values also reflect the priorities that are set in the system. The higher the value, the higher the priority. For the smart house use case, we want to prioritise comfortable temperature; therefore $QA_{CT}^{req} > QA_{AP}^{req}$, see Figure I.6. We denote this objective as $Obj(QA_{CT}^{req}, QA_{AP}^{req})$. Observe that, in general, there is one objective for each function since the objective is an instance of a function.

Function design. We denote each function design in Figure I.6 as

$$fd^{id}(QA_{CT}^{est_{id}}, QA_{AP}^{est_{id}}, Cmp_{id}) ,$$

where

- id is one of the identifiers of the controllers,
- QA_{CT}^{est} and QA_{AP}^{est} are *estimated* quality attribute values for comfortable temperature and air purity, and
- $Cmp \subseteq \{heater, waterheater, window\}$ are *required* components.

Function grounding. If a controller is selected, we *ground* its function design. This instance is then called *function grounding*. In the function grounding, we store the *measured* quality attributes QA_{CT}^{meas} and QA_{AP}^{meas} that reflect the degree to which the quality attributes are fulfilled at the moment.

If one of the actuators fails or the measured quality attribute values are lower than the required ones, then the function grounding will be marked as in error, which will trigger reconfiguration in the metacontroller. This includes grounding a (new) function design.

Quality attribute values. Figure I.6 includes the required and estimated values for the quality attributes used in the use case. The function design fd^{comf} captures the goal of the comfort controller: to prioritise a comfortable temperature in the room and then, if the temperature is good enough, try to maintain a reasonable air quality, thus $QA_{CT}^{estcomf} > QA_{AP}^{estcomf}$. The function design fd^{eco} focuses on a good air quality, therefore $QA_{CT}^{esteco} < QA_{AP}^{esteco}$.

For simplicity all degraded controllers have the same estimated values in our model, which are assumed to be lower than the estimated values for the other controllers, and prioritise room temperature.

All the above elements are used by the metacontroller to decide when to switch between the different controllers, so that the system does its best to meet both requirements R_3 and R_4 .

I.4.2 Metacontrol Operation

We now detail the operation of the metacontroller, following the MAPE-K loop, where the knowledge base of the MAPE-K loop includes all the elements and relationships among the elements defined in Section I.4.1.

Monitor. During the monitor step, the metacontroller uses some collected logs in the system to calculate the current measured values of the quality attributes. In particular, we assume that the heating system has a log for the temperature values in the room $T = t_0, t_1, \dots, t_n$ and a log for the air quality values in the room $A = a_0, a_1, \dots, a_n$, where $t_i, a_i \in \mathbb{Q}$ for all $i = 0, \dots, n$. They were retrieved from the sensors every time step $0, \dots, n$, where n is the current time step. Let $N \in \mathbb{N}$ be the size of the time window that we want to consider for the computation of the current values of the quality attributes. We now decide whether a temperature t_i is considered to be a comfortable temperature. Let $CT: \mathbb{Q} \rightarrow \{0, 1\}$ be defined as

$$CT(t_i) = \begin{cases} 1 & \text{if } 19.1 \leq t_i \leq 21.5, \\ 0 & \text{otherwise.} \end{cases} \quad (\text{I.1})$$

The measured quality attribute QA_{CT}^{meas} for comfortable temperature is then given by the function $QA_{CT}^{meas}: \mathbb{N} \times \mathbb{Q}^n \rightarrow [0, 1]$, defined as follows:

$$QA_{CT}^{meas}(N, T) = \frac{\sum_{i=(n-N)}^n CT(t_i)}{N}. \quad (\text{I.2})$$

We compute the measured air purity as a function $AP: \mathbb{Q} \rightarrow \{0, 1\}$, defined as:

$$AP(a_i) = \begin{cases} 1 & \text{if } a_i \geq 0.7, \\ 0 & \text{otherwise.} \end{cases} \quad (\text{I.3})$$

The measured quality attribute QA_{AP}^{meas} for air purity is then given by a function $QA_{AP}^{meas}: \mathbb{N} \times \mathbb{Q}^n \rightarrow [0, 1]$, defined as follows:

$$QA_{AP}^{meas}(N, A) = \frac{\sum_{i=(n-N)}^n AP(a_i)}{N}. \quad (\text{I.4})$$

Observe that the thresholds for comfortable temperature and air purity in CT and AP differ from the system requirements in Section I.3. This difference anticipates the adaptation that will be done the metacontroller. Finally, if $n < N$, we only consider the first n time steps in both QA_{CT}^{meas} and QA_{AP}^{meas} .

Analyse. During the analysis step, the metacontroller first checks whether one of the actuators is in error. If yes, then the current function grounding is marked to be in error. If all actuators work, the metacontroller compares the measured quality attribute values QA_{CT}^{meas} and QA_{AP}^{meas} with the required QA values in the objective $Obj(QA_{CT}^{req}, QA_{AP}^{req})$. If at least one measured QA value is lower than the respective required QA value, then the current function grounding is marked to be in error. The QAs that have a lower measured than required value are marked as underachieved. For example, if $QA_{CT}^{meas} = 0.2$, then QA_{CT} is marked as underachieving since $QA_{CT}^{req} = 0.3$, see Figure I.6.

Plan. If the metacontroller marked the function grounding to be in error, then during the planning step the metacontroller needs to select a new function design to ground. In the case where both quality attributes are underachieved, the objective $Obj(QA_{CT}^{req}, QA_{AP}^{req})$ prioritises the QA for comfortable temperature since $QA_{CT}^{req} > QA_{AP}^{req}$.

Assuming that the system is at time step n , we define $available_n$ as the set of function designs that are available at time step n , i.e., the set of function designs whose required components only contain components that are not broken. Furthermore, we define $poorQA_n$ as the set of QAs that are underachieved at time step n . The planning step in the metacontroller acts slightly different depending on two cases:

Case 1: *The metacontroller only needs to look at the required components to choose a function design.* In our particular scenario, this is the case in which either the heater or the water heater is failing.

In this case $available_n \subseteq \{fd^{degrA}, fd^{degrB}\}$. Since in our scenario only one of the heaters can be broken, $available_n$ will have exactly one element, which the metacontroller chooses to be grounded in the execute step.

Case 2: *The metacontroller needs to look into the QAs that are marked as underachieved.* In our particular scenario, this is the case in which all the actuators are working normally. Thus, $available_n$ consists of all function designs. In this case $poorQA_n \subseteq \{QA_{CT}, QA_{AP}\}$. Observe that $poorQA_n$ contains at least one element. Let $QA_x \in poorQA_n$ be the QA with the highest priority in $poorQA_n$. The metacontroller searches for the function design in $available_n$ with the highest estimated value for QA_x , i.e., the function design with identifier i such that for all function designs in $available_n$ with identifier j it holds that $QA_x^{est_i} \geq QA_x^{est_j}$.

Execute. During the execute step, the chosen function design is grounded and the controller associated with this function design is activated, triggering execution in the controller layer as described in Section I.3.1.

I.5 Modelling the Smart House Use Case in Maude

We now describe the Maude model of the metacontrolled smart house application, including all elements described in Sections I.3 and I.4. A multiset **Configuration** is used to represent the components of the Metacontrol ecosystem, as suggested in Section I.2.2. A **Configuration** contains all objects that are crucial for the model of the smart house, detailed in the sequel. Figures I.7 and I.11 give the syntax of a selection of these objects. The full model can be found in the online repository.

I.5.1 Model Dynamics

The following elements model the execution of the entire self-adaptive system, as described in Figure I.1.

Scheduler is an object of sort **Scheduler** (see Figure I.7, Line 1) that captures the MAPE-K loop and ensures that rules are applied in the correct order, according to Figure I.8. It keeps track of which sets of rules can be applied, using the attribute **Status**, and schedules which components can apply rules. The attribute **RuleApplied** indicates whether one of the rules in the active set has already been applied. The scheduler object is present in all rules. A rule can only be applied if the scheduler object has the right value in the attribute **Status**, and if the attribute **RuleApplied** is false, e.g., see the rule in Figure I.9. The scheduler makes the model deterministic. However, different configurations will trigger different rules inside the sets.

Time is modelled with an object of sort **Clock** (see Figure I.7, Line 2) that captures the passage of time in the system, which works together with a rule for modelling how time advances. It has attributes

- **Timesteps:** a natural number representing the current time step in the system,

```

1 op < Scheduler | Status: _ , RuleApplied: _ > : ScheduleComp Bool → Scheduler .
2 op < _ : Clock | Timesteps: _ , Time: _ , TempLog: _ , AqLog: _ >
3   : Oid Timesteps Time TVPList TVPList → Clock .
4 op < _ : Thermometer | Degrees: _ > : Oid Temperature → Thermometer .
5 op < _ : Airquality | Value: _ > : Oid AirqualityStatus → Airquality .
6 op < _ : Heater | _ > : Oid Attribute → Heater .
7 op < _ : Waterheater | _ > : Oid Attribute → Waterheater .
8 op < _ : Window | _ > : Oid Attribute → Window .
9 op < _ : ComfortController | Selected: _ > : Oid Selected → ComfortController .
10 op < _ : EcoController | Selected: _ > : Oid Selected → EcoController .
11 op < _ : DegradedContrA | Selected: _ > : Oid Selected → DegradedContrA .
12 op < _ : DegradedContrB | Selected: _ > : Oid Selected → DegradedContrB .
13 op < Environment | Version: _ > : Nat → Environment .
    
```

Figure I.7: Selected objects of the smart house heating system, modelled in Maude.

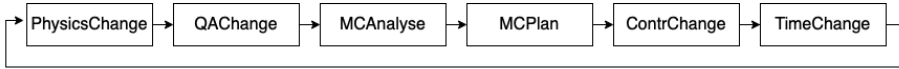


Figure I.8: The order in which the different set of rules are applied

- Time: the current time, and
- TVPList: a list of (Timestep, Float)-pairs.

After each time step, a pair representing the current time step and temperature (air quality) is added to the temperature log `TempLog` (respectively to the air quality log `AqLog`). As described in Section I.3, we assume that one time step is one hour, so the time is computed as the current time steps modulo 24. Furthermore, a sort `TimedConfiguration` is introduced. It is used in certain rules to ensure that rules are applied to the full global state configuration and not only to subsets. If the scheduler indicates that the time should advance, then a rule that increases the `Timesteps` by one, computes the new `Time` and adds new pairs to `TempLog` and `AqLog` is applied.

I.5.2 The Smart House Model

In this section, we present the model of the managed system, as displayed in Figure I.1 and described in Section I.3.

Thermometer models the measurement of the current temperature in the room. An object of sort `Thermometer` (see Figure I.7, Line 4) has the attribute `Temperature` that is a float value that reflects the current temperature in the smart house.

Air quality sensor models the measurement of the current air quality value in the room. An object of sort `Airquality` (see Figure I.7, Line 5) has the attribute

```

1 crl [CContrTempOkAqOk] :
2   < H : Heater | A > < WH : Waterheater | A1 > < W : Window | A2 >
3   < T : Thermometer | Degrees: DG > < AQ : Airquality | Value: AQS >
4   < C : Clock | Timesteps: TS, Time: TI, TempLog: TL, AqLog: AL >
5   < CC : ComfortController | Selected: true >
6   < Scheduler | Status: ContrChange, RuleApplied: false >
7   ⇒
8   (msg hOff from CC to H)(msg whOff from CC to WH)(msg closed from CC to W)
9   < H : Heater | A > < WH : Waterheater | A1 > < W : Window | A2 >
10  < T : Thermometer | Degrees: DG > < AQ : Airquality | Value: AQS >
11  < C : Clock | Timesteps: TS, Time: TI, TempLog: TL, AqLog: AL >
12  < CC : ComfortController | Selected: true >
13  < Scheduler | Status: ContrChange, RuleApplied: true >
14  if morning(TI) == false /\ hot(DG) == false /\ cold(DG) == false
15  /\ aqok(AQS) == true .

```

Figure I.9: A rule for the comfort controller in Maude (coloured text highlights changed or new elements).

AirqualityStatus that is a float value that reflects the current air quality value in the smart house.

Actuators have an object identifier *Oid* and a multiset of *Attributes* (see Figure I.7, Lines 6–8). Thus, the attributes that are necessary for an actuator are not explicitly specified. However, each actuator should have exactly one instance of the following attributes:

- **Status:** can be of sort *HeaterStatus*, *WaterheaterStatus* or *WindowStatus*, depending on the actuator, and represents the current status of the actuator;
- **EffectTemp:** the effect of the actuator on the temperature, specified as a float value;
- **EffectAQ:** the effect of the actuator on the air quality, specified as a float value; and
- **Broken:** indicates whether the actuator is broken or not.

See Figure I.4 for further details about the effect of the actuators on the temperature and air quality.

Physics. If the scheduler indicates that the temperature and air quality should be changed, then a rule is applied to change them according to the status of the actuators and the time, as specified in Section I.3. We also implemented three different ways the environment can influence the temperature and air quality in the room, where two of them do not only depend on the time of the day, but also on the time step. The environment behaviour can be selected via the attribute *Value* of the object *Environment* (see Figure I.7, Line 13).

```

1 rl [AnalyseDegA] :
2   { { H : Heater | Broken: yes, A }
3   < MC : Metacontroller | MetaLog: ML >
4   < Scheduler | Status: MCAnalyse, RuleApplied: false >
5   < ErrorPropagation | FgError: false, QaCtError: B, QaApError: B1,
6     ActError: AIE >
7   CONF}
8   =>
9   { { H : Heater | Broken: yes, A }
10  < MC : Metacontroller | MetaLog: ML >
11  < Scheduler | Status: MCAnalyse, RuleApplied: true >
12  < ErrorPropagation | FgError: true, QaCtError: false, QaApError: false,
13    ActError: heater >
14  deselect(CONF)} .

```

Figure I.10: A rule for the metacontroller in Maude. The rule is applied when the heater is broken (coloured text highlights changed or new elements).

I.5.3 The Controller Layer

The controllers, defined in Section I.3.1, are modelled as objects; the attribute `Selected` indicates if a controller is currently active (see Figure I.7, Lines 9–12).

The policies of the controllers are captured as rules. If a controller is active and the scheduler indicates that the controller should apply a rule, then the appropriate rule, which aims to change the status of the actuators, is applied. However, if an actuator is broken, its status should not be changed. We use a sort `Msg` for passing messages from a controller to an actuator, defined as `op msg_from_to : ActuatorStatus Oid Oid → Msg`. A broken actuator will not change its status when consuming the message. Otherwise it will change its status to the desired one.

The Maude model has rules that cover all the cases of the decision diagrams of the controllers. A sample rule for the comfort controller, given in Figure I.9, captures a path in the decision diagram of Figure I.5. This rule is applied when it is not morning and the temperature and air quality are OK. The pattern on the left hand side of the rule describes a configuration with the involved actuators, sensors, controllers. The scheduler has status `ContrChange` and a `RuleApplied` attribute with value `false`. The rule creates three messages and changes the attribute `RuleApplied` to `true`, which ensures that the controller only applies one rule per time step. The rule uses auxiliary functions; e.g., the function `morning` determines whether the current time is between 6 and 12 o'clock, `cold`, the functions `cold` and `hot` whether it is cold (i.e., less than 19.0°C) or hot (i.e., more than 21.0°C), and the function `aqok` whether the current air quality is above 1.0.

I.5.4 The Metacontrol Layer

We now present the model of the managing system, as shown in Figure I.1 and described in Section I.4.

```

1 op ⟨ _ : Metacontroller | MetaLog: _ ⟩ : Oid MetaLog → Metacontroller .
2 op ⟨ RequiredQAs | requQaCT: _, requQaAP: _ ⟩ : Rat Rat → Objective .
3 op ⟨ _ : QaComfTemp | _ ⟩ : Oid QaAttributes → QaComfTemp .
4 op ⟨ _ : QaAirPurity | _ ⟩ : Oid QaAttributes → QaAirPurity .
5 op ⟨ _ : FDContr | _ ⟩ : Oid FDAttributes → FunctionDesign .
6 op ⟨ ErrorPropagation | FgError: _, QaCtError: _, QaApError: _, ActError: _ ⟩
7   : Bool Bool Bool ActuatorList → InError .

```

Figure I.11: Selected objects used by the metacontroller in Maude.

Metacontroller is captured by an object of sort *Metacontroller* (see Figure I.11, Line 1). The attribute *MetaLog* contains a list of pairs (Timestep, Value), where Value can be Eco, Comf, DegA, or DegB. At a time step i , the metacontroller might switch between controllers. If so, it adds a pair (i, C_i) to the *MetaLog*, where C_i represents the controller which has been activated.

A sample rule of the metacontroller in Maude is displayed in Figure I.10. This rule is part of the set of rules for the analysis step. The rule is applied when the heater is broken to propagate the error and to signal in the Metacontrol layer that the system needs reconfiguration. Note that the curly brackets around the configuration enforce that this rule can only be applied to the configuration as a whole. The pattern on the left hand side of the rule describes a configuration with a broken heater, all the necessary components used by the metacontroller, as well as the whole configuration CONF which is not further specified. The scheduler has status *MCAnalyse* and a *RuleApplied* attribute with value *false*. The *ErrorPropagation* object has a *FgError* attribute with value *false*, *QaCtError* and *QaApError* attributes with arbitrary boolean values, and an attribute *ActError* with an arbitrary list of actuators as value. The rule changes the *RuleApplied* attribute to *true*, which ensures that only one analyse rule can be applied every time step. Furthermore, it changes the attribute *FgError* to *true* because the function grounding is in error, the attributes *QaCtError* and *QaApError* to *false*, and the attribute *ActError* to heater because the heater is broken. Lastly, it uses the auxiliary function *deselect* to deselect the currently active controller.

Quality attributes can be *required*, *expected* and *measured* QAs, see Section I.4.1. They occur in different parts of the model in Maude. The *required* QA values are captured in an object of sort *Objective* (see Figure I.11, Line 2), where *requQaCT* and *requQaAP* are the required QA values for comfortable temperature and air purity, respectively. The *expected* QA values are captured in the function designs (described later). The *measured* QA values are captured in objects of sort *QaComfTemp* and *QaAirPurity* (see Figure I.11, Lines 3–4), where *QaAttributes* is a multiset sort of attributes. Each quality attribute should include each of the following attributes exactly once:

- **Consider**: a natural number that reflects how many time steps should be considered for the computation of the quality attribute;

Controllers	NoViol			IntViol		
	Tmp.	AirQu.	Total	Tmp.	AirQu.	Total
Comfort	0	48	48	0.00	16.45	16.45
Eco	29	0	29	15.19	0.00	15.19
Metacontroller	10	14	24	4.815	4.65	9.465

Figure I.12: Collected metrics from running Scenario 1 in Maude.

- **Past**: a list of Boolean values recording $CT(t_i)$ in **QaComfTemp** and $AP(a_i)$ in **QaAirPurity**, defined in Equations I.1 and I.3, for each time step i ;
- **Status**: the current value of the quality attribute, computed according to Equation I.2 for **QaComfTemp** and according to Equation I.4 for **QaAirPurity**;
- **QaComputed**: a Boolean value that indicates whether the QA was computed in the current time step.

The rules that update the attribute **Past** and compute the new measured QA values are applied. when enabled by the scheduler.

Function design objects of sort **FunctionDesign** are defined for every controller **Contr**, where **FDAAttributes** is a multiset sort of attributes (see Figure I.11, Line 5). Function design objects include the following attributes:

- **ConContr**: the **Oid** of the controller connected to this function design;
- **ExpQaCT**: the expected QA value for comfortable temperature;
- **ExpQaAP**: the expected air purity QA value; and
- **RequActuators**: a list of required actuators.

Function grounding is represented implicitly. Error propagation happens when certain actuators are in error and when quality attributes are underachieved. This is captured with an object of sort **InError** (see Figure I.11, Lines 6 and 7), where the Boolean values indicate whether the function grounding, the QA comfortable temperature, or the QA air purity are in error, and **ActError** is a (possibly empty) list of actuators that are in error.

An **Analyse** rule is applied before a **Plan** rule, when enabled by the scheduler. The rules follow the procedure described in Section I.4.2.

I.6 Evaluating the Executable Model of the Smart House

This section reports on simulation experiments to evaluate the performance of the controllers and the impact of adding the Metacontrol framework to the system using the developed Maude model. The experiments consist of running the Maude model of the smart house for 200 time steps, measuring relevant metrics,

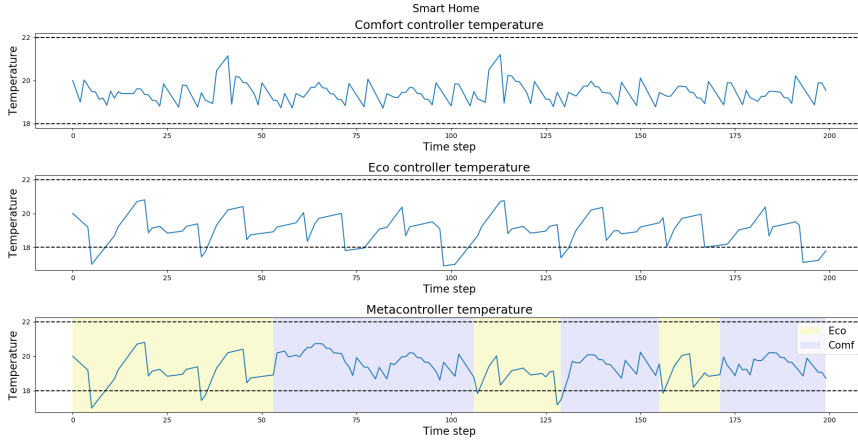


Figure I.13: The evolution of temperature for Scenario 1.

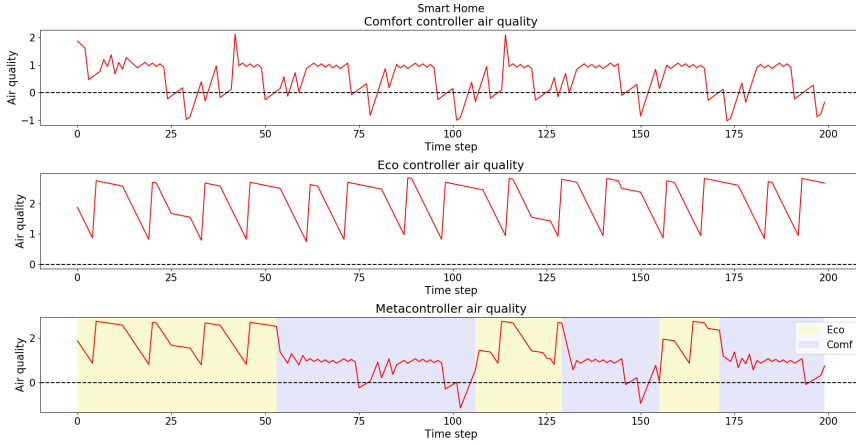


Figure I.14: The evolution of air quality for Scenario 1.

and comparing how the system behaves with the comfort and eco controllers, and with the Metacontrol layer. The experiments were done for the following scenarios: (1) all actuators are working, (2) the heater breaks at time step 75, and (3) the water heater breaks at time step 75. The experiments were also repeated for two other variations of the environment, which gave similar results to the ones reported in the paper. Note that given an initial configuration, the model is deterministic due to the round-robin scheduling policy and the simplification of the variability in the environment. Therefore, one simulation is enough to capture the behaviour of the system in the different scenarios.

The requirements R_3 and R_4 reported in Figure I.3 are instantiated as follows: We assume that a comfortable temperature is between 18 and 22 degrees and

a good air quality level is above 0. The results obtained for Scenario 1 are summarised in Figures I.12, I.13, and I.14, respectively, where we considered an initial temperature of 19°C and an initial air quality of 1.3. We collect metrics that measure the following: *NoViol* counts how many time steps the requirements R_3 and R_4 presented in Figure I.3 are violated. If, for example, the air quality drops below 0 for 2 time steps, *NoViol* will be equal to 2. *IntViol* sums up the severity of the violation with respect to the thresholds. If for example the air quality is -0.5 in time step 1, -1 in time step 2, and 0 in time step 3, it would result in an intensity of violation of -1.5. The collected metrics are shown in Figure I.12.

Figures I.13 and I.14 show that the comfort controller does not violate R_3 , but violates R_4 , and the eco controller violates R_3 , but not R_4 . When Metacontrol is used, both requirements are violated, but the number of times and the intensity of the violations are lower than when the comfort or eco controllers are used in isolation. Similar results were also obtained with the Metacontrol layer for Scenarios 2 and 3. Further experiments injected variability in the environment, where the metacontroller still performed better than the comfort and eco controller in Scenario 1 and at least as good as the comfort controller in Scenarios 2 and 3. All the experiments can be reproduced following the instructions in the online repository.

The results show that for the smart house use case, the additional Metacontrol layer, as modelled in this work, allowed the smart house system to adapt and improve its performance in all the experiments reported in this section.

I.7 Related Work

Multiple works have addressed formal models of self-adaptive systems in the context of providing assurances [Wey+17]. ActivFORMS [IW14] is a self-adaptation approach that explicitly uses formal models at runtime in the form of timed automata to promote adaptation, thus ensuring that the properties verified at design time are guaranteed at runtime. Compared with Metacontrol, ActivFORMS requires to develop the entire system, i.e. first and second order self-adaptation, using formal methods, while Metacontrol allows to leverage existing domain controllers. ENTRUST [Cal+17] is a generic methodology to design reliable self-adaptive software and its associated assurance cases, based on developing verifiable models, such as the automata in ActivFORMS, to check if they satisfy certain properties, such as that the controllers are deadlock free. Our work relates to ActivFORMS and ENTRUST regarding the development and verification of the self-adaptive system models. Our aim is to use Maude to formally model Metacontrol architectures and use it to analyse their properties.

The analysis of self-adaptive systems has been addressed using various formal methods. For example, an abstract modelling framework based on automata is introduced in [BK18], several papers [CC21; FK18; MA17] use (a subclass of) Petri nets to model self-adaptive systems, and a domain specific language

has been proposed to support compositional verification of self-adaptive cyber-physical systems [Bor+18].

In our work, we took inspiration from the work of Arcaina et al. [ARS16] on modelling MAPE-K feedback loops with Abstract State Machines, which introduced the Smart House Case Study that we adapted in our paper. However, the controllers they considered in their work correspond to the managed system in our paper; i.e., they model the control level rather than the Metacontrol level considered in our work. Furthermore, they consider a scenario in which multiple controllers can be active at the same time, which differs from our work which only considered the Metacontrol level activating and deactivating different controllers. Thus, they considered a different form of adaptation than in our work. Their work is further developed, for example, in [Arc+20].

Maude has previously been used to formalise and study reflection. In particular, rewriting logic allows a universal theory of reflection [Cla00], which is also supported by Maude. Maude has also been used to formalise and study reflection in actor models using a “Russian dolls” architecture [MT02], especially in the context of reflective middleware [VTA04]. Bruni et. al. [Bru+12] proposed a conceptual framework for self-adaptation that they realised in Maude [Bru+15]. Their work resembles ours in using a layered model of self-adaptation by means of MAPE-K feedback loops, but differs from ours in that they use a white box approach for implementing self-adaptation, whereas Metacontrol uses a black box approach.

I.8 Conclusion and Future Work

In this paper, we present the first executable formal model of a self-adaptive system based on Metacontrol. The use of Maude has allowed us to model a use case based on the smart house use case by Arcani et al. [ARS16], including all the layers in such a self-adaptive system of systems, from the physics of the environment, via the first-order adaptation layer formed by the sensors, actuators and controllers, to the second-order adaptation layer defined by the Metacontrol architecture. The results presented in Section I.6 for different scenarios of environment inputs and system failures validate our model of second-order self-adaptation with Metacontrol. They show an improved performance of the smart house system with respect to its requirements when the metacontroller performs architectural adaptation using the alternative controllers, compared to when the system’s behaviour is limited to one of the individual controllers.

We highlight that the goal of this work is not to model the best controllers in the market, but to demonstrate that for scenarios where there are different controllers with specific advantages and limitations, Metacontrol can be used to capture and exploit this knowledge in order to improve the overall system performance. However, the potential to improve performance is deeply dependent on how well-designed the TOMASys metamodel is, especially the QAs.

In this paper, we have used simulations to analyse the benefits of Metacontrol in scenarios with fairly deterministic environments. In future work, we plan to

use stronger analysis techniques, such as model checking and the exploration of what-if scenarios, to verify safety as well as liveness properties of the Metacontrol layer’s abilities for self-adaptation in more unpredictable environments. For example, it would be interesting to verify properties related to the responsiveness and coverage; e.g., the Metacontrol layer’s ability to always eventually regain correct behaviour, the maximal delay in this process, and whether the choice of active controllers will always fluctuate or eventually stabilise. Our long term goal is to analyse Metacontrol under assumptions about the managed system, and identify minimal requirements to the managed system such that the Metacontrol layer can guarantee desired overall correctness properties. A first step in this direction will be to generalise the formal model of the Metacontrol framework by replacing the executable models of the managed system and its environment by declarative specifications of their behaviour.

From the perspective of the design of the Metacontrol framework, our work on the Maude model has already provided insights into the current limitations of both the Metacontrol self-adaptation reasoning based on the TOMASys conceptual model and its execution model. In future work, we plan to extend the TOMASys metamodel to evaluate and prioritise conflicting requirements. Furthermore, we plan to explore the use of behavioural specifications of system components, as discussed above, to enrich the knowledge base of the Metacontrol architecture and integrate predictive analyses by means of formal models in the Metacontrol’s self-adaptation layer.

Acknowledgements. This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200) and the ROBOMINERS project (Grant Agreement No. 820971).

References

- [Arc+20] Paolo Arcaini, Raffaella Mirandola, Elvinia Riccobene, and Patrizia Scandurra. “MSL: A Pattern Language for Engineering Self-Adaptive Systems”. In: *Journal of Systems and Software* vol. 164 (2020), p. 110558. DOI: [10.1016/j.jss.2020.110558](https://doi.org/10.1016/j.jss.2020.110558).
- [ARS16] Paolo Arcaini, Elvinia Riccobene, and Patrizia Scandurra. “Formal Design and Verification of Self-Adaptive Systems with Decentralized Control”. In: *ACM Transactions on Autonomous and Adaptive Systems*. 2016.
- [BK18] Aimee Borda and Vasileios Koutavas. “Self-Adaptive Automata”. In: *Proc. 6th Conference on Formal Methods in Software Engineering (FormaliSE 2018)*. ACM, 2018, pp. 64–73. DOI: [10.1145/3193992.3194001](https://doi.org/10.1145/3193992.3194001).

- [Bor+18] Aimee Borda, Liliana Pasquale, Vasileios Koutavas, and Bashar Nuseibeh. “Compositional Verification of Self-Adaptive Cyber-Physical Systems”. In: *Proc. 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2018)*. 2018, pp. 1–11.
- [Bru+09] Yuriy Brun, Giovanna Di Marzo Serugendo, Cristina Gacek, Holger Giese, Holger M. Kienle, Marin Litoiu, Hausi A. Müller, Mauro Pezzè, and Mary Shaw. “Engineering Self-Adaptive Systems through Feedback Loops”. In: *Software Engineering for Self-Adaptive Systems*. Vol. 5525. Lecture Notes in Computer Science. Springer, 2009, pp. 48–70.
- [Bru+12] Roberto Bruni, Andrea Corradini, Fabio Gadducci, Alberto Lluch-Lafuente, and Andrea Vandin. “A Conceptual Framework for Adaptation”. In: *Proc. 15th International Conference on Fundamental Approaches to Software Engineering (FASE 2012)*. Ed. by Juan de Lara and Andrea Zisman. Vol. 7212. Lecture Notes in Computer Science. Springer, 2012, pp. 240–254. DOI: [10.1007/978-3-642-28872-2_17](https://doi.org/10.1007/978-3-642-28872-2_17).
- [Bru+15] Roberto Bruni, Andrea Corradini, Fabio Gadducci, Alberto Lluch-Lafuente, and Andrea Vandin. “Modelling and Analyzing Adaptive Self-Assembly Strategies with Maude”. In: *Science of Computer Programming* vol. 99 (2015), pp. 75–94. DOI: [10.1016/j.scico.2013.11.043](https://doi.org/10.1016/j.scico.2013.11.043).
- [BS03] Egon Börger and Robert F. Stärk. *Abstract State Machines. A Method for High-Level System Design and Analysis*. Springer, 2003.
- [Cal+17] Radu Calinescu, Danny Weyns, Simos Gerasimou, Muhammad Usman Iftikhar, Ibrahim Habli, and Tim Kelly. “Engineering trustworthy self-adaptive software with dynamic assurance cases”. In: *IEEE Transactions on Software Engineering* vol. 44, no. 11 (2017), pp. 1039–1069.
- [CC21] Matteo Camilli and Lorenzo Capra. “Formal Specification and Verification of Decentralized Self-Adaptive Systems Using Symmetric Nets”. In: *Discrete Event Dynamic Systems* vol. 31, no. 4 (2021), pp. 609–657. DOI: [10.1007/s10626-021-00343-3](https://doi.org/10.1007/s10626-021-00343-3).
- [Cla+07] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn L. Talcott. *All About Maude - A High-Performance Logical Framework, How to Specify, Program and Verify Systems in Rewriting Logic*. Vol. 4350. Lecture Notes in Computer Science. Springer, 2007. DOI: [10.1007/978-3-540-71999-1](https://doi.org/10.1007/978-3-540-71999-1).
- [Cla00] Manuel Clavel. *Reflection in Rewriting Logic: Metalogical Foundations and Metaprogramming Applications*. CSLI Publications, 2000.

- [Cor13] Carlos Hernández Corbato. “Model-based self-awareness patterns for autonomy”. PhD thesis. Universidad Politécnica de Madrid, 2013.
- [FK18] Muhammad Ilyas Fakhir and Syed Asad Raza Kazmi. “Formal Specification and Verification of Self-Adaptive Concurrent Systems”. In: *IEEE Access* vol. 6 (2018), pp. 34790–34803. DOI: [10.1109/ACCESS.2018.2849821](https://doi.org/10.1109/ACCESS.2018.2849821).
- [Gar21] David Garlan. “The Unknown Unknowns Are Not Totally Unknown”. In: *Proc. International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2021)*. IEEE, 2021, pp. 264–265. DOI: [10.1109/SEAMS51251.2021.00047](https://doi.org/10.1109/SEAMS51251.2021.00047).
- [HBS18] Carlos Hernández, Julita Bermejo-Alonso, and Ricardo Sanz. “A self-adaptation framework based on functional knowledge for augmented autonomy in robots”. In: *Integrated Computer-Aided Engineering* vol. 25, no. 2 (2018), pp. 157–172.
- [IW14] M Usman Iftikhar and Danny Weyns. “ActivFORMS: Active formal models for self-adaptation”. In: *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2014)*. ACM, 2014, pp. 125–134.
- [KC03] Jeffrey O. Kephart and David M. Chess. “The Vision of Autonomic Computing”. In: *IEEE Computer* vol. 36, no. 1 (2003), pp. 41–50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
- [MA17] Natash Ali Mian and Farooq Ahmad. “Modeling and Analysis of MAPE-K Loop in Self Adaptive Systems Using Petri Nets”. In: *International Journal of Computer Science and Network Security (IJCSNS)* vol. 17 (2017), p. 6.
- [Mes12] José Meseguer. “Twenty years of rewriting logic”. In: *The Journal of Logic and Algebraic Programming*. vol. 81, no. 7-8 (2012), pp. 721–781. DOI: [10.1016/j.jlap.2012.06.003](https://doi.org/10.1016/j.jlap.2012.06.003).
- [MT02] José Meseguer and Carolyn L. Talcott. “Semantic Models for Distributed Object Reflection”. In: *ECOOP*. Vol. 2374. Lecture Notes in Computer Science. Springer, 2002, pp. 1–36.
- [VTA04] Nalini Venkatasubramanian, Carolyn Talcott, and Gul A Agha. “A Formal Model for Reasoning About Adaptive QoS-Enabled Middleware”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* vol. 13, no. 1 (2004), pp. 86–147.
- [Wey+17] Danny Weyns, Nelly Bencomo, Radu Calinescu, Javier Camara, Carlo Ghezzi, Vincenzo Grassi, Lars Grunske, Paola Inverardi, Jean-Marc Jezequel, Sam Malek, Raffaella Mirandola, Marco Mori, and Giordano Tamburrelli. “Perpetual Assurances for Self-Adaptive Systems”. In: *Software Engineering for Self-Adaptive Systems III*.

- Assurances*. Ed. by Rogério de Lemos, David Garlan, Carlo Ghezzi, and Holger Giese. Cham: Springer, 2017, pp. 31–63.
- [Wir90] Martin Wirsing. “Algebraic Specification”. In: *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics (B)*. Elsevier and MIT Press, 1990, pp. 675–788.
- [Ölv17] Peter Csaba Ölveczky. *Designing Reliable Distributed Systems - A Formal Methods Approach Based on Executable Modeling in Maude*. Springer, 2017. DOI: 10.1007/978-1-4471-6687-0.

Analysing Self-Adaptive Systems as Software Product Lines

**Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani,
Einar Broch Johnsen, S. Lizeth Tapia Tarifa**

Published in *Journal of Systems and Software*. 2025, vol. 222, p. 112324. DOI: 10.1016/j.jss.2024.112324.

Abstract

Self-adaptation is a crucial feature of autonomous systems that must cope with uncertainties in, e.g., their environment and their internal state. Self-adaptive systems (SASs) can be realised as two-layered systems, introducing a separation of concerns between the domain-specific functionalities of the system (the *managed* subsystem) and the adaptation logic (the *managing* subsystem), i.e., introducing an external feedback loop for managing adaptation in the system. We present an approach to model SASs as dynamic software product lines (SPLs) and leverage existing approaches to SPL-based analysis for the analysis of SASs. To do so, the functionalities of the SAS are modelled in a feature model, capturing the SAS's variability. This allows us to model the managed subsystem of the SAS as a family of systems, where each family member corresponds to a valid feature configuration of the SAS. Thus, the managed subsystem of an SAS is modelled as an SPL model; more precisely, a probabilistic featured transition system. The managing subsystem of an SAS is modelled as a control layer capable of dynamically switching between these valid configurations, depending on both environmental and internal conditions. We demonstrate the approach on a small-scale evaluation of a self-adaptive autonomous underwater vehicle used for pipeline inspection, which we model and analyse with the feature-aware probabilistic model checker ProFeat. The approach allows us to analyse probabilistic reward and safety properties for the SAS, as well as the correctness of its adaptation logic.

II.1 Introduction

This paper proposes an approach to model and analyse self-adaptive systems (SASs) [Abb+23; Wey20] as dynamic software product lines (DSPLs) [Hal+13; HPS12]. To demonstrate the usefulness and feasibility of the approach, we

conduct a small-scale evaluation [WR22] using a self-adaptive autonomous underwater vehicle (AUV).

SASs form a fast-evolving field of research and range from applications like smart houses to autonomous robots. They can be realised as two-layered systems with a *managed subsystem* responsible for the domain concerns and a *managing subsystem* responsible for the adaptation logic, i.e., adapting the managed subsystem to changes in the system and its environment. Self-adaptation is extensively applied in industry, for example in the web, mobile and cloud domains and for embedded, cyber-physical and IoT systems [Abb+23]. An SAS is able to adapt its behaviour during runtime to, e.g., changes in the environment, failures of the system, or varying user requirements by exploiting existing redundancies of the system. Many SASs include design-time as well as runtime variability. Considering, e.g., self-adaptive robots, the hardware as well as the software can be configured during design time according to, e.g., customers' preferences and the application domain [Gar+19]. During runtime, variability in the robot could for example include different localisation techniques that can be selected according to the current environmental conditions. Software product lines (SPLs) have previously been proposed to model static variability of robotic systems [GB14], i.e., variability during design time, and DSPLs have been proposed to manage variability during runtime for self-adaptive robots [BCH15; GH15].

Although the idea of using DSPLs to manage runtime variability for self-adaptive robots is appealing [BCH15], it is still considered an unsolved challenge [Gar+19]. Managing runtime variability for SASs is in general very difficult and “there is a need to validate the proposals, either in an industrial environment or in different test cases, expanding the application areas”, according to a review on variability management in DSPLs for SASs [AS22], which considers 84 papers published during 2010–2021. While DSPLs have been used before to manage runtime variability of self-adaptive robots, formal verification techniques that have been applied in the past to DSPLs have not been used to analyse SASs: a recent literature review [AMV23] on testing, validation, and verification of robotic and autonomous systems does not discuss any work using family-based analysis techniques for SASs.

One way to specify SPL models is by means of featured transition systems (FTSs) [Cla+13]. An FTS models the behaviour of all configurations of an SPL in a single transition system by associating transitions with features that condition the possibility of executing the transition in specific configurations, governed by a feature (variability) model [Ape+13; BSE19; Sch+06]. Thus, an FTS realises a so-called 150% family model that contains many superimposed variants [Sch+12; Thü+19]. When analysing the SPL, the compact structure of an FTS is exploited to reason about the whole SPL, i.e., all variants, at once. This approach was extended to DSPLs by allowing dynamic feature reconfiguration [Cor+13a] and by supporting probabilistic and non-deterministic choices as well as allowing quantitative analysis [DKB14].

Methodology

In this paper, we use family-based techniques from DSPLs to model and analyse a two-layered SAS focusing on runtime variability in the form of activating and deactivating features. To do so, we present a methodology that shows how two-layered SASs can be seen as DSPLs and how existing techniques for modelling and analysing DSPLs can be applied to SASs. Thereby, we highlight the natural correspondence between SASs and family-based DSPL modelling, and show in particular how family-based analysis methods can be used for SASs. In this paper, we thus propose to use existing techniques and tools known for modelling and analysing DSPLs, to model and analyse SASs. More specifically, we highlight the correspondence between the features of a feature model and the configurations of a managed subsystem, as well as the correspondence between dynamically changing the feature configuration of the managed subsystem and the managing subsystem of the SAS. Using these correspondences, existing tools for the analysis of DSPLs can be leveraged for the analysis of SASs. To demonstrate this, we conduct a small-scale evaluation that exemplifies our approach.

In the small-scale evaluation used in this paper, an AUV is modelled as a probabilistic FTS with dynamic feature switching. As the behaviour of the AUV depends on external conditions, which are hard to control, we opted for a probabilistic model in which uncontrolled events, like a thruster failure, occur with given probabilities. Using a probabilistic model enables us to provide safety guarantees of the system even in the presence of uncertainties. The managed subsystem of the AUV, handling the domain concerns, is modelled as a family of systems whose family members correspond to valid feature configurations, and a feature model captures the domain-specific functionalities of the AUV, making the dependencies and requirements between the components of the AUV explicit. The adaptation behaviour of the AUV (the managing subsystem) is modelled as a control layer with dynamic *feature switches*, changing the feature configurations of the managed subsystem according to input from a probabilistic environment model, a probabilistic hardware failures model and the managed subsystem. As our focus in this paper is on how family-based DSPL modelling and family-based analysis can be used to analyse SASs, our small-scale evaluation presents a simplified version of an AUV, with limited variability. Although the presented model can be extended to a more realistic underwater robot, the small-scale evaluation is sufficient to illustrate the idea.

In our small-scale evaluation, the analysis can give system operators an estimate of the mission duration and of the AUV's energy consumption, provide safety guarantees, and ensure that the implementation of the adaptation logic satisfies certain correctness properties. Our analyses have been performed with ProFeat [Chr+18], a tool for probabilistic family-based model checking. Family-based model checking provides a means to simultaneously model check, in a single run, properties of a family of models, each representing a different configuration [Thü+14]. We describe families of probabilistic systems as probabilistic FTS models, augmented with *costs* and *rewards*, which are useful for quantitative analysis, i.e., the resulting FTSs are featured Markov decision processes [BK08].

Contributions. This paper is an extension of Päßler et al. [Päß+23a], with related artifact [Päß+23b], recently published as an original software publication [Päß+25], which introduced a small-scale evaluation of an SAS from the underwater robotics domain, modelled as a probabilistic FTS with dynamic feature switching, and used family-based analysis to verify some essential (quantitative) properties. The contributions of that paper are here extended by including:

- a background discussion and context for the proposed approach to modelling and analysing two-layered SASs as DSPLs (Section II.2);
- an extended small-scale evaluation of an SAS from the underwater robotics domain, adding sensors used with different priorities as a possibility for the need to abort the mission and as another reason for adaptation (Section II.3);
- an extended probabilistic FTS model with dynamic feature switching, including a new module for modelling sensor failures, an extended managed subsystem and a more complex managing subsystem (Section II.4);
- new analyses, including analysing sensor failures and correctness issues of the adaptation logic (Section II.5);
- an evaluation of how the proposed approach supports extensibility of the model, to what extent it supports the analysis of an SAS, and how the correctness of the adaptation logic with respect to its specification can be analysed (Section II.6).

Overall, we propose to leverage existing techniques for modelling and analysing DSPLs, and apply them to the modelling and analysis of SASs instead of focusing on modelling and analysing a realistic SAS. To show the feasibility of this proposed approach by means of a proof of concept, we conduct a small-scale evaluation of a simplified AUV mission that allows us to highlight the correspondence between a two-layered self-adaptive robotic system and an FTS with its corresponding feature controller for dynamic feature switching.

Outline. The paper is structured as follows. Section II.2 discusses the correspondence between SASs and family-based DSPL modelling. Section II.3 presents the extended small-scale evaluation of a pipeline inspection with an AUV. Section II.4 substantiates the discussion in terms of our small-scale evaluation and how it can be modelled as a family-based system, while Section II.5 presents the analysis results. Section II.6 presents an evaluation of our approach. Section II.7 provides an overview of related work, and Section II.8 discusses our results and some ideas for future work.

II.2 Family-Based Modelling and Analysis of SASs

In this section, we consider SASs from the perspective of runtime variability and family-based analysis. Since SPLs offer a high-level, structured view of a configuration space, self-adaptation can be understood in terms of runtime variability and DSPLs. However, taking this view on self-adaptation triggers the following question: how can we use family-based DSPL modelling and family-based analysis techniques to analyse self-adaptive behaviour in an SAS to increase its reliability? We argue that analysis techniques for family-based models like FTSs, combined with a model that captures the adaptation logic for runtime variability, moving between different valid products of the product family, can be a natural fit for the analysis of SASs.

In the remainder of this section we detail our argument by (1) giving an overview of what SASs are and the uncertainties that trigger reconfiguration, in particular, uncertainties in physical environments, a very important aspect of resilience in the context of autonomous robots; (2) giving an overview of product variability approaches at runtime and family-based analysis techniques that mostly explore techniques to analyse a family of products under known contexts; and (3) discussing the natural fit between SASs and DSPLs and summarising our proposal for how to leverage family-based modelling and analysis techniques to explore properties in the reconfiguration space that are valid under certain uncertainties that are captured in the model.

II.2.1 SASs and Uncertainties that Trigger Reconfiguration

What are SASs? Many software systems are subject to different forms of uncertainty like changes in the surrounding environment, internal failures and changing user requirements. Often, manually maintaining and adapting these systems during runtime by a system operator is prohibitively expensive and error-prone. Enabling systems to adapt themselves provides several advantages. For example, a system that is able to perform self-adaptation can also be deployed in environments where communication between an operator and the system is very limited or impossible, e.g., in space or under water. Self-adaptation gives a system a higher level of autonomy.

SASs can be realised by *internal* or *external self-adaptation* [KC03; ST09; Wey20]. An internal self-adaptation mechanism embeds the adaptation logic within the application logic of the system itself using, e.g., exception handling or fault-tolerance mechanisms. Internal self-adaptation has been criticised for poor maintainability and scalability. In contrast, external self-adaptation introduces a separation of concerns between the application logic and the adaptation logic of the SAS through an external feedback loop. Using external self-adaptation has “the advantage that [it] localize[s] the adaptation concerns in sparable system elements that can be analyzed, modified, and reused across different self-adaptive systems” [Wey20]. In this paper, we focus on external self-adaptation.

SASs with external self-adaptation can be implemented using a two-layered approach which decomposes the system into a *managed* and a *managing*

subsystem [KC03] (see Figure II.1). The *managed* subsystem deals with the domain concerns of the application logic and tries to reach the goals set by the user of the system, e.g., navigating a robot to a specific location. The *managing* subsystem handles the adaptation concerns and defines an adaptation logic that specifies a strategy on how the system can fulfil the goals under uncertainty [Wey20], e.g., adapting to changing environmental conditions. While the managed subsystem may affect the environment with its actions, the managing subsystem monitors the environment and the internal state of the managed subsystem. Using the adaptation logic, the managing subsystem deduces whether and which reconfiguration of the managed subsystem is needed and adapts the managed subsystem accordingly, exploiting so-called hardware and software redundancies of the managed subsystem to do so. For example, for robots, this can be realised in practice [Rez+23] using, e.g., the System Modes package [NLR21] of the Robot Operating System (ROS) [Qui+09]. Self-adaptive mission control has been realised in several studies (e.g., [Her13; Rom+22]), enabling on-the-fly manipulation of control strategies (e.g., by activation or deactivation). There are also implementations of some of these systems in ROS [Boz+22; Bru20]. We refer to [SK16, Chapter 12: Robotic Systems Architectures and Programming] for further robot systems control architectures.

In two-layered SASs, managing subsystems can be realised using a MAPE-K feedback loop [KC03]. Systems implementing a MAPE-K loop *Monitor* the environment and the managed subsystem during runtime (collecting, e.g., sensor readings and information about the internal state of the managed subsystem); this information is then *Analysed* to decide whether an adaptation of the managed subsystem is needed; if this is the case, the managing subsystem *Plans* a reconfiguration of the managed subsystem, which it afterwards *Executes*. These steps are all performed in the context of a shared *Knowledge base*.

The adaptation logic of the *Plan* phase of the MAPE-K loop is typically specified using either rule-based or goal-based approaches [BHA12], relating the monitored data about the managed subsystem and the environment (e.g., component status information and sensor data) to models in the *Knowledge base* (e.g., valid component configurations and metrics) to systematically plan how to achieve a goal (e.g., safely navigating a robot to a location). Rule-based approaches provide deterministic, predetermined actions in response to changes in the environment and the managed subsystem, whereas goal-based approaches provide more flexibility for optimisation, possibly by systematically exploring all possible configurations before making a decision.

SAS in Physical Environments. One of the most important challenges for an SAS that operates in a physical environment (see Figure II.1) is to handle the uncertainties that are induced by this environment [Hez+21]. SASs that operate in a physical environment, such as, e.g., robots, have to face and overcome this additional uncertainty. Examples of uncertainties in the environment that a robot needs to overcome include unexpected obstacles, erosion or avalanches, and climate events like rain or wind, which affect the

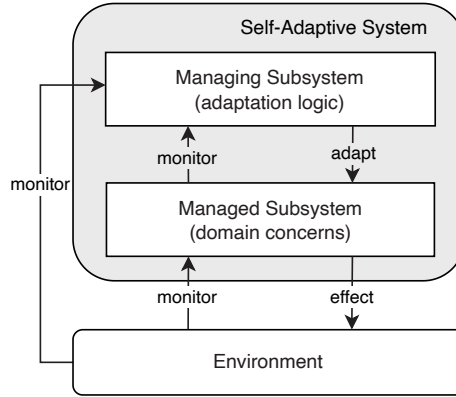


Figure II.1: A two-layered architecture for an SAS ([Päß+23a])

robot’s ability to manoeuvre or to inspect an asset. While there is no established approach for how to handle uncertainty in general, many practitioners use formal techniques and learning, along with general paradigms like the MAPE-K feedback loop, to develop an adaptation logic that can handle the uncertainties of these environments [Hez+21].

To understand how changes in the environment affect the behaviour of the SAS, it is crucial to analyse behavioural requirements when developing an SAS that operates under the uncertainties of a physical environment. These requirements often use quantitative metrics that change during runtime, such as monitored information about the managed subsystem and the environment. For instance, for behavioural requirements concerning energy consumption, the metric should be minimised, while for safety requirements, the metric should be maximised. Both rule-based and goal-based adaptation logics can be used to enable the SAS to meet its behavioural requirements. Many practitioners rely on formal methods to provide evidence for the system’s compliance with such requirements [Luc+19; Wey+12], but many different methods are used [AMV23; Hez+21].

II.2.2 Towards Managing Variability for a Family of Products

What are DSPLs? DSPLs are a generalisation of SPLs for developing highly-configurable, runtime-adaptive systems, i.e., they allow to switch from one product to another at runtime [BHA12; HPS12; Sch+08]. In particular, DSPLs structure the configuration space for runtime variability as an SPL. They can, e.g., be realised by means of a logic for self-adaptation that respects the structure of a feature model; different solutions to support runtime variability mechanisms have been proposed [Cap+14; Val+22]. DSPLs may be found in application domains such as service-oriented systems, mobile software, ecosystems and SASs [Cap+14].

A crucial distinction within the DSPL design landscape [BHA12] is between

bounded adaptivity, which deals with context variation that is anticipated at design time, and *open adaptivity*, which deals with context variation that is not planned at design time and requires model extension. For bounded adaptivity, DSPLs have been advocated as a means to constrain the evolution of SASs and to enable the assessment of important properties of a system even before starting its implementation [Bar14]. For open adaptivity, dynamic feature models have been proposed to model variability during runtime [Din+10]. More recently, DSPLs have been advocated to provide a conceptual framework for managing the variability in cyber-physical systems which need to frequently reconfigure their software components (e.g., due to the addition, update or removal of physical components) without being shut down [Qui+21].

Analysis of Family-Based Models. A range of techniques have been explored for the analysis of SPLs, including type checking, static analysis, theorem proving, and model checking. Thüm et al. [Thü+14] provided an overview of these analysis techniques, emphasising in particular *family-based analysis* techniques for SPLs. Family-based techniques are all-in-one techniques, according to which the behaviour of all product configurations (variants) of the SPL is examined in the same analysis, simultaneously. Family-based analysis differs from brute-force enumerative product-based analysis, in which the behaviour of every product (variant) is examined individually, one-by-one.

Family-based model checking is a prominent family-based analysis technique that provides a means to simultaneously model check, in one single run, properties of multiple models, each representing the behaviour of a different product configuration (variant) of the SPL. FTSs were introduced to model the behaviour of SPLs for family-based model checking [Cla+10]. Their action-labelled transitions are equipped with feature expressions that condition the presence of transitions in specific product configurations, which are represented by ordinary labelled transition systems obtainable by projection. The compact structure of an FTS enables reasoning on the behaviour of the whole SPL at once. The properties of FTSs can be verified by dedicated family-based model-checking tools such as SNIP [Cla+12; Cla+13], ProVeLines [Cor+13b], (FTS4)VMC [Bee+16; Bee+22; BM14; BMS12] and fNuSMV [Cla+14; DLW19]. Statistical family-based model checking is supported by QFLan [Bee+20a; Van+18] and probabilistic family-based model checking by ProFeat [Chr+18; DBK15], which is designed to analyse quantitative properties for families of stochastic systems. Next to dedicated SPL model-checking tools, suitable abstractions or encodings have made well-known classical model checkers such as SPIN [Dim+15; Dim+17; DW17], PRISM [DBK15], Maude [Loc+16], mCRL2 [Bee+20b; BVW17] and NuSMV [Dim20] amenable to family-based model checking. The advantage of using well-established and typically highly optimised off-the-shelf model checkers with a broad user base is to avoid having to maintain dedicated SPL model checkers. For example, ProFeat uses PRISM as a back-end tool: after translating its input models to PRISM input models, PRISM is invoked for the actual analysis, after which the output of PRISM is post-processed by ProFeat.

The development of family-based analysis techniques for DSPLs has so far received limited attention. Model-based approaches to handle feature changes at the code level in DSPLs have been proposed [ASN14; Dam+18; MCP13], combining delta- and feature-oriented programming [Ape+12; Ape+13; Sch+10] with techniques for dynamic software updates such as object migration. More recently, a family-based algorithm, based on modelling the configuration space of a DSPL as an FTS, estimates the quality of service (expressed as the sum of weights) provided by each configuration of a weighted FTS over an observed execution trace [Ola+18]; the authors analyse the quality of service obtained for the trace using different configurations of the DSPL but do not analyse a self-adaptation policy as such. In contrast, *feature controllers*, which enable the dynamic activation or deactivation of features in the FTS, are supported by the family-based probabilistic model checker ProFeat [Chr+18]. Observe that such a feature controller, expressed as a transition system, is restricted to a finite state space and therefore to bounded adaptivity.

II.2.3 Family-Based Modelling and Analysis of Two-Layered SASs

In this paper, we propose a family-based approach to model and analyse two-layered SASs. The managed subsystem of an SAS needs to include hardware and software redundancies to enable self-adaptation. These redundancies, which can, e.g., be (partially) redundant components, algorithms, and parameters, can be used by the managing subsystem to adapt the managed subsystem if a redundant part does not work as expected or cannot be used in the current (environmental) conditions. For example, an underwater robot might include different sensors that can be used to observe the environment, like a camera and a sonar. Due to energy restrictions, the managing subsystem might only want to use one sensor at a time. However, if one of the sensors breaks or the environmental conditions change, it can still adapt the managed subsystem to use the other sensor even though it might not provide optimal performance. From the SPL perspective, these redundancies can be seen as features of the managed subsystem. Each configuration of the managed subsystem then corresponds to a choice of features. Thus, a managed subsystem can be considered as a family of configurations that share the same core functionalities (features) and differ in their optional features. Consequently, the managed subsystem can be understood as an SPL where a product (variant) corresponds to a configuration of the managed subsystem. The managing subsystem can then be understood as an adaptation logic for runtime variability for the SPL of the managed subsystem.

Note that we consider all points of variability of the managed subsystem that can be adapted by the managing subsystem to be features¹. For example, if the managing subsystem adapts parameters of the managed subsystem, these parameters are considered to be features since we want to analyse the interaction between the managed and managing subsystem. While the more common SPL

¹The opinions on what constitutes a feature differ between the SPL and the SAS communities.

modelling approaches typically would not consider all such variability points to be features, there exist many different definitions of what constitutes a feature [CHS08] and our approach is in line with that of [CE00, Chapter 4: Feature Modeling]: “anything users or client programs might want to control about a concept is a feature”.

Modelling the behaviour of a managed subsystem of an SAS as a 150% SPL model, such as an FTS model, enables analysing the behaviour of all possible configurations of the managed subsystem in one run. However, this 150% SPL model is not sufficient to analyse the two-layered SAS; the model still lacks the managing subsystem. The managing subsystem of the SAS can be seen as a controller that switches between different configurations of the managed subsystem when needed. Triggered by, e.g., changes in the physical environment, the managing subsystem changes the active features of the managed subsystem. Considering an FTS model of the behaviour of a managed subsystem, the managing subsystem enables and disables transitions (i.e., behaviour) dynamically during runtime. Hence, an SAS can be modelled as a 150% SPL model together with a controller that enables and disables behaviour during runtime [DKB14]. To capture the uncertainties in the managed subsystem, a probabilistic model can be chosen.

Therefore, we propose to model a two-layered SAS as a family-based system by modelling the managed subsystem of an SAS as a probabilistic FTS and the managing subsystem as a *non-deterministic controller* activating and deactivating features. This enables the analysis of all possible configurations and re-configurations of the managed subsystem in one single run. The family-based model of the two-layered SAS makes it possible to analyse optimal strategies for the managing subsystem with respect to different desired properties of the managed subsystem. Furthermore, since FTSs have been in use for SPL analysis for some time now, there are already existing, mature tools that can be exploited for the analysis of SASs.

In his keynote address “The 20-year journey of SPLE in Hitachi and the next” at the 2023 SPL Conference (SPLC 2023), Kentaro Yoshimura, chief researcher at Hitachi, presented the use of DSPLs for autonomous robotic systems as a new industrial challenge. He mentioned that the dynamicity is in the runtime behaviour of the autonomous robots that need to adapt and reconfigure based on input perceived from the environment without continuous human guidance. To meet this challenge, we capture the uncertainties of the environment as a probabilistic transition system that we compose with the probabilistic FTS model of the SAS; the idea is that we do not model how or why changes occur in the environment but rather the probabilities with which the SAS observes these changes.

By combining the family-based model of the two-layered SAS with a stochastic model of its environment, we obtain a Markov decision process that can be enriched with costs and rewards, and model checked also for non-functional properties as well as for optimal adaptation strategies. It is worth remarking that this approach is restricted to bounded adaptivity in the sense that we do not extend the models. However, the underspecified (or non-deterministic) adaptation logic enables the proposed family-based analysis to find the optimal

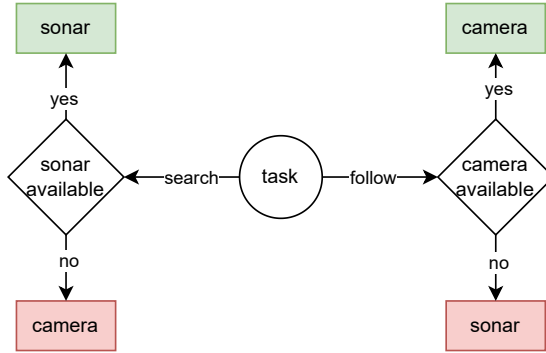


Figure II.2: The priorities for choosing the vision sensors; if both vision sensors fail, the mission is aborted

adaptation logic for a given environment model. The adaptation logic need not be fully understood or specified a priori for all environments; rather, the adaptation logic of the two-level SAS can be automatically adapted to different environment models by means of the proposed family-based analysis.

To demonstrate the proposed methodology of family-based analysis of two-level SASs, Section II.3 presents a small-scale evaluation of an SAS with a rule-based managing subsystem. Goal-based systems, which may need to consider multiple possible configurations, fit equally well with family-based models by exploring multiple variants at a time. Section II.3 also details how the managed subsystem of the SAS can be viewed as a family-based system, and the managing subsystem as a controller switching between configurations of the managed subsystem. Section II.4 then details how the proposed methodology was implemented in the tool ProFeat [Chr+18].

II.3 Small-Scale Evaluation: Pipeline Inspection by AUV

In this section, we introduce our small-scale evaluation of an AUV used for pipeline inspection, which was inspired by the exemplar SUAVE [Rez+23]. This work extends the small-scale evaluation described and modelled in our previous work [Päß+23a; Päß+23b] by including two sensors for vision, a *camera* and a (*side-scan*) *sonar*.

II.3.1 An Overview of the Small-Scale Evaluation

An AUV has the mission to first find and then inspect a pipeline located on a seabed. During system operation, it can use either the camera or the sonar as a vision sensor. Both can be used for searching for the pipeline and for inspecting it. However, the sonar is preferred for searching because it can cover a wider area and operate at a higher altitude, while the camera is preferred for following and inspecting the pipeline because it is easier to detect faults in the pipeline

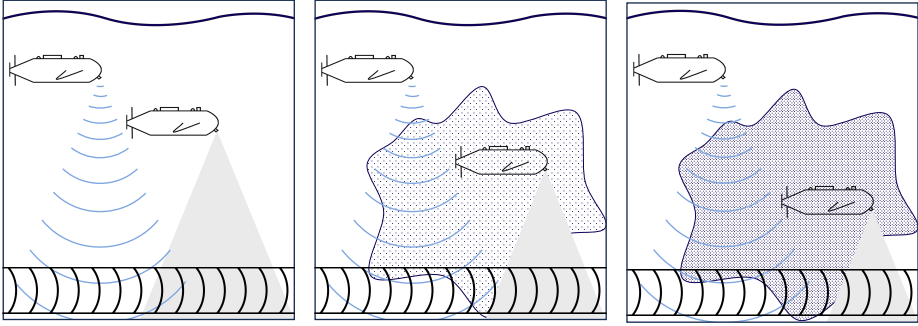


Figure II.3: The maximum altitudes in different water visibilities when operating with the sonar (left AUV in each image) and with the camera (right AUV in each image). The left image shows good visibility, the middle medium visibility and the right bad visibility.

with the camera, see Figure II.2. The water visibility (i.e., the distance in meters within which the AUV can perceive objects with the camera) might change during runtime (e.g., due to currents that swirl up the seabed), affecting at which altitude the AUV can operate when using the camera. Furthermore, one or more of the AUV's thrusters might fail and need to be restarted before the mission can be continued. In addition, the camera and the sonar can have a permanent software or hardware failure, making it necessary to use the other vision sensor or abort the mission if both sensors fail. The camera can also get blocked, e.g., because of natural or human waste sticking to the camera. In this case, the camera is only temporarily unavailable and there is a probability of the camera getting unblocked, thus the camera becoming available again.

The AUV can choose to operate at four different altitudes, *low*, *med* (for medium), *high* and *very high*. A higher altitude allows the AUV to have a wider field of view and thus increases its chances of finding the pipeline during its search. The probability of a thruster failure is lower at a higher altitude because, e.g., seaweed typically might wrap around the thrusters at a lower altitude. However, when using the camera, the altitude at which the AUV can perceive the seabed depends on the water visibility, and the AUV can never perceive the seabed from a very high altitude. With low water visibility, the AUV cannot perceive the seabed with the camera from a high or medium altitude. Thus, it is not always possible to operate at a high or medium altitude, and the altitude of the AUV needs to be changed during the search when using the camera, depending on the current environmental conditions. However, when using the sonar for searching, the AUV can always operate at a very high altitude because the water visibility does not affect the sonar; see Figure II.3 for an illustration of the maximum altitudes when operating with the sonar and the camera in different environmental conditions.

Once the pipeline is found, the AUV will follow it at a low altitude to increase the resolution of the camera and sonar images. However, the AUV can also lose

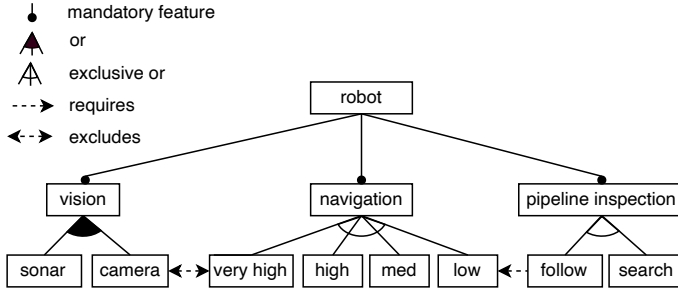


Figure II.4: Feature model of the small-scale evaluation

the pipeline again, e.g., when the pipeline is partly covered by sand or the AUV's thrusters failed for some time causing the AUV to drift off its path. In this case, the AUV has to search for the pipeline again, enabling all four altitudes.

II.3.2 Two-layered View of the AUV as a Family-based Model

Considering the AUV as a two-layered SAS, the AUV's managed subsystem is responsible for the following tasks: (1) the search for the pipeline and (2) the inspection of the pipeline. Depending on the current task, altitude, and available sensors of the AUV, a different configuration of the managed subsystem must be chosen. Thus, the managed subsystem can be seen as a family of systems where each family member corresponds to a valid configuration of the AUV, i.e., it can be seen as an SPL as discussed in Section II.2. To do so, the different altitudes for navigation (*low*, *med*, *high*, and *very high*), the tasks *search* and *follow*, and the vision sensors *camera* and *sonar* can be seen as *features* of the managed subsystem that adhere to the *feature model* in Figure II.4, which models the dependencies and constraints among the features [Ape+13; Mei+17]. Each configuration of the AUV contains exactly one feature for navigation and one for pipeline inspection, and feature *follow* requires feature *low*. The configuration also includes one or no feature for vision, but the feature *camera* excludes the feature *very high*. This yields nine different configurations of the managed subsystem of the AUV, i.e., nine different products of the SPL.

The managing subsystem of the AUV switches between these configurations during runtime by activating and deactivating the subfeatures of *navigation*, *pipeline inspection* and *vision*, while the resulting feature configuration has to adhere to the feature model in Figure II.4, i.e., a valid product of the SPL has to be chosen. The managing subsystem chooses the feature *sonar* when searching for the pipeline if the sonar did not fail. If the sonar fails, it chooses the feature *camera* also for searching for the pipeline. When using the camera for the search, the features *low*, *med* and *high* are activated and deactivated according to the current water visibility. If the water visibility is good, the features *low*, *med*, and *high* can be activated; if the water visibility is average, *high* cannot be activated; and if the water visibility is poor, only *low* can be activated. The managing

subsystem switches from the feature *search* to *follow* if the pipeline was found, and from *follow* to *search* if the pipeline was lost. When the AUV is following the pipeline, the managing subsystem prefers to choose the feature *camera* for inspecting it if the camera did not fail and is not blocked. If the camera fails during or before the inspection or is blocked during the inspection, the managing subsystem chooses the feature *sonar* for inspecting the pipeline. It deactivates both vision subfeatures if both the camera and the sonar failed, resulting in a configuration without an active vision subfeature and in the need to abort the mission.

Thus, we do not explicitly model the MAPE-K loop discussed in Section II.2.1 but abstract from it. This is because the MAPE-K loop can be used for realising a managing subsystem, but it does not impact the analysis we want to conduct with the model. In this paper, the analysis focuses on the interaction between the managed and managing subsystem, independent of the realisation of the managing subsystem.

II.3.3 Separation of Concerns between Managed and Managing Subsystem

As described before, the managed subsystem of the AUV consists of several different hardware and software components that can be combined in predefined ways to form a valid configuration of the AUV, modelled by the feature model in Figure II.4. The feature model specifies all possible valid configurations of the managed subsystem. However, the managed subsystem does not know whether these configurations make sense in certain scenarios, such knowledge is captured by a separate subsystem that implements how to adapt the AUV. The managing subsystem implements such an adaptation logic. Thus, it chooses configurations of the managed subsystem depending on the current environmental and internal conditions. Depending on the non-functional requirements of the system, a different managing subsystem can be implemented, e.g., minimising the energy consumption or the time taken for the mission. Therefore, a managing subsystem might not choose certain configurations of the managed subsystem that are possible since they are not useful for the strategy of the managing subsystem. Thus, there is a separation of concerns between the managed and the managing subsystem.

In this small-scale evaluation, the configurations using the sonar for the search at a high, medium or low altitude are permitted by the managed subsystem because they do not cause problems. However, the managing subsystem implemented here does not use these configurations because it prefers searching for the pipeline with the sonar at a very high altitude. Nonetheless, another managing subsystem, implementing a different strategy, might choose them.

II.4 Modelling the AUV Small-Scale Evaluation with ProFeat

In this section, we describe the behavioural models of the managed and managing subsystems, of the environment, and the hardware failures, i.e., we describe the behaviour of how the camera and the sonar may run into hardware and software failures and of how the camera may get blocked. Furthermore, we model the small-scale evaluation with the family-based model checker ProFeat.

ProFeat^{2,3} [Chr+18] provides a means to both specify probabilistic system families and perform family-based (quantitative) analysis on them. Thus, different from classical FTSs, it also caters for probabilistic models as well as costs and rewards for quantitative analysis. ProFeat extends the probabilistic model checker PRISM⁴ [KNP11] with functionalities such as family models, features and feature switches. Thereby, it enables family-based modelling and (quantitative) analysis of probabilistic systems in which feature configurations may dynamically change during runtime. Thus, ProFeat fits well for modelling DSPLs as considered in this paper. The whole model can be analysed with probabilistic family-based model checking using PRISM. The probabilities used in our model are estimates and have not been validated by experiments, since in this paper our goal is not to make a model that is as realistic as possible, but rather to show the feasibility of our methodology.

Similar to an SAS, a ProFeat model can be seen as a two-layered model, as illustrated in Figure II.1. The behaviour of a family of systems that differ in their features, such as the managed subsystem of an SAS, or more generally a DSPL, can be specified. Then a so-called *feature controller* can activate and deactivate the features during runtime, and thus change the behaviour of the system, such as the managing subsystem of an SAS that changes the configuration of the managed subsystem. Furthermore, the environment and other parts of the system like, e.g., the behaviour of the hardware, e.g., failures, can be specified as separate modules that interact with the managed and managing subsystem. Thus, ProFeat is well-suited to model and analyse the small-scale evaluation described in Section II.3.

A ProFeat model consists of three parts: an obligatory feature model that specifies features and their relations and constraints, obligatory modules that specify the behaviour of the features, and an optional feature controller that activates or deactivates features.

The pipeline inspection small-scale evaluation was modelled as a Markov decision process in ProFeat⁵. The ProFeat model consists of (1) the implementation of the feature model of Figure II.4, which is explained in Section II.4.1; (2) modules describing the behaviour of the managed subsystem of the AUV, called *AUV module* (see Figure II.6), that of the environment (see Figure II.7) and that of the hardware failures (see Table II.1), which are explained in Sections II.4.2, II.4.3, and II.4.4, respectively; and (3) the feature

²<https://pchrson.github.io/profeat>

³<https://github.com/pchrson/profeat>

⁴<https://www.prismmodelchecker.org/manual>

⁵The complete ProFeat model of the small-scale evaluation is publicly available [Päß+24].

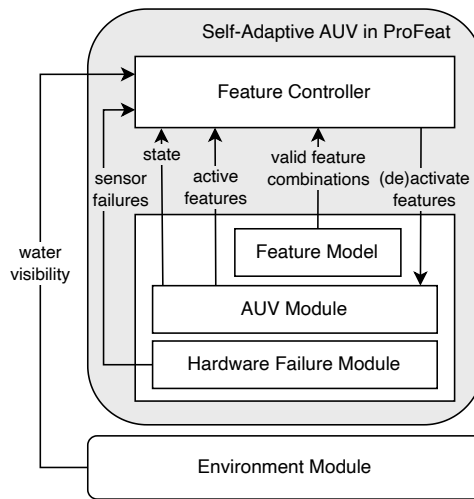


Figure II.5: The two-layered view (cf. Figure II.1) of the self-adaptive AUV modelled in ProFeat, including the components of the ProFeat model and how information concerning features, environmental conditions, failures and states flows between them

controller that switches between features during runtime, corresponding to the managing subsystem of the AUV (see Figure II.8), whose behaviour is explained in Section II.4.5.

An overview of the different components, and how they fit into the two-layered view of an SAS as presented in Figure II.1, is shown in Figure II.5. The managing subsystem is modelled by the feature controller; the managed subsystem is modelled by the feature model, the AUV module, and the hardware failure module; and the environment is modelled by the environment module. The feature controller monitors the environment by receiving information about the current water visibility, and it monitors the managed subsystem by receiving information about sensor failures (which sensors failed or are blocked) from the hardware failure module; information about the state of the AUV and the active features from the AUV module; and information about valid feature combinations from the feature model. Using this information, it decides non-deterministically which features to activate and deactivate and updates the information about active and inactive features in the AUV module, corresponding to an adaptation of the managed subsystem. This updated information determines which transition the AUV module can take. In our model, there is no direct information flow between the environment and the managed subsystem.

Starting from Section II.4.2, the subsequent sections are all structured in the same way: first, we describe the behaviour of the module and, second, we describe its ProFeat implementation. Therefore, a reader only interested in the high-level behaviour can easily navigate the sections without reading the implementation details.

```

1 root feature
2   all of robot;
3   modules auv, environment, hardware;
4   rewards "time"
5     [step] true : 1;
6   endrewards
7   rewards "energy"
8     // Costs for being in a recovery state
9     (s=recover_very_high) : 2;
10    // .. omitted code ..
11
12    // Costs for switching altitudes
13    (s=search_very_high) & active(high) : 2;
14    (s=search_very_high) & active(med) : 4;
15    (s=search_very_high) & active(low) : 6;
16    // .. omitted code ..
17
18    // Costs for going to low altitude when the pipeline is found
19    (s=found) & active(very_high) : 6;
20    (s=found) & active(high) : 4;
21    (s=found) & active(med) : 2;
22    // .. omitted code ..
23
24    // Additional costs for sonar because it takes more energy than camera
25    active(sonar) : 3;
26  endrewards
27 endfeature

```

Listing II.1: An excerpt of the declaration of the root feature of the small-scale evaluation

II.4.1 The Feature Model

This section explains how the feature model of the small-scale evaluation is expressed in ProFeat, including connections and constraints among features. Each feature is specified within a **feature** ... **endfeature** block, whereas the declaration of the root feature is done in a (unique) **root feature** ... **endfeature** block.

The Root Feature. An excerpt of the implementation of the root feature of the pipeline inspection small-scale evaluation according to Figure II.4 is displayed in Listing II.1. The root feature can be decomposed into subfeatures; in this case only one, the subfeature *robot*, see Line 2. The *all of* keyword indicates that all subfeatures have to be included in the feature configuration if the parent feature (in this case the root feature) is included. It is, e.g., also possible to use the *one of* keyword if exactly one subfeature has to be included, see Line 2 of Listing II.2, or *[m..n]* if between *m* and *n* features have to be included, see Line 7 of Listing II.2. It is possible to specify the behaviour of a (set of) features in so-called *modules*. The modules implementing a feature's behaviour can be specified following the keyword **modules**. In this small-scale evaluation, the root feature is the only feature specifying modules, thus the behaviour of all features is modelled in the modules *auv*, *environment* and *hardware* described later. Using separate modules enables to introduce a separation of concerns between the different parts of the model.

Unlike common feature models, ProFeat allows to specify feature-specific rewards in the declaration of a feature. Rewards are real values that can be attached to transitions and states to analyse quantitative properties of the system, e.g., the expected energy consumption or mission time. Even though they are called “rewards” in ProFeat and PRISM, they can also be interpreted as costs, i.e., they can be used both to motivate and to penalise going to a state or taking a transition. Each reward is encapsulated in a **rewards ... endrewards** block. In the small-scale evaluation, we consider the rewards *time* and *energy*, see Lines 4–26 of Listing II.1. During each transition that the AUV module takes, the reward *time* is increased by 1; it is a transition-based reward, see Line 5. We assume that one time step corresponds to one minute, allowing us to compute an estimate of a mission’s duration.

The reward *energy* is a state-based reward and can be used to estimate the necessary battery level for a mission completion. If a thruster of the AUV fails and needs to be recovered, a reward of 2 is given, see Line 9. The model also reflects that switching between the search altitudes requires significant energy. Since the altitude is switched if the AUV is in a search state and a navigation subfeature that does not correspond to the current search altitude that is active, a higher energy reward is given in these states. If the AUV needs to switch between low and very high altitudes, such as, e.g., in Line 15, an energy reward of 6 is given; if it needs to switch between very high and medium or between high and low altitude, such as, e.g., in Line 14, an energy reward of 4 is given; while all other altitude switches receive a reward of 2, see, e.g., Line 13. Since the altitude must be changed to *low* once the pipeline is found, these cases also receive an energy reward as explained above, see Lines 19–21. All other states, apart from the state in which the mission is aborted, receive an energy reward of 1. Since the sonar uses more energy than the camera, an additional energy reward of 3 is given if the sonar is used, see Line 25. We use the function *active* to determine which feature is active, i.e., included in the current feature configuration; given a feature, the function returns *true* if it is active and *false* otherwise. Note that both *time* and *energy* rewards are interpreted as costs.

Ordinary Features. The remainder of the feature model is implemented similarly to the root feature, but the features do not contain feature-specific modules or rewards. The features are implemented and named according to the feature model in Figure II.4. To have only one initial state, we initialise the model with the features *search*, *very_high* and *sonar* active, using the keyword *initial constraint*, see for example Lines 3 and 8 of Listing II.2. As examples of the implementation of other features, the declarations of the features *navigation* and *vision* are given in Listing II.2.

```

1 feature navigation
2   one of low, med, high, very_high;
3   initial constraint active(very_high);
4 endfeature
5
6 feature vision
7   [0..1] of sonar, camera;
8   initial constraint active(sonar);
9 endfeature

```

Listing II.2: The declaration of the navigation and vision features of the small-scale evaluation

II.4.2 The Managed Subsystem

The Behavioural Model of the Managed Subsystem. The behaviour of the managed subsystem of the AUV can be described by a probabilistic transition system equipped with features that guard transitions (a probabilistic FTS). Only if the feature guarding a transition is included in the current configuration of the managed subsystem of the AUV, the transition can be taken. This transition system adheres to the feature model in Figure II.4 and is depicted in Figure II.6, where some details have been omitted to avoid cluttering (in particular, all probabilities and some guards such as *camera* or *sonar*). The details can be obtained from the publicly available model [PäB+24]. The probabilistic model allows to easily model the possibilities of, e.g., finding and losing the pipeline depending on the system configuration.

The transition system can roughly be divided into two parts, one concerning the search for and one the following of the pipeline, as shown by the grey boxes in Figure II.6. When the AUV starts its mission, i.e., in state *start task*, the AUV can either immediately start following the pipeline if it was deployed right above it, or start searching for it. During the search for the pipeline, i.e., when the AUV is in the grey area labelled *search*, the feature *search* should be active and remain active until the state *found* is reached. The managing subsystem can switch between the features *low*, *med* *high* and *very high* during every transition, but the state *search very high* can only be reached when the feature *sonar* is active, not with the feature *camera*. Note that this probabilistic FTS does not reflect that the altitude should only be changed according to the water visibility if the feature *camera* is active, as described in Section II.3, since this logic is encoded in the managing subsystem, see Section II.4.5.

Once the pipeline is found, the managing subsystem has to deactivate the feature *search* and activate the feature *follow*, which also implies activating the feature *low* and deactivating *med*, *high* and *very high* due to the feature constraints in Figure II.4. We assume that the managing subsystem activates and deactivates features during transitions, so the features *follow* and *low* should be activated during the transition from the state *found* to the state *start task*. When the AUV is following the pipeline, i.e., in the grey area labelled *follow*, it can lose the pipeline again, e.g., because of sand covering it or because it drifted off its path due to thruster failures. Then the managing subsystem has

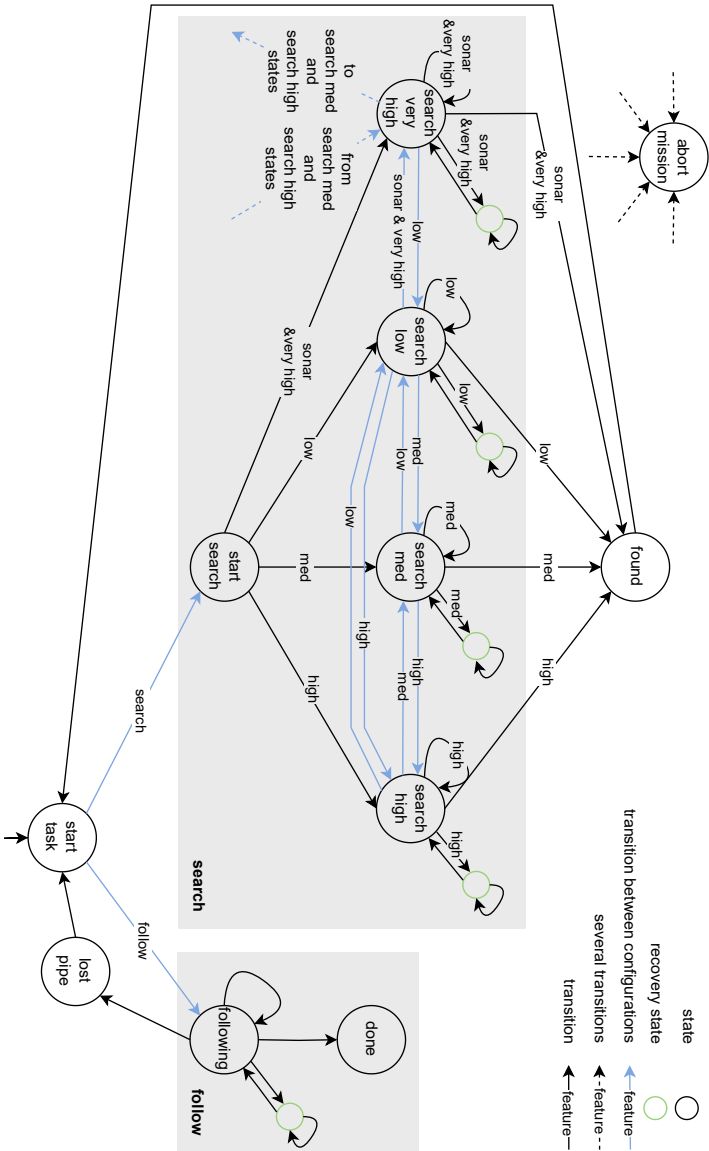


Figure II.6: The managed subsystem of the AUV, focusing on the tasks and the search altitude

to activate the feature *search* during the transition from *lost pipe* to *start task*.

Observe that the feature model does not require to have a vision feature active, see Line 7 of Listing II.2, capturing that both vision sensors can have a hardware or software failure and can no longer be used. If both vision sensors fail, then the pipeline inspection cannot be accomplished and the managed subsystem goes to the state *abort mission*, which can be reached from every state of the probabilistic FTS.

Additionally, observe that apart from the state *search very high* and its recovery state, which can only be reached when the feature *sonar* is active, the probabilistic FTS does not reflect which vision feature is currently active. It would also be possible to duplicate the search states to, e.g., have a *search low camera* and a *search low sonar* state, but this would increase the state space considerably. Since we are not interested in analysing properties associated with these different states, we opt for a probabilistic FTS with no explicit representation of the vision features.

In the model, we distinguish between two kinds of transitions: transitions that model the behaviour of a certain configuration of the managed subsystem (black transitions) and (featured) transitions that switch between configurations, enabled by the managing subsystem during runtime (blue transitions). The labels *search*, *follow*, *low*, *med*, *high*, *very high*, *sonar* and implicitly *camera* on the transitions represent the features that have to be active to execute the respective transition. The transitions between configurations (blue) implicitly carry the action to start the task or go to the altitude specified by the feature associated with the transition. For instance, the transitions from *search low* to *search medium* can be taken if the feature *med* is active because the transition has the guard *med*. When taking this transition, the AUV should perform the action of going to a medium altitude. The transitions inside a configuration (black) with a feature label contain the implicit action to stay at the current altitude because the navigation subfeature has not been changed during the previous transition. However, as described above, the blue transitions do not represent all transitions between configurations. If the configuration is changed from the *camera* to the *sonar* or the other way around, this is not reflected in the probabilistic FTS in Figure II.6.

Whether a transition inside the configuration or between configurations is executed in the search states *search low*, *search medium*, *search high* and *search very high* depends on the managing subsystem, i.e., the controller switching between features (see Section II.4.5). If the managing subsystem switched between the features *low*, *med*, *high* and *very high* during the last transition, a transition to the search state corresponding to the new feature will be executed, i.e., the configuration will be changed (for switching to the *search very high* state, also the feature *sonar* has to be active). Otherwise, a transition inside the configuration will be executed. For instance, consider the state *search low*. If the feature *low* is active, a black transition will be executed. If, however, the managing subsystem deactivated the feature *low* during the last transition and activated *med*, *high* or *very high*, then the AUV will perform a transition to the state *search medium*, *search high* or *search very high*, respectively.

The Implementation of the Managed Subsystem in ProFeat. The module `auv` models the behaviour of the managed subsystem of the AUV as displayed in Figure II.6, see Listing II.3 for an excerpt of the model. As depicted in Figure II.6, there are sixteen enumerated states in the ProFeat module with names that correspond to the state labels in the figure. The recovery states are named according to the state they are connected to (e.g., the recovery state connected to `search_high` is called `recover_high`). The variable `s` in Line 2 represents the current state of the AUV and is initialised using the keyword `init` with the state `start_task`. To record how many meters of the pipeline have already been inspected, the variable `d_insp` in Line 3 represents the distance the AUV has already inspected the pipeline, it is initialised by 0. The variable `inspect` represents the desired inspection length and can be set by the user during design time. Since the number of times a thruster failed impacts how much the AUV deviates from its path, the variable `t_failed` in Line 4 can be increased if a thruster fails while the AUV follows the pipeline. It is bounded by the influence a thruster failure can have on the system (`infl_tf`) that can be set by the user during design time.

The behaviour of the module is specified with *guarded commands*, corresponding to possible, probabilistic transitions of the following form:

```
[action] guard -> prob_1: update_1 + ... + prob_n: update_n;
```

A command may have an optional label `action` to annotate it or to synchronise with other modules. In PRISM, the `guard` is a predicate over global and local variables of the model, which can also come from other modules. ProFeat extends the guards by, e.g., enabling the use of the function `active`. If the guard is true, then the system state is changed with probability `prob_i` using `update_i` for all *i*. An update describes how the system should perform a transition by giving new values for variables, either directly or as a function using other variables.

For instance, consider the command in Lines 14–17, which can be read as follows. If the system is in state `search_high`, the feature `high` and at least one of the features `sonar` and `camera` is active, then with a probability of 0.59, the system changes its state to `found`, with a probability of 0.4 it changes to `search_high` and with a probability of 0.01 it changes to `recover_high`. These are exactly the black transitions shown in Figure II.6 exiting from state *search_high*. This command also has an action label, `step`. Using this action label, the managed subsystem synchronises with the environment and hardware failures modules, and with the feature controller, as described later.

The blue transitions exiting from the state *search_high* in Figure II.6 are modelled in Lines 18–23, where the transition to state `search_very_high` is only represented implicitly in Figure II.6. If the model is in state `search_high`, but the feature `low`, `med` or `very_high` is active, indicating that the AUV should go to the respective altitude, then the state is changed to the respective search state. The transitions exiting the states `search_very_high`, `search_med` and `search_low` are modelled similarly. However, the probability of going to the state `found` is highest from state `search_very_high` and lowest from `search_low` because the

```

1 module auv
2   s : [0..15] init start_task;
3   d_insp : [0..inspect] init 0;
4   t_failed : [0..infl_tf] init 0;
5
6   // To the correct task
7   [step] (s=start_task & active(search) & (active(camera) | active(sonar)))
8     -> 1: (s'=start_search);
9   [step] (s=start_task & active(follow) & (active(camera) | active(sonar)))
10    -> 1: (s'=following);
11
12   // .. omitted code ..
13   // From search state to another state
14   [step] (s=search_high & active(high) & (active(camera) | active(sonar)))
15     -> 0.59:(s'=found)
16     + 0.4:(s'=search_high)
17     + 0.01:(s'=recover_high);
18   [step] (s=search_high & active(very_high) & active(sonar))
19     -> 1:(s'=search_very_high);
20   [step] (s=search_high & active(med) & (active(camera) | active(sonar)))
21     -> 1:(s'=search_med);
22   [step] (s=search_high & active(low) & (active(camera) | active(sonar)))
23     -> 1:(s'=search_low);
24   // .. omitted code ..
25
26   // Go to other task if pipeline is found
27   [step] (s=found) & (active(camera) | active(sonar)) -> 1:(s'=start_task);
28
29   // Following the pipeline
30   [step] (s=following) & (d_insp<inspect) & (t_failed=0)
31     & (active(camera) | active(sonar))
32     -> 0.92: (s'=following) & (d_insp'=d_insp+1)
33     + 0.05: (s'=lost_pipe)
34     + 0.03:(s'=recover_following)
35     & (t_failed'=(t_failed<infl_tf? t_failed+1 : t_failed));
36   [step] (s=following) & (d_insp<inspect) & (t_failed>0)
37     & (active(camera) | active(sonar))
38     -> 0.92*(1-t_failed/infl_tf): (s'=following)
39     & (d_insp'=d_insp+1) & (t_failed'=t_failed-1)
40     + 0.05*(1+((0.92*t_failed)/(0.05*infl_tf))): (s'=lost_pipe)
41     + 0.03:(s'=recover_following)
42     & (t_failed'=(t_failed<infl_tf? t_failed+1 : t_failed));
43   [step] (s=following) & (d_insp=inspect) & (active(camera) | active(sonar))
44     -> (s'=done);
45
46   // Lost the pipeline
47   [step] (s=lost_pipe) & (active(camera) | active(sonar))
48     -> 1: (s'=start_task) & (t_failed'=0);
49
50   // Recovery states
51   [step] (s=recover_high) & (active(camera) | active(sonar))
52     -> 0.5:(s'=recover_high) + 0.5:(s'=search_high);
53   // .. omitted code ..
54
55   // Abort mission if both sensors failed
56   // (if the mission has not been finished yet)
57   [step] (s!=done) & (s!=abort_mission) & (camera_failed & sonar_failed)
58     -> 1: (s'=abort_mission);
59
60   // Wait that the camera gets unblocked if it is blocked
61   // but didn't fail, sonar failed
62   [step] (s!=done) & (s!=abort_mission)
63     & !active(sonar) & !active(camera) & !camera_failed -> 1: true;
64   // .. omitted code ..
65 endmodule

```

Listing II.3: An excerpt of the ProFeat AUV module of the small-scale evaluation

II. Analysing Self-Adaptive Systems as Software Product Lines

AUV has a wider field of view when performing the search at a higher altitude. Furthermore, the probability of a thruster failure, i.e., of going to the respective recover state, is highest in state `search_low` and lowest in states `search_high` and `search_very_high` because the probability of seaweed getting stuck in the thrusters is higher at a lower altitude. If the AUV found the pipeline, then a transition to `start_task` is taken, see Line 27.

From the state `start_task`, a transition to either `start_search` or `following` can be taken, depending on which subfeature of `pipeline_inspection` is currently active, see Lines 7–9.

From the `following` state, the transitions that can be taken depend on the variables `d_insp` and `t_failed`. Lines 30–35 consider the case where the distance of the pipeline that has already been inspected (`d_insp`) is less than the distance the pipeline should be inspected (`inspect`) and the variable `t_failed` is 0, indicating that there were no recent thruster failures. Then the AUV stays in the `following` state and inspects the pipeline one more meter, it loses the pipeline, or a thruster fails and it transitions to the failure state and increases `t_failed` if `t_failed` is not at its maximum. Lines 36–42 consider the case where `d_insp` is less than `inspect` and `t_failed` is greater than 0. In this case, the probabilities of following and of losing the pipeline depend on the value of `t_failed`. The bigger the value, the more likely it is to lose the pipeline because it indicates that the AUV's thrusters did not work for some time, causing it to drift off its path. If the already inspected distance is equal to the required inspection distance, the AUV transitions to the `done` state (see Line 43) and finishes the pipeline inspection. If the AUV lost the pipeline (see Line 47), then a transition to `start_task` is taken and the variable `t_failed` is set to 0 again.

When the AUV is in a recovery state, it can either stay there for another time step or exit it again to the state from where the recovery was triggered, see Lines 51–52.

Note that all transitions have a guard `active(camera) | active(sonar)` (apart from the transitions to the `search_very_high` state which have a guard `active(sonar)`). Thus, these transitions can only be taken if one of the two features is active, i.e., they cannot be taken if both the sonar and camera failed or if the sonar failed and the camera is blocked. If both vision sensors failed, indicated by the variables `camera_failed` and `sensor_failed`, then the pipeline inspection has to be aborted, modelled in Lines 57–58. Note that the check `camera_failed & sonar_failed` cannot be replaced by `!active(camera) & !active(sonar)` because the feature `camera` is also inactive if the camera is blocked, which is not a permanent fault but can be unblocked. Thus, if both features `camera` and `sonar` are inactive but the camera did not fail, then the managed subsystem does nothing and waits for the camera to get unblocked, see Lines 62–63.

All commands in the module `auv` are labelled with `step`. Thus, every transition receives a time reward of 1, i.e., the time advances with every transition the AUV takes, see Lines 4–6 of Listing II.1.

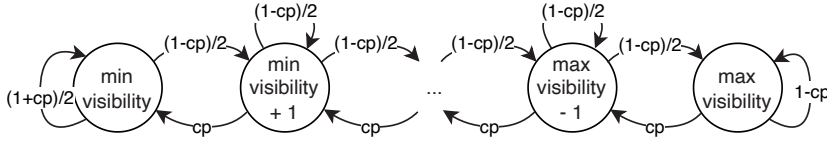


Figure II.7: The behaviour of the environment ([Päß+23a])

```

1 module environment
2   water_visib : [min_visib..max_visib] init round((max_visib-min_visib)/2);
3   [step] true -> current_prob: (water_visib'= (water_visib=min_visib?
4     min_visib:water_visib-1)) + (1-current_prob)/2: (water_visib'=
5     (water_visib=max_visib? max_visib:water_visib+1))
6     + (1-current_prob)/2: true;
7 endmodule
    
```

Listing II.4: The ProFeat environment module of the small-scale evaluation

II.4.3 The Environment

The Behavioural Model of the Environment. The only parameter of the environment considered in this small-scale evaluation is the water visibility, which influences at which altitude the AUV can operate when using the camera as a vision sensor. We assume that there is a minimum and maximum visibility of the environment, depending on where the AUV is deployed and set by the user during design time. Furthermore, different environments also have different probabilities of currents that influence the water visibility. This can also be set during design time. The behaviour of the environment is then modelled as depicted in Figure II.7, where cp represents the *current probability*. With the probability cp of currents, the water visibility decreases by 1, while it stays the same or increases by 1 with probability $(1-cp)/2$. If the water visibility is already at minimum visibility, the water visibility stays the same with probability $(1+cp)/2$ and, at maximum visibility, it stays the same with probability $(1-cp)$.

The Implementation of the Environment in ProFeat. The environment is modelled in a separate environment module, see Listing II.4. The variable `water_visib` in Line 2 reflects the current water visibility and is initialised parametrically, depending on the minimum and maximum visibility, see Line 2. The function `round()` is pre-implemented in the PRISM language and rounds to the nearest integer. The environment module synchronises with the other modules and the feature controller via the label of its command, `step`. Since the guard of the only command in the environment module is `true`, the environment executes a transition every time the other modules and the feature controller do. By decoupling the environment module from the AUV module, we obtain a separation of concerns which makes it easier to change the model of the environment if needed.

II. Analysing Self-Adaptive Systems as Software Product Lines

Table II.1: Transition matrix, *aw* stands for “all working”, *cf* for “camera failure”, *sf* for “sonar failure”, and *cb* for “camera blocked”; a 1 indicates that there is a transition, a 0 that there is none

from \ to	aw	cf	sf	cb	cf sf	cf cb	sf cb	cf sf cb
aw	1	1	1	1	1	1	1	1
cf	0	1	0	0	1	1	0	1
sf	0	0	1	0	1	0	1	1
cb	1	1	1	1	1	1	1	1
cf, sf	0	0	0	0	1	0	0	1
cf, cb	0	1	0	0	1	1	0	1
sf, cb	0	0	1	0	1	0	1	1
cf, sf, cb	0	0	0	0	1	0	0	1

II.4.4 The Hardware Failures

The Behavioural Model of the Hardware Failures. The hardware failures module models how the vision sensors, the camera and the sonar, can fail and get blocked (in case of the camera) during runtime, causing the need for switching between vision sensors or aborting the mission. In our model, we assume to know probabilities for the failure of the sonar and the camera that could be provided by the sensor manufacturers in practice. The probability of the camera getting blocked can be determined depending on the environment the AUV is deployed in, e.g., depending on the amount of natural and human waste, while the probability of the camera getting unblocked can be determined depending on the probability of currents as the currents might take away the blockage of the camera.

Every time step, the camera and sonar can fail and the camera can get blocked or unblocked (if it was blocked). In fact, between none and all of these things can happen simultaneously. Table II.1 shows the states and transitions of the hardware failures module. It is possible that all sensors are working (*aw*), that the camera or sonar failed (*cf* and *sf*, respectively), that the camera is blocked (*cb*), or a combination of the last three. The table contains a 1 if the respective transition exists and a 0 otherwise. For example, there exists a transition from sonar failed and camera blocked (*sf*, *cb*) to sonar failed (*sf*), namely, if the camera gets unblocked, but no transition to camera blocked (*cb*) because the sonar cannot recover once it failed.

Note that the thruster failures are not modelled in this module since they are embedded in the managed subsystem. This is because we only consider internal self-adaptation for thruster failures, i.e., exception handling; if a thruster failure occurs, the system goes to the corresponding recovery state and tries to restart the thruster. On the other hand, adapting to failed sensors (or changes in the water visibility) is realised with external self-adaptation and is thus not included in the managed subsystem but in separate modules to create a separation of

concerns between the behaviour of the managed subsystem and the uncertainties it is subject to. This also makes it easier to include more uncertainties, requiring external adaptation, to the small-scale evaluation.

The Implementation of the Hardware Failures in ProFeat. The hardware failures are modelled in a separate hardware module, see Listing II.5. The Boolean variables `sonar_failed`, `camera_failed` and `camera_blocked` in Lines 2–4 indicate whether the sonar failed, the camera failed or the camera is blocked, respectively. They are initialised to be false.

If both the sonar and the camera did not fail and the camera is not blocked, one of the sensors can fail or get blocked (see Lines 8–15), two sensors can fail or get blocked at the same time (see Lines 18–29) or both sensors fail and the camera gets blocked at the same time (see Lines 32–36). These commands reflect the transitions in the first line of Table II.1. Similar commands exist for the other transitions indicated in the table. Like the environment module, the hardware failures module synchronises with the other modules and the feature controller via the label of its commands, `step`.

II.4.5 The Managing Subsystem

The Behavioural Model of the Managing Subsystem. As described in Section II.3, the managing subsystem of the AUV implements the AUV’s adaptation logic, which corresponds to activating and deactivating the features of the managed subsystem. The behaviour of the managing subsystem of the AUV is depicted in Figure II.8. As in the figure of the managed subsystem (Figure II.6), the transition system is divided into two parts, one part for *searching* for the pipeline and one for *following* it, indicated by the grey boxes. For better readability, both parts are again divided into a part for using the *camera* and one for using the *sonar*, indicated by the black-framed boxes. Each transition contains a guard, written in black, and an action, written in grey separated by a vertical bar. The guards of transitions in our model are Boolean operations over the states of the managed subsystem, checks over active features and values of variables. The action means activating the indicated feature and implicitly deactivating the other subfeatures of the same category, e.g., activating the feature *camera* and deactivating the feature *sonar*. The dashed transitions connect every state of the box/part where they start to the box where they end. For example, the dashed transition between the *search* part and the *camera* box in the *following* part connects the states *search altitude low*, *search altitude medium*, *search altitude high* and *search altitude very high* to the state *follow camera*.

All transitions in the *search* part of the figure contain the guards $s! = \text{abort mission}$, $s! = \text{found}$ and $\text{active}(\text{search})$, where s refers to the current state of the managed subsystem. Since the sonar is the preferred sensor for searching for the pipeline, see Figure II.2, the *camera* part of *search* also contains the guard *sonar failed*, so transitions can only be taken if the sonar failed, and there are only transitions from the *sonar* part of *search* to the *camera* part of *search*.

```

1 module hardware
2   sonar_failed : bool init false;
3   camera_failed : bool init false;
4   camera_blocked : bool init false;
5
6   ///// Everything is working
7   // One of the sensors fails/gets blocked
8   [step] (!sonar_failed & !camera_failed & !camera_blocked)
9     -> sonar_fail_prob: (sonar_failed' = true) + (1-sonar_fail_prob): true;
10  [step] (!sonar_failed & !camera_failed & !camera_blocked)
11    -> camera_fail_prob: (camera_failed' = true)
12      + (1-camera_fail_prob): true;
13  [step] (!sonar_failed & !camera_failed & !camera_blocked)
14    -> camera_block_prob: (camera_blocked' = true)
15      + (1-camera_block_prob): true;
16
17  // Two sensors fail/get blocked at the same time
18  [step] (!sonar_failed & !camera_failed & !camera_blocked)
19    -> camera_fail_prob*sonar_fail_prob: (camera_failed' = true)
20      & (sonar_failed' = true)
21      + (1-camera_fail_prob*sonar_fail_prob): true;
22  [step] (!sonar_failed & !camera_blocked & !camera_blocked)
23    -> sonar_fail_prob*camera_block_prob: (camera_blocked' = true)
24      & (sonar_failed' = true)
25      + (1-sonar_fail_prob*camera_block_prob): true;
26  [step] (!camera_failed & !camera_blocked & !camera_blocked)
27    -> camera_fail_prob*camera_block_prob: (camera_blocked' = true)
28      & (camera_failed' = true)
29      + (1-camera_fail_prob*camera_block_prob): true;
30
31  // Everything fails/gets blocked at the same time
32  [step] (!camera_failed & !camera_blocked & !sonar_failed)
33    -> camera_fail_prob*camera_block_prob*sonar_fail_prob:
34      (camera_blocked' = true) & (camera_failed' = true)
35      & (sonar_failed' = true)
36      + (1-camera_fail_prob*camera_block_prob*sonar_fail_prob): true;
37  // .. omitted code ..
38 endmodule

```

Listing II.5: An excerpt of the ProFeat hardware failures module of the small-scale evaluation

Furthermore, all transitions in the *camera* part of *search* contain the guard *!camera unavailable* because the camera cannot be used if it is unavailable.

When searching with the sonar, the managing subsystem always chooses to activate the feature *very high*, as indicated in grey on the transition, since this gives the biggest field of view, and thus the highest probability of finding the pipeline. Implicitly, the other subfeatures of *navigation* are deactivated. Since the transition in the *sonar* part contains the guard *!sonar failed*, it can only be taken if the sonar did not fail. If the sonar failed but the camera is available, i.e., it did not fail and is not blocked, a transition to the *camera* part is taken, depending on the current water visibility. When searching with the camera, the managing subsystem activates and deactivates the features *low*, *med*, and *high* according to the current water visibility as described in Section II.3. Both when searching with the sonar and the camera, a transition to the state *wait* is taken if the sonar failed and the camera did not fail but is blocked, and a transition to the state *mission aborted* is taken if both the sonar and the camera failed.

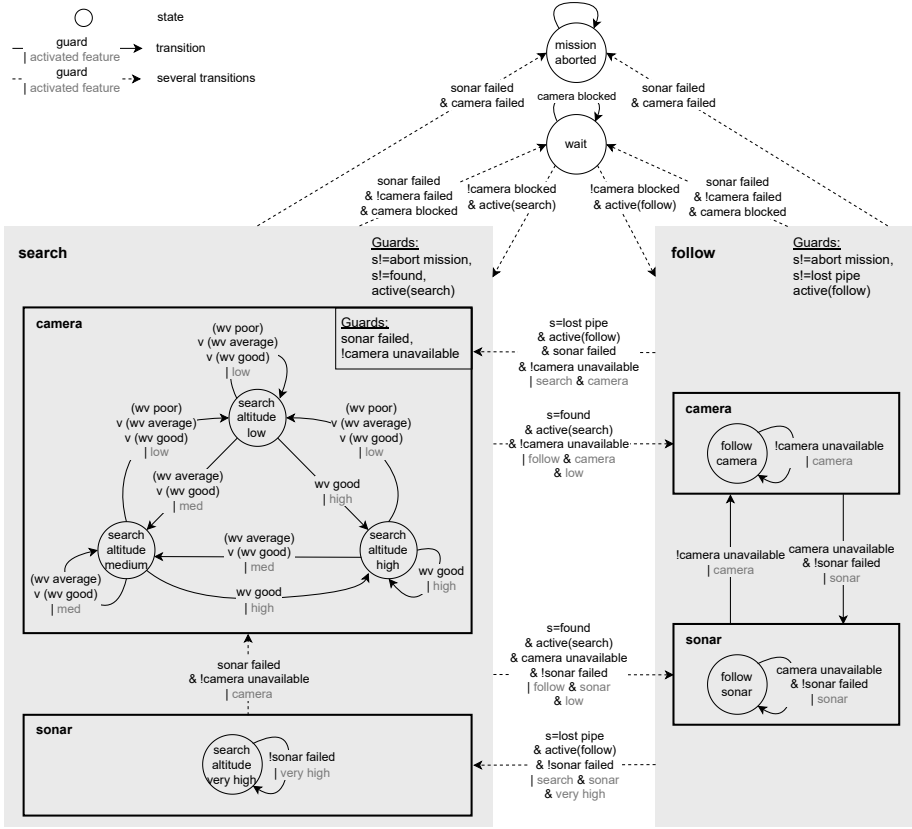


Figure II.8: The managing subsystem of the AUV, showing the transitions between feature configurations; the dashed transitions connect every state of the box where the transition starts to every state of the box where the transition ends

Once the pipeline has been found, i.e., the managed subsystem is in state *found*, one of the transitions between the *search* and *follow* part is taken. These transitions include the action of activating *follow*, *low*, and either *camera* or *sonar* (and deactivating the other features). The camera is preferred for following the pipeline, see Figure II.2, so a transition to the *camera* part of *follow* is taken if the camera is available. Otherwise, a transition to the *sonar* part is taken. In the *follow* part, every transition contains the guards $s! = \text{abort mission}$, $s! = \text{lost pipe}$ and $\text{active}(\text{follow})$, where s again refers to the current state of the managed subsystem. When following the pipeline, the managing subsystem switches to the feature *sonar* if the camera becomes unavailable, and back to the feature *camera* if the camera becomes available again. As in the *search* part, a transition to the state *wait* or *mission aborted* is taken if the respective guards are satisfied.

When the AUV loses the pipeline, i.e., the managed subsystem is in state

lost pipe, the managing subsystem activates *search* and either *sonar* or *camera*, depending on whether the sonar failed or not. If the sonar did not fail, the managing subsystem also activates the feature *very high* to search at a very high altitude. Otherwise, the AUV will start searching for the pipeline with the camera at a low altitude.

From the state *wait*, the managing subsystem will transition to the *search* or *follow* part, depending on the active feature, if the camera gets unblocked.

The Implementation of the Managing Subsystem in ProFeat. The managing subsystem of the AUV is implemented as a feature controller in ProFeat. The feature controller can use *commands* to change the state of the system. Such commands are similar to those used in a module; they are mostly of the form [action] guard -> update. Each command can have an optional label action to synchronise with the modules, and its guard is a predicate of global and local variables of the model and can also contain the function *active*. In contrast to the commands in the modules, the feature controller can *activate* and *deactivate* features in the *update* of a command. Several features can be activated and deactivated at the same time, but this cannot be done probabilistically and the resulting feature configuration has to adhere to the feature model.

In the pipeline inspection small-scale evaluation, subfeatures of *navigation* (i.e., the different altitudes at which the AUV can operate), subfeatures of *pipeline_inspection* (i.e., the tasks the robot has to fulfil), and the subfeatures of *vision* (i.e., the vision sensors of the AUV) can be switched by the feature controller while the AUV is operating (at runtime), see Listing II.6.

When the feature *search* is active and the pipeline has not been found yet, the feature controller prefers activating the sonar and only activates the camera if the sonar failed. This is achieved by checking if the Boolean *sonar_failed*, which can be changed probabilistically by the *hardware* module, is true, see Line 9. If it is false, the features *very high* and *sonar* are activated and the other *navigation* and *vision* subfeatures are deactivated, see Lines 10–11. Note that it is also possible to activate or deactivate a feature if it is already active or inactive, respectively.

If *sonar_failed* is true, the feature controller checks if the camera is available, i.e., it did not fail and is not blocked, enabled by the variable *camera_unavailable*, see Line 3 (in PRISM and ProFeat, a *formula* can be used to assign an identifier to an expression). If the camera is available, the feature *camera* is activated and the feature controller activates and deactivates the altitude features *low*, *med*, and *high* non-deterministically, but according to the current water visibility, as described before. The minimum and maximum water visibility can be set by the user during design time and influence the altitudes associated with the features *low*, *med*, and *high*; i.e., it influences when the feature controller can switch features (the altitude associated with the feature *very high* does not depend on the water visibility). To reflect this, the variables *med_visib* and *high_visib* are declared as in Lines 1–2. If the water visibility is less than *med_visib*, the feature controller activates the feature *low* because the AUV cannot perceive the seabed

```

1 formula med_visib = (max_visib-min_visib)/3;
2 formula high_visib = 2*(max_visib-min_visib)/3;
3 formula camera_unavailable = camera_failed | camera_blocked;
4
5 controller
6 /////// Searching
7 // Prefer searching with the sonar if it is available
8 // Sonar should search on a very high altitude
9 [step] (s!=abort_mission) & (s!=found) & active(search) & !sonar_failed
10     -> activate(very_high) & deactivate(high) & deactivate(med)
11         & deactivate(low) & activate(sonar) & deactivate(camera);
12
13 // Use camera if sonar is not available and camera is available
14 // Change altitude depending on water visibility
15 // (not possible at very high altitude)
16 [step] (s!=abort_mission) & (s!=found) & active(search) & sonar_failed
17     & !camera_unavailable & med_visib<=water_visib
18     & water_visib<high_visib
19     -> activate(low) & deactivate(med) & deactivate(high)
20         & deactivate(very_high) & activate(camera) & deactivate(sonar);
21 [step] (s!=abort_mission) & (s!=found) & active(search) & sonar_failed
22     & !camera_unavailable & med_visib<=water_visib
23     & water_visib<high_visib
24     -> activate(med) & deactivate(low) & deactivate(high)
25         & deactivate(very_high) & activate(camera) & deactivate(sonar);
26 // .. omitted code ..
27
28 /////// Switch task from "search" to "follow"
29 // Prefer camera for following if it is available
30 [step] (s=found) & active(search) & !camera_unavailable
31     -> deactivate(search) & activate(follow) & activate(camera)
32         & deactivate(sonar) & activate(low) & deactivate(med)
33         & deactivate(high) & deactivate(very_high);
34 [step] (s=found) & active(search) & camera_unavailable & !sonar_failed
35     -> deactivate(search) & activate(follow) & activate(sonar)
36         & deactivate(camera) & activate(low) & deactivate(med)
37         & deactivate(high) & deactivate(very_high);
38
39 /////// Switch task from "follow" to "search"
40 // Prefer sonar for searching if it is available
41 [step] (s=lost_pipe) & active(follow) & !sonar_failed
42     -> deactivate(follow) & activate(search) & activate(sonar)
43         & deactivate(camera) & activate(very_high) & deactivate(high)
44         & deactivate(med) & deactivate(low);
45 [step] (s=lost_pipe) & active(follow) & sonar_failed & !camera_unavailable
46     -> deactivate(follow) & activate(search) & activate(camera) & deactivate(sonar);
47
48 /////// Follow the pipeline
49 // Use the camera for following/inspecting if it is available
50 [step] (s!=abort_mission) & (s!=lost_pipe) & active(follow)
51     & !camera_unavailable
52     -> activate(camera) & deactivate(sonar);
53 [step] (s!=abort_mission) & (s!=lost_pipe) & active(follow)
54     & camera_unavailable & !sonar_failed
55     -> activate(sonar) & deactivate(camera);
56
57 // Deactivate vision sensors when they are broken
58 [step] (s!=abort_mission) & camera_failed & sonar_failed
59     -> deactivate(camera) & deactivate(sonar);
60
61 // Wait for camera to get unblocked
62 // (sonar failed and camera is blocked but did not fail)
63 [step] sonar_failed & camera_blocked & !camera_failed -> 1: true;
64
65 // Avoid deadlock with broken sensors
66 [step] (s=abort_mission) -> 1: true;
67 endcontroller

```

Listing II.6: An excerpt of the ProFeat feature controller of the small-scale evaluation

from a higher altitude. If the water visibility is between `med_visib` and `high_visib`, it chooses non-deterministically between `low` and `med` (see Lines 16–24), whereas it chooses non-deterministically between all three altitudes if the water visibility is above `high_visib`.

When the pipeline is found, i.e., the managed subsystem is in state `found`, the feature controller activates the features `follow` and `low` and deactivates the other pipeline inspection and navigation subfeatures since the AUV should follow the pipeline at a low altitude. The camera is the preferred sensor for following the pipeline, so it is activated if it is available, and otherwise the sonar is activated, see Lines 30–37. The same holds when the AUV is following the pipeline, see Lines 50–55.

If the AUV loses the pipeline, i.e., the managed subsystem is in state `lost_pipe`, the feature controller activates `search` and deactivates `follow` to start the search for the pipeline. Since the sonar is the preferred sensor for searching for the pipeline, the feature controller activates the feature `sonar` if the sonar is available and `camera` otherwise, see Lines 41–46. Note that the feature controller also activates the feature `very_high` and deactivates the other navigation subfeatures if the sonar is activated, see Lines 43–44, because the AUV should search for the pipeline at a very high altitude with the sonar. However, if the sonar is not available and the feature controller activates the feature `camera`, it does not change the navigation subfeature, i.e., the AUV will start searching for the pipeline at a low altitude with the camera.

All commands described so far have guards to ensure that it is not the case that both the sonar and the camera are unavailable. If both of them failed, the features `sonar` and `camera` should both be deactivated, see Lines 58–59, leading to the managed subsystem going to state `abort_mission`. If the sonar failed and the camera is blocked but did not fail, then the feature controller waits for the camera to get unblocked, i.e., it does nothing, see Line 63.

The feature controller synchronises with the `auv`, `environment`, and `hardware` modules via action label `step`. Since all transitions of the modules and feature controller have the same action label, they can only execute a transition if there is a transition with a guard evaluating to true in all modules and in the feature controller. Thus, the feature controller needs to include a transition doing nothing if the managed subsystem is in state `abort_mission`, see Line 66.

II.5 Analysis

ProFeat automatically converts models to PRISM for probabilistic model checking. To analyse a PRISM model, properties can be specified in the PRISM property specification language, which includes several probabilistic temporal logics like PCTL, CSL, and probabilistic LTL. For family-based analysis, ProFeat extends this specification language to include, e.g., the function `active`. The properties specified with ProFeat are automatically translated into the PRISM property specification language such that the PRISM engine can be used for probabilistic, family-based model checking. When specifying properties

Table II.2: Two different scenarios used for analysis

Scenario	min_visib	max_visib	current_prob	inspect	c_block_prob
1 (North Sea)	1	10	0.6	10	0.05
2 (Caribbean Sea)	3	20	0.3	30	0.03

Table II.3: The probabilities for finishing the inspection and aborting the mission for both scenarios

Scenario	Finish inspection		Abort mission	
	min	max	min	max
1 (North Sea)	0.962	1.0	0.0	0.038
2 (Caribbean Sea)	0.747	1.0	0.0	0.253

in ProFeat, ProFeat constructs, including variables and features, have to be specified in $\{\dots\}$ to be correctly translated to the PRISM property specification language.

The operators used for analysis in this paper are $\mathbf{P}$ and $\mathbf{R}$, which reason about probabilities of events and about expected rewards, respectively. Since we use Markov decision processes that involve non-determinism, these operators must be further specified to ask for the *minimum* or *maximum* probability and expected cost, respectively, for all possible resolutions of non-determinism.

The analysis of the model considers four different aspects. First, we analyse properties related to sensor failures in Section II.5.1. Second, the rewards *energy* and *time* are used to compute some safety guarantees that can be used for the deployment of the AUV, which is detailed in Section II.5.2. We analyse safety properties concerning unsafe states in Section II.5.3. Lastly, correctness issues of the adaptation logic with respect to its specification described in Section II.3 are analysed in Section II.5.4. Note that it is not necessary to analyse whether the model satisfies the constraints of the feature model because this is automatically ensured by ProFeat. In addition to the analyses described in this section, complementary complex analyses could be performed, e.g., comparing different implementations of the feature controller. In this paper, we just give a taste of possible analyses to demonstrate the feasibility of our methodology.

Deployment Scenarios. We analyse two different scenarios, the values used in these scenarios are reported in Table II.2. Scenario 1 is intended to capture a deployment of the AUV in the North Sea, and Scenario 2 in the Caribbean Sea. In Scenario 1, the minimum and maximum water visibility (in 0.5 meter units) are relatively low and the probability of currents that decrease the water visibility is relatively high. In this case, only 10 meters of the pipeline have to be inspected, and the probability of the camera getting blocked is higher than in the Caribbean Sea since we assume that in this scenario, there is more natural and human waste in the water than in the other scenario. Scenario 2, in the Caribbean Sea, has a higher minimum and maximum visibility and a lower probability of currents compared to the North Sea. In this scenario, 30 meters

II. Analysing Self-Adaptive Systems as Software Product Lines

```
1 // The minimum and maximum probability that the sonar does not fail until
2 // the pipeline inspection is finished
3 Pmin=? [!sonar_failed] U {s=done};;
4 Pmax=? [!sonar_failed] U {s=done};;
5
6 // The minimum and maximum probability that the camera does not fail until
7 // the pipeline inspection is finished
8 Pmin=? [!camera_failed] U {s=done};;
9 Pmax=? [!camera_failed] U {s=done};;
```

Listing II.7: Analysis of finishing the inspection without a sensor failure

Table II.4: The probabilities for finishing an inspection without a sonar failure or a camera failure

Scenario	No sonar failure		No camera failure	
	min	max	min	max
1 (North Sea)	0.809	1.0	0.899	1.0
2 (Caribbean Sea)	0.592	1.0	0.721	1.0

of pipeline have to be inspected, and it has a lower probability of the camera getting blocked than in the North Sea.

For both scenarios, we first analyse whether it is always possible to finish the pipeline inspection, i.e., reach the state `done`, and how likely it is to abort the mission. The results are reported in Table II.3. As expected, it holds that the minimum probability of aborting the mission is equal to 1 minus the maximum probability of finishing the inspection, and similarly for the maximum probability of aborting the mission.

II.5.1 Sensor Failures

Sensor failures can lessen the results the end-user gets from a pipeline inspection. If the sonar fails and the AUV has to search for the pipeline with the camera, then it will probably take more time because the search cannot be done at a very high altitude and it will probably take more energy since the AUV has to change altitudes depending on the water visibility. Thus, since the battery of the AUV is limited, less pipeline can be inspected compared to when both sensors work. If, on the other hand, the camera fails, then the inspection results when using the sonar will not be as good because fewer details can be seen when inspecting with the sonar. For example, detecting tiny holes in the pipeline which could be found when inspecting with the camera because of bubbles rising from the pipeline, cannot be discovered when inspecting with the sonar. Thus, in both cases, the results from the pipeline inspection will probably not be as good as with working sensors.

Therefore, it is interesting to know how likely it is for the camera and sonar to fail before the inspection is finished. The commands in Listing II.7 give the minimum and maximum probability of reaching the state `done` without a sonar failure (Lines 3–4) and without a camera failure (Lines 8–9). The results for

```

1 R{"energy"}min=? [F ${s=done}];
2 R{"energy"}max=? [F ${s=done}];

```

Listing II.8: Analysis using the rewards

Table II.5: Expected minimum/maximum rewards for completing the mission for both scenarios

Scenario	Energy		Time	
	min	max	min	max
1 (North Sea)	49.28	∞	22.29	∞
2 (Caribbean Sea)	97.33	∞	53.78	∞

Scenarios 1 and 2 are reported in Table II.4. It can be seen that the length of pipeline that has to be inspected (a longer pipeline in Scenario 2) has an impact on the minimum probability of finishing the inspection without a camera or sonar failure.

II.5.2 Reward Properties

The rewards time and energy are used to analyse some safety properties related to the execution of the AUV. Since the AUV has only a limited amount of battery, an estimation of the energy needed to complete the mission is required. This ensures that the AUV is only deployed for the mission if it has sufficient battery to complete it. The commands in Listing II.8 are used to compute the minimum and maximum expected energy (for all resolutions of non-determinism) to complete the mission. Since the model includes two reward structures, the name of the reward has to be specified in {"..."} after the `R` operator. Similarly, the minimum and maximum expected time to complete the mission is analysed to give the system operators an estimate of how long the mission might last. The results for Scenarios 1 and 2 are reported in Table II.5. It can be seen that the variation of the parameters in the two scenarios strongly influences the expected energy consumption and time duration of the mission, leading to an almost doubled minimum energy consumption and a more than doubled minimum time duration for Scenario 2. The maximum rewards for completing the mission are infinite. In PRISM, if a reachability reward is computed and “the probability of satisfying the formula is less than 1, the expected reward is defined to be infinite”⁶. Thus, the maximum rewards are infinite because the minimum probability for completing the pipeline inspection is less than 1, see Table II.3, i.e., there are cases in which the pipeline inspection cannot be completed. Note that this is different from the results reported in previous work [Päß+23a] because the mission can be aborted in the extended small-scale evaluation considered here.

⁶<https://www.prismmodelchecker.org/manual/PropertySpecification/Reward-basedProperties>

```

1 label "unsafe" = s=recover_very_high | s=recover_high | s=recover_med
2               | s=recover_low | s=recover_following | s=abort_mission;
3 label "safe" = s=lost_pipe | s=start_task | s=start_search
4               | s=search_very_high | s=search_high | s=search_med | s=search_low
5               | s=found | s=following | s=done;
6 label "thruster_failure" = s=recover_very_high | s=recover_high | s=recover_med
7               | s=recover_low | s=recover_following;
8 Pmin=? [G "safe"];
9 Pmax=? [F "unsafe"];
10 filter(max, Pmax=? [ F<=k "thruster_failure" ], "safe");
11 filter(avg, Pmax=? [ F<=k "thruster_failure" ], "safe");

```

Listing II.9: Analysis of unsafe states

II.5.3 Unsafe States

Thruster failures, although we assume that they can be repaired, pose a threat to the AUV. Unforeseen events like strong currents might cause the AUV to be damaged, e.g., by causing it to crash into a rock. To analyse this, the states are grouped into *safe*, *unsafe* and *thruster failure* states, where the unsafe states contain all thruster failure states and the *abort mission* state, and all other states are *safe*. The grouping of states is achieved by using labels, see Lines 1–7 of Listing II.9. These labels are then used to calculate the probability of several properties. The minimum probability of only taking safe states (see Line 8) is shown to be 0.686 for Scenario 1 and 0.29 for Scenario 2. As expected, the probability of only taking safe states is higher for a shorter pipeline inspection. Furthermore, the maximum probability of finally reaching an unsafe state (see Line 9), i.e., a state where either a thruster failed or the mission was aborted, is reported to be 0.31 in Scenario 1 and 0.71 in Scenario 2. Again, the fact that the probability is much higher for Scenario 2 probably comes from the fact that the length of pipeline that needs to be inspected is much higher in Scenario 2 than in Scenario 1.

The probability of going to a thruster failure state from a safe state should be as small as possible. This is analysed with the properties in Lines 10–11. First, the maximum probability (over all possible resolutions of non-determinism) for reaching a thruster failure state from a safe state in k time steps is calculated, and then the maximum (or average) over all these probabilities is taken. PRISM experiments allow analysing this property automatically for a specified range of k ; the plotted graphs for Scenarios 1 and 2 are displayed in Figure II.9. They show that the probability of reaching a thruster failure state from a safe state increases with the number of considered time steps. Furthermore, the maximum probability of reaching a thruster failure state from a safe state stabilises much later and at a higher value than the average probability (for both scenarios). While the maximum probability of reaching a thruster failure state from a safe state stabilises after about 270 time steps at ≈ 0.84 in Scenario 1 and after about 280 time steps at ≈ 0.91 in Scenario 2, the average probability stabilises after about 180 time steps at ≈ 0.16 in Scenario 1 and after about 250 time steps at ≈ 0.31 in Scenario 2.

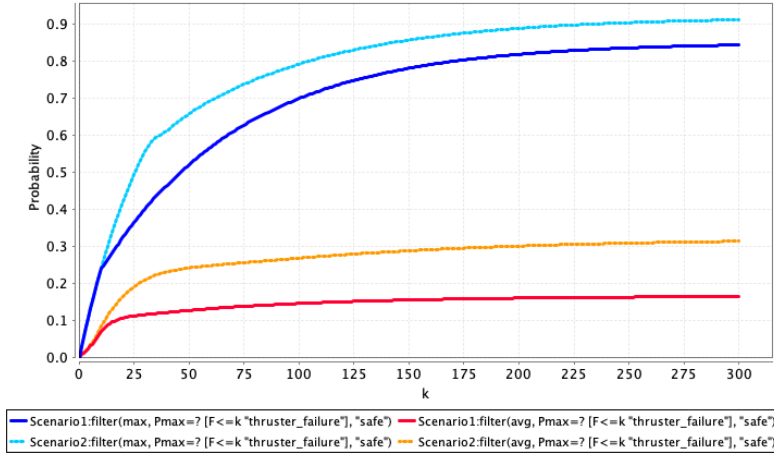


Figure II.9: Results for reaching a thruster failure state from a safe state in k time steps

II.5.4 Correctness Issues of the Adaptation Logic

Since adaptation is a crucial part of the small-scale evaluation, it is important to check that (1) the adaptation logic described in Section II.3 was correctly implemented, and that (2) the managing subsystem is able to adapt the managed subsystem correctly. Both are analysed with the properties in Listing II.10.

First, we want to ensure that, if the camera or sonar failed, the features camera and sonar, respectively, are deactivated. For the camera, this was achieved by checking that it always holds that if the camera failed, i.e., `camera_failed` is true, eventually the feature camera will always be inactive, see Line 2. By writing $P \geq 1.0$ in front of this property, ProFeat will return true, if the property always holds, i.e., if its minimum probability is 1.0, and false otherwise. The same is analysed for the sonar, see Line 3.

Similarly, it is important to ensure that, if the camera is blocked, the feature camera will eventually be inactive or the camera will get unblocked, see Line 7, and that the camera will eventually be unblocked if it was blocked, see Line 10.

Furthermore, the properties in Lines 15–16 and Lines 17–19 ensure that, when searching for the pipeline with the camera, the managing subsystem will eventually choose a correct altitude according to the water visibility. To do so, we ask whether the probability is greater than or equal to 1 that it is always the case that if the features camera and search are active (i.e., the AUV is searching for the pipeline with the camera) and the water visibility is low (i.e., `water_visib < med_visib`), then eventually the feature low will be active (i.e., the AUV goes to a low altitude), or the water visibility will have increased (i.e., `water_visib >= med_visib`), see Lines 15–16. A similar property should hold if the water visibility is medium, see Lines 17–19, except that in this case we need to ensure that either the feature low or med will be active. Note that for the case

```
1 // The features camera and sonar are deactivated if the respective sensor failed
2 P>=1.0 [G ({camera_failed} => (F G ${!active(camera)}))];
3 P>=1.0 [G ({sonar_failed} => (F G ${!active(sonar)}))];
4
5 // The feature camera is deactivated if the camera is blocked
6 // (unless the camera got unblocked again)
7 P>=1.0 [G ({camera_blocked} => (F (${!active(camera)} | ${!camera_blocked})) );]
8
9 // If the camera is blocked, it will eventually be unblocked
10 P>=1.0 [G ({camera_blocked} => (F ${!camera_blocked}))];
11
12 // When using the camera for searching, the managing subsystem will eventually
13 // choose a correct altitude according to the water visibility
14 // (unless the the water visibility changed again)
15 P>=1.0 [G (($active(camera)} & ${active(search)} & ${water_visib<med_visib})
16 => (F (${active(low)} | ${water_visib>=med_visib}))];
17 P>=1.0 [G (($active(camera)} & ${active(search)}
18 & ${water_visib>=med_visib} & ${water_visib<high_visib})
19 => (F (${active(med)} | ${active(low)} | ${water_visib>=high_visib}))];
20
21 // The sonar is used for searching for the pipeline if it did not fail
22 P>=1.0 [G (($active(search)} & ${!sonar_failed})
23 => (F (${active(sonar)} | ${sonar_failed}))];
24
25 // The camera is used for following the pipeline
26 // if it did not fail and is not blocked
27 P>=1.0 [G (($active(follow)} & ${!camera_unavailable})
28 => (F (${active(camera)} | ${camera_unavailable}))];
```

Listing II.10: Analysis of the correctness of the adaptation logic

of the water visibility being high, i.e., `water_visib >= high_visib`, no analysis is needed since all three altitudes can be chosen, depending on the strategy of the managing subsystem.

Lastly, we need to check whether the managing subsystem maintains the priorities defined in Figure II.2, i.e., the sonar is used for searching for the pipeline if it did not fail and the camera is used for following the pipeline if it did not fail. In Lines 22–23, this is analysed by checking if the probability is greater than or equal to 1 that it is always the case that, if the feature `search` is active and the sonar did not fail, then eventually the feature `sonar` will be active or the sonar will fail. This is analysed similarly for the camera, see Lines 27–28, except that the variable `camera_failed` is not used, but instead the variable `camera_unavailable` is used, since the camera should only be used if it did not fail and is not blocked, i.e., is not unavailable.

The analysis showed that the properties described in this subsection are true for both scenarios.

II.6 Evaluation

To evaluate our methodology with the help of the small-scale evaluation detailed in Section II.4 and analysed in Section II.5, we consider the following research questions (RQs):

- RQ1** How does the proposed methodology of modelling an SAS as a DSPL support extensibility?
- RQ2** To what extent does the proposed methodology enable the analysis of the reliability of SASs?
- RQ3** How can one analyse whether the adaptation logic has been realised according to its specification?

II.6.1 RQ1: How Does the Proposed Methodology of Modelling an SAS As a DSPL Support Extensibility?

We consider different kinds of model extensions:

- (1) Adding variability to the managed subsystem;
- (2) Adding behaviour of the managed subsystem;
- (3) Adding uncertainties;
- (4) Extending the adaptation logic (to account for new uncertainties).

All of these extensions have been covered by the extension of the small-scale evaluation in comparison to the model in our previous paper [Päß+23a], hereafter called the “old model”.

Adding Variability. Adding variability to the managed subsystem is done by extending the feature model with new features. The variability can be restricted by adding new constraints to the feature model. Both of these changes have been done in the extension presented in this paper by adding four new features (*very high*, *vision*, *sonar* and *camera*) where the camera excludes the feature *very high*, adding a new constraint. Adding these features and constraints to the ProFeat model was straightforward, see Section II.4.1.

Adding Behaviour. New features are introduced to model new behaviour and new constraints can restrict the already existing behaviour. These behavioural changes have to be included in the behavioural model of the managed subsystem where the behaviour of all valid configurations and the possible re-configurations are modelled. All changes mentioned here have to be done in the model of the behaviour of the managed subsystem (the *auv* module of our small-scale evaluation).

If a new constraint has been added to the previous feature model, this is reflected by including additional guards to some of the transitions of the module modelling the behaviour of the system. If new features (and constraints relating to these features) are included, this can also be reflected in the guards of transitions. For example, in this paper, we added the features *camera* and *sonar* to the feature model of the old model in [Päß+23a], where either of them has to be active to be able to perform the mission. If none is active, then the

mission should be aborted. Thus, all transitions present in the old model need to receive a guard stating that either the camera or the sonar is active (see, e.g., Line 7 of Listing II.3).

Furthermore, introducing new behaviour can also require introducing new states and transitions. In our model, we had to include the states *search very high* and its corresponding recovery state, as well as the state *abort mission*, see Figure II.6. The new states and the states from the old model had to be connected with transitions whose feature guards may include new features. For example, from each of the search states of the old model, a transition guarded by the features *very high* and *sonar* was added to reflect going to a very high altitude if the features *very high* and *sonar* are active (see, e.g., Lines 18–19 of Listing II.3).

Extending our model with new behaviour was straightforward. It mostly required to keep track of the states, transitions, and constraints that have already been added and the ones that still need to be added.

Adding Uncertainties. Uncertainties in the managed subsystem can be added by including probabilistic updates in transitions or in the same way as uncertainties were added in the environment, by including a module that models these uncertainties and synchronises with the other modules. In this paper, we added the uncertainty of failures (and blockages) of the camera and sonar. These failures can occur with given probabilities and are modelled in a separate module that only models the sensor failures and blockages but synchronises with the other modules. Thus, uncertainties can be added to the model without changing the already existing model.

Extending the Adaptation Logic. When uncertainties or variability in the managed subsystem are introduced, the adaptation logic (modelled in the feature controller) will often have to be updated too. As for adding behaviour to the managed subsystem, this can include an extension of the guards of already existing transitions, restricting when the adaptation rule is applicable, and adding new transitions, corresponding to new adaptation rules. In our model extension, the transitions (adaptation rules) were modified to reflect the priorities for searching with the sonar and inspecting with the camera. This included extending the guards with the variables changed by the hardware failures module that reflect whether the sonar or camera failed or the camera is blocked (see Listing II.6). Extending the adaptation logic was straightforward and, similar to adding behaviour to the managed subsystem, it required keeping track of which adaptations have already been implemented.

In conclusion, the proposed methodology supports extensibility of the variability model by adding features to the feature model; it caters for an extension of the behaviour of the managed subsystem by adding new states and transitions; it supports an extension of uncertainties by including new modules that synchronise with the already existing ones; and it provides extensibility of the adaptation logic by including new adaptation rules in the form of guarded

commands. All of the mentioned extensions were implemented into the model in a straightforward manner.

II.6.2 RQ2: To What Extent Does the Proposed Methodology Enable the Analysis of SASs?

In our small-scale evaluation, we consider different kinds of reliability analysis:

- (1) Analysing how likely it is for a mission to be successful;
- (2) Analysing sensor failures;
- (3) Analysing rewards;
- (4) Analysing unsafe states and thruster failures.

Successful Mission. We analysed both best- and worst-case scenarios for finishing the pipeline inspection and aborting the mission (see Table II.3). It can be seen that, depending on the chosen scenario, the probability of finishing the inspection and the probability of aborting the mission vary. While the minimum probability of a successful mission is 0.962 in Scenario 1, it is only 0.747 in Scenario 2. Therefore, the chosen scenario can have a big impact on the probability of a successful mission and it can thus be useful to analyse the impact of the scenario on a successful mission beforehand.

Sensor Failures. In Section II.5.1, we analyse the probability of sonar and camera failures in the different scenarios. Table II.4 reports the results of this analysis. It can be seen that the scenario has a significant impact on the probability of achieving a mission without a sensor failure. Even though the maximum probability of finishing the mission without a camera or sonar failure is always 1.0, the minimum probability of finishing without a sensor failure is much lower in Scenario 2 compared to Scenario 1 (for the sonar, 0.592 in Scenario 2 vs. 0.809 in Scenario 1). Furthermore, the probability of a camera failure during the mission is less than the probability of a sonar failure, where the difference varies again with the scenarios.

Rewards. In Section II.5.2, we analysed the minimum and the maximum expected energy consumption as well as the minimum and the maximum expected mission time for finishing the mission. As in the previously described analyses, the scenario has an impact on the analysis results such that the minimum rewards for both energy and time are approximately doubled in Scenario 2 compared to Scenario 1. Since the probability of finishing the pipeline inspection is not 1.0 (because the mission can be aborted if both the camera and the sonar failed), the maximum expected energy and time are defined to be infinite.

Unsafe States and Thruster Failures. By grouping the set of states into different parts, it can be analysed how likely it is to go to these states. In Section II.5.3, we analysed how likely it is to eventually reach an unsafe state, i.e., a thruster failure state or the mission aborted state, and how likely it is to only take safe states. As in the previously presented analyses, the scenario has a big impact on the analysis results, making unsafe states in a mission execution more likely in Scenario 2 than in Scenario 1. Furthermore, PRISM experiments could be exploited to show how the probability of going to a thruster failure state increases with the amount of considered time steps.

To conclude, these reliability analyses are enabled by the two-layered modelling approach that provides a separation of concerns between the managed and the managing subsystem, i.e., between the application logic and the adaptation logic. Of course, the analysis presented here is not exhaustive, we just give a taste of the possible analyses. In Section II.8, we provide further ideas of analyses possible for the model.

II.6.3 RQ3: How Can One Analyse Whether the Adaptation Logic Has Been Realised According to Its Specification?

To analyse whether the adaptation logic has been implemented according to its specification, we considered two aspects in our small-scale evaluation (see Section II.5.4): we analysed whether changes in the system like sensor failures and changing water visibility triggered the correct adaptations and whether the managing subsystem is able to maintain the desired sensor priorities. Using this analysis, we could determine that the adaptation logic was implemented according to its specification. Depending on the implemented adaptation logic, other properties could be analysed to determine that the adaptation logic was implemented according to its specification, such as, e.g., that an adaptation happens within a certain amount of time.

Again, the two-layered approach enabled this analysis and also makes it straightforward to modify the adaptation logic in case the analysis reveals that it has not been realised according to its specification.

II.6.4 Threats to Validity

Formal methods research typically ignores the main empirical research strategies as defined in software engineering [SF18]. A summary of these strategies, specifically adapted for their application in formal methods, including guidelines for selecting the most appropriate research strategy in light of the peculiarity of formal methods research, is reported in [BF22].

Typical threats to validity of research involving case studies or small-scale evaluations are related to the representativeness of the data and analysis (*external validity*), i.e., to what extent can the results be applied to contexts other than the one of the study (*scope of validity*), the soundness of the design (*internal validity*), including, e.g., the expectations or inclinations of the researcher that may have impacted the design or model (*researcher bias*), and the definition of

variables and associated measures (*construct validity*), i.e., to what extent the abstract constructs of interest are well-defined as variables that can effectively be measured quantitatively or evaluated qualitatively. There are many well-known trade-offs between internal and external validity, between the knowledge depth that can be achieved, and concerning the generalisability of the results (e.g., a model may have realistic elements, yet its results are typically hardly applicable to real-world cases).

Considering the small-scale evaluation presented in this paper, there might be other kinds of model extensions that we did not consider and that are more complicated or not possible to realise, limiting the scope of validity. It also remains to be seen how well our methodology scales. One limitation is that the different cases that have to be considered are sometimes hard to track and cases can easily be forgotten. For the adaptation logic, such kind of mistakes can be caught using the analysis presented in Section II.5.4. However, catching mistakes in the managed subsystem might require more effort. Further construct validity follows from the fact that more complicated requirements for the adaptation logic might be harder to analyse because they are not expressible in one of the logics supported by PRISM. Finally, since the development of the modelling and analysis methodology as well as its evaluation have been performed by the same group of researchers, there might also be a researcher bias.

II.7 Related Work

A comprehensive literature study of state-of-the-art techniques for testing, validation, and verification of robotic and autonomous systems is given in a recent survey [AMV23] and recently AUV behaviour has been modelled and analysed with timed automata and UPPAAL [QVW24]; however, work using family-based analysis and DSPLs models was not found in this survey, which indicates the novelty of the general approach taken in our work. In this section, complementing the related work already discussed throughout the paper, we provide further references and context and consider more detail of work related to the family-based analysis of SPL models, the modelling of DSPLs, using SPLs for robotics, and the analysis of SASs as DSPLs.

Family-Based Analysis of SPL Models. As already discussed in Section II.2, the properties of FTSs can be verified with dedicated SPL model-checking tools. Most of these tools accept FTS-like input models, but VMC for instance accepts as input either an FTS through its front-end tool FTS4VMC for static analysis and family-based model checking, or a Modal Transition System (MTS) with a set of logical variability constraints (MTS ν), akin to an FTS' feature expressions. MTSS were introduced to capture the refinement of partial descriptions into more detailed ones [Kře17; LT88]. MTS ν s were introduced to compactly model product family behaviour, whose individual variant (product) behaviour can be obtained through a special-purpose refinement relation or by an equivalent operational derivation procedure [Bee+16]. Such MTSS are equally expressive

as FTSS [Bee+19]. Also other well-known formalisms have been extended with features for SPL modelling and analysis, including variable (modal) I/O automata [LNW07; LPT09], feature (Petri) nets [MCP10; MPC16], and featured team automata [Bee+21]. Yet, to the best of our knowledge, none of these can natively deal with DSPLs. While featured modal contract automata [Bas+17; Bas+20] can be used to synthesise the dynamic composition and orchestration of SPLs valid products, the associated toolset [BB22] does not offer family-based model checking nor quantitative analysis.

Modelling of DSPLs. Cordy et al. [Cor+13a] showed how to model DSPLs with Adaptive FTSS, which are FTSS whose set of features is partitioned into (adaptable) system features and environment features, and which implicitly capture the environment by means of macrostates that are triples formed of the system's state and the configurations of both system and environment. No specific tooling is available and failures are modelled as subtypes of environment features, which allows one to describe failure modes and effects but not their probabilities. In this paper, we therefore used ProFeat [Chr+18], a software tool built on top of PRISM for the analysis of feature-aware probabilistic models. It provides a guarded-command language to model SPL models of probabilistic systems as well as an automatic translation of SPL models to the input language of PRISM (i.e., featureless models). ProFeat can deal with probabilistic DSPLs by offering dynamic feature switching (i.e., activation and deactivation of features at runtime), as we have seen in this paper, and with feature attributes. Moreover, a separate feature controller, i.e., orthogonal to the behavioural model, is responsible for feature switching. We used this in this paper, as it neatly fits the concept adopted here to split an SAS into a managed and a managing subsystem (see Figure II.1). Alternatively, QFLan [Bee+20a; Van+18] offers probabilistic simulations to yield statistical approximations, thus trading 100% precision for scalability. However, in QFLan features can be (un)installed or replaced by operational store actions, i.e., as part of the behavioural model, which interact with a declarative store of constraints. Also approaches based on dynamic Delta-Oriented Programming [Dam+18; DS11] or reconfiguration automata [Bür+14; Loc+17] allow the installation of new features as part of (staged) reconfigurations.

Saller et al. [SLR13] proposed a model-based approach for designing context-aware DSPLs, i.e., DSPLs with a feature model enriched during design time with context information that is exploited during runtime. The focus is on mobile devices and resource constraint systems in a non-probabilistic setting. Model-based approaches for engineering supervisory controllers for DSPLs in non-probabilistic settings have been proposed in [Bas+17; Bas+20; TR23]. Supervisory controllers control (i.e., manage) a system by guaranteeing the maximally permissive behaviour allowed under a given set of constraints.

Using SPLs for Robotics. There are several approaches that model, but do not analyse, SASs as (dynamic) SPLs (e.g., [Aya+21; Ben+08; DGR07; Hal+06]).

For robotics, Gherardi et al. [GB14] proposed their toolchain HyperFlex to model robotic systems as SPLs. HyperFlex supports the design and reuse of reference architectures for robotic systems and was extended with the Robot Perception Specification Language for robotic perception systems by Brugali and Hochgeschwender [BH17]. It supports the representation of variability at different abstraction levels, and feature models from different parts of the system can be composed in several different ways. However, contrary to the approach used in this paper, HyperFlex only considers design time variability. Furthermore, it is only used for modelling robotic systems, not for analysing them. Gherardi and Hochgeschwender [GH15] provide an approach to model both design time and runtime variability of robotic systems based on (D)SPLs. They also use features to model the functionalities of the robot and change features during runtime depending on the environment. However, the approach is focused on achieving adaptation in a real robot and does not consider analysis of the system.

The relevance of SPLs for robotic systems is underlined by Brugali [Bru21]. He argues that most of the costs for robotic systems come from non-reusable software. A robotic system mostly contains software that is tailored to the specific application and embodiment of the robot, and often even software libraries for common robotic functionalities are not reusable. Therefore, they have to be re-developed each and every time. The author thus proposes a new approach for the development of robotic software using SPLs.

Analysis of SASs as DSPLs. Chrszon et al. [Chr+20; Chr+23] model and analyse configurable systems as role-based systems, an extension of feature-oriented systems, with a focus on feature interaction; in contrast to our paper, a separation between managed and managing subsystem is not considered.

A number of studies [Cor+13c; Göt+20; Göt+21; Pfa+17] address the analysis of SASs specified as DSPLs along a dimension orthogonal to the one considered in our work, namely, they focus on reconfiguration costs and real-time constraints of reconfiguration decisions. Sousa et al. [SRD17] proposed the use of temporal constraints and reconfiguration operations to model the reconfiguration lifecycle of a DSPL. They address modelling the variability of cloud systems and identifying reconfigurations that meet given criteria.

II.8 Discussion and Future Work

The work presented in this paper shows the natural correspondence between an SAS that is implemented using external self-adaptation with a managed and a managing subsystem, and a DSPL in the form of a 150% SPL family model with a controller switching between features during runtime. We exploit this correspondence to analyse a two-level SAS operating in an uncontrolled environment, using family-based analysis techniques.

To showcase how an SAS in an uncontrolled environment can be modelled as an FTS model, we used a feature model together with a probabilistic FTS to model the managed subsystem of an AUV used for pipeline inspection, and a

controller switching between these features to model the managing subsystem of the AUV. This allowed modelling the managed subsystem of the AUV as a family of systems, where each family member corresponds to a valid feature configuration of the AUV. The managing subsystem could then be considered as a control layer capable of dynamically switching between these feature configurations depending on both environmental and internal conditions. Furthermore, we showed that these kinds of models can be analysed by means of probabilistic family-based model checking. In our small-scale evaluation, the tool ProFeat was used for this, analysing reward and safety properties.

ProFeat allowed to model the two different layers of abstraction of an SAS, the managed and managing subsystem, which also makes it easier to understand the model and the adaptation logic. As shown in Section II.6, this type of modelling enables extensions of the model in a straightforward manner (see RQ1). Furthermore, it makes analysing all configurations of the managed subsystem more efficient by enabling family-based model checking. It also enabled us to analyse not only whether the adaptation logic was implemented correctly (see RQ3), but also enabled an analysis of important system properties (see RQ2). We are unaware of other work that exploits the family-based modelling and analysis capabilities of ProFeat for SASs, but we believe this is a natural fit.

The family-based analysis proposed in this paper is subject to the usual restrictions of automata-based analyses, and it remains to study the degree to which these can be circumvented. In particular, automata modelling of complex systems is known to suffer from the state-space explosion (i.e., the number of states needed to accurately model a system may exceed the amount of available computer memory). Techniques to mitigate the state-space explosion can be based on compositionality or abstraction. Compositional analysis techniques for FTSs have not been extensively studied so far (but see, e.g., [BV14; Dub19]). In our work, it has so far been sufficient to use finite representations of the managed subsystem and of the environment. Techniques to discretise continuous systems exist [Alu+00; GP07], but, to the best of our knowledge, it is an open problem to what extent these techniques reduce the precision of optimised strategies produced by the family-based analysis.

In addition to the analyses showcased in this paper, the models allow for many other kinds of analysis, some of which we list here. By implementing different environment and hardware failure modules, it can be analysed how the behaviour of the environment and the hardware failures influences the behaviour of the AUV. Furthermore, parametric model checking, where certain variables are left unspecified, can be used to, e.g., analyse in which scenarios the AUV can be deployed in while still ensuring certain safety properties. Lastly, it is possible to use multi-objective analysis to analyse properties where several objectives have to be satisfied, e.g., minimising both time and energy.

It remains to be seen how the modelling and analysis scale with more complex systems. Concerning the modelling, we believe that the compositional approach we took makes it easier to develop models of complex systems compared to a monolithic approach, because different concerns can be modelled in distinct, synchronising modules. This makes both developing and modifying the model

easier. One problem for modelling might be that the adaptation logic of a complex system can get very complicated and introducing errors when modelling becomes more likely. Here, the proposed analysis for analysing the adaptation logic can help to find mistakes that were introduced when modelling the adaptation logic. As described above, the analysis of complex systems often suffers from a state-space explosion, and developing a finite representation of the different parts of the system might become a challenge. It would be interesting to investigate if the proposed techniques for mitigating these risks can be applied to our methodology.

The small-scale evaluation in this paper is of course a highly simplified model of an AUV and its mission. However, we showed that it is feasible to model and analyse a two-layered self-adaptive cyber-physical system as a family of configurations with a controller switching between them. There are many ways of extending the model. First, more functionalities and variability points of the AUV as well as new tasks can be incorporated in the model by including more features in the feature model. This also necessitates adapting the model of the managed subsystem to include the new behaviour, as well as adapting the model of the managing subsystem to include the new variability points in the adaptation logic. Furthermore, including new uncertainties that lead to adaptation needs (like the environment and the hardware failures in this paper) can be done by including new modules that model the uncertainties and including them in the adaptation logic of the managing subsystem. It would also be possible to consider a swarm of AUVs working together by leveraging ProFeat's functionality of module parametrisation. However, of the extensions mentioned here, this is probably the most difficult one and likely requires remodelling of most parts of the small-scale evaluation. Furthermore, the model of the environment could easily be exchanged with a more realistic one. All these changes would require some modelling effort, however, there should not be challenges concerning the analysis of the extended models. To analyse a real AUV, the models of the AUV and of the environment, and in particular the probabilities, have to be adapted to the robot and the environment with the help of real data and domain experts. We plan to investigate this together with an industrial partner of the MSCA network REMARO (Reliable AI for Marine Robotics).

In the future, we plan to find optimal strategies for the managing subsystem, i.e., the controller switching between features, e.g., to minimise energy consumption. We would also like to find patterns between choosing a certain feature configuration and the effect of this on quality criteria of the system. Finding such control patterns could help to improve the adaptation logic of the managing subsystem to be more resilient towards faults. Furthermore, it would be interesting to investigate more complex ways of representing relations between features, for example, representing in the feature model that the AUV has to follow the pipeline at a low altitude when using the camera and at a medium altitude when using the sonar. Finally, analysing a system with and without adaptation to determine both the gains and the costs of using self-adaptation would be interesting.

Acknowledgements. We would like to thank Clemens Dubslaff for explaining ProFeat and its usage to us, and for answering numerous questions. This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200), by the Italian project NODES (which has received funding from the MUR – M4C2 1.5 of PNRR with grant agreement no. ECS00000036) and by the Italian MUR PRIN 2020TL3X8X project T-LADIES (Typeful Language Adaptation for Dynamic, Interacting and Evolving Systems).

References

- [Abb+23] Nadeem Abbas, Jesper Andersson, Ilias Gerostathopoulos, Patricia Lago, Stefan Biffl, Juergen Musil, Premek Brada, Tomas Bures, Amleto Di Salle, Matthias Galster, Grace Lewis, Marin Litoiu, Panos Patros, and Patrizio Pelliccione. “Self-Adaptation in Industry: A Survey”. In: *ACM Transactions on Autonomous and Adaptive Systems* (2023). DOI: 10.1145/3589227.
- [Alu+00] Rajeev Alur, Thomas A. Henzinger, Gerardo Lafferriere, and George J. Pappas. “Discrete Abstractions of Hybrid Systems”. In: *Proceedings of the IEEE* vol. 88, no. 7 (2000), pp. 971–984. DOI: 10.1109/5.871304.
- [AMV23] Hugo Araujo, Mohammad Reza Mousavi, and Mahsa Varshosaz. “Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review”. In: *ACM Transactions on Software Engineering and Methodology* vol. 32, no. 2 (2023), 51:1–51:61. DOI: 10.1145/3542945.
- [Ape+12] Sven Apel, Sergiy S. Kolesnikov, Jörg Liebig, Christian Kästner, Martin Kuhlemann, and Thomas Leich. “Access control in feature-oriented programming”. In: *Science of Computer Programming* vol. 77, no. 3 (2012), pp. 174–187. DOI: 10.1016/J.SCICO.2010.07.005.
- [Ape+13] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Germany: Springer, 2013. DOI: 10.1007/978-3-642-37521-7.
- [AS22] Oscar Aguayo and Samuel Sepúlveda. “Variability Management in Dynamic Software Product Lines for Self-Adaptive Systems—A Systematic Mapping”. In: *Applied Sciences* vol. 12, no. 20 (2022). DOI: 10.3390/app122010240.
- [ASN14] Stephan Adelsberger, Stefan Sobernig, and Gustaf Neumann. “Towards Assessing the Complexity of Object Migration in Dynamic, Feature-oriented Software Product Lines”. In: *Proceedings of the 8th International Workshop on Variability Modelling of Software-intensive Systems (VaMoS 2014)*. ACM, 2014, 17:1–17:8. DOI: 10.1145/2556624.2556645.

-
- [Aya+21] Inmaculada Ayala, Alessandro V. Papadopoulos, Mercedes Amor, and Lidia Fuentes. “ProDSPL: Proactive self-adaptation based on Dynamic Software Product Lines”. In: *Journal of Systems and Software* vol. 175 (2021). DOI: 10.1016/J.JSS.2021.110909.
- [Bar14] Luciano Baresi. “Self-Adaptive Systems, Services, and Product Lines”. In: *Proceedings of the 18th International Software Product Line Conference (SPLC 2014)*. USA: ACM, 2014, pp. 2–4. DOI: 10.1145/2648511.2648512.
- [Bas+17] Davide Basile, Maurice H. ter Beek, Felicita Di Giandomenico, and Stefania Gnesi. “Orchestration of Dynamic Service Product Lines with Featured Modal Contract Automata”. In: *Proceedings of the 21st International Systems and Software Product Line Conference (SPLC 2017)*. Vol. 2. ACM, 2017, pp. 117–122. DOI: 10.1145/3109729.3109741.
- [Bas+20] Davide Basile, Maurice H. ter Beek, Pierpaolo Degano, Axel Legay, Gian-Luigi Ferrari, Stefania Gnesi, and Felicita Di Giandomenico. “Controller synthesis of service contracts with variability”. In: *Science of Computer Programming* vol. 187 (2020). DOI: 10.1016/j.scico.2019.102344.
- [BB22] Davide Basile and Maurice H. ter Beek. “Contract Automata Library”. In: *Science of Computer Programming* vol. 221 (2022). DOI: 10.1016/j.scico.2022.102841.
- [BCH15] Davide Brugali, Rafael Capilla, and Mike Hinchey. “Dynamic Variability Meets Robotics”. In: *IEEE Computer* vol. 48, no. 12 (2015), pp. 94–97. DOI: 10.1109/MC.2015.354.
- [Bee+16] Maurice H. ter Beek, Alessandro Fantechi, Stefania Gnesi, and Franco Mazzanti. “Modelling and Analysing Variability in Product Families: Model Checking of Modal Transition Systems with Variability Constraints”. In: *Journal of Logical and Algebraic Methods in Programming* vol. 85, no. 2 (2016), pp. 287–315. DOI: 10.1016/j.jlamp.2015.11.006.
- [Bee+19] Maurice H. ter Beek, Ferruccio Damiani, Stefania Gnesi, Franco Mazzanti, and Luca Paolini. “On the Expressiveness of Modal Transition Systems with Variability Constraints”. In: *Science of Computer Programming* vol. 169 (2019), pp. 1–17. DOI: 10.1016/j.scico.2018.09.006.
- [Bee+20a] Maurice H. ter Beek, Axel Legay, Alberto Lluch Lafuente, and Andrea Vandin. “A Framework for Quantitative Modeling and Analysis of Highly (Re)configurable Systems”. In: *IEEE Transaction on Software Engineering* vol. 46, no. 3 (2020), pp. 321–345. DOI: 10.1109/TSE.2018.2853726.

- [Bee+20b] Maurice H. ter Beek, Sijf van Loo, Erik P. de Vink, and Tim A.C. Willemse. “Family-Based SPL Model Checking Using Parity Games with Variability”. In: *Proceedings of the 23rd International Conference on Fundamental Approaches to Software Engineering (FASE 2020)*. Ed. by Heike Wehrheim and Jordi Cabot. Vol. 12076. Lecture Notes in Computer Science. Germany: Springer, 2020, pp. 245–265. DOI: [10.1007/978-3-030-45234-6_12](https://doi.org/10.1007/978-3-030-45234-6_12).
- [Bee+21] Maurice H. ter Beek, Guillermina Cledou, Rolf Hennicker, and José Proença. “Featured Team Automata”. In: *Proceedings of the 24th International Symposium on Formal Methods (FM 2021)*. Ed. by Marieke Huisman, Corina S. Pasareanu, and Naijun Zhan. Vol. 13047. Lecture Notes in Computer Science. Springer, 2021, pp. 483–502. DOI: [10.1007/978-3-030-90870-6_26](https://doi.org/10.1007/978-3-030-90870-6_26).
- [Bee+22] Maurice H. ter Beek, Ferruccio Damiani, Michael Lienhardt, Franco Mazzanti, and Luca Paolini. “Efficient Static Analysis and Verification of Featured Transition Systems”. In: *Empirical Software Engineering* vol. 22, no. 1 (2022), 10:1–10:43. DOI: [10.1007/s10664-020-09930-8](https://doi.org/10.1007/s10664-020-09930-8).
- [Ben+08] Nelly Bencomo, Peter Sawyer, Gordon S. Blair, and Paul Grace. “Dynamically Adaptive Systems are Product Lines too: Using Model-Driven Techniques to Capture Dynamic Variability of Adaptive Systems”. In: *Proceedings of the 12th International Conference on Software Product Lines (SPLC 2008)*. Ed. by Steffen Thiel and Klaus Pohl. Vol. 2. Ireland: Lero, University of Limerick, 2008, pp. 23–32.
- [BF22] Maurice H. ter Beek and Alessio Ferrari. “Empirical Formal Methods: Guidelines for Performing Empirical Studies on Formal Methods”. In: *Software* vol. 1, no. 4 (2022), pp. 381–416. DOI: [10.3390/software1040017](https://doi.org/10.3390/software1040017).
- [BH17] Davide Brugali and Nico Hochgeschwender. “Managing the Functional Variability of Robotic Perception Systems”. In: *Proceedings of the 1st International Conference on Robotic Computing (IRC 2017)*. USA: IEEE, 2017, pp. 277–283. DOI: [10.1109/IRC.2017.20](https://doi.org/10.1109/IRC.2017.20).
- [BHA12] Nelly Bencomo, Svein O. Hallsteinsen, and Eduardo Santana de Almeida. “A View of the Dynamic Software Product Line Landscape”. In: *IEEE Computer* vol. 45, no. 10 (2012), pp. 36–41. DOI: [10.1109/MC.2012.292](https://doi.org/10.1109/MC.2012.292).
- [BK08] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. USA: MIT Press, 2008.
- [BM14] Maurice H. ter Beek and Franco Mazzanti. “VMC: Recent Advances and Challenges Ahead”. In: *Proceedings of the 18th International Software Product Line Conference (SPLC 2014)*. Vol. 2. USA: ACM, 2014, pp. 70–77. DOI: [10.1145/2647908.2655969](https://doi.org/10.1145/2647908.2655969).

- [BMS12] Maurice H. ter Beek, Franco Mazzanti, and Aldi Sulova. “VMC: A Tool for Product Variability Analysis”. In: *Proceedings of the 18th International Symposium on Formal Methods (FM 2012)*. Ed. by Dimitra Giannakopoulou and Dominique Méry. Vol. 7436. Lecture Notes in Computer Science. Germany: Springer, 2012, pp. 450–454.
- [Boz+22] Darko Bozhinoski, Mario Garzon Oviedo, Nadia Hammoudeh Garcia, Harshavardhan Deshpande, Gijs van der Hoorn, Jon Tjerngren, Andrzej Wasowski, and Carlos Hernández Corbato. “MROS: Runtime Adaptation for Robot Control Architectures”. In: *Advanced Robotics* vol. 36, no. 11 (2022), pp. 502–518. DOI: 10.1080/01691864.2022.2039761.
- [Bru20] Davide Brugali. “Runtime reconfiguration of robot control systems: a ROS-based case study”. In: *Proceedings of the 4th International Conference on Robotic Computing (IRC 2020)*. IEEE, 2020, pp. 256–262. DOI: 10.1109/IRC.2020.00047.
- [Bru21] Davide Brugali. “Software Product Line Engineering for Robotics”. In: *Software Engineering for Robotics*. Ed. by Ana Cavalcanti, Brijesh Dongol, Rob Hierons, Jon Timmis, and Jim Woodcock. Germany: Springer, 2021, pp. 1–28. DOI: 10.1007/978-3-030-66494-7_1.
- [BSE19] Maurice H. ter Beek, Klaus Schmid, and Holger Eichelberger. “Textual Variability Modeling Languages: An Overview and Considerations”. In: *Proceedings of the 23rd International Systems and Software Product Line Conference (SPLC 2019)*. Vol. 2. USA: ACM, 2019, 82:1–82:7. DOI: 10.1145/3307630.3342398.
- [BV14] Maurice H. ter Beek and Erik P. de Vink. “Towards Modular Verification of Software Product Lines with mCRL2”. In: *Proceedings of the 6th International Symposium on Leveraging Applications of Formal Methods: Technologies for Mastering Change (ISoLA 2014)*. Ed. by Tiziana Margaria and Bernhard Steffen. Vol. 8802. Lecture Notes in Computer Science. Springer, 2014, pp. 368–385. DOI: 10.1007/978-3-662-45234-9_26.
- [BVW17] Maurice H. ter Beek, Erik P. de Vink, and Tim A. C. Willemse. “Family-Based Model Checking with mCRL2”. In: *Proceedings of the 20th International Conference on Fundamental Approaches to Software Engineering (FASE 2017)*. Ed. by Marieke Huisman and Julia Rubin. Vol. 10202. Lecture Notes in Computer Science. Germany: Springer, 2017, pp. 387–405. DOI: 10.1007/978-3-662-54494-5_23.
- [Bür+14] Johannes Bürdek, Sascha Lity, Malte Lochau, Markus Berens, Ursula Goltz, and Andy Schürr. “Staged Configuration of Dynamic Software Product Lines with Complex Binding Time Constraints”. In: *Proceedings of the 8th International Workshop on Variability*

- Modelling of Software-intensive Systems (VaMoS 2014)*. ACM, 2014, 16:1–16:8. DOI: [10.1145/2556624.2556627](https://doi.org/10.1145/2556624.2556627).
- [Cap+14] Rafael Capilla, Jan Bosch, Pablo Trinidad, Antonio Ruiz-Cortés, and Mike Hinchey. “An overview of Dynamic Software Product Line architectures and techniques: Observations from research and industry”. In: *Journal of Systems and Software* vol. 91 (2014), pp. 3–23. DOI: <https://doi.org/10.1016/j.jss.2013.12.038>.
- [CE00] Krzysztof Czarnecki and Ulrich W. Eisenecker. *Generative Programming: Methods, Tools, and Applications*. USA: Addison-Wesley, 2000.
- [Chr+18] Philipp Chrszon, Clemens Dubsloff, Sascha Klüppelholz, and Christel Baier. “ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking”. In: *Formal Aspects of Computing* vol. 30, no. 1 (2018), pp. 45–75. DOI: [10.1007/s00165-017-0432-4](https://doi.org/10.1007/s00165-017-0432-4).
- [Chr+20] Philipp Chrszon, Christel Baier, Clemens Dubsloff, and Sascha Klüppelholz. “From Features to Roles”. In: *Proceedings of the 24th International Systems and Software Product Line Conference (SPLC 2020)*. USA: ACM, 2020, 19:1–19:11. DOI: [10.1145/3382025.3414962](https://doi.org/10.1145/3382025.3414962).
- [Chr+23] Philipp Chrszon, Christel Baier, Clemens Dubsloff, and Sascha Klüppelholz. “Interaction Detection in Configurable Systems – A Formal Approach Featuring Roles”. In: *Journal of Systems and Software* vol. 196 (2023), p. 111556. DOI: [10.1016/j.jss.2022.111556](https://doi.org/10.1016/j.jss.2022.111556).
- [CHS08] Andreas Classen, Patrick Heymans, and Pierre-Yves Schobbens. “What’s in a Feature: A Requirements Engineering Perspective”. In: *International Conference on Fundamental Approaches to Software Engineering (FASE)*. Ed. by José Luiz Fiadeiro and Paola Inverardi. Vol. 4961. Lecture Notes in Computer Science. Germany: Springer, 2008, pp. 16–30. DOI: [10.1007/978-3-540-78743-3_2](https://doi.org/10.1007/978-3-540-78743-3_2).
- [Cla+10] Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, Axel Legay, and Jean-François Raskin. “Model Checking Lots of Systems: Efficient Verification of Temporal Properties in Software Product Lines”. In: *Proceedings of the 32nd International Conference on Software Engineering (ICSE 2010)*. USA: ACM, 2010, pp. 335–344. DOI: [10.1145/1806799.1806850](https://doi.org/10.1145/1806799.1806850).
- [Cla+12] Andreas Classen, Maxime Cordy, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Model Checking Software Product Lines with SNIP”. In: *International Journal on Software Tools for Technology Transfer* vol. 14, no. 5 (2012), pp. 589–612. DOI: [10.1007/s10009-012-0234-1](https://doi.org/10.1007/s10009-012-0234-1).

- [Cla+13] Andreas Classen, Maxime Cordy, Pierre-Yves Schobbens, Patrick Heymans, Axel Legay, and Jean-François Raskin. “Featured Transition Systems: Foundations for Verifying Variability-Intensive Systems and Their Application to LTL Model Checking”. In: *IEEE Transaction on Software Engineering* vol. 39, no. 8 (2013), pp. 1069–1089. DOI: [10.1109/TSE.2012.86](https://doi.org/10.1109/TSE.2012.86).
- [Cla+14] Andreas Classen, Maxime Cordy, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Formal Semantics, Modular Specification, and Symbolic Verification of Product-Line Behaviour”. In: *Science of Computer Programming* vol. 80 (2014), pp. 416–439. DOI: [10.1145/2499777.2499781](https://doi.org/10.1145/2499777.2499781).
- [Cor+13a] Maxime Cordy, Andreas Classen, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Model Checking Adaptive Software with Featured Transition Systems”. In: *Assurances for Self-Adaptive Systems: Principles, Models, and Techniques*. Ed. by Javier Cámara, Rogério de Lemos, Carlo Ghezzi, and Antónia Lopes. Vol. 7740. Lecture Notes in Computer Science. Germany: Springer, 2013, pp. 1–29.
- [Cor+13b] Maxime Cordy, Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, and Axel Legay. “ProVeLines: A Product Line of Verifiers for Software Product Lines”. In: *Proceedings of the 17th International Software Product Line Conference (SPLC 2013)*. Vol. 2. USA: ACM, 2013, pp. 141–146. DOI: [10.1145/2499777.2499781](https://doi.org/10.1145/2499777.2499781).
- [Cor+13c] Maxime Cordy, Axel Legay, Pierre-Yves Schobbens, and Louis-Marie Traonouez. “A Framework for the Rigorous Design of Highly Adaptive Timed Systems”. In: *Proceedings of the 1st FME Workshop on Formal Methods in Software Engineering (FormaliSE 2013)*. USA: IEEE, 2013, pp. 64–70. DOI: [10.1109/FormaliSE.2013.6612279](https://doi.org/10.1109/FormaliSE.2013.6612279).
- [Dam+18] Ferruccio Damiani, Luca Padovani, Ina Schaefer, and Christoph Seidl. “A core calculus for dynamic delta-oriented programming”. In: *Acta Informatica* vol. 55, no. 4 (2018), pp. 269–307. DOI: [10.1007/S00236-017-0293-6](https://doi.org/10.1007/S00236-017-0293-6).
- [DBK15] Clemens Dubslaff, Christel Baier, and Sascha Klüppelholz. “Probabilistic Model Checking for Feature-Oriented Systems”. In: *Transactions on Aspect-Oriented Software Development XII*. Ed. by Shigeru Chiba, Éric Tanter, Erik Ernst, and Robert Hirschfeld. Vol. 8989. Lecture Notes in Computer Science. Germany: Springer, 2015, pp. 180–220. DOI: [10.1007/978-3-662-46734-3_5](https://doi.org/10.1007/978-3-662-46734-3_5).
- [DGR07] Deepak Dhungana, Paul Grünbacher, and Rick Rabiser. “Domain-Specific Adaptations of Product Line Variability Modeling”. In: *Proceedings of the IFIP WG 8.1 Working Conference on Situational Method Engineering: Fundamentals and Experiences (ME 2007)*.

- Ed. by Jolita Ralyté, Sjaak Brinkkemper, and Brian Henderson-Sellers. Vol. 244. IFIP. Germany: Springer, 2007, pp. 238–251. DOI: 10.1007/978-0-387-73947-2_19.
- [Dim+15] Aleksandar S. Dimovski, Ahmad Salim Al-Sibahi, Claus Brabrand, and Andrzej Wąsowski. “Family-Based Model Checking Without a Family-Based Model Checker”. In: *Proceedings of the 22nd International Symposium on Model Checking Software (SPIN 2015)*. Ed. by B. Fischer and J. Geldenhuys. Vol. 9232. Lecture Notes in Computer Science. Springer, 2015, pp. 282–299. DOI: 10.1007/978-3-319-23404-5_18.
- [Dim+17] Aleksandar S. Dimovski, Ahmad Salim Al-Sibahi, Claus Brabrand, and Andrzej Wąsowski. “Efficient Family-Based Model Checking via Variability Abstractions”. In: *International Journal on Software Tools for Technology Transfer* vol. 5, no. 19 (2017), pp. 585–603. DOI: 10.1007/s10009-016-0425-2.
- [Dim20] Aleksandar S. Dimovski. “CTL* Family-Based Model Checking using Variability Abstractions and Modal Transition Systems”. In: *International Journal on Software Tools for Technology Transfer* vol. 22, no. 1 (2020), pp. 35–55. DOI: 10.1007/s10009-019-00528-0.
- [Din+10] Tom Dinkelaker, Ralf Mitschke, Karin Fetzer, and Mira Mezini. “A Dynamic Software Product Line Approach Using Aspect Models at Runtime”. In: *Proceedings of the 1st International Workshop on Composition: Objects, Aspects, Components, Services and Product Lines*. Ed. by Philippe Lahire, Geri Georg, Mourad Oussalah, Jon Whittle, Naouel Moha, and Stefan Van Baelen. Vol. 564. CEUR Workshop Proceedings. 2010.
- [DKB14] Clemens Dubslaff, Sascha Klüppelholz, and Christel Baier. “Probabilistic Model Checking for Energy Analysis in Software Product Lines”. In: *Proceedings of the 13th International Conference on Modularity (MODULARITY 2014)*. USA: ACM, 2014, pp. 169–180. DOI: 10.1145/2577080.2577095.
- [DLW19] Aleksandar S. Dimovski, Axel Legay, and Andrzej Wąsowski. “Variability Abstraction and Refinement for Game-Based Lifted Model Checking of Full CTL”. In: *Proceedings of the 22nd International Conference on Fundamental Approaches to Software Engineering (FASE 2019)*. Ed. by R. Hähnle and W. van der Aalst. Vol. 11424. Lecture Notes in Computer Science. Springer, 2019, pp. 192–209. DOI: 10.1007/978-3-030-16722-6_11.
- [DS11] Ferruccio Damiani and Ina Schaefer. “Dynamic Delta-Oriented Programming”. In: *Proceedings of the 15th International Software Product Line Conference (SPLC 2011)*. Vol. 2. ACM, 2011. DOI: 10.1145/2019136.2019175.

- [Dub19] Clemens Dubslaff. “Compositional Feature-Oriented Systems”. In: *Proceedings of the 17th International Conference on Software Engineering and Formal Methods (SEFM 2019)*. Ed. by Peter Csaba Ölveczky and Gwen Salaün. Vol. 11724. Lecture Notes in Computer Science. Springer, 2019, pp. 162–180. DOI: 10.1007/978-3-030-30446-1_9.
- [DW17] Aleksandar S. Dimovski and Andrzej Wasowski. “Variability-Specific Abstraction Refinement for Family-Based Model Checking”. In: *Proceedings of the 20th International Conference on Fundamental Approaches to Software Engineering (FASE 2017)*. Ed. by M. Huisman and J. Rubin. Vol. 10202. Lecture Notes in Computer Science. Springer, 2017, pp. 406–423. DOI: 10.1007/978-3-662-54494-5_24.
- [Gar+19] Sergio García, Daniel Strüder, Davide Brugali, Alessandro Di Fava, Philipp Schillinger, Patrizio Pelliccione, and Thorsten Berger. “Variability Modeling of Service Robots: Experiences and Challenges”. In: *Proceedings of the 13th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS 2019)*. USA: ACM, 2019, 8:1–8:6. DOI: 10.1145/3302333.3302350.
- [GB14] Luca Gherardi and Davide Brugali. “Modeling and Reusing Robotic Software Architectures: the HyperFlex Toolchain”. In: *Proceedings of the International Conference on Robotics and Automation (ICRA 2014)*. USA: IEEE, 2014, pp. 6414–6420. DOI: 10.1109/ICRA.2014.6907806.
- [GH15] Luca Gherardi and Nico Hochgeschwender. “RRA: Models and Tools for Robotics Run-time Adaptation”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2015)*. IEEE, 2015, pp. 1777–1784. DOI: 10.1109/IROS.2015.7353608.
- [GP07] Antoine Girard and George J. Pappas. “Approximation Metrics for Discrete and Continuous Systems”. In: *IEEE Transactions on Automatic Control* vol. 52, no. 5 (2007), pp. 782–798. DOI: 10.1109/TAC.2007.895849.
- [Göt+20] Hendrik Göttmann, Lars Luthmann, Malte Lochau, and Andy Schürr. “Real-Time-Aware Reconfiguration Decisions for Dynamic Software Product Lines”. In: *Proceedings of the 24th Systems and Software Product Line Conference (SPLC 2020)*. USA: ACM, 2020, 13:1–13:11. DOI: 10.1145/3382025.3414945.
- [Göt+21] Hendrik Göttmann, Isabelle Bacher, Nicolas Gottwald, and Malte Lochau. “Static Analysis Techniques for Efficient Consistency Checking of Real-Time-Aware DSPL Specifications”. In: *Proceedings of the 15th International Working Conference on Variability Modelling of Software-Intensive Systems (VaMoS 2021)*. USA: ACM, 2021. DOI: 10.1145/3442391.3442409.

- [Hal+06] Svein Hallsteinsen, Erlend Stav, Arnor Solberg, and Jacqueline Floch. “Using Product Line Techniques to Build Adaptive Systems”. In: *Proceedings of the 10th International Software Product Line Conference (SPLC 2006)*. USA: IEEE, 2006, pp. 141–150. DOI: 10.1109/SPLINE.2006.1691586.
- [Hal+13] Svein Hallsteinsen, Mike Hinchey, Sooyong Park, and Klaus Schmid. “Dynamic Software Product Lines”. In: *Systems and Software Variability Management: Concepts, Tools and Experiences*. Ed. by Rafael Capilla, Jan Bosch, and Kyo-Chul Kang. Germany: Springer, 2013, pp. 253–260. DOI: 10.1007/978-3-642-36583-6_16.
- [Her13] Carlos Hernández Corbato. “Model-Based Self-awareness Patterns for Autonomy”. PhD thesis. Universidad Politécnica de Madrid, 2013. DOI: 10.20868/UPM.thesis.23178.
- [Hez+21] Sara M. Hezavehi, Danny Weyns, Paris Avgeriou, Radu Calinescu, Raffaella Mirandola, and Diego Perez-Palacin. “Uncertainty in Self-adaptive Systems: A Research Community Perspective”. In: *ACM Transactions on Autonomous and Adaptive Systems* vol. 15, no. 4 (2021), 10:1–10:36. DOI: 10.1145/3487921.
- [HPS12] Mike Hinchey, Sooyong Park, and Klaus Schmid. “Building Dynamic Software Product Lines”. In: *IEEE Computer* vol. 45, no. 10 (2012), pp. 22–26. DOI: 10.1109/MC.2012.332.
- [KC03] Jeffrey O. Kephart and David M. Chess. “The Vision of Autonomic Computing”. In: *IEEE Computer* vol. 36, no. 1 (2003), pp. 41–50. DOI: 10.1109/MC.2003.1160055.
- [KNP11] Marta Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of Probabilistic Real-Time Systems”. In: *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV 2011)*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Germany: Springer, 2011, pp. 585–591. DOI: 10.1007/978-3-642-22110-1_47.
- [Kře17] Jan Křetínský. “30 Years of Modal Transition Systems: Survey of Extensions and Analysis”. In: *Models, Algorithms, Logics and Tools*. Ed. by L. Aceto, G. Bacci, G. Bacci, A. Ingólfssdóttir, A. Legay, and R. Mardare. Vol. 10460. Lecture Notes in Computer Science. Springer, 2017, pp. 36–74. DOI: 10.1007/978-3-319-63121-9_3.
- [LNW07] Kim G. Larsen, Ulrik Nyman, and Andrzej Wąsowski. “Modal I/O Automata for Interface and Product Line Theories”. In: *Proceedings of the 16th European Symposium on Programming (ESOP 2007)*. Ed. by Rocco De Nicola. Vol. 4421. Lecture Notes in Computer Science. Springer, 2007, pp. 64–79. DOI: 10.1007/978-3-540-71316-6_6.

- [Loc+16] Malte Lochau, Stephan Mennicke, Hauke Baller, and Lars Ribbeck. “Incremental Model Checking of Delta-Oriented Software Product Lines”. In: *Journal of Logical and Algebraic Methods in Programming* vol. 85, no. 1 (2016), pp. 245–267. DOI: 10.1016/j.jlamp.2015.09.004.
- [Loc+17] Malte Lochau, Johannes Bürdek, Stefan Hölzle, and Andy Schürr. “Specification and automated validation of staged reconfiguration processes for dynamic software product lines”. In: *Software & Systems Modeling* vol. 16, no. 1 (2017), pp. 125–152. DOI: 10.1007/S10270-015-0470-4.
- [LPT09] Kim Lauenroth, Klaus Pohl, and Simon Töhning. “Model Checking of Domain Artifacts in Product Line Engineering”. In: *Proceedings of the 24th International Conference on Automated Software Engineering (ASE 2009)*. IEEE, 2009, pp. 269–280. DOI: 10.1109/ASE.2009.16.
- [LT88] Kim G. Larsen and Bent Thomsen. “A Modal Process Logic”. In: *Proceedings of the 3rd Symposium on Logic in Computer Science (LICS 1988)*. IEEE, 1988, pp. 203–210. DOI: 10.1109/LICS.1988.5119.
- [Luc+19] Matt Luckcuck, Marie Farrell, Louise A. Dennis, Clare Dixon, and Michael Fisher. “Formal Specification and Verification of Autonomous Robotic Systems: A Survey”. In: *ACM Computing Surveys* vol. 52, no. 5 (2019), 100:1–100:41. DOI: 10.1145/3342355.
- [MCP10] Radu Muschevici, Dave Clarke, and José Proença. “Feature Petri Nets”. In: *Proceedings of the 1st International Workshop on Formal Methods in Software Product Line Engineering (FMSPLE 2010)*. Technical Report, University of Lancaster, 2010.
- [MCP13] Radu Muschevici, Dave Clarke, and José Proença. “Executable Modelling of Dynamic Software Product Lines in the ABS Language”. In: *Proceedings of the 5th International Workshop on Feature-Oriented Software Development (FOSD 2013)*. ACM, 2013, pp. 17–24. DOI: 10.1145/2528265.2528266.
- [Mei+17] Jens Meinicke, Thomas Thüm, Reimar Schröter, Fabian Benduhn, Thomas Leich, and Gunter Saake. *Mastering Software Variability with FeatureIDE*. Springer, 2017. DOI: 10.1007/978-3-319-61443-4.
- [MPC16] Radu Muschevici, José Proença, and Dave Clarke. “Feature Nets: behavioural modelling of software product lines”. In: *Software & Systems Modeling* vol. 15, no. 4 (2016), pp. 1181–1206. DOI: 10.1007/s10270-015-0475-z.

- [NLR21] Arne Nordmann, Ralph Lange, and Francisco Martín Rico. “System Modes – Digestible System (Re-)Configuration for Robotics”. In: *Proceedings of the 3rd International Workshop on Robotics Software Engineering (RoSE 2021)*. USA: IEEE, 2021, pp. 19–24. DOI: 10.1109/RoSE52553.2021.00010.
- [Ola+18] Rafael Olaechea, Joanne Atlee, Axel Legay, and Uli Fahrenberg. “Trace Checking for Dynamic Software Product Lines”. In: *Proceedings of the 13th International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2018)*. ACM, 2018, pp. 69–75. DOI: 10.1145/3194133.3194143.
- [Pfa+17] Martin Pfannemuller, Christian Krupitzer, Markus Weckesser, and Christian Becker. “A Dynamic Software Product Line Approach for Adaptation Planning in Autonomic Computing Systems”. In: *Proceedings of the International Conference on Autonomic Computing (ICAC 2017)*. USA: IEEE, 2017, pp. 247–254. DOI: 10.1109/ICAC.2017.18.
- [Päß+23a] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, S. Lizeth Tapia Tarifa, and Einar Broch Johnsen. “Formal Modelling and Analysis of a Self-Adaptive Robotic System”. In: *Proceedings of the 18th International Conference on integrated Formal Methods (iFM 2023)*. Ed. by Paula Herber and Anton Wijs. Vol. 14300. Lecture Notes in Computer Science. Springer, 2023, pp. 343–363. DOI: 10.1007/978-3-031-47705-8_18.
- [Päß+23b] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, S. Lizeth Tapia Tarifa, and Einar Broch Johnsen. *Formal Modelling and Analysis of a Self-Adaptive Robotic System (Artifact)*. Version 1.0.0. 2023. DOI: 10.5281/zenodo.8275533.
- [Päß+24] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, and Silvia Lizeth Tapia Tarifa. *Analysing Self-Adaptive Systems as Software Product Lines (Artifact)*. Zenodo. Version 1.0.0. 2024. DOI: 10.5281/zenodo.14230735.
- [Päß+25] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, and S. Lizeth Tapia Tarifa. “A Configurable Software Model of a Self-Adaptive Robotic System”. In: *Science of Computer Programming* vol. 240 (2025), p. 103221. DOI: 10.1016/j.scico.2024.103221.
- [Qui+09] Morgan Quigley, Brian Gerkey, Ken Conley, Josh Faust, Tully Foote, Jeremy Leibs, Eric Berger, Rob Wheeler, and Andrew Ng. “ROS: An Open-Source Robot Operating System”. In: *Proceedings of the Open-Source Software Workshop of the International Conference on Robotics and Automation (ICRA 2009)*. USA: IEEE, 2009.

- [Qui+21] Clément Quinton, Michael Vierhauser, Rick Rabiser, Luciano Baresi, Paul Grünbacher, and Christian Schuhmayer. “Evolution in dynamic software product lines”. In: *Journal of Software: Evolution and Process* vol. 33, no. 2 (2021), e2293:1–e2293:25. DOI: 10.1002/SMR.2293.
- [QVW24] Sergio Quijano, Mahsa Varshosaz, and Andrzej Wąsowski. “Modeling and Safety Analysis of Autonomous Underwater Vehicles Behaviors”. In: *Proceedings of the 17th International Conference on Software Testing, Verification and Validation Workshops (ICSTW 2024)*. USA: IEEE, 2024, pp. 63–67. DOI: 10.1109/ICSTW60967.2024.00022.
- [Rez+23] Gustavo Rezende Silva, Juliane Päßler, Jeroen Zwanepol, Elvin Alberts, S. Lizeth Tapia Tarifa, Ilias Gerostathopoulos, Einar Broch Johnsen, and Carlos Hernández Corbato. “SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles”. In: *Proceedings of the 18th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2023)*. IEEE, 2023, pp. 181–187. DOI: 10.1109/SEAMS59076.2023.00031.
- [Rom+22] Adrián Romero-Garcés, Renan Salles De Freitas, Rebeca Marfil, Cristina Vicente-Chicote, Jesús Martínez, Juan F. Inglés-Romero, and Antonio Bandera. “QoS metrics-in-the-loop for endowing runtime self-adaptation to robotic software architectures”. In: *Multimedia Tools and Applications* vol. 81, no. 3 (2022), pp. 3603–3628. DOI: 10.1007/S11042-021-11603-7.
- [Sch+06] Pierre-Yves Schobbens, Patrick Heymans, Jean-Christophe Trigaux, and Yves Bontemps. “Feature Diagrams: A Survey and a Formal Semantics”. In: *Proceedings of the 14th International Conference on Requirements Engineering (RE 2006)*. USA: IEEE, 2006, pp. 136–145. DOI: 10.1109/RE.2006.23.
- [Sch+08] Klaus Schmid, Sooyong Park, Mike Hinchey, and Svein Hallsteinsen. “Dynamic Software Product Lines”. In: *IEEE Computer* vol. 41, no. 4 (2008), pp. 93–95. DOI: 10.1109/MC.2008.123.
- [Sch+10] Ina Schaefer, Lorenzo Bettini, Viviana Bono, Ferruccio Damiani, and Nico Tanzarella. “Delta-Oriented Programming of Software Product Lines”. In: *Proceedings of the 14th International Software Product Line Conference (SPLC 2010)*. Ed. by Jan Bosch and Jaejoon Lee. Vol. 6287. Lecture Notes in Computer Science. Springer, 2010, pp. 77–91. DOI: 10.1007/978-3-642-15579-6_6.
- [Sch+12] Ina Schaefer, Rick Rabiser, Dave Clarke, Lorenzo Bettini, David Benavides, Goetz Botterweck, Animesh Pathak, Salvador Trujillo, and Karina Villela. “Software diversity: state of the art and perspectives”. In: *International Journal on Software Tools for Technology Transfer* vol. 14, no. 5 (2012), pp. 477–495. DOI: 10.1007/S10009-012-0253-Y.

- [SF18] Klaas-Jan Stol and Brian Fitzgerald. “The ABC of software engineering research”. In: *ACM Transactions on Software Engineering and Methodology* vol. 27, no. 3 (2018), pp. 1–51. DOI: [10.1145/3241743](https://doi.org/10.1145/3241743).
- [SK16] Bruno Siciliano and Oussama Khatib, eds. *Springer Handbook of Robotics*. Springer Handbooks. Springer, 2016. DOI: [10.1007/978-3-319-32552-1](https://doi.org/10.1007/978-3-319-32552-1).
- [SLR13] Karsten Saller, Malte Lochau, and Ingo Reimund. “Context-Aware DSPLs: Model-Based Runtime Adaptation for Resource-Constrained Systems”. In: *Proceedings of the 17th International Software Product Line Conference (SPLC 2013)*. Vol. 2. USA: ACM, 2013, pp. 106–113. DOI: [10.1145/2499777.2500716](https://doi.org/10.1145/2499777.2500716).
- [SRD17] Gustavo Sousa, Walter Rudametkin, and Laurence Duchien. “Extending Dynamic Software Product Lines with Temporal Constraints”. In: *Proceedings of the 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2017)*. USA: IEEE, 2017, pp. 129–139. DOI: [10.1109/SEAMS.2017.6](https://doi.org/10.1109/SEAMS.2017.6).
- [ST09] Mazeiar Salehie and Ladan Tahvildari. “Self-Adaptive Software: Landscape and Research Challenges”. In: *ACM Transactions on Autonomous and Adaptive Systems* vol. 4, no. 2 (2009), 14:1–14:42. DOI: [10.1145/1516533.1516538](https://doi.org/10.1145/1516533.1516538).
- [Thü+14] Thomas Thüm, Sven Apel, Christian Kästner, Ina Schaefer, and Gunter Saake. “A Classification and Survey of Analysis Strategies for Software Product Lines”. In: *ACM Computing Surveys* vol. 47, no. 1 (2014), 6:1–6:45. DOI: [10.1145/2580950](https://doi.org/10.1145/2580950).
- [Thü+19] Thomas Thüm, André van Hoorn, Sven Apel, Johannes Bürdek, Sinem Getir, Robert Heinrich, Reiner Jung, Matthias Kowal, Malte Lochau, Ina Schaefer, and Jürgen Walter. “Performance Analysis Strategies for Software Variants and Versions”. In: *Managed Software Evolution*. Ed. by Ralf H. Reussner, Michael Goedicke, Wilhelm Hasselbring, Birgit Vogel-Heuser, Jan Keim, and Lukas Martin. Springer, 2019. Chap. 8, pp. 175–206. DOI: [10.1007/978-3-030-13499-0_8](https://doi.org/10.1007/978-3-030-13499-0_8).
- [TR23] Sander Thuijsman and Michel Reniers. “Supervisory Control for Dynamic Feature Configuration in Product Lines”. In: *ACM Transactions on Embedded Computing Systems* (2023). DOI: [10.1145/3579644](https://doi.org/10.1145/3579644).
- [Val+22] Alejandro Valdezate, Rafael Capilla, Jonathan Crespo, and Ramón Barber. “RuVa: A Runtime Software Variability Algorithm”. In: *IEEE Access* vol. 10 (2022), pp. 52525–52536. DOI: [10.1109/ACCESS.2022.3175505](https://doi.org/10.1109/ACCESS.2022.3175505).

-
- [Van+18] Andrea Vandin, Maurice H. ter Beek, Axel Legay, and Alberto Lluch Lafuente. “QFLan: A Tool for the Quantitative Analysis of Highly Reconfigurable Systems”. In: *Proceedings of the 22nd International Symposium on Formal Methods (FM 2018)*. Ed. by Klaus Havelund, Jan Peleska, Bill Roscoe, and Erik de Vink. Vol. 10951. Lecture Notes in Computer Science. Germany: Springer, 2018, pp. 329–337. DOI: [10.1007/978-3-319-95582-7_19](https://doi.org/10.1007/978-3-319-95582-7_19).
- [Wey+12] Danny Weyns, M. Usman Iftikhar, Didac Gil de la Iglesia, and Tanvir Ahmad. “A Survey of Formal Methods in Self-Adaptive Systems”. In: *Proceedings of the 5th International C* Conference on Computer Science and Software Engineering (C3S2E 2012)*. USA: ACM, 2012, pp. 67–79. DOI: [10.1145/2347583.2347592](https://doi.org/10.1145/2347583.2347592).
- [Wey20] Danny Weyns. *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons, 2020.
- [WR22] Claes Wohlin and Austen Rainer. “Is it a case study?—A critical analysis and guidance”. In: *Journal of Systems and Software* vol. 192 (2022). DOI: [10.1016/j.jss.2022.111395](https://doi.org/10.1016/j.jss.2022.111395).

Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Clemens Dubsloff, Einar Broch Johnsen, S. Lizeth Tapia Tarifa

Published in *Formal Aspects of Computing*. Association for Computing Machinery, 2025. DOI: 10.1145/3709159.

Abstract

Improved autonomy in robotic systems is needed for innovation in, e.g., the marine sector. Autonomous robots that are let loose in hazardous environments, such as underwater, need to handle uncertainties that stem from both their environment and internal state. While self-adaptation is crucial to cope with these uncertainties, bad decisions may cause the robot to get lost or even to cause severe environmental damage. Autonomous, self-adaptive robots that operate in uncontrolled environments full of uncertainties need to be reliable! Since these uncertainties are hard to replicate in test deployments, we need methods to formally analyse self-adaptive robots operating in uncontrolled environments. In this paper, we show how feature-oriented techniques can be used to formally model and analyse self-adaptive robotic systems in the presence of such uncertainties. Self-adaptive systems can be organised as two-layered systems with a *managed* subsystem handling the domain concerns and a *managing* subsystem implementing the adaptation logic. We consider a case study of an autonomous underwater vehicle (AUV) for pipeline inspection, in which the managed subsystem of the AUV is modelled as a family of systems, where each family member corresponds to a valid configuration of the AUV which can be seen as an operating mode of the AUV's behaviour. The managing subsystem of the AUV is modelled as a control layer that is capable of dynamically switching between such valid configurations, depending on both environmental and internal uncertainties. These uncertainties are captured in a probabilistic and highly configurable model. Our modelling approach allows us to exploit powerful formal methods for feature-oriented systems, which we illustrate by analysing safety properties, energy consumption, and multi-objective properties, as well as performing parameter synthesis to analyse to what extent environmental conditions affect the AUV. The case study is realised



in the probabilistic feature-oriented modelling language and verification tool ProFeat, and in particular exploits family-based probabilistic and parametric model checking.

III.1 Introduction

More than 70 percent of the surface of Earth is covered in water. The blue economy, which includes expanding sectors such as marine energy, bio-industry and food production, water desalination, mining, underwater construction and ecosystem monitoring, is crucially dependent on underwater vehicles that are able to operate in environments that are inaccessible or hostile to humans. The sustained growth of the blue economy and control of undesirable environmental impacts essentially depends on two factors: increased *autonomy* in underwater robotics and *reliability* of underwater operations, as control shifts from human operators to the robots themselves, expanding their range of activities. However, this growth is slowed down by a gap between what the underwater robotics industry delivers today and the required level of autonomy and reliability. While there is currently a significant push towards increased autonomy, many underwater robots are still controlled by human operators at the mission level. The need for human operators significantly increases cost and restricts the activities in which underwater robots can engage. Due to low bandwidth underwater, such control goes via cable between the underwater robot and the operator on land or on ship.

For autonomous mission control in underwater robotics, a particular challenge stems from the environments in which the robots operate. Generally, these environments are not fully understood and may be subject to change, potentially causing surprises for the robot. For example, a robot performing underwater pipeline inspection may need to change how it performs its mission because of unexpected underwater currents, reduced water visibility, or even a landslide. One approach to dealing with such environments is through *self-adaptation* [Wey20]. Self-adaptation enables the robots to autonomously adapt their mission-level control strategies to the environment in which they operate—during the mission itself. Enabling robots to adapt in this way provides several advantages. Self-adaptive mission control expands the range of scenarios in which the robots are able to operate, resulting in a higher level of autonomy. Self-adaptive mission control has been realised in, e.g., *Metacontrol* [Her13], in which control strategies can be manipulated by activating, deactivating or adjusting them on the fly; MROS [Boz+22] implements Metacontrol for the Robot Operating System (ROS) [Qui+09].

Let us briefly recall how a self-adaptive system (SAS) can be realised using a two-layered architecture which decomposes the system into a *managed* and a *managing* subsystem [KC03], see Figure III.1. The *managed* subsystem deals with the domain concerns and tries to reach the goals set by the system's user, e.g., navigating a robot to a specific location. The *managing* subsystem handles the adaptation concerns and defines an adaptation logic that specifies a strategy

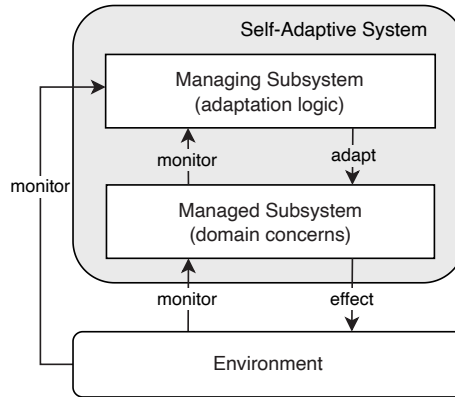


Figure III.1: The architecture of a two-layered SAS with a managed subsystem implementing the application logic and a managing subsystem implementing the adaptation logic ([PäB+23a])

on how the system can fulfil the goals under uncertainty [Wey20], e.g., adapting to changing environmental conditions. While the managed subsystem may affect the environment via its actions, the managing subsystem monitors the environment and the internal state of the managed subsystem. By using the adaptation logic, the managing subsystem decides whether and which changes are needed and adapts the managed subsystem accordingly.

Unless done in the right way, autonomous controllers for underwater vehicles can potentially be both damaging and expensive, as vehicles may get lost and even, in the worst case, cause harm to installations or the marine environment. However, it is cumbersome to validate new designs for autonomous mission-level control of underwater vehicles in situ: vehicles may get lost during validation and only a few real-life scenarios get tested in practice. To address this problem, it is useful to validate the mission control in simulation. This leads to the emergence of configurable simulators that target the mission control layer of robotic software stacks. Using simulation scenarios, the autonomous mission control layer of the underwater robot can be tested and validated much more efficiently. Simulators, such as the open-source exemplars UNDERSEA [Ger+17] and SUAVE [Rez+23], allow scenarios for mission-level control to be tested and compared in simulation, based on a realistic robot software stack, and enable the comparison of different strategies for autonomous mission control.

However, testing and validation might not be enough to assure the system operators that the autonomous robots will operate as desired. Therefore, in addition to testing and validation, rigorous analysis techniques such as model checking [BK08] are often applied in industry [Bee+24] to help provide safety guarantees for these and other (mission-critical) systems even in the presence of uncertainties. Our work considers analysis techniques for autonomous underwater robots, based on model checking, and complements simulation-based validation techniques for autonomous underwater vehicles (AUVs). Specifically,

our goal in this paper is to model and analyse a case study of mission control for an AUV, realised as a two-layered self-adaptive system which was taken from the exemplar SUAVE. To this aim, we organise the adaptation space of the self-adaptive system as a reconfiguration problem, using feature-oriented modelling techniques [Chr+18; Chr+23]. The functionalities of the *managed subsystem* of the AUV are modelled in a feature model, making the dependencies and requirements between the components of the AUV explicit. The behaviour of the managed subsystem is modelled as a featured Markov decision process, i.e., a probabilistic transition system where feature-guarded transitions can only be taken in a system configuration with matching features. Thus, the managed subsystem of the AUV is modelled as a family of systems whose family members correspond to valid feature configurations which can be understood as modes of operation of the AUV. As the behaviour of the AUV depends on environmental and internal conditions, which are both hard to control, we opted for a probabilistic model in which uncontrolled events, like a thruster failure, occur with given probabilities. We model the behaviour of the *managing subsystem* as a control layer that switches between the feature configurations of the managed subsystem according to input from the probabilistic environment model and the managed subsystem. In our case study, we consider an abstract version of an AUV, focusing on core functionalities of the AUV with limited features and variability. The resulting model can be refined and extended in several ways depending on the necessary degree of realism to verify specific requirements. Finally, we model the *environment* as a probabilistic transition system which abstracts from the real environments in which AUVs operate, yet captures essential aspects of changes in the environment that can affect the behaviour of the AUV.

The analysis of the resulting model considers five different aspects: (1) safety guarantees concerning a mission's duration and energy usage that can be used for the deployment of the AUV; (2) safety guarantees concerning unsafe states; (3) the impact of different environments on the safety guarantees; (4) parameter synthesis to analyse to what extent environmental conditions affect the AUV; and (5) trade-offs between mission duration and energy usage. Together, the analyses provide a good indication of the feasibility of model checking as a viable technique to analyse AUVs. The case study is modelled in ProFeat [Chr+18], a tool for probabilistic family-based model checking, which provides a means to simultaneously model check, in a single run, properties of all different configurations in a *family model* [Thü+14]. The analysis of the model was carried out in the backends for ProFeat, in PRISM [KNP11] and Storm [Hen+22], depending on the analysed properties. We close the paper with a discussion about the suitability of family-based modelling techniques for analysing self-adaptive systems such as AUVs, and the relation of these techniques to dynamic *software product lines* (SPLs) [BHA12; DS11; Dub19; GH03; Sch+08].

Contribution. This paper is an extension of the conference publication by Päßler et al. [Päß+23a] and the related artefact [Päß+23b], recently published as an original software publication [Päß+25], which used techniques from SPLs

to model and analyse the case study of a self-adaptive AUV as a two-layered system, providing the following contributions:

- a case study of an SAS from the underwater robotics domain, modelled as a probabilistic feature-guarded transition system with dynamic feature switching [Päß+23a];
- automated verification of (quantitative) properties that are important for roboticists, using family-based analysis [Päß+23a];
- a configurable software model of a dynamic SPL, reflecting the self-adaptive AUV [Päß+23b; Päß+25].

In more detail, this extended version of the work additionally provides the following contributions:

- enabling more variants of the case study which can include another environment and an AUV that can operate at more altitudes (see Section III.4), enabling analysis of how the environment and the number of possible altitudes influence the analysis results;
- extending the analysis with
 - an analysis of how the number of possible altitudes influences the analysis results using PRISM (see Sections III.5.2 and III.5.3),
 - an analysis of how the environments influence the analysis results using PRISM experiments (see Section III.5.4),
 - an analysis of how the probability of currents influences the analysis results using parametric model checking with Storm (see Section III.5.5), and
 - multi-objective queries where we explore achievability, numerical and Pareto queries using Storm (see Section III.5.6);
- a discussion about the suitability of using SPL techniques and tools for modelling and analysing SASs (see Section III.7.1).

To the best of our knowledge, we are the first to showcase the applicability of feature-oriented verification using Storm and performing parameter synthesis.

Outline. Section III.2 presents the case study of pipeline inspection with an AUV, setting up the context of the SAS analysed in this article. The analysis technique of family-based model checking and a brief account on (dynamic) SPLs and ProFeat is provided in Section III.3. Section III.4 explains both the behaviour of the managed and managing subsystem of the AUV and the environment, as well as their implementation in ProFeat. Section III.5 presents quantitative analyses conducted on the case study, involving the tools ProFeat, PRISM, and Storm. Section III.6 summarises related work and Section III.7 discusses our results and ideas for future work.

III.2 Case Study: Pipeline Inspection by AUV

In this section, we introduce the case study of this article, an AUV used for pipeline inspection. The case study is inspired by the case study of the exemplar SUAVE [Rez+23]. In SUAVE, the AUV has been implemented in a simulator using ROS2 [Mac+22] to enable the comparison of different self-adaptation strategies, i.e., different implementations of a managing subsystem. The full SUAVE model would clearly exceed the limits when turning towards a formal analysis. We hence abstracted the case study to its essential characteristics, aiming towards verification of the core functionalities and self-adaptivity of the AUV in various exemplary environments.

As within the SUAVE case study, an AUV has the mission to first find and then follow and inspect a pipeline located on a seabed. For moving in different directions, the AUV has several thrusters. All thrusters are needed for a reliable operation of the AUV, however, thruster failures are possible. Furthermore, the AUV can operate at different altitudes, where a higher altitude provides a wider field of view and thus increases the AUV's chances of finding the pipeline. We assume that switching between altitudes increases the energy consumption. The only aspect of the environment considered in this case study is the water visibility: it reflects the maximum distance at which the AUV can perceive objects; we assume that it changes probabilistically during runtime, e.g., due to currents that swirl up the seabed. By assuming probabilistic behaviour of the environment, we assume to have partial knowledge of the environment which is in line with the fact that it is in general difficult to understand.

The AUV's operation is influenced by thruster failures, the water visibility, and its tasks (searching for the pipeline and following it). When a thruster fails, the AUV starts drifting off its intended path and the thruster has to be restarted before the AUV can continue its mission. Thruster failures are more likely at a lower altitude because seaweed might wrap around the thrusters. If a thruster failure happens while following the pipeline and the AUV drifts too much off its path, the pipeline can be lost and the AUV needs to search for it again. The water visibility determines the maximum altitude at which the AUV can perceive the seabed, and thus the maximum altitude at which it can operate. The higher the visibility, the higher the maximum possible operational altitude. The task determines whether the AUV switches between altitudes or stays at one altitude. When following the pipeline, the AUV does not benefit from a wider field of view and should thus operate at the lowest possible altitude to keep the energy consumption low by not switching between altitudes. However, when searching for the pipeline, the AUV can operate at every altitude.

As in SUAVE, the AUV is separated into a managed and a managing subsystem. The managed subsystem is responsible for searching for and following the pipeline. It has different configurations that consist of a task and an altitude where a configuration can be understood as a mode of operation of the AUV. Thus, the managed subsystem can in fact be seen as a family of systems where each family member corresponds to a system configuration. The managing subsystem chooses a configuration of the managed subsystem depending on the

current task and the water visibility. When following the pipeline, the lowest possible altitude is chosen to decrease the energy consumption. When searching for the pipeline, a higher altitude is preferred because it increases the probability of finding the pipeline and it decreases the probability of a thruster failure. However, the altitudes at which the AUV can operate are determined by the water visibility. The higher the visibility, the higher the maximum operational altitude. Considering, e.g., three possible altitudes at which the AUV can operate, the managed subsystem consists of four family members: one family member for searching at each possible altitude and one for following the pipeline at the lowest altitude. The managing subsystem switches between the first three during the search for the pipeline depending on the current water visibility and activates the last one when the pipeline should be followed. Note that, unlike in SUAVE, we assume that thruster failures are handled by the managed subsystem and not the managing subsystem.

In this article, we present an extension of the first formal modelling of the AUV case study by Päßler et al. [Päß+23a]. While the original case study only considered three possible altitudes, we additionally provide a setting with five altitudes to invest the impact of a more fine-grained implementation of adaptation by the managing subsystem. These settings are hereafter referred to as the *three-altitudes* case study and the *five-altitudes* case study, respectively; SUAVE corresponds to the three-altitudes case study.

III.3 Software Product Lines and Family-Based Model Checking

Software product line (SPL) engineering concerns the management of *commonality* and *variability*, usually defined in terms of core and optional features, across families of software products [Ape+13; CN01; PBL05]. There exist many different definitions of what constitutes a *feature* [CHS08], ranging from “an increment in product functionality” [BBC06] to the approach adopted in this paper, namely “anything users or client programs might want to control about a concept is a feature” [CE00, Chapter 4: Feature Modeling]. Selected features provide a *feature configuration*, from which those that give rise to an actual software product are collected in a set of *valid configurations*. Such sets of valid configurations are commonly modelled in a *variability* or *feature model*, most prominently using structures such as feature diagrams [Kan+90] (see, e.g., [BSE19; Sch+06] for surveys). The actual behaviour of software products to be deployed can be modelled choosing annotative [Cla+13], compositional [Sch+10], or hybrid approaches [Dub19; KA08]. In annotative approaches, the behaviour of the whole SPL is modelled in one behavioural model where parts of behaviour are annotated by features, specifying the behaviour of the products containing this feature. Compositional approaches specify the behaviour of each single feature in an isolated feature module, and the feature modules are then composed to determine the behaviour of the software product. In this paper, we opt for a hybrid approach, where the behaviour of (sets of) features is modelled in feature

modules that may also contain feature guarded behaviour to specify feature interactions or fine-grained variability [Dub19].

Since the number of potential software products is exponential in the number of features, the analysis of SPLs is a challenging task [Thü+14]. Many different analysis approaches for SPLs have been proposed using model checking [Cla+10], static analysis [Bee+22], theorem proving [Thü+12], or testing [BVM16]. A one-by-one analysis, investigating every product in isolation, easily becomes infeasible. Hence, when giving a behavioural model of the SPL, an all-in-one analysis is to be preferred, where the behaviour of all products is modelled in a *family model* and analysed in a single analysis step [Cla+13]. As products of an SPL typically share a large amount of features and behaviour, this family model can usually be compactly represented and analysed by exploiting symbolic representations and analysis techniques [Cla+14]. In this paper, we consider the *SPL model-checking problem*, which was first recognised in the seminal paper of [Cla+10]. It generalises the classical model-checking problem as follows: given a logic formula φ , determine for every valid configuration whether φ is satisfied (and provide a counterexample for each configuration not satisfying φ). The straightforward one-by-one approach to solve this problem is by *product-based model checking*, applying classical model checking to each of the products in the SPL. *Family-based model checking* implements an all-in-one approach where all products of a family model, modelling the combined behaviour of all products, are simultaneously checked [Thü+14]. The answer to the SPL model-checking problem is obtained by projecting the results gained from the family model to single configurations. This approach is also well-suited for analysing reliability or other quantitative aspects of SPLs through *family-based probabilistic model checking* [Bee+20a; DBK15]. Another dimension where the use of family models for SPLs excels is in modelling and analysing *dynamic SPLs* where feature configurations can be switched during runtime [GH03; Hal+13]. Such reconfigurations can be modelled in a similar way as modelling actual behaviour in feature modules, leading to *feature controllers* where each feature corresponds to a variable of the module (see, e.g., [Dub19]). As a family model contains the behaviour of all valid feature configurations, a feature controller can select the possible behaviour of the current feature configuration within the family model during runtime, and change between the behaviours of the products.

The tool ProFeat¹ [Chr+18] implements this kind of analysis for dynamic SPLs through choosing a hybrid modelling approach combined with family-based probabilistic model checking. Thereby, all possible products and reconfigurations between products can be analysed in one run. In particular, ProFeat provides a means to model probabilistic system families in an intuitive specification language as well as perform family-based quantitative analysis on them. It extends the probabilistic model checker PRISM² [KNP11] with functionalities such as family models, feature modules, and the feature controller. Technically,

¹<https://pchrzon.github.io/profeat>

²<https://www.prismmodelchecker.org/manual>

ProFeat performs a translation of the feature-oriented model to PRISM’s input language, which is then analysed with family-based probabilistic model checking. Hence, all of the analysis capabilities of PRISM carry over to ProFeat as well. Further, note that this translational approach also enables the use of other analysis tools supporting PRISM’s input language, e.g., the state-of-the-art probabilistic model checker Storm [Hen+22]. The translation ensures feature-specific properties such as reconfiguration within the valid configurations and respecting the rules for reconfiguration. Analysis results from the model checkers are translated back automatically in a feature-aware representation, including compact symbolic representations through decision diagrams [Chr+18].

III.4 Feature-oriented Modelling of the AUV Case Study

In this section, we show that feature-oriented modelling is well-suited to formally specify the structure and behaviour of the AUV case study. First, we observe that the formal model to describe the behaviour of the AUV has to support quantitative aspects such as failure probabilities, time, and energy consumption. Hence, *probabilistic* feature-oriented modelling approaches [Bee+20a; DBK15] have to be preferred compared to non-probabilistic ones [Bee+16; Cla+13]. Further, the managing subsystem has to be able to switch between configurations of the managed subsystem such that *dynamic reconfigurations* are supported as well [Bee+20a; Chr+18; Van+18]. Due to the clear separation of concerns between behaviour of features and the reconfiguration component, we decided to model the case study with the probabilistic feature-oriented engineering framework ProFeat.

Similar to an SAS, a ProFeat model can be seen as a two-layered model, as illustrated in Figure III.1. The behaviour of a family of systems that differ in their features, such as the managed subsystem of an SAS, can be specified. Then a so-called *feature controller* can activate and deactivate the features during runtime, and thus change the behaviour of the system, such as the managing subsystem of an SAS that changes the configuration of the managed subsystem. Furthermore, the environment can be specified as a separate module that interacts with the managed and managing subsystem. Thus, ProFeat is well suited to model and analyse the case study described in Section III.2.

Technically, a ProFeat model consists of three parts: an obligatory feature model that specifies features and their relations and constraints, obligatory modules that specify the behaviour of the features, and an optional feature controller that activates or deactivates features. This structure is also reflected in our implementation of the two case study variants in ProFeat.³

Both pipeline inspection case studies ultimately specify a Markov decision process, i.e., a state-based operational model that includes non-deterministic and probabilistic state transitions. The probabilities and parameters used in our model are estimates and have not been validated by experiments. This article aims to show the practical feasibility of using feature-oriented modelling

³The models are publicly available [Päß+24].

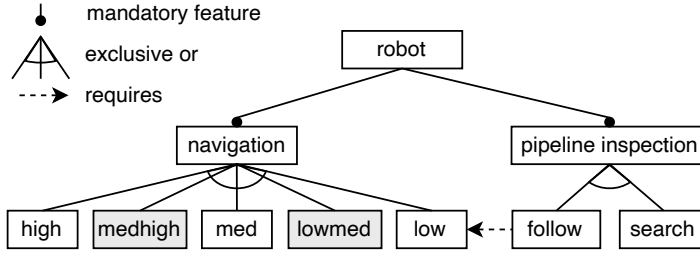


Figure III.2: Feature model of the five-altitudes model; the three-altitudes model does not contain the features with a grey background

for quantitative SAS verification, not to model a realistic AUV. More realistic values can be obtained by doing experiments with a real AUV and by consulting domain experts and are left for future work.

In this section, we follow the modelling structure of the two case study variants in ProFeat. First, we introduce the feature-oriented view of the case study along with its implementation as a feature model in Section III.4.1. Then we explain the behaviour of the managing subsystem, i.e., how the managing subsystem switches between valid feature configurations of the feature model, and its implementation as a feature controller in ProFeat in Section III.4.2. Lastly, we describe the behaviour of the managed subsystem and the environment as well as their implementation as modules in ProFeat in Sections III.4.3 and III.4.4, respectively. For each of the modelling components, we describe both case studies by focusing on the three-altitudes case study and explaining the additions for the five-altitudes case study.

III.4.1 The Feature Model

Feature-Oriented View of the Case Study. The important characteristics of the managed subsystem for self-adaptation are the tasks of the AUV and the possible altitudes. Thus, we consider them to be the AUV's *features*. The features corresponding to the tasks of the AUV are called *search* and *follow*, and the features corresponding to the possible altitudes are called *low*, *med* (for medium), and *high* in the three-altitudes case study with additional features *lowmed* and *medhigh* in the five-altitudes case study. The basis for feature-oriented modelling is the so-called *feature model* that abstractly relates how features can be combined, i.e., which activation of features yields a valid configuration of the system. *Feature diagrams* [Ape+13; Kan+90] constitute the standard feature model formalism. They hierarchically structure features to specify the high-level dependencies between the features. Specifically, an activation of a subfeature implies the activation of its parent feature. To create a feature model for the AUV case study, the altitude features are considered to be subfeatures of the feature *navigation*, the task features to be subfeatures of the feature *pipeline*

inspection, in both cases connected by an exclusive or relation, and *navigation* and *pipeline inspection* are considered to be mandatory subfeatures of the root feature *robot*. Furthermore, since following the pipeline requires a low altitude, the feature *follow* is connected to the feature *low* with a requires relationship. The resulting feature model can be seen in Figure III.2 where the feature model of the three-altitudes case study does not contain the features with the grey background.

Each configuration of the AUV contains the features *robot*, *navigation*, and *pipeline_inspection* as well as exactly one sub-feature for navigation and one for pipeline inspection, while moreover the feature *follow* requires the feature *low*. This yields four different configurations of the managed subsystem of the AUV in the three-altitudes case study (one including the leaf features *search* and *high*, one including *search* and *med*, one including *search* and *low*, and the last one including *follow* and *low*) and two additional configurations in the five-altitudes case study (one including the leaf features *search* and *medhigh* and including one *search* and *lowmed*). Therefore, the valid configurations of the three-altitudes case study are the following (where the leaf features are displayed in bold font):

$\{robot, navigation, pipeline_inspection, \mathbf{search}, \mathbf{high}\},$
 $\{robot, navigation, pipeline_inspection, \mathbf{search}, \mathbf{med}\},$
 $\{robot, navigation, pipeline_inspection, \mathbf{search}, \mathbf{low}\},$
 $\{robot, navigation, pipeline_inspection, \mathbf{follow}, \mathbf{low}\}.$

The five-altitudes case study includes the following configurations in addition to those of the three-altitudes case study:

$\{robot, navigation, pipeline_inspection, \mathbf{search}, \mathbf{medhigh}\},$
 $\{robot, navigation, pipeline_inspection, \mathbf{search}, \mathbf{lowmed}\}.$

By differentiating between configurations depending on the selected features, distinct behaviour of the AUV can be specified depending on the configuration as described in Section III.4.3.

The ProFeat Implementation of the Feature Model. We show how the feature model of the case study is expressed in ProFeat, including connections and constraints among features. Each feature is specified within a **feature ... endfeature** block, and the declaration of the root feature is done in a **root feature ... endfeature** block. An excerpt of the implementation of the root feature of the pipeline inspection case study according to Figure III.2 is displayed in Listing III.1. The root feature can be decomposed into subfeatures; in this case, in only one, the subfeature *robot*, see Line 2. The **all of** keyword indicates that all subfeatures have to be included in the feature configuration if the parent feature, in this case the root feature, is included. It is, e.g., also possible to use the **one of** keyword if exactly one subfeature has to be included, see Line 2 of Listing III.2. The modules modelling the behaviour of the root feature are specified after the keyword **modules**. In our case studies, the root feature is the

```
1 root feature
2   all of robot;
3   modules auv, environment;
4   rewards "time"
5     [step] true : 1;
6   endrewards
7   rewards "energy_3alt"
8     // Costs for being in a recovery state of the three-altitudes model
9     (s=recover_high) : 2;
10    // .. omitted code ..
11
12    // Costs for switching altitudes low-med-high
13    (s=search_high) & active(low) : 4;
14    (s=search_high) & active(med) : 2;
15    (s=found) & active(high) : 4;
16    (s=found) & active(med) : 2;
17    // .. omitted code ..
18  endrewards
19  rewards "energy_5alt"
20    // .. omitted code ..
21
22    // Costs for switching altitudes low-lowmed-med-medhigh-high
23    (s=search_high) & active(medhigh) : 1;
24    (s=search_high) & active(lowmed) : 3;
25    (s=found) & active(medhigh) : 3;
26    (s=found) & active(lowmed) : 1;
27    // .. omitted code ..
28  endfeature
```

Listing III.1: An excerpt of the declaration of the root feature of the five-altitudes model; the root feature of the three-altitudes model does not contain the reward energy_5alt

```
1 feature navigation
2   one of low, lowmed, med, medhigh, high;
3   initial constraint active(low);
4 endfeature
```

Listing III.2: The declaration of the navigation feature of the five-altitudes model; the navigation feature in the three-altitudes model does not contain the features lowmed and medhigh

only feature specifying modules, thus, the behaviour of all features is modelled in the modules `auv` and `environment` described in Sections III.4.3 and III.4.4, respectively.

Contrary to an ordinary feature model, ProFeat allows to specify feature-specific rewards in the declaration of a feature. Reward structures allow to annotate quantitative measures to behaviour and can be interpreted in various ways. Reward structures whose aim is to be reduced are implementing costs such as energy or time. Differently, reward structures with favourable properties usually have an objective to be increased, such as utility, successfully transferred packages, points earned, or alike. Each reward structure is encapsulated in a **rewards ... endrewards** block. In the three-altitudes case study, we consider the rewards `time` and `energy_3alt`, see Lines 4–18 of Listing III.1. Additionally to

these rewards, the five-altitudes case study contains the reward `energy_5alt`, see Lines 19–28. During each transition the AUV module takes, the reward `time` is increased by 1; it is a transition-based reward, see Line 5. We assume that one time step corresponds to one minute, allowing us to compute an estimate of a mission’s duration. The energy rewards are state-based rewards, which are gained when entering a state. We distinguish between the one that takes all states of the three-altitudes model into account (`energy_3alt`), and the one that additionally gives rewards to the states of the five-altitudes model (`energy_5alt`).

Both energy rewards can be used to estimate the necessary battery level for mission completion. Since the reward `energy_3alt` does not take all states of the five-altitudes model into account, using it with the five-altitudes model is only done to compare the two models. In both reward structures, a reward of two energy units is given if a thruster of the AUV failed and needs to be recovered, see, e.g., Line 9. Furthermore, switching between the search altitudes requires significant energy. Since the altitude is switched if the AUV is in a search state and a navigation subfeature that does not correspond to the current search altitude is active, a higher energy reward is given in these states. The given energy reward corresponds to the number of switched altitude levels. For example, if the AUV needs to switch between high and low, lowmed, med, or medhigh altitude, a reward of 4, 3, 2, or 1 energy units, respectively, is given (see Lines 13, 24, 14, or 23, respectively). The reward `energy_3alt` only contains rewards for the switches between `low`, `med`, and `high`, whereas the reward `energy_5alt` contains all possible switches. Since the altitude must be changed to `low` once the pipeline has been found, these cases also receive an energy reward as explained above, see Lines 15–16 and 25–26. All other states receive an energy reward of 1, where the reward `energy_3alt` only assigns rewards to states of the three-altitudes model. We use the function `active` to determine which feature is active, i.e., included in the current feature configuration; given a feature, the function returns true if it is active and false otherwise.

The remainder of the feature model is implemented similarly to the root feature, but the features do not contain feature-specific modules or rewards. The features are implemented and named according to the feature model in Figure III.2. To have only one initial state, we initialise the model with the features `search` and `low` active, using the keyword `initial constraint`, see Line 3 of Listing III.2. As an example of the implementation of another feature, the declaration of the feature `navigation` of the five-altitudes model can be seen in Listing III.2.

III.4.2 The Managing Subsystem

The Behavioural Model of the Managing Subsystem. The managing subsystem implements the AUV’s adaptation logic. Here, adaptation during runtime is achieved by switching between the different configurations of the managed subsystem, activating and deactivating the subfeatures of *navigation* and *pipeline inspection*. Note that adaptations have to adhere to the feature model in Figure III.2, i.e., the resulting feature configuration has to be valid. By

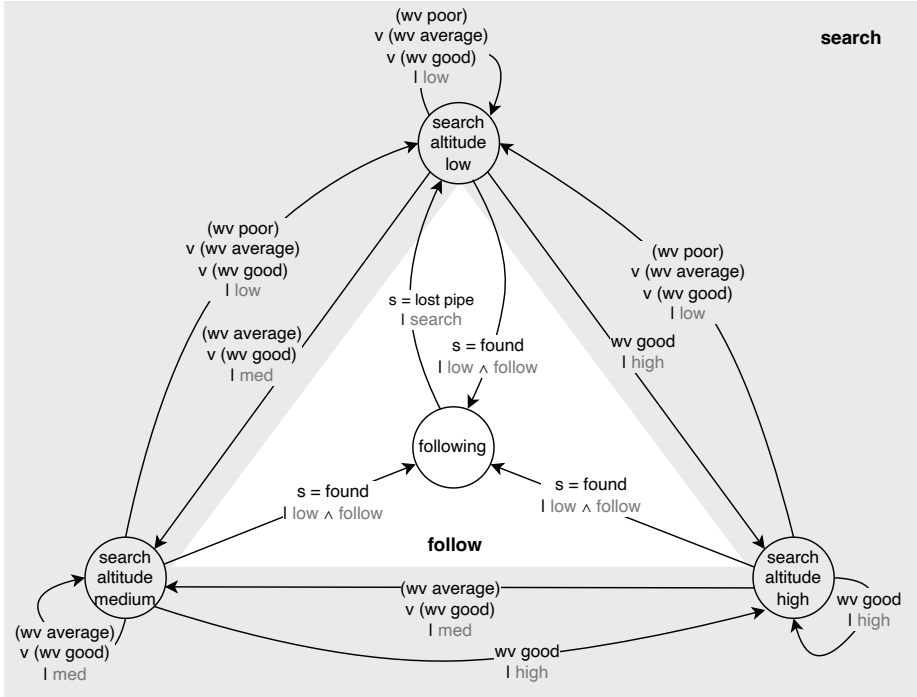


Figure III.3: The behaviour of the managing subsystem of the AUV in the three-altitudes case study ([Päß+23a]); transition guards are written in black, actions in grey following a vertical bar; actions reflect the activated features, conflicting features are deactivated; by (de)activating features, transitions of the managed subsystem are (de)activated

switching between the configurations of the managed subsystem, the managing subsystem enables different behaviour.

The behaviour of the managing subsystem in the three-altitudes model is displayed in Figure III.3. The grey area of the figure includes the transitions that can be taken during the search for the pipeline, and the white area the transitions once the pipeline has been found. Each transition contains a guard, written in black, and an action, written in grey after a vertical bar. The managing subsystem of the five-altitudes model contains two more states, *search altitude lowmed* and *search altitude medhigh*, as well as transitions from these states to each other and to all states in Figure III.3. The actions on these transitions follow the same pattern as in Figure III.3 and the guards are assigned as described below and shown in Table III.1.

During the search for the pipeline, i.e., when the feature *search* is active, reflected in the grey area of Figure III.3 (plus the states *search altitude lowmed* and *search altitude medhigh* and their transitions in the five-altitudes model), the managing subsystem activates and deactivates the subfeatures of *navigation*

Table III.1: The possible altitudes for different water visibilities in the five-altitudes case study, where the water visibility reflects the maximum distance at which the AUV can perceive objects

water visibility (wv)	Possible Altitude				
	low	lowmed	med	medhigh	high
$wv < \text{lowmed_visib}$	✓	✗	✗	✗	✗
$\text{lowmed_visib} \leq wv < \text{med_visib}$	✓	✓	✗	✗	✗
$\text{med_visib} \leq wv < \text{medhigh_visib}$	✓	✓	✓	✗	✗
$\text{medhigh_visib} \leq wv < \text{high_visib}$	✓	✓	✓	✓	✗
$\text{high_visib} \leq wv$	✓	✓	✓	✓	✓

according to the current water visibility (which reflects the maximum distance at which the AUV can perceive objects). To do so, thresholds *lowmed_visib*, *med_visib*, *medhigh_visib*, and *high_visib* that depend on the environment that the AUV operates in are introduced. In the three-altitudes case study, only *low* can be activated if the water visibility is below *med_visib*, both *low* and *med* can be activated if it is between *med_visib* and *high_visib*, and all of them can be activated if it is above *high_visib*. In the five-altitudes case study, the possible altitudes for different water visibilities can be found in Table III.1. In Figure III.3, the activated feature is displayed in grey on the transition, implicitly the other subfeatures of *navigation* and *pipeline_inspection* are deactivated. Note that the transitions in the grey area implicitly carry the guard $s \neq \text{found}$, i.e., the AUV is not in the state *found*, because they represent the transitions during the search for the pipeline. This guard was omitted for better readability.

Once the pipeline has been found, i.e., the managed subsystem is in the state *found*, one of the transitions in the white area (plus the transitions from *search altitude lowmed* and *search altitude medhigh* to *following* in the five-altitudes model), guarded by $s = \text{found}$, is taken. These transitions include the action of activating *low* and *follow*, and thus deactivating the other navigation subfeatures and *search*. When the AUV loses the pipeline, i.e., it is in the state *lost pipe*, the managing subsystem activates *search* and deactivates *follow*. Since the AUV is following the pipeline at a low altitude, the AUV will start searching at a low altitude.

The Implementation of the Managing Subsystem in ProFeat. As usual in the PRISM input language and thus in ProFeat as well, behaviour is specified in *modules*, containing a set of *probabilistic guarded commands* of the following form:

[action] guard -> prob_1: update_1 + ... + prob_n: update_n;

A command may have an optional label *action* to annotate it or to synchronise with other modules. In PRISM, the *guard* is a predicate over global and local variables of the model, which can also come from other modules. ProFeat extends the capability of guards to also support a predicate *active* that yields

```

1 formula lowmed_visib = (4*min_visib+max_visib)/5;
2 formula med_visib = (3*min_visib+2*max_visib)/5;
3 formula medhigh_visib = (2*min_visib+3*max_visib)/5;
4 formula high_visib = (min_visib+4*max_visib)/5;
5
6 controller
7   // Change altitude depending on water visibility
8   [step] (s!=found) & active(search) & water_visib < med_visib
9     -> activate(low) & deactivate(high) & deactivate(med);
10  [step] (s!=found) & active(search)
11    & med_visib <= water_visib & water_visib < high_visib
12    -> activate(low) & deactivate(med) & deactivate(high);
13  [step] (s!=found) & active(search)
14    & med_visib <= water_visib & water_visib < high_visib
15    -> activate(med) & deactivate(low) & deactivate(high);
16  // .. omitted code ..
17
18  // Switch task from "search" to "follow"
19  [step] (s=found) & active(search)
20    -> deactivate(search) & activate(follow) & activate(low)
21    & deactivate(med) & deactivate(high);
22
23  // Switch task from "follow" to "search"
24  [step] (s=lost_pipe) & active(follow)
25    -> deactivate(follow) & activate(search);
26
27  // Enable transitions when following the pipeline
28  [step] (s!=lost_pipe) & active(follow) -> true;
29 endcontroller

```

Listing III.3: An excerpt of the ProFeat feature controller of the three-altitudes model

whether a given feature is active or not. If the guard is true, then the system state is changed with probability prob_i using update_i for all i . An update describes how the system should perform a transition by giving new values for variables, either directly or as a function using other variables. To model feature switches, ProFeat provides a specific module, called *feature controller*, that specifies commands over features. Through keywords *activate* and *deactivate*, the feature controller can activate and deactivate features in the *update* of a command. Several features can be activated and deactivated simultaneously. If the resulting feature configuration does not adhere to the feature model, then ProFeat blocks this transition.

Due to its close correspondence, the managing subsystem of the AUV is implemented as a feature controller in ProFeat. In the pipeline inspection case study, subfeatures of *navigation* (i.e., the different altitudes at which the AUV can operate) and subfeatures of *pipeline_inspection* (i.e., the tasks the robot has to fulfil) can be switched by the feature controller during runtime, see Listing III.3 for an excerpt of the three-altitudes feature controller. The feature controller of the five-altitudes model is implemented similarly, respecting the restrictions in Table III.1. We will describe the implementation of the three-altitudes model here, briefly highlighting additions in the five-altitudes model in brackets.

When the feature *search* is active and the pipeline has not been found yet, the feature controller activates and deactivates the altitudes non-deterministically,

but according to the current water visibility, as described before. Whether or not the pipeline has been found is reflected in the state `s` of the managed subsystem (as described in Section III.4.3) which the feature controller accesses. The minimum and maximum water visibility can be set by the user during design time and influence the altitudes associated with the features `low`, `med` and `high` (plus `lowmed` and `medhigh` in the five-altitudes model); i.e., it influences when the feature controller is able to switch features. To reflect this, the variables `med_visib` and `high_visib` are declared as in Lines 2 and 4 (plus the variables `lowmed_visib` and `medhigh_visib` in Lines 1 and 3 in the five-altitudes model). A *formula* in PRISM and ProFeat can be used to assign an identifier to an expression. If the water visibility is less than `med_visib`, the feature controller activates the feature `low` (see Lines 8–9) because the AUV cannot perceive the seabed from a higher altitude. The variable `water_visib` reflects the current water visibility and is declared in the environment module which also updates this variable, see Section III.4.4. If the water visibility is between `med_visib` and `high_visib`, the feature controller chooses non-deterministically between `low` and `med` (see Lines 10–15), whereas it chooses non-deterministically between all three altitudes if the water visibility is above `high_visib`. (In the five-altitudes model, it activates the features non-deterministically according to Table III.1.) Note that it is also possible to deactivate or activate a feature if it is already inactive or active, respectively. The computation of `med_visib` and `high_visib` is different from the one in our previous paper to align the three and five-altitudes models.

When the pipeline is found, i.e., the managed subsystem is in state `found`, the feature controller activates the feature `follow` and deactivates `search`, see Lines 19–21. Since the AUV should be at a low altitude while following the pipeline, the feature controller also deactivates the features `high` and `med` (plus `lowmed` and `medhigh` in the five-altitudes model) and activates `low`. If the AUV lost the pipeline, i.e., the managed subsystem is in state `lost_pipe`, the feature controller deactivates `follow` and activates `search` to start the search for the pipeline, see Lines 24–25.

The feature controller synchronises with the `auv` and `environment` modules via the action label `step`. Since all transitions of the modules and feature controller have the same action label, they can only execute a transition if there is a transition with a guard evaluating to true in both modules and in the feature controller. Thus, the feature controller needs to include a transition doing nothing if the feature `follow` is active and the AUV is not in state `lost_pipe`, see Line 28.

Let us reflect on technical details how ProFeat ensures a sound encoding of the SAS into PRISM models. During the translation of the ProFeat model into the PRISM language, features are expressed by standard PRISM variables in the feature controller module, where the value of the variable reflects whether the feature is active or inactive. To ensure that the system adapts only within valid configurations, ProFeat automatically includes several constraints into guards and updates of PRISM commands. The constraints on valid feature configurations are added as guards to every command in the feature controller and to the initial state expression of the model. Further, these constraints are also incorporated into the potential updates of the feature controller (for details,

see [Chr+18]). For every feature module, guards of commands are extended by the corresponding feature to be active, while an additional unblocking command is added that is only enabled if the corresponding feature is inactive (for proofs of this translation to be sound, we refer to [Dub19; Dub21]). These transformations ensure that the SAS is always in a valid configuration during runtime, even when adapting to changing environments.

III.4.3 The Managed Subsystem

The Behavioural Model of the Managed Subsystem. The behaviour of the managed subsystem of the AUV is described as a *featured Markov decision process (fMDP)* [DBK15; Dub21]. Here, each transition results from a guarded command comprising a feature guard and a state guard on the state variables of the system. A transition can be taken if and only if both the feature guard is fulfilled in the current configuration of the managed subsystem and the state guard is fulfilled by the current values of the state variables. The possible configuration switches done by the managing subsystem can resolve the feature guards, leading to an MDP where features are encoded into the state-space of the system [DBK15]. Figure III.4 depicts such an MDP for the AUV in the three-altitudes setting, where some details have been omitted to avoid cluttering (in particular, all probabilities). The details can be obtained from the publicly available model.⁴ The five-altitudes model contains four more states, *search_lowmed*, and *search_medhigh* together with their respective recovery states and the transitions following the same pattern as in the three-altitudes model, which is further explained below. The probabilistic model allows to easily model the possibilities of, e.g., finding and losing the pipeline depending on the system configuration.

The MDP consists of the behaviour of the configurations of the managed subsystem together with the switches between the configurations induced by the managing subsystem. Therefore, we distinguish between two kinds of transitions in Figure III.4: black transitions that model the behaviour of a certain configuration of the managed subsystem and blue transitions that switch between configurations, enabled by the managing subsystem during runtime. The labels *search*, *follow*, *low*, *med*, and *high* on the transitions represent the features that have to be active to execute the respective transition, i.e., they represent the feature guard. The blue transitions between configurations implicitly carry the action to start the task or go to the altitude specified by the feature associated with the transition.

The behaviour of one configuration, i.e., the behaviour associated to a set of features F compliant with the feature model, is modelled by the projection of the MDP on the sub-MDP containing only features in F . More concretely, it is modelled by the MDP consisting of all black transitions with feature guards satisfied by F and the states that these transitions connect. Consider for example the configuration with the features *robot*, *navigation*, *pipeline_inspection*, *low*,

⁴The models are publicly available [Päß+24].

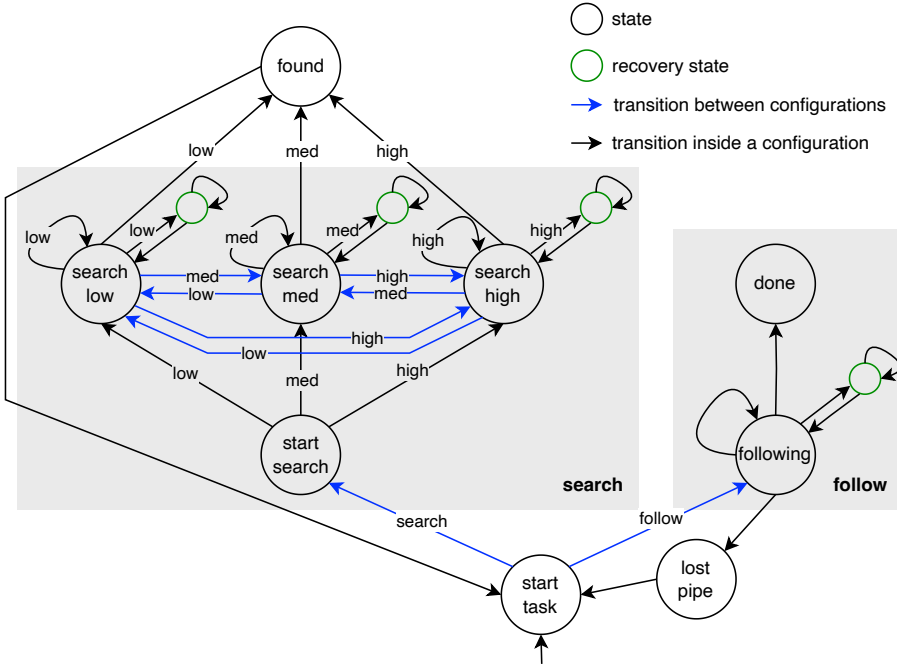


Figure III.4: The behaviour of the configurations of the managed subsystem and the switches between configurations in the three-altitudes case study ([PäB+23a]); transitions with a feature annotation are guarded by the respective feature; transitions without annotation are not guarded by features; blue transitions are enabled by a feature (de)activation of the managing subsystem; black transitions are taken if the managing subsystem did not (de)activate features; probabilities are omitted for better readability

and *search*. Then the behaviour of this configuration is represented by the states *start_task*, *search_low* with its associated recovery state, and *found* as well as the transitions between them and the self-loops of *search_low* and its recovery state. Hence, the behaviour of the MDP corresponding to this configuration consists of the AUV starting its task *search* and going to the state *search_low* where it remains until either a failure occurs and it goes to the associated failure state, or until it finds the pipeline and transitions to the state *found* and then again to *start_task*.

The managing subsystem can switch between configurations (and thus change behaviour) by activating another subfeature of *navigation* during the search for the pipeline, or by activating another subfeature of *pipeline_inspection* if the pipeline was found or lost, reflected by the states *found* and *lost pipe* of the managed subsystem. These switches between configurations are reflected by the blue transitions in Figure III.4. Consider for example the same configuration as before which consists of the features *robot*, *navigation*, *pipeline_inspection*,

low, and *search*. Then the managing subsystem can activate another subfeature of *navigation* during the search of the pipeline, i.e., in the state *search_low* which implies deactivating the feature *low*. If this happens, the black transitions within the current configuration cannot be taken anymore because the feature *low* is inactive. Only the blue transition with the newly activated *navigation* subfeature to a new configuration can be taken. If, for example, the feature *med* has been activated, the blue transition from *search_low* to *search_med* will be taken which implicitly carries the action to go to a medium altitude.

The behaviour of the AUV can then be described as follows. At deployment time, i.e., in state *start task*, the AUV can either immediately start following the pipeline if it was deployed above it, indicated by the feature *follow* in the initial configuration, or start searching for it, indicated by the feature *search* in the initial configuration. During the search for the pipeline, i.e., when the AUV is in the grey area labelled *search*, the managing subsystem might switch between the subfeatures of *navigation* during a transition as described in Section III.4.2 which causes a configuration change of the managed subsystem, reflected by taking a blue transition. If the managing subsystem does not switch between configurations, then the AUV keeps searching for the pipeline, it transitions to a thruster failure state to repair its broken thrusters, or it finds the pipeline and transitions to the state *found*. Once the pipeline has been found, the AUV transitions to the state *start task* again. As described in Section III.4.2, the managing subsystem deactivates the feature *search* during this transition and activates the feature *follow*. When the AUV is following the pipeline, i.e., in the grey area labelled *follow*, it can also lose the pipeline again, transitioning to the state *lost_pipe*, e.g., because of sand covering the pipeline or because the AUV drifted off its path due to thruster failures. In this case, the managing subsystem activates the feature *search* during the transition from *lost_pipe* to *start task* (see Section III.4.2) such that the AUV will start its search again.

Additionally to the states and transitions displayed in Figure III.4, the five-altitudes model contains the states *search_lowmed* and *search_medhigh* with their respective recovery states. The states follow the same pattern as the ones of the three-altitudes model: the search states are connected with configuration changing (blue) transitions to the other search states, guarded by the corresponding feature, and they contain a self-loop and a transition to their respective recovery state, guarded by their respective feature (black transitions). The recovery states each contain a self-loop and a (black) transition back to the corresponding search state.

The ProFeat Implementation of the Managed Subsystem. In ProFeat, each feature can be implemented by several *feature modules*, each comprising a set of guarded probabilistic commands (see Section III.4.2). We encapsulate the behaviour of the managed subsystem of the AUV in a module *auv*. Feature modules in ProFeat can exhibit guards in commands that include the function *active*, evaluating to true if the corresponding feature is active. To this end, configuration-dependent behaviour can easily be specified in feature modules.

```

1 module auv
2   s : [0..12] init start_task;
3   d_insp : [0..inspect] init 0;
4   t_failed : [0..infl_tf] init 0;
5
6   // To the correct task
7   [step] (s=start_task & active(search)) -> 1: (s'=start_search);
8   [step] (s=start_task & active(follow)) -> 1: (s'=following);
9
10  // .. omitted code ..
11  // From search state to another state
12  [step] (s=search_high & active(high))
13    -> 0.59: (s'=found)
14    + 0.4: (s'=search_high)
15    + 0.01: (s'=recover_high);
16  [step] (s=search_high & active(med)) -> 1: (s'=search_med);
17  [step] (s=search_high & active(low)) -> 1: (s'=search_low);
18  // .. omitted code ..
19
20  // Go to other task if pipeline is found
21  [step] (s=found) -> 1: (s'=start_task);
22
23  // Following the pipeline
24  [step] (s=following) & (d_insp<inspect) & (t_failed=0)
25    -> 0.92: (s'=following) & (d_insp'=d_insp+1)
26    + 0.05: (s'=lost_pipe)
27    + 0.03: (s'=recover_following)
28    & (t_failed'=(t_failed<infl_tf? t_failed+1 : t_failed));
29  [step] (s=following) & (d_insp=inspect) & (t_failed>0)
30    -> 0.92*(1-t_failed/infl_tf): (s'=following)
31    & (d_insp'=d_insp+1) & (t_failed'=t_failed-1)
32    + 0.05*(1+((0.92*t_failed)/(0.05*infl_tf))): (s'=lost_pipe)
33    + 0.03: (s'=recover_following)
34    & (t_failed'=(t_failed<infl_tf? t_failed+1 : t_failed));
35  [step] (s=following) & (d_insp=inspect) -> (s'=done);
36
37  // Lost the pipeline
38  [step] (s=lost_pipe) -> 1: (s'=start_task) & (t_failed'=0);
39
40  // Recovery states
41  [step] (s=recover_high) -> 0.5: (s'=recover_high) + 0.5: (s'=search_high);
42  // .. omitted code ..
43 endmodule

```

Listing III.4: An excerpt of the ProFeat AUV module of the case study

Figure III.4 shows the behavior of the module together with feature switches through the feature controller for the three-altitudes model and with the additional states in the five-altitudes model as described before. In this section, we will describe the implementation of the three-altitudes model (see Listing III.4 for an excerpt). As in Section III.4.2, the additions for the five-altitudes model as described above are briefly introduced in brackets.

As in Figure III.4, there are thirteen enumerated states (seventeen in the five-altitudes model) in the ProFeat module with names that correspond to the state labels in the figure. The recovery states are named according to the state they are connected to (e.g., the recovery state connected to `search_high` is called `recover_high`). The variable `s` in Line 2 in Listing III.4 represents the current state of the AUV and is initialised using the keyword `init` with the state `start_task`. To record how many meters of the pipeline have already been inspected, the

variable `d_insp` in Line 3 represents the distance of the pipeline that the AUV has already inspected, it is initialised with 0. The variable `inspect` represents the desired inspection length and it can be set by the user during design time. Since the number of times a thruster failed impacts how much the AUV deviates from its path, the variable `t_failed` can be increased if a thruster fails while the AUV follows the pipeline. It is bounded by the influence a thruster failure can have on the system (`infl_tf`) that can be set by the user during design time.

As an example for the use of the active keyword, consider the command in Lines 12–15, which can be read as follows. If the system is in state `search_high` and the feature `high` is active, then with a probability of 0.59, the system changes its state to `found`, with a probability of 0.4 it changes to `search_high` and with a probability of 0.01 it changes to `recover_high`. These are exactly the black transitions shown in Figure III.4 exiting from state `search_high`. This command also has an action label, `step`. Using this action label, it synchronises with the environment module and the feature controller. The blue transitions exiting state `search_high` in Figure III.4 are modelled in Lines 16–17 (in the five-altitudes model, additional transitions to the states `search_lowmed` and `search_medhigh` with the respective guards are added). If the model is in state `search_high`, but the feature `low` or `med` is active, indicating that the AUV should go to the respective altitude, then the state is changed to the respective search state. The transitions exiting the states `search_med` and `search_low` are modelled similarly (as well as the transitions existing the states `search_lowmed` and `search_medhigh` in the five-altitudes model). However, the probability of going to the state `found` is highest from state `search_high` and lowest from `search_low` because the AUV has a wider field of view when performing the search at a higher altitude. Furthermore, the probability of a thruster failure, i.e., of going to the respective `recover` state, is highest in state `search_low` and lowest in state `search_high` because the probability of seaweed getting stuck in the thrusters is higher at a lower altitude. If the AUV found the pipeline, then a transition to `start_task` is taken, see Line 21.

From the state `start_task`, a transition to either `start_search` or `following` can be taken, depending on which subfeature of `pipeline_inspection` is currently active, see Lines 7–8.

From the `following` state, the transitions that can be taken depend on the variables `d_insp` and `t_failed`. Lines 24–28 consider the case where the distance of the pipeline that has already been inspected (`d_insp`) is less than the distance the pipeline should be inspected (`inspect`) and the variable `t_failed` is 0, indicating that there were no recent thruster failures. Then the AUV stays in the `following` state and inspects the pipeline one more meter, it loses the pipeline, or a thruster fails and it transitions to the failure state and increases `t_failed` if `t_failed` is not at its maximum. Lines 29–34 consider the case where `d_insp` is less than `inspect` and `t_failed` is greater than 0. In this case, the probabilities of following and of losing the pipeline depend on the value of `t_failed`. The bigger the value, the more likely it is to lose the pipeline because it indicates that the AUV's thrusters did not work for some time, causing it to drift off its path. If the already inspected distance is equal to the required inspection distance, the AUV transitions to the `done` state (see Line 35) and finishes the pipeline inspection. If

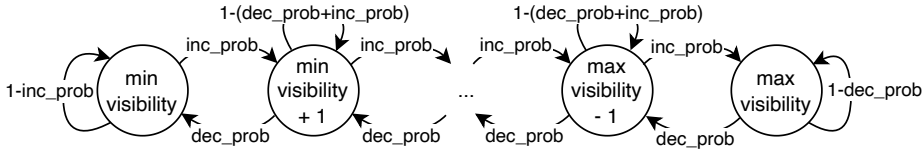


Figure III.5: The behaviour of the environment (modified from [PäB+23a]); the probabilities of increasing and decreasing water visibility (inc_prob and dec_prob , respectively) can be used to model different behaviour of the environment; the minimum and maximum visibility depend on where (in which kind of water conditions) the AUV is deployed

the AUV lost the pipeline (see Line 38), then a transition to `start_task` is taken and the variable `t_failed` is set to 0 again.

When the AUV is in a recovery state, it can either stay there for another time step or exit it again to the state from where the recovery was triggered (see Line 41).

All commands in the module `auv` are labelled with `step`. Thus, every transition receives a time reward of 1, i.e., the time advances with every transition the AUV takes, see Lines 4–6 of Listing III.1.

III.4.4 The Environment

The Behavioural Model of the Environment. The only parameter of the environment we consider in our model is the water visibility. We model it with a value that we assume to be bounded by a minimum and a maximum visibility that depend on where the AUV is deployed and can be set by the user during design time. We assume that changes in the water visibility occur probabilistically with a probability of currents which also depends on the place of deployment of the AUV and can be set by the user during design time. Thereby, we reflect the uncertainty about how the environment will behave. We used two different behavioural models for the environment, a *symmetric* and an *asymmetric* one. Their behaviour is depicted in Figure III.5, where inc_prob denotes the probability of an increasing water visibility and dec_prob the probability of a decreasing water visibility. With the asymmetric behaviour of the environment, the water visibility decreases by 1 with the probability of currents cp , i.e., $dec_prob=cp$, while it stays the same or increases by 1 with probability $(1-cp)/2$, i.e., $inc_prob=(1-cp)/2$. If the water visibility is already at the minimum visibility, the water visibility stays the same with probability $(1+cp)/2 = 1-inc_prob$ and, at maximum visibility, it stays the same with probability $1-cp = 1-dec_prob$. With the symmetric behaviour, we set $inc_prob=dec_prob=cp$ and calculate the cases for minimum and maximum visibility as shown in Figure III.5. The focus of this paper is on modelling and verifying an abstract variant of the AUV, not prioritising a realistic model. Therefore, we chose two simple environment behaviours to analyse if and how the environment can influence the behaviour of the AUV. The environment

```
1 module environment
2   water_visib : [min_visib..max_visib] init round((max_visib-min_visib)/2);
3   [step] true -> dec_prob: (water_visib'=
4     (water_visib=min_visib? min_visib:water_visib-1))
5     + inc_prob: (water_visib'=
6     (water_visib=max_visib? max_visib:water_visib+1))
7     + (1-(dec_prob+inc_prob)): true;
8 endmodule
```

Listing III.5: The ProFeat environment module of the case study, where the variables `dec_prob` and `inc_prob` have to be instantiated to model different behaviours of the environment

model can easily be enhanced with more realistic specifications. Instantiating the values of minimal and maximal visibility as well as probability of currents of the environment together with selecting a desired length of pipeline that should be inspected provides *scenarios* which we will use for analysis, see Section III.5.1.

The Implementation of the Environment in ProFeat. The environment is modelled in a separate environment module, see Listing III.5. The variable `water_visib` in Line 2 reflects the current water visibility and is initialised parametrically, depending on the minimum and maximum visibility. The function `round()` is pre-implemented in the PRISM language and rounds to the nearest integer. The environment module synchronises with the AUV module via the label of its action, `step`. Since the guard of the only action in the environment module is `true`, the environment executes a transition every time the AUV module does. By decoupling the environment module from the AUV module, we obtain a separation of concerns which makes it easier to change the model of the environment if needed.

For the asymmetric environment, the variables for increasing and decreasing the water visibility are instantiated as `inc_prob=(1-current_prob)/2` and `dec_prob=current_prob` whereas for the symmetric environment these variables are instantiated as `inc_prob=dec_prob=current_prob/2` where `current_prob` is the variable reflecting the probability of currents.

III.5 Analysis

ProFeat automatically converts models to the PRISM input language for quantitative analysis, e.g., by statistical or probabilistic model checking. To analyse a PRISM model, properties can be specified in the PRISM property specification language, which includes several probabilistic temporal logics like PCTL [HJ94], CSL [Azi+96], or probabilistic LTL [Bai98]. For family-based analysis, ProFeat extends this specification language to include properties depending on feature selection through the keyword `active`.⁵

⁵ProFeat constructs within PRISM properties are specified using bracketing `${...}` to ensure a well-formed translation to the PRISM property specification language.

The model translated into the PRISM input and property specification language can be interpreted by various (probabilistic) model checking tools. In our experiments, we report on analyses with PRISM and Storm⁶ [Hen+22], depending on the properties to be investigated. Since PRISM works well together with ProFeat, we used it whenever possible. However, for parametric model checking and multi-objective reasoning, Storm is considered as the state-of-the-art [Deh+15; Jun+24], providing in general better functionality and performance. We thus opted to use Storm for parameter synthesis and multi-objective reasoning experiments (see reporting in Sections III.5.5 and III.5.6).

In our analysis, we used the standard settings for both PRISM and Storm. For the parametric analysis with Storm, the chosen parameters are reported in the respective section. All experiments were conducted on a MacBook Pro with an Apple M1 chip and 16GB memory.

The analysis of the model considered five different aspects: (1) the rewards `time`, `energy_3alt` and `energy_5alt` were used to compute safety guarantees that can be used for the deployment of the AUV using PRISM, see Section III.5.2; (2) reachability of safe/unsafe states was analysed using PRISM, see Section III.5.3; (3) we analysed how the different environments introduced in Section III.4.4 influence the analysis results using PRISM, see Section III.5.4; (4) minimal and maximal rewards with unspecified probability of currents were computed to determine the possible range of the rewards using Storm, see Section III.5.5; (5) finally, we analysed trade-off properties between the time and energy rewards using Storm in Section III.5.6. Note that it is not necessary to analyse whether the model satisfies the constraints of the feature model because ProFeat automatically ensures this. Of course, a more complex analysis can be done in addition to the analyses presented in this paper. Our intention in this section is to give a taste of possible analyses that demonstrate the feasibility and applicability of our approach.

III.5.1 Parameter Setup for the Scenarios

The operators used for analysis in this paper are $\mathbf{P}$ and $\mathbf{R}$, which reason about probabilities of events and about expected rewards, respectively. Since we use Markov decision processes (which involve non-determinism), these operators must be further specified to ask for the *minimum* or *maximum* probability and *minimum* or *maximum* expected cost, respectively, for all possible resolutions of non-determinism.

In our analyses, we used four different scenarios, two of them with an undefined value for the probability of currents. The values used in these scenarios are reported in Table III.2. Scenario 1 is in the North Sea, where the minimum and maximum water visibility (in 0.5 meter units) is relatively low and the probability of currents is relatively high. In this case, only 10 meters of the pipeline have to be inspected. Scenario 2 is in the Caribbean Sea with a higher minimum and maximum visibility and a lower probability of currents compared

⁶<https://www.stormchecker.org/index.html>

III. Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

Table III.2: Four different scenarios used for analysis; in Scenarios 3 and 4 the probability of currents `current_prob` is left as a parameter

Scenario	min_visib	max_visib	current_prob	inspect
1 (North Sea)	1	10	0.6	10
2 (Caribbean Sea)	3	20	0.3	30
3 (Parametric North Sea)	1	10	x	10
4 (Parametric Caribbean Sea)	3	20	x	30

to the North Sea, as well as 30 meters of pipeline that have to be inspected. Scenarios 3 and 4, are parametric versions of Scenarios 1 and 2, respectively. That is, they are the North Sea and the Caribbean Sea scenarios, where the value of the variable `current_prob` is left unspecified. We use Scenarios 3 and 4 for the parametric analyses in Sections III.5.4 and III.5.5 and Scenarios 1 and 2 for the other analyses.

The probability of currents of the scenario influences how the water visibility changes. The probabilities for an increasing, decreasing or stagnant water visibility in Scenarios 1 and 2 with symmetric and asymmetric environments are listed in Table III.3. It can be seen that with the symmetric environment, staying at the same water visibility has by far the highest probability in Scenario 2 while the difference between increasing/decreasing and staying at the same water visibility is much smaller in Scenario 1. With the asymmetric environment, however, the situation is reversed. Here, decreasing the water visibility has a much higher probability than increasing or stagnating in Scenario 1, while the difference between the probabilities is very small in Scenario 2, with the probability of an increasing or stagnant water visibility being a bit higher than the probability of decreasing.

Furthermore, the minimum and maximum visibilities of the scenarios influence the thresholds at which the managing subsystem can choose specific altitudes, they are listed in Table III.4 for both scenarios. The altitude difference between the thresholds for Scenarios 1 and 3 is 1.8, while the difference is 3.4 for Scenarios 2 and 4. Therefore, an increasing or decreasing water visibility can cause an increasing or decreasing number of possible altitudes of the AUV much faster in Scenarios 1 and 3 than in Scenarios 2 and 4. Consider for example a situation in the five-altitudes case study where the water visibility is 6.4 which allows all altitudes apart from `high` in Scenarios 1 and 3 and the altitudes `low` and `lowmed` in Scenarios 2 and 4. Then it is possible that `high` can be activated in two time steps in Scenarios 1 and 3 if the water visibility increases in both time steps, but it is only possible to activate `med` after at least four time steps in Scenarios 2 and 4.

We first analysed for Scenarios 1 and 2 in both models whether it is always possible to finish the pipeline inspection, i.e., to reach the state `done`. This could be confirmed since the minimum probability for all resolutions of non-determinism of eventually reaching the state `done` is 1.0.

Table III.3: The probabilities of a water visibility change in Scenarios 1 and 2

Environment	Scenario 1			Scenario 2		
	inc.	dec.	stag.	inc.	dec.	stag.
symmetric	0.30	0.30	0.40	0.15	0.15	0.70
asymmetric	0.20	0.60	0.20	0.35	0.30	0.35

Table III.4: The thresholds for possible altitudes for all four scenarios

Threshold	Scenarios	
	1&3	2&4
lowmed	2.8	6.4
med	4.6	9.8
medhigh	6.4	13.2
high	8.2	16.6

Note that the analysis results of the three-altitudes model with the asymmetric environment reported in the following sections are different from those obtained in our previous paper [Päß+23a] where we also used the asymmetric environment. The difference between the model from our previous paper (hereafter called the “old model”) and the three-altitudes model in this paper are the thresholds at which adaptation can be triggered. In this paper, we divided the space in which the AUV can operate (between `min_visib` and `max_visib`) into five equally big parts to accommodate switches between the five different altitudes in the five-altitudes model. To enable a comparison between the analysis results of the three- and five-altitudes models, we used a subset of the thresholds defined for the five-altitudes model also for the three-altitudes model instead of the thresholds defined in the old model. Thus, the thresholds at which the managing subsystem can switch between altitudes are different, allowing different configurations for the two versions of the model at certain water visibilities. For example, using Scenario 2, the threshold `med_visib` is defined as approximately 5.7 in the old model but as 9.8 in this paper’s models. Similarly, the threshold `high_visib` is defined as approximately 11.3 in the old model and as 16.6 in this paper’s models for Scenario 2. Therefore, using the old model with Scenario 2, the managing subsystem can switch between the features `low` and `med` if the water visibility is above 5.7, increasing the energy consumption. However, in the three-altitudes model of this paper, the managing subsystem has to choose the feature `low` if the water visibility is below 9.8. Similarly, the managing subsystem of the old model can switch between the three altitudes with lower water visibilities than the managing subsystem of the three-altitudes case study of this paper. This enables the managing subsystem of the old model to choose a strategy that results in a higher maximum energy consumption than is possible for the three-altitudes model in this paper because this strategy is not possible for the managing subsystem of this paper. Therefore, the maximum energy consumption with the old model is much higher than the energy consumption with the three-altitudes model of this paper.

```
1 R{"energy_3alt"}min=? [F ${s=done}];  
2 R{"energy_3alt"}max=? [F ${s=done}];
```

Listing III.6: Analysis using the rewards

III.5.2 Reward Properties

The rewards `time`, `energy_3alt`, and `energy_5alt` were used to analyse safety properties related to the execution of the AUV in the model. Since the AUV has only a limited amount of battery, an estimation of the energy needed to complete the mission is required. This ensures that the AUV is only deployed for the mission if it has sufficient battery to complete it. The commands in Listing III.6 were used to compute the minimum and maximum expected `energy_3alt` reward (for all resolutions of non-determinism) to complete the mission. Since the model includes three reward structures, the name of the reward has to be specified in `{"..."}` after the `R` operator. Similarly, the minimum and maximum expected `energy_5alt` and `time` rewards to complete the mission were analysed, the latter to give the system operators an estimate of how much time the mission might take. The execution time of the analysis in PRISM for the three-altitudes case study took at most 3.8 seconds while the analysis for the five-altitudes case study took at most 15.5 seconds. For both case studies, the most time-consuming analysis was analysing the maximum time of Scenario 2 with the asymmetric environment.

The results for the two models in Scenarios 1 and 2 as well as with symmetric and asymmetric environments are reported in Table III.5. For analysing the three-altitudes model, the reward `energy_5alt` was not used and therefore marked as N/A (Not Applicable) because it refers to states that are not present in the three-altitudes model. It can be seen that the variation of the parameters in the two scenarios strongly influences the expected energy and time of the mission. The difference between minimum and maximum expected energy and minimum and maximum expected time for Scenario 2 are significantly bigger than for Scenario 1. In particular, the maximum expected energy and time are much higher for Scenario 2 than for Scenario 1.

Furthermore, the behaviour of the environment has an interesting impact on the analysis results. In Scenario 1, the minimum rewards are slightly higher in the asymmetric environment while the maximum rewards are at least 2.4 times higher with the symmetric environment for all reward structures. However, in Scenario 2, the minimum rewards are slightly higher in the symmetric environment while the maximum rewards are at least 2 times higher with the asymmetric environment. Several factors influence these results and have to be consulted to explain this phenomenon:

- (1) The probability of currents is higher in Scenario 1 than in Scenario 2 which leads to different probabilities for increasing or decreasing the water visibility, see Table III.3;
- (2) The inspection length is longer in Scenario 2 than in Scenario 1;

Table III.5: Expected min-/maximum rewards for completing the mission for the two models concerning both scenarios and environments

Scenario	Environment	Property	3-Altitudes	5-Altitudes
			Result	Result
1	symmetric	energy_3alt	min	24.7839
			max	139.6440
		energy_5alt	min	N/A
			max	N/A
	asymmetric	time	min	23.7951
			max	66.2743
		energy_3alt	min	24.7844
			max	31.4667
2	symmetric	energy_5alt	min	N/A
			max	N/A
		time	min	23.8917
			max	27.1616
	asymmetric	energy_3alt	min	59.0823
			max	382.1825
		energy_5alt	min	N/A
			max	N/A
	symmetric	time	min	56.4871
			max	173.2865
	asymmetric	energy_3alt	min	59.0823
			max	963.8585
		energy_5alt	min	N/A
			max	N/A
	asymmetric	time	min	56.3243
			max	341.9468

- (3) The difference between the minimum and maximum visibility is higher in Scenario 2 than in Scenario 1, which means that the water visibility has to increase or decrease more to allow for more or less possible altitudes, respectively, see Table III.4;
- (4) With the symmetric environment, increasing and decreasing the water visibility has the same probability, whereas with the asymmetric environment, increasing and staying at the same altitude has the same probability.

In Scenario 1, a decrease of the water visibility leads to a decrease in the number of possible altitudes very fast because the thresholds at which different altitudes can be activated are relatively close together compared to Scenario 2, see Table III.4. Therefore, the asymmetric environment, where the probability of decreasing the water visibility is very high for Scenario 1, see Table III.3, does possibly not allow as many altitude switches as the symmetric environment because the water visibility might often be low. This might lead to a higher maximum energy consumption with the symmetric environment since it allows

more altitude switches. The higher maximum time for the symmetric environment can be explained similarly. Time advances with every transition of the combined transition system of the managed and managing subsystem and the environment. Switching between altitudes of the managed subsystem is one such time step during which it is impossible to find the pipeline. Therefore, increasing the switches between altitudes increases the maximum time the AUV takes to finish the pipeline inspection. As described before, altitude switches are more likely in the symmetric environment which thus leads to a higher maximum time reward.

In Scenario 2, the water visibility has to increase much more than in Scenario 1 to allow a higher altitude of the AUV, see Table III.4. With the symmetric environment, the probability of the water visibility staying the same is much higher than an increase or a decrease in the water visibility, see Table III.3. Therefore, it is less likely to increase the water visibility to a point where a higher altitude is allowed, which leads to a higher maximum energy and time with the asymmetric environment (the argumentation is similar to the one for Scenario 1).

It can also be seen that in all experiments, the five-altitudes model enables a mission to happen in less time and with less energy compared to the three-altitudes model, independent of the chosen energy reward. Thus, in these scenarios and with these environment behaviours, it is beneficial to enable the AUV to operate at five instead of three altitudes.

It might seem counter-intuitive that the five-altitudes model with the reward `energy_3alt` does not give the same results as the three-altitudes model with this reward; we give an explanation for this here. Considering the minimum expected energy, searching at a `lowmed` or `medhigh` altitude or switching to and from these altitudes does not cost energy with the `energy_3alt` reward because these altitudes are not part of the three-altitudes model. Therefore, the five-altitudes model, which allows these altitudes, enables a lower minimum `energy_3alt` reward than the three-altitudes model since the managing subsystem can choose a strategy with `lowmed` and `medhigh` altitudes which do not increase the `energy_3alt` reward. Concerning the maximum expected `energy_3alt` reward, we investigated the strategy for obtaining this maximum reward in both models. Interestingly, the strategy in the five-altitudes model includes going to `lowmed` altitude if the water visibility is below `med_visib`. If the water visibility is below `med_visib`, the managing subsystem can hinder the AUV from finding the pipeline by continually switching between `low` and `lowmed`. To understand this, note that there is only a probability of finding the pipeline if the current search altitude matches the active `navigation` subfeature as, e.g., in Lines 12-15 of Listing III.4. If, on the other hand, the current search altitude is different from the active `navigation` subfeature, then the AUV will change with probability 1 to the search state corresponding to the active `navigation` subfeature, see, e.g., Line 16 of Listing III.4, and there is no probability of finding the pipeline during this transition. Therefore, unless the water visibility drops below `lowmed_visib`, the managing subsystem in the five-altitudes model can choose a strategy that increases the `energy_3alt` reward without enabling it to find the pipeline. It can be observed that here, again, the scenario and the behaviour of the environment influence how much impact this

```

1 label "unsafe" = s=recover_high | s=recover_medhigh | s=recover_med
2                 | s=recover_lowmed | s=recover_low | s=recover_following;
3 label "safe" = s=lost_pipe | s=start_task | s=start_search
4               | s=search_high | s=search_medhigh | s=search_med
5               | s=search_lowmed | s=search_low
6               | s=found | s=following | s=done;
7
8 Pmin=? [G "safe"];
9 filter(avg, Pmax=? [ F<=k "unsafe" ], "safe");
10 filter(min, Pmin=? [ F<=k "safe" ], "unsafe");

```

Listing III.7: Analysis of unsafe states

strategy can have on the maximal `energy_3alt` reward. The highest impact can be achieved in Scenario 2 with the asymmetric environment where the maximum `energy_3alt` reward in the five-altitudes model is ≈ 2.86 times higher than in the three-altitudes model.

Note that analysing the maximum rewards is a worst-case analysis. Any reasonable adaptation strategy would not choose to, e.g., maximise the energy consumption by continually switch between altitudes if the goal of the system is to do a pipeline inspection. Nonetheless, it can be interesting to do this worst-case analysis.

In conclusion, the analysis revealed that the scenario and the environment behaviour can have a big impact on the expected rewards. It can therefore be beneficial to evaluate a model with different scenarios and environment behaviours. Furthermore, increasing the granularity of the model by enabling more altitudes at which the AUV can operate provided lower expected minimum rewards in our model, giving an indication that increasing the granularity of the model can be beneficial.

III.5.3 Unsafe States

Thruster failures, although we assume that they can be repaired, pose a threat to the AUV since unforeseen events like strong currents might cause the AUV to be damaged, e.g., by causing it to crash into a rock. Even though the possibility of, e.g., a crash is not modelled in our model, we showcase how thruster failure states can be analysed. To analyse this, the state space was partitioned into two parts, *safe* and *unsafe* states. This was achieved by using labels, see Lines 1–6 of Listing III.7 for the labels of the five-altitudes model. The labels of the three-altitudes model only contain the states present in this model. These labels were then used to calculate the probability of several properties regarding reachability of safe/unsafe states. Each of the analyses in PRISM took less than half a second.

Safe States. The minimum probability of only taking safe states (see Line 8) was shown to be ≈ 0.65 for Scenario 1 and ≈ 0.32 for Scenario 2 in the two models and with both symmetric and asymmetric environments. As expected, the probability of only taking safe states is higher for a shorter pipeline inspection.

It is interesting to see that the probability of only taking safe states is influenced by neither the behaviour of the environment nor the number of possible altitudes of the AUV even though the probability of going to an unsafe state depends on the altitude of the AUV, i.e., it is higher the lower the altitude of the AUV, see Section III.2. The altitude at which the AUV can operate then depends on the environment. However, it seems that both of them do not influence the probability of only taking safe states as the results are always exactly the same.

Safe to Unsafe. We analysed this phenomenon further by analysing the probability of going from a safe state to an unsafe state in the two models with both scenarios and environments. In this way, we hope to find a relation between probabilities concerning safe and unsafe states and the two models and environments. This is analysed with the property in Line 9 of Listing III.7. First, the maximum probability (over all possible resolutions of non-determinism) for reaching an unsafe state from a safe state is calculated, and then the average is taken. PRISM experiments allow analysing this property automatically for a specified range of k , in our experiments, the time steps. The plotted graphs for Scenarios 1 and 2 with the three- and five-altitudes models are displayed in Figure III.6. The values for the symmetric and asymmetric environments were the same so they are not annotated separately.

The graphs show that the probability of reaching an unsafe state from a safe state increases with the number of considered time steps. Furthermore, in Scenario 2, the probability stabilises much later and at a higher value than in Scenario 1. While the average probability of reaching an unsafe state from a safe state stabilises after about 42 time steps at ≈ 0.21 in Scenario 1, it stabilises after about 70 time steps at ≈ 0.42 in Scenario 2. As can be seen from the graphs, the differences between the three- and five-altitudes models are marginal. For example, for Scenario 2, after 50 time steps the probabilities differ by only 0.0022. In conclusion, for these experiments concerning safe and unsafe states, the environment did not influence the calculated probabilities, and the number of possible altitudes only had a minor effect on them.

Unsafe to Safe. It is also important to ensure that a safe state will be reached from an unsafe state after a short time, as, e.g., in Line 10, where k is an integer. For every unsafe state, the minimum probability (for all possible resolutions of non-determinism) of reaching a safe state within k time steps is calculated. Then the minimum over all these probabilities is taken. Thus, it gives the minimum probability of reaching a safe state from an unsafe state in k time steps. PRISM experiments showed that for the two models in both scenarios and environments, the probability of reaching a safe state from an unsafe state is above 0.95 after 5 time steps and above 0.99 after 7 time steps. Thus, for this analysis about safe and unsafe states, the behaviour of the environment and the number of altitudes again does not influence the result.

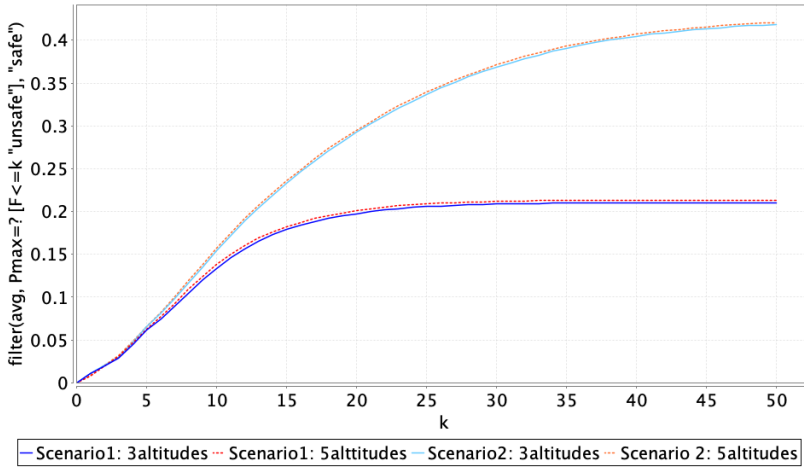


Figure III.6: Results for reaching an unsafe state from a safe state in k time steps for Scenarios 1 and 2 with the three-altitudes and five-altitudes model; more concretely, for a fixed k , the maximum probability of reaching an unsafe state from a safe state is calculated over all possible resolutions of non-determinism, after which the average value is taken

III.5.4 Different Environments

To analyse how distinct implementations of the environment, as described in Section III.4.4, influence the difference between the minimum and maximum rewards for the three- and five-altitudes models, we conducted PRISM experiments with the models for Scenarios 3 and 4. That is, the probability of currents, which is a parameter in Scenarios 3 and 4, was left unspecified, and we asked PRISM to plot the graphs for minimum and maximum energy as in Listing III.6. Each of the experiments in PRISM took less than 35 seconds when concerning a step size of 0.01 for the probability of currents. Considering the same model and scenario, the experiments for minimal energy took less time than the experiments for maximal time, and the experiments with the symmetric environment took less time than the ones with the asymmetric environment.

Figures III.7 and III.8 show the plotted graphs for minimum and maximum energy in Scenario 3, respectively, with the symmetric and asymmetric environments. The dotted lines show the values for the five-altitudes model with `energy_5alt` which assigns rewards to all states, including the intermediate ones. The solid lines show the values for the three- and five-altitudes models with `energy_3alt` which only assigns rewards to states present in both models. The graphs for Scenario 4 look similar and are therefore not displayed here.

It can be seen that the five-altitudes model can achieve a lower expected minimal energy reward with both environments, especially with the `energy_5alt` reward. Therefore, considering the scenarios and environments described in this paper, it seems that enabling the AUV to operate at more altitudes

III. Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

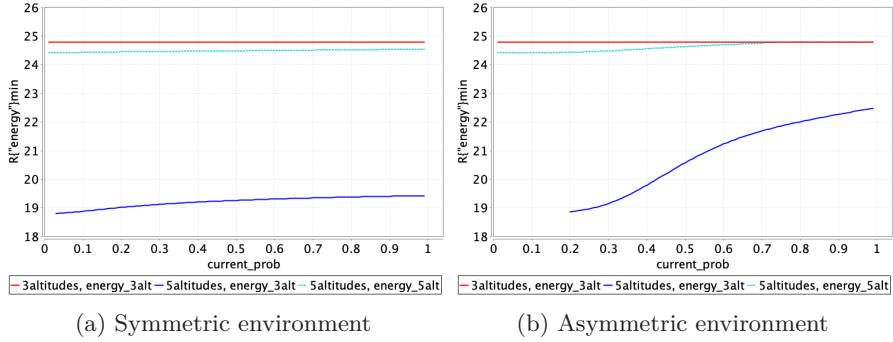


Figure III.7: The graphs for minimum expected energy in Scenario 3 where the probability of currents $current_prob$ is left as a parameter; they show the dependency of the minimum expected energy on the parameter $current_prob$

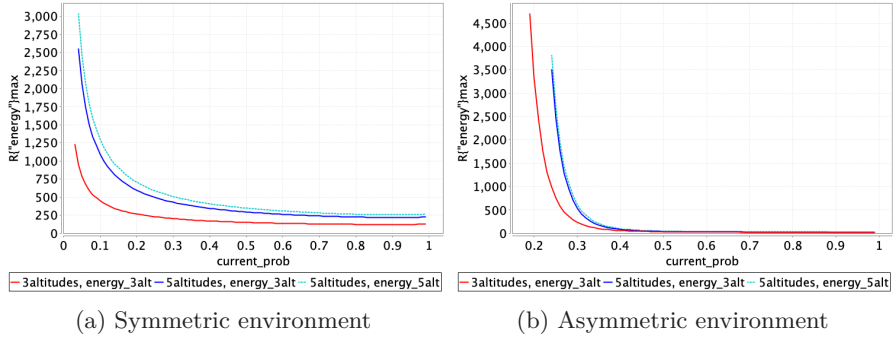


Figure III.8: The graphs for maximum expected energy in Scenario 3 where the probability of currents $current_prob$ is left as a parameter; they show the dependency of the maximum expected energy on the parameter $current_prob$

provides an advantage. However, the influence of the number of altitudes varies with the chosen environment model. With the asymmetric environment, the difference between the three- and five-altitudes model decreases much more with an increasing probability of currents compared to the symmetric environment.

For the maximum expected energy reward, the results for the five-altitudes model are again higher than the ones for the three-altitudes models. However, with increasing probability of currents, the difference between the results of the three- and five-altitudes models decreases, with the asymmetric environment they are almost the same with probabilities of currents above 0.4. Furthermore, for probabilities of currents less than 0.1 with the symmetric environment and less than 0.3 with the asymmetric environment, the maximum expected energy consumption is very high, independent of the chosen model. Therefore, one should consider using a different model of the AUV when employing it in scenarios with low probabilities of currents. If, however, the probability of currents is above 0.3, the experiments suggest that a higher number of possible altitudes for

```

1 storm-pars --prism model.prism --prop "R{\"energy_3alt\"}min=? [F _s = done]"
2   --mode feasibility --feasibility:method pla --direction min
3   --region "0.3<=current_prob<=0.8" --guarantee 0.0001 abs

```

Listing III.8: Analysis of rewards with parametric probability of currents using Storm

the AUV could provide an overall better performance: The minimum expected energy is lower and the maximum expected energy consumption is not so much higher. In future work, it would be interesting to investigate if these results can be replicated for even more possible altitudes.

III.5.5 Reward Properties with Varying Probability of Currents

The probability of currents as specified in our scenarios is only an estimate and can also vary depending on the season. However, the AUV should be able to operate in all different environments with all possible probabilities of currents. Further, it would be useful to have information on the extreme probabilities of currents for the AUV, i.e., obtain worst- and best-case probabilities of currents, to schedule pipeline inspection missions depending on the environmental settings. In this case, it can be useful to leave the probability of currents underspecified, i.e., as a parameter, and analyse the range of minimum and maximum expected rewards depending on the instantiation of the probability of currents. *Parametric model checking* [Hah+10] provides dedicated methods to analyse Markovian models where transition probabilities may involve parameters. While PRISM supports parametric model checking through a reimplementaion of the seminal parametric model checker PARAM [Hah+10], recent advancements in the field are mainly achieved within the Storm model checker [Jun+24]. In fact, already predecessors of the parametric engine of Storm clearly outperformed both, PRISM and PARAM in both functionality and speed [Deh+15]. This could also be witnessed in the following experimental studies on parametrised AUV models, where PRISM was not able to provide results within a reasonable time. With Storm, however, the analysis of all reported properties took less than 11 seconds where the analysis for the three-altitudes model took less than 4 seconds.

In the models for parametric model checking, we left the probability of currents as a single parameter. To analyse the minimum expected `energy_3alt` reward depending on the instantiation of `current_prob`, the command in Listing III.8 was used. The binary `storm-pars` invokes the parametric model-checking engine of Storm that can be used with PRISM files by specifying the PRISM files to be analysed after `--prism`. The property to be analysed can be specified after `--prop`. Here, the same property as specified in Line 1 of Listing III.6 is used. Since the property does not contain a bound for the reward, we have to specify with `--direction min` that the reward should be minimised. That is, for all possible instantiations of `current_prob`, Storm computes the minimal expected reward for `energy_3alt`, providing an interval of values whose

III. Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

Table III.6: Parametric model checking in Scenario 3 with asymmetric environment for parametric probability of currents with $0.3 \leq \text{current_prob} \leq 0.8$

Property	Direction		3-Altitudes		5-Altitudes	
			Result	current_prob	Result	current_prob
energy_3alt	min	min	24.7844	0.55	19.1614	0.300015259
		max	24.7844	0.55	22.0147	0.799992371
	max	min	27.9799	0.799999762	28.0336	0.799999523
		max	239.8359	0.300003815	545.114	0.300007629
energy_5alt	min	min	N/A	N/A	24.4879	0.30012207
		max	N/A	N/A	24.7843	0.76875
	max	min	N/A	N/A	30.8068	0.799998093
		max	N/A	N/A	621.3432	0.300007629
time	min	min	23.7638	0.30012207	23.5507	0.300061035
		max	23.8936	0.7375	23.8937	0.76875
	max	min	25.4847	0.799996185	28.3323	0.799996185
		max	97.5207	0.300003815	226.3386	0.300007629

lower bound is given. The upper bound can be found by using `--direction max`.

Storm supports several different engines for `storm-pars` which are called *modes* and have to be specified after `--mode`. In the experiments, we use the *feasibility* mode to find parameter values such that the specified property holds. In the feasibility mode, either *parameter lifting (pla)* [Qua+16] or *gradient decent (gd)* can be used and specified after `--feasibility:method`. The `--region` option can be used to specify the region of the parameter, and `--guarantee` to specify how close the result should be to the optimal value.

We used Scenario 3 with the asymmetric environment and parametric probability of currents for parametric analysis where we restricted the probability of currents to be between 0.3 and 0.8. The used interval for the probability of currents is an estimate and can be made more realistic by consulting domain experts. Similarly to the command in Listing III.8, we analysed the minimum and maximum rewards for `energy_3alt`, `energy_5alt` and `time` both in minimum and maximum direction, providing an interval of possible values for each of them.

The results of the analysis are reported in Table III.6 and are two-fold: they provide an interval of possible values for each property and optimal values for the probability of currents to minimise or maximise the property. Consider for example minimising the `energy_3` reward for finishing the pipeline inspection as in Line 1 of Listing III.6. The results of the analysis can be found in the first two lines of Table III.6 where the “min direction” provides the lower bound of possible values, and the “max direction” an upper bound. Therefore, in the three-altitudes model, the `energy_3alt` reward is always 24.7844, whereas it is in the interval [19.1614, 22.0147] in the five-altitudes model with the lower bound achieved with `current_prob` ≈ 0.3 and the upper bound achieved with `current_prob` ≈ 0.8 . Thus, the lower and upper bounds in the five-altitudes model are achieved with the minimal and maximal possible value of `current_prob`.

Note that the results of Table III.6 are also reflected in the graphs of Figures III.7b and III.8b. As an example, consider the maximum `energy_3alt` reward in the three-altitudes model. Table III.6 shows that the minimum of all

```

1 multi(R{"time"}<=23.8918 [F _s=done], R{"energy_3alt"}<=24.7845 [F _s = done])
2 multi(R{"time"}min=? [F _s=done ], R{"energy_3alt"}<=24.7845 [F _s = done])
3 multi(R{"time"}min=? [F _s=done ], R{"energy_3alt"}min=? [F _s = done])
4 --multiobjective:exportplot plot/

```

Listing III.9: Multi-objective queries

maximal rewards (with probability of currents between 0.3 and 0.8) is achieved with a probability of currents of almost 0.8 at a value of approximately 24, and the maximum of all maximal rewards is achieved with a probability of currents of ≈ 0.3 at a value of ≈ 239 . The red line in Figure III.8b also shows these values. However, while PRISM experiments provide a way to see how one or several parameters influence the model over the whole parameter space, parametric model checking with Storm focuses on computing only the extreme values. With both techniques, it is possible to specify how close one wants to be to the optimal solution. In the case of parametric model checking with Storm, this can be done explicitly by defining a *guarantee*, while it can only be done implicitly with PRISM experiments by setting the step size for the parameter. When using PRISM experiments, the (close to) optimal result has to be determined by inspecting the calculated values which can be tedious. Parametric model checking with Storm, on the other hand, provides this information by default. Therefore, depending on the required results, one or the other technique should be chosen.

We only display the results for Scenario 3 with the asymmetric environment here to give a taste of the possible analysis. Analysing the same properties with Scenario 3 and the symmetric environment or with Scenario 4 is possible but takes much more time. For example, the analysis in Listing III.8 for Scenario 3 with the asymmetric environment took less than two seconds, while it took approximately 25 minutes for Scenario 4 with the same environment. Since Scenario 4 represents a longer pipeline inspection than Scenario 3, the model is much bigger which could explain this difference. However, further investigation is needed to explain why the analysis with the symmetric environment is so much slower.

It would also be interesting to use parametric model checking with a completely parametric scenario. However, the values of `min_visib` and `max_visib` are used to calculate the thresholds for changing between the search altitudes. Therefore, they cannot be left unspecified, making parametric model checking with a completely parametric scenario unfeasible for our case study.

III.5.6 Multi-Objective Queries

The queries considered until now only considered one property at a time. For example, in Section III.5.2, we asked for the minimum and maximum time and minimum and maximum energy rewards for finishing a pipeline inspection. However, intuitively, there is a trade-off between finishing the pipeline inspection in a short time (minimising the reward `time`) and consuming less energy (minimising the rewards `energy_3alt` and `energy_5alt`). To minimise

time, one wants to go to a higher altitude if possible because the probability of finding the pipeline is higher. On the other hand, switching between altitudes takes a lot of energy and should be avoided when minimising the energy rewards. Therefore, it is interesting to analyse the time and energy rewards in combination, which can be done using *multi-objective queries*.

PRISM and Storm allow three different types of multi-objective queries^{7 8}: (1) *achievability* queries, asking whether it is possible to achieve two properties at the same time, (2) *numerical* queries, leaving the threshold of one property open and asking for the optimal value of the threshold, and (3) *Pareto* queries, where both thresholds are open. To analyse the trade-off between time and energy rewards we employed the probabilistic model checker Storm, as it provided superior functionality and performance for multi-objective analysis compared to PRISM. For all analyses in this section, we used the three-altitudes model with Scenario 1 and asymmetric environment to exemplify the possible analyses. The multi-objective analysis in Storm took at most a bit more than a millisecond.

Achievability Queries. First, we wanted to know if there really is a trade-off between minimising time and energy. To analyse this, we used the property in Line 1 of Listing III.9 for the achievability query. This represents the question: “Does there exist a strategy for finishing the inspection such that the expected time reward is less than or equal to 23.8918 *and* the expected energy_3alt reward is less than or equal to 24.7845?”. The values used in this query are close to the optimal values as reported in Table III.5. The result of this query is *false*, so it is not possible to finish the inspection with both minimal time and minimal energy. However, if we change the values to 23.9 for the reward time and to 24.8 for the reward energy_3alt, then the result is *true*. Thus, even though it is not possible to achieve minimal time and energy, it is almost possible.

Numerical Queries. Using numerical queries, we can determine how close to optimal the strategy can be. More specifically, we ask the following question: “What is the minimal expected time reward for finishing the inspection over all strategies for which the expected energy_3alt reward for finishing the inspection is less than or equal to 24.7845?”, see Line 2 of Listing III.9. The result is ≈ 23.8937 which differs from the optimal result by less than 0.002. It is also interesting to know the *maximum* time reward given an almost optimal energy_3alt reward. This is achieved by replacing $R\{\text{"time"}\}_{\min}$ with $R\{\text{"time"}\}_{\max}$ in Line 2. Interestingly, the result only differs from the minimal time reward by ≈ 0.00007 .

Similarly, we analysed the minimal and maximal expected energy_3alt reward over all strategies for which the expected time reward is less than or equal to 23.8918, which both returned ≈ 24.7893 as a result. Thus, also in this case, the difference between the optimal and the expected value is minimal and the minimal and maximal expected rewards are the same.

⁷<https://www.prismmodelchecker.org/manual/PropertySpecification/Multi-objectiveProperties>

⁸<https://www.stormchecker.org/documentation/background/properties.html#multi-objective-model-checking>

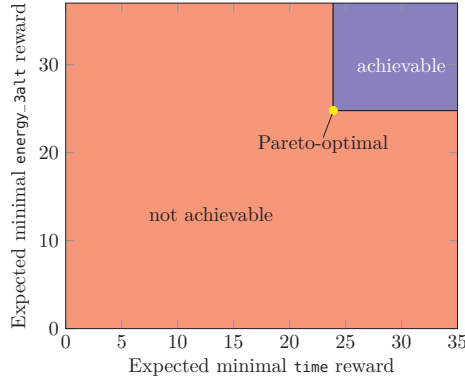


Figure III.9: Achievable points for minimising the rewards `time` and `energy_3alt` with the three-altitudes model, Scenario 1 and asymmetric environment

Pareto Queries. The previous paragraphs described how to determine whether two properties can be achieved at the same time, and how well one property can be achieved given a threshold for the second property. Another interesting problem is to leave the thresholds for both properties open and ask for the *achievable* and *Pareto-optimal* points, we only consider them informally here and refer the interested reader to [FKP12; QJK22] for additional explanations.

Consider the case where we want to minimise two objectives as before in this section. Informally, a point $P = (x, y)$ is *achievable*, if there exists a strategy such that the expected value for the first objective is less or equal to x and the expected value for the second objective is less or equal to y . The point P is *Pareto-optimal* if it is achievable and every point $P' = (x', y') \neq P$ with $x' \leq x$ and $y' \leq y$ is not achievable. To analyse achievable and Pareto-optimal points, we used the property in Lines 3–4 of Listing III.9 which also gives the files for plotting the achievable and Pareto-optimal points as shown in Figure III.9.

The five Pareto-optimal (`time`, `energy_3alt`) points computed by Storm are:

$$\begin{aligned} & (23.89184492, 24.78901283), (23.89174364, 24.7958376), \\ & (23.89373384, 24.78443418), (23.89280945, 24.78621552), \\ & (23.89338681, 24.78505311) \end{aligned}$$

Comparing these pairs with the (non-achievable) optimal pair $(23.89170189, 24.78439125)$, we see that the `time` reward diverges by at most 0.00203195 and the `energy_3alt` reward by at most 0.01144635, and the sum of the pair has at most a difference of 0.0114881 to the optimal pair. Therefore, contrary to our initial intuition, it is almost possible to achieve the optimal result for both minimal time and minimal energy.

III.6 Related Work

The analysis of behavioural requirements is often crucial when developing an SAS that operates in the uncertainty of a physical environment. These requirements often use quantitative metrics that change during runtime. Both rule-based and goal-based adaptation logics can be used to enable the SAS to meet its behavioural requirements. Many practitioners rely on formal methods to provide evidence for the system's compliance with such requirements [Luc+19; Wey+12], also outside the context of SASs [Bee+24], but a plethora of different methods are used [AMV23; Hez+21]. Recently, AUV behaviour was modelled and analysed with timed automata and UPPAAL [QVW24].

III.6.1 SPLs for Self-Adaptive and Robotic Systems

SPLs have previously been proposed to model variability for robotic systems, focusing on static variability modelled during design time [GB14]. In [Bru21], it is argued that most of the costs for robotic systems come from non-reusable software. A robotic system mostly contains software tailored to the specific application and embodiment of the robot, and often even software libraries for common robotic functionalities are not reusable, so they must be re-developed all the time. Thus, they proposed a new approach for the development of robotic software using SPLs. For robotics, the authors in [GB14] propose the toolchain HyperFlex to model robotic systems as SPLs; it supports the design and reuse of reference architectures for robotic systems and was extended with the Robot Perception Specification Language for robotic perception systems in [BH17]. It allows to represent variability at different abstraction levels, and feature models from different parts of the system can be composed in several different ways. However, contrary to the approach used in this paper, HyperFlex only considers design time variability. Furthermore, it is only used for modelling robotic systems, not for analysing them.

Dynamic SPLs (DSPLs) have been proposed to manage variability during runtime [DS11; GH03; Hal+13; HPS12]. Several approaches model SASs as DSPLs to account for runtime variability such as, e.g., [Ben+08; Cor+13a; DGR07; GS10; Hal+06; LM09; SLR13]. These approaches focus on modelling SASs using DSPLs and not on formal analysis. DSPLs have also been proposed to manage variability during runtime in the context of self-adaptive robots [BCH15].

III.6.2 Family-Based Model Checking

The benefits of family-based model checking using an annotative approach are outlined in the aforementioned seminal work on FTSs by Classen et al. [Cla+10; Cla+13; Cla+14]. Properties of FTSs can be verified by dedicated family-based model-checking tools such as SNIP [Cla+12], ProVeLines [Cor+13b], fNuSMV [Cla+14], and FTS4VMC [Bee+22]. Based on this work on FTSs, several other approaches with various modelling languages and formalisms have been proposed [Bee+16; BVM16; FSM19; VBM18]. Furthermore, well-known

model checkers have been made amenable to family-based model checking by suitable abstractions and encodings applied to the input languages of, e.g., PRISM [Chr+18; DBK15], Maude [Loc+16], SPIN [Dim+17; DW17], mCRL2 [Bee+20b; BVW17], and NuSMV [Dim20]. A clear advantage of relying on such well-established and highly optimised model checkers is to avoid having to maintain dedicated SPL model checkers.

Also parametric model checking [Daw04; Hah+10] has been used for the family-based analysis of SPLs [GS11; Rod+15]. While we use the techniques to synthesise optimal parameters for a SAS environment, the existing techniques use probability parameters to encode variability.

In this paper, we used ProFeat [Chr+18], a software tool built on top of the de-facto standard input language of PRISM for the analysis of feature-aware probabilistic models. ProFeat hence enables all functionalities provided by the probabilistic model checkers PRISM [KNP11] and Storm [Hen+22] also for feature-oriented systems, including analyses by statistical, parametric, and probabilistic model checking. Besides ProFeat, also QFLan [Bee+20a; Van+18] offers a powerful modelling language and tool to analyse quantitative SPLs, focusing on probabilistic simulations to yield statistical approximations, trading 100% precision for scalability.

III.6.3 Feature-Oriented Analysis of SASs

Dubslaff et al. [DBK15] modelled and analysed a self-adaptive energy-aware server system using feature-oriented formalisms. There, the feature controller for dynamically switching features only sets constraints on self-adaptation and does not actively perform the adaptation actions as the managing subsystem does in our paper. Chrszon et al. [Chr+20] have modelled and analysed SASs using feature-oriented approaches by means of *multi-product lines*. Specific for their approach is the concept of *roles* that correspond to features but besides activating (“admitting a role”) also distinguish actively performing role actions (“role playing”). While not explicitly separating managed and managing subsystem, their *role manager* takes on the purpose of managing subsystem and allows for multiple managed subsystems (e.g., multiple robots in a production pipeline). This approach has also been extended to the quantitative setting and family-based model checking was used to analyse the reliability of a self-adaptive robotic cell [Chr+23].

III.7 Discussion and Future Work

In this paper, we have demonstrated how a self-adaptive AUV system can be modelled and analysed using feature-oriented approaches. To this aim, we considered a two-layered SAS architecture of a self-adaptive AUV used for pipeline inspection. The variability of the managed subsystem was modelled as a feature model and the behaviour of the resulting (feature) configurations was modelled as a probabilistic (featured) transition system that also encodes the

transitions between configurations. The managing subsystem was modelled as a feature controller switching between features and thereby enabling different behaviour (i.e., operation modes) of the managed subsystem.

We modelled the case study in the tool ProFeat and used it for family-based probabilistic model checking of both reward and safety properties via both PRISM and Storm. In particular, we analysed safety guarantees concerning an AUV's mission duration and energy usage as well as concerning the reachability of (un)safe states, the impact that different environments have on these safety guarantees, to what extent environmental conditions affect the AUV, and trade-offs between mission duration and energy usage. For a more realistic analysis of a AUV, both the models of the AUV and of the environment, and in particular the probabilities, will have to be adapted to the robot and the environment with the help of real data and domain experts. The models will have to be extracted from the real system at the correct level of abstraction for verifying the desired properties. We plan to further investigate this together with an industrial partner of the MSCA network REMARO (Reliable AI for Marine Robotics).

III.7.1 Using SPL Techniques for Modelling and Analysing SASs

In this paper, we have used a feature-oriented approach for modelling and analysing the two-layered self-adaptive AUV. This allowed us to model the managed subsystem of the AUV as a family of systems (or products), where each family member corresponds to a valid configuration of the AUV. The managing subsystem could then be considered as a control layer capable of dynamically switching between these feature configurations depending on both environmental and internal conditions. ProFeat was used for probabilistic family-based model checking, analysing reward and safety properties.

The ProFeat tool allowed us to directly model the two different layers of the SAS, the managed and managing subsystem, which also makes it easier to understand the model as well as the adaptation logic. Furthermore, it made analysing all configurations of the managed subsystem more efficient by enabling family-based (SPL) model checking. However, it remains to be seen how this scales with larger models than the ones considered in this paper. Alternatively, it would also be possible to model the case study without a feature-oriented approach by encoding the features as variables. Besides others, the transitions of the managed and managing subsystem, which would then use these variables, would need to explicitly take care of staying within valid feature configurations. ProFeat conducts this approach in an automated manner, internally transforming the feature-oriented model into a PRISM model agnostic to features. The theoretical underpinning of this transformation requires several technicalities, which are difficult to ensure when done manually [Dub21]. Even in our small case study, the number of transitions in the managed subsystem increased from 44 to 351 when transforming the ProFeat model into a PRISM model. This increase is mainly due to the required synchronisation between the

managed and managing subsystems, where a correct modelling is ensured by the transformations.

A particular aspect of our work is the use of a two-layered model to analyse self-adaptation in the managing subsystem of the AUV. This allows us to not only model and analyse a family of configurations of the managed subsystem by means of family-based model checking, but also model and analyse the self-adaptation strategy by means of a feature controller which switches between these configurations depending on the so-called uncertainties of the uncontrolled environment in which the AUV operates. This feature controller, which dynamically changes between products in the configuration space of the managed subsystem, has similarities to variability management in dynamic SPLs; in our work, we explore this approach to not only model but also analyse self-adaptive behaviour in AUVs. Our case study suggests that family-based modelling and analysis of SASs using ProFeat and (probabilistic) model checking is both a very natural and feasible approach.

The case study in this paper is of course an abstract model of an AUV and its mission. However, we showed through an extensive set of analyses that it is feasible to model and analyse a two-layered self-adaptive cyber-physical system as a family of configurations with a controller switching between them.

III.7.2 Future Work

In the future, we plan to investigate which kinds of SASs can be modelled and analysed following the overall approach applied to this case study, aiming to propose a general methodology for modelling and analysing SASs as family-based systems. Furthermore, we plan to find optimal strategies for the managing subsystem, i.e., the controller switching between features to, e.g., minimise energy consumption. We would also like to find patterns between choosing a certain feature configuration and the effect of this choice on quality criteria for the system. Finding such control patterns could help to improve the adaptation logic of the managing subsystem to be more resilient towards faults. Another interesting research direction would be to include the model into a real system and update, e.g., the probabilities using real-time information. Then the model could be run regularly to re-verify system properties upon model updates. Finally, we would like to investigate how our approach scales with larger system models than the ones considered in this paper.

Acknowledgements. We would like to thank Matthias Volk and Tim Quatmann for answering various questions about Storm and multi-objective model checking.

This work was partially supported by the European Union's Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200), by the Italian project NODES (which has received funding from the MUR – M4C2 1.5 of PNRR with grant agreement no. ECS00000036), by the Italian MUR PRIN 2020TL3X8X project T-LADIES (Typeful Language Adaptation for Dynamic, Interacting and Evolving Systems), by the German DFG under the projects EXC 2050/1 (CeTI, project ID

390696704, as part of Germany’s Excellence Strategy) and TRR 248 (see <https://perspicuous-computing.science>, project ID 389792660), and by the Dutch NWO through Veni grant VI.Veni.222.431.

References

- [AMV23] Hugo Araujo, Mohammad Reza Mousavi, and Mahsa Varshosaz. “Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review”. In: *ACM Transactions on Software Engineering and Methodology* vol. 32, no. 2 (2023), 51:1–51:61. DOI: 10.1145/3542945.
- [Ape+13] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Germany: Springer, 2013. DOI: 10.1007/978-3-642-37521-7.
- [Azi+96] Adnan Aziz, Kumud Sanwal, Vigyan Singhal, and Robert K. Brayton. “Verifying Continuous Time Markov Chains”. In: *International Conference on Computer Aided Verification (CAV)*. Ed. by Rajeev Alur and Thomas A. Henzinger. Vol. 1102. Lecture Notes in Computer Science. Germany: Springer, 1996, pp. 269–276. DOI: 10.1007/3-540-61474-5_75.
- [Bai98] Christel Baier. *On Algorithmic Verification Methods for Probabilistic Systems*. Universität Mannheim. Habilitation thesis. 1998.
- [BBC06] Don Batory, David Benavides, and Antonio Ruiz Cortés. “Automated Analysis of Feature Models: Challenges Ahead”. In: *Communications of the ACM* vol. 49, no. 12 (2006), pp. 45–47. DOI: 10.1145/1183236.1183264.
- [BCH15] Davide Brugali, Rafael Capilla, and Mike Hinchey. “Dynamic Variability Meets Robotics”. In: *IEEE Computer* vol. 48, no. 12 (2015), pp. 94–97. DOI: 10.1109/MC.2015.354.
- [Bee+16] Maurice H. ter Beek, Alessandro Fantechi, Stefania Gnesi, and Franco Mazzanti. “Modelling and Analysing Variability in Product Families: Model Checking of Modal Transition Systems with Variability Constraints”. In: *Journal of Logical and Algebraic Methods in Programming* vol. 85, no. 2 (2016), pp. 287–315. DOI: 10.1016/j.jlamp.2015.11.006.
- [Bee+20a] Maurice H. ter Beek, Axel Legay, Alberto Lluch Lafuente, and Andrea Vandin. “A Framework for Quantitative Modeling and Analysis of Highly (Re)configurable Systems”. In: *IEEE Transaction on Software Engineering* vol. 46, no. 3 (2020), pp. 321–345. DOI: 10.1109/TSE.2018.2853726.

- [Bee+20b] Maurice H. ter Beek, Sijf van Loo, Erik P. de Vink, and Tim A.C. Willemse. “Family-Based SPL Model Checking Using Parity Games with Variability”. In: *Proceedings of the 23rd International Conference on Fundamental Approaches to Software Engineering (FASE 2020)*. Ed. by Heike Wehrheim and Jordi Cabot. Vol. 12076. Lecture Notes in Computer Science. Germany: Springer, 2020, pp. 245–265. DOI: [10.1007/978-3-030-45234-6_12](https://doi.org/10.1007/978-3-030-45234-6_12).
- [Bee+22] Maurice H. ter Beek, Ferruccio Damiani, Michael Lienhardt, Franco Mazzanti, and Luca Paolini. “Efficient Static Analysis and Verification of Featured Transition Systems”. In: *Empirical Software Engineering* vol. 22, no. 1 (2022), 10:1–10:43. DOI: [10.1007/s10664-020-09930-8](https://doi.org/10.1007/s10664-020-09930-8).
- [Bee+24] Maurice H. ter Beek, Rod Chapman, Rance Cleaveland, Hubert Garavel, Rong Gu, Ivo ter Horst, Jeroen J. A. Keiren, Thierry Lecomte, Michael Leuschel, Kristin Yvonne Rozier, Augusto Sampaio, Cristina Seceleanu, Martyn Thomas, Tim A. C. Willemse, and Lijun Zhang. “Formal Methods in Industry”. In: *Formal Aspects of Computing* (2024). DOI: [10.1145/3689374](https://doi.org/10.1145/3689374).
- [Ben+08] Nelly Bencomo, Peter Sawyer, Gordon S. Blair, and Paul Grace. “Dynamically Adaptive Systems are Product Lines too: Using Model-Driven Techniques to Capture Dynamic Variability of Adaptive Systems”. In: *Proceedings of the 12th International Conference on Software Product Lines (SPLC 2008)*. Ed. by Steffen Thiel and Klaus Pohl. Vol. 2. Ireland: Lero, University of Limerick, 2008, pp. 23–32.
- [BH17] Davide Brugali and Nico Hochgeschwender. “Managing the Functional Variability of Robotic Perception Systems”. In: *Proceedings of the 1st International Conference on Robotic Computing (IRC 2017)*. USA: IEEE, 2017, pp. 277–283. DOI: [10.1109/IRC.2017.20](https://doi.org/10.1109/IRC.2017.20).
- [BHA12] Nelly Bencomo, Svein O. Hallsteinsen, and Eduardo Santana de Almeida. “A View of the Dynamic Software Product Line Landscape”. In: *IEEE Computer* vol. 45, no. 10 (2012), pp. 36–41. DOI: [10.1109/MC.2012.292](https://doi.org/10.1109/MC.2012.292).
- [BK08] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. USA: MIT Press, 2008.
- [Boz+22] Darko Bozhinoski, Mario Garzon Oviedo, Nadia Hammoudeh Garcia, Harshavardhan Deshpande, Gijs van der Hoorn, Jon Tjerngren, Andrzej Wąsowski, and Carlos Hernández Corbato. “MROS: Runtime Adaptation for Robot Control Architectures”. In: *Advanced Robotics* vol. 36, no. 11 (2022), pp. 502–518. DOI: [10.1080/01691864.2022.2039761](https://doi.org/10.1080/01691864.2022.2039761).

- [Bru21] Davide Brugali. “Software Product Line Engineering for Robotics”. In: *Software Engineering for Robotics*. Ed. by Ana Cavalcanti, Brijesh Dongol, Rob Hierons, Jon Timmis, and Jim Woodcock. Germany: Springer, 2021, pp. 1–28. DOI: 10.1007/978-3-030-66494-7_1.
- [BSE19] Maurice H. ter Beek, Klaus Schmid, and Holger Eichelberger. “Textual Variability Modeling Languages: An Overview and Considerations”. In: *Proceedings of the 23rd International Systems and Software Product Line Conference (SPLC 2019)*. Vol. 2. USA: ACM, 2019, 82:1–82:7. DOI: 10.1145/3307630.3342398.
- [BVM16] Harsh Beohar, Mahsa Varshosaz, and Mohammad Reza Mousavi. “Basic Behavioral Models for Software Product Lines: Expressiveness and Testing Pre-orders”. In: *Science of Computer Programming* vol. 123 (2016), pp. 42–60. DOI: 10.1016/j.scico.2015.06.005.
- [BVW17] Maurice H. ter Beek, Erik P. de Vink, and Tim A. C. Willemse. “Family-Based Model Checking with mCRL2”. In: *Proceedings of the 20th International Conference on Fundamental Approaches to Software Engineering (FASE 2017)*. Ed. by Marieke Huisman and Julia Rubin. Vol. 10202. Lecture Notes in Computer Science. Germany: Springer, 2017, pp. 387–405. DOI: 10.1007/978-3-662-54494-5_23.
- [CE00] Krzysztof Czarnecki and Ulrich W. Eisenecker. *Generative Programming: Methods, Tools, and Applications*. USA: Addison-Wesley, 2000.
- [Chr+18] Philipp Chrszon, Clemens Dubslaff, Sascha Klüppelholz, and Christel Baier. “ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking”. In: *Formal Aspects of Computing* vol. 30, no. 1 (2018), pp. 45–75. DOI: 10.1007/s00165-017-0432-4.
- [Chr+20] Philipp Chrszon, Christel Baier, Clemens Dubslaff, and Sascha Klüppelholz. “From Features to Roles”. In: *Proceedings of the 24th International Systems and Software Product Line Conference (SPLC 2020)*. USA: ACM, 2020, 19:1–19:11. DOI: 10.1145/3382025.3414962.
- [Chr+23] Philipp Chrszon, Christel Baier, Clemens Dubslaff, and Sascha Klüppelholz. “Interaction Detection in Configurable Systems – A Formal Approach Featuring Roles”. In: *Journal of Systems and Software* vol. 196 (2023), p. 111556. DOI: 10.1016/j.jss.2022.111556.
- [CHS08] Andreas Classen, Patrick Heymans, and Pierre-Yves Schobbens. “What’s in a Feature: A Requirements Engineering Perspective”. In: *International Conference on Fundamental Approaches to Software Engineering (FASE)*. Ed. by José Luiz Fiadeiro and Paola Inverardi. Vol. 4961. Lecture Notes in Computer Science. Germany: Springer, 2008, pp. 16–30. DOI: 10.1007/978-3-540-78743-3_2.

- [Cla+10] Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, Axel Legay, and Jean-François Raskin. “Model Checking Lots of Systems: Efficient Verification of Temporal Properties in Software Product Lines”. In: *Proceedings of the 32nd International Conference on Software Engineering (ICSE 2010)*. USA: ACM, 2010, pp. 335–344. DOI: 10.1145/1806799.1806850.
- [Cla+12] Andreas Classen, Maxime Cordy, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Model Checking Software Product Lines with SNIP”. In: *International Journal on Software Tools for Technology Transfer* vol. 14, no. 5 (2012), pp. 589–612. DOI: 10.1007/s10009-012-0234-1.
- [Cla+13] Andreas Classen, Maxime Cordy, Pierre-Yves Schobbens, Patrick Heymans, Axel Legay, and Jean-François Raskin. “Featured Transition Systems: Foundations for Verifying Variability-Intensive Systems and Their Application to LTL Model Checking”. In: *IEEE Transaction on Software Engineering* vol. 39, no. 8 (2013), pp. 1069–1089. DOI: 10.1109/TSE.2012.86.
- [Cla+14] Andreas Classen, Maxime Cordy, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Formal Semantics, Modular Specification, and Symbolic Verification of Product-Line Behaviour”. In: *Science of Computer Programming* vol. 80 (2014), pp. 416–439. DOI: 10.1145/2499777.2499781.
- [CN01] Paul Clements and Linda Northrop. *Software Product Lines: Practices and Patterns*. USA: Addison-Wesley, 2001.
- [Cor+13a] Maxime Cordy, Andreas Classen, Patrick Heymans, Axel Legay, and Pierre-Yves Schobbens. “Model Checking Adaptive Software with Featured Transition Systems”. In: *Assurances for Self-Adaptive Systems: Principles, Models, and Techniques*. Ed. by Javier Cámara, Rogério de Lemos, Carlo Ghezzi, and Antónia Lopes. Vol. 7740. Lecture Notes in Computer Science. Germany: Springer, 2013, pp. 1–29. DOI: 10.1007/978-3-642-36249-1_1.
- [Cor+13b] Maxime Cordy, Andreas Classen, Patrick Heymans, Pierre-Yves Schobbens, and Axel Legay. “ProVeLines: A Product Line of Verifiers for Software Product Lines”. In: *Proceedings of the 17th International Software Product Line Conference (SPLC 2013)*. Vol. 2. USA: ACM, 2013, pp. 141–146. DOI: 10.1145/2499777.2499781.
- [Daw04] Conrado Daws. “Symbolic and Parametric Model Checking of Discrete-Time Markov Chains”. In: *International Colloquium on Theoretical Aspects of Computing (ICTAC)*. Ed. by Zhiming Liu and Keijiro Araki. Vol. 3407. Lecture Notes in Computer Science. Germany: Springer, 2004, pp. 280–294. DOI: 10.1007/978-3-540-31862-0_21.

III. Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

- [DBK15] Clemens Dubsloff, Christel Baier, and Sascha Klüppelholz. “Probabilistic Model Checking for Feature-Oriented Systems”. In: *Transactions on Aspect-Oriented Software Development XII*. Ed. by Shigeru Chiba, Éric Tanter, Erik Ernst, and Robert Hirschfeld. Vol. 8989. Lecture Notes in Computer Science. Germany: Springer, 2015, pp. 180–220. DOI: [10.1007/978-3-662-46734-3_5](https://doi.org/10.1007/978-3-662-46734-3_5).
- [Deh+15] Christian Dehnert, Sebastian Junges, Nils Jansen, Florian Corzilius, Matthias Volk, Harold Bruintjes, Joost-Pieter Katoen, and Erika Ábrahám. “PROPhESY: A PRObabilistic ParamETER SYnthesis Tool”. In: *Proceedings of the 27th International Conference on Computer Aided Verification (CAV 2015)*. Ed. by Daniel Kroening and Corina S. Pasareanu. Vol. 9206. Lecture Notes in Computer Science. Germany: Springer, 2015, pp. 214–231. DOI: [10.1007/978-3-319-21690-4_13](https://doi.org/10.1007/978-3-319-21690-4_13).
- [DGR07] Deepak Dhungana, Paul Grünbacher, and Rick Rabiser. “Domain-Specific Adaptations of Product Line Variability Modeling”. In: *Proceedings of the IFIP WG 8.1 Working Conference on Situational Method Engineering: Fundamentals and Experiences (ME 2007)*. Ed. by Jolita Ralyté, Sjaak Brinkkemper, and Brian Henderson-Sellers. Vol. 244. IFIP. Germany: Springer, 2007, pp. 238–251. DOI: [10.1007/978-0-387-73947-2_19](https://doi.org/10.1007/978-0-387-73947-2_19).
- [Dim+17] Aleksandar S. Dimovski, Ahmad Salim Al-Sibahi, Claus Brabrand, and Andrzej Wąsowski. “Efficient Family-Based Model Checking via Variability Abstractions”. In: *International Journal on Software Tools for Technology Transfer* vol. 5, no. 19 (2017), pp. 585–603. DOI: [10.1007/s10009-016-0425-2](https://doi.org/10.1007/s10009-016-0425-2).
- [Dim20] Aleksandar S. Dimovski. “CTL* Family-Based Model Checking using Variability Abstractions and Modal Transition Systems”. In: *International Journal on Software Tools for Technology Transfer* vol. 22, no. 1 (2020), pp. 35–55. DOI: [10.1007/s10009-019-00528-0](https://doi.org/10.1007/s10009-019-00528-0).
- [DS11] Ferruccio Damiani and Ina Schaefer. “Dynamic Delta-oriented Programming”. In: *Proceedings of the 15th Software Product Line Conference (SPLC 2011)*. Vol. 2. USA: ACM, 2011, 34:1–34:8. DOI: [10.1145/2019136.2019175](https://doi.org/10.1145/2019136.2019175).
- [Dub19] Clemens Dubsloff. “Compositional Feature-Oriented Systems”. In: *Proceedings of the 17th International Conference on Software Engineering Engineering and Formal Methods (SEFM 2019)*. Ed. by Peter Csaba Ölveczky and Gwen Salaün. Vol. 11724. Lecture Notes in Computer Science. Germany: Springer, 2019, pp. 162–180. DOI: [10.1007/978-3-030-30446-1_9](https://doi.org/10.1007/978-3-030-30446-1_9).
- [Dub21] Clemens Dubsloff. “Quantitative Analysis of Configurable and Reconfigurable Systems”. PhD thesis. TU Dresden, 2021.

- [DW17] Aleksandar S. Dimovski and Andrzej Wąsowski. “Variability-Specific Abstraction Refinement for Family-Based Model Checking”. In: *Proceedings of the 20th International Conference on Fundamental Approaches to Software Engineering (FASE 2017)*. Ed. by M. Huisman and J. Rubin. Vol. 10202. Lecture Notes in Computer Science. Springer, 2017, pp. 406–423. DOI: [10.1007/978-3-662-54494-5_24](https://doi.org/10.1007/978-3-662-54494-5_24).
- [FKP12] Vojtech Forejt, Marta Z. Kwiatkowska, and David Parker. “Pareto Curves for Probabilistic Model Checking”. In: *Proceedings of the 10th International Symposium on Automated Technology for Verification and Analysis (ATVA 2012)*. Ed. by Supratik Chakraborty and Madhavan Mukund. Vol. 7561. Lecture Notes in Computer Science. Germany: Springer, 2012, pp. 317–332. DOI: [10.1007/978-3-642-33386-6_25](https://doi.org/10.1007/978-3-642-33386-6_25).
- [FSM19] Vanderson Hafemann Fragal, Adenilso da Silva Simão, and Mohammad Reza Mousavi. “Hierarchical Featured State Machines”. In: *Science of Computer Programming* vol. 171 (2019), pp. 67–88. DOI: [10.1016/J.SCICO.2018.10.001](https://doi.org/10.1016/J.SCICO.2018.10.001).
- [GB14] Luca Gherardi and Davide Brugali. “Modeling and Reusing Robotic Software Architectures: the HyperFlex Toolchain”. In: *Proceedings of the International Conference on Robotics and Automation (ICRA 2014)*. USA: IEEE, 2014, pp. 6414–6420. DOI: [10.1109/ICRA.2014.6907806](https://doi.org/10.1109/ICRA.2014.6907806).
- [Ger+17] Simos Gerasimou, Radu Calinescu, Stepan Shevtsov, and Danny Weyns. “UNDERSEA: An Exemplar for Engineering Self-Adaptive Unmanned Underwater Vehicles”. In: *Proceedings of the 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2017)*. USA: IEEE, 2017, pp. 83–89. DOI: [10.1109/SEAMS.2017.19](https://doi.org/10.1109/SEAMS.2017.19).
- [GH03] Hassan Gomaa and Mohamed Hussein. “Dynamic Software Reconfiguration in Software Product Families”. In: *Revised Papers of the 5th International Workshop on Software Product-Family Engineering (PFE 2003)*. Ed. by Frank van der Linden. Vol. 3014. Lecture Notes in Computer Science. Germany: Springer, 2003, pp. 435–444. DOI: [10.1007/978-3-540-24667-1_33](https://doi.org/10.1007/978-3-540-24667-1_33).
- [GS10] Carlo Ghezzi and Amir Molzam Sharifloo. “Dealing with Non-Functional Requirements for Adaptive Systems via Dynamic Software Product-Lines”. In: *Software Engineering for Self-Adaptive Systems II*. Ed. by Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw. Vol. 7475. Lecture Notes in Computer Science. Germany: Springer, 2010, pp. 191–213. DOI: [10.1007/978-3-642-35813-5_8](https://doi.org/10.1007/978-3-642-35813-5_8).

- [GS11] Carlo Ghezzi and Amir Molzam Sharifloo. “Verifying Non-functional Properties of Software Product Lines: Towards an Efficient Approach Using Parametric Model Checking”. In: *Proceedings of the 15th International Software Product Lines Conference (SPLC 2011)*. USA: IEEE, 2011, pp. 170–174. DOI: 10.1109/SPLC.2011.33.
- [Hah+10] Ernst Moritz Hahn, Holger Hermanns, Björn Wachter, and Lijun Zhang. “PARAM: A Model Checker for Parametric Markov Models”. In: *Proceedings of the 22nd International Conference on Computer Aided Verification (CAV 2010)*. Ed. by Tayssir Touili, Byron Cook, and Paul B. Jackson. Vol. 6174. Lecture Notes in Computer Science. Germany: Springer, 2010, pp. 660–664. DOI: 10.1007/978-3-642-14295-6_56.
- [Hal+06] Svein Hallsteinsen, Erlend Stav, Arnor Solberg, and Jacqueline Floch. “Using Product Line Techniques to Build Adaptive Systems”. In: *Proceedings of the 10th International Software Product Line Conference (SPLC 2006)*. USA: IEEE, 2006, pp. 141–150. DOI: 10.1109/SPLINE.2006.1691586.
- [Hal+13] Svein Hallsteinsen, Mike Hinchey, Sooyong Park, and Klaus Schmid. “Dynamic Software Product Lines”. In: *Systems and Software Variability Management: Concepts, Tools and Experiences*. Ed. by Rafael Capilla, Jan Bosch, and Kyo-Chul Kang. Germany: Springer, 2013, pp. 253–260. DOI: 10.1007/978-3-642-36583-6_16.
- [Hen+22] Christian Hensel, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. “The Probabilistic Model Checker Storm”. In: *International Journal on Software Tools for Technology Transfer* vol. 24, no. 4 (2022), pp. 589–610. DOI: 10.1007/S10009-021-00633-Z.
- [Her13] Carlos Hernández Corbato. “Model-Based Self-awareness Patterns for Autonomy”. PhD thesis. Universidad Politécnica de Madrid, 2013. DOI: 10.20868/UPM.thesis.23178.
- [Hez+21] Sara M. Hezavehi, Danny Weyns, Paris Avgeriou, Radu Calinescu, Raffaella Mirandola, and Diego Perez-Palacin. “Uncertainty in Self-adaptive Systems: A Research Community Perspective”. In: *ACM Transactions on Autonomous and Adaptive Systems* vol. 15, no. 4 (2021), 10:1–10:36. DOI: 10.1145/3487921.
- [HJ94] Hans Hansson and Bengt Jonsson. “A Logic for Reasoning about Time and Reliability”. In: *Formal Aspects of Computing* vol. 6, no. 5 (1994), pp. 512–535. DOI: 10.1007/BF01211866.
- [HPS12] Mike Hinchey, Sooyong Park, and Klaus Schmid. “Building Dynamic Software Product Lines”. In: *IEEE Computer* vol. 45, no. 10 (2012), pp. 22–26. DOI: 10.1109/MC.2012.332.

- [Jun+24] Sebastian Junges, Erika Ábrahám, Christian Hensel, Nils Jansen, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. “Parameter Synthesis for Markov Models: Covering the Parameter Space”. In: *Formal Methods in System Design* vol. 62, no. 1 (2024), pp. 181–259. DOI: [10.1007/s10703-023-00442-x](https://doi.org/10.1007/s10703-023-00442-x).
- [KA08] Christian Kästner and Sven Apel. “Integrating Compositional and Annotative Approaches for Product Line Engineering”. In: *Proceedings of the Workshop on Modularization, Composition and Generative Techniques for Product Line Engineering (McGPLE 2008)*. Ed. by Neil Loughran, Iris Groher, Roberto Lopez-Herrejon, Sven Apel, and Christa Schwanninger. Germany: University of Passau, 2008, pp. 35–40.
- [Kan+90] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Tech. rep. CMU/SEI-90-TR-21. Carnegie Mellon University, 1990.
- [KC03] Jeffrey O. Kephart and David M. Chess. “The Vision of Autonomic Computing”. In: *IEEE Computer* vol. 36, no. 1 (2003), pp. 41–50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
- [KNP11] Marta Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of Probabilistic Real-Time Systems”. In: *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV 2011)*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Germany: Springer, 2011, pp. 585–591. DOI: [10.1007/978-3-642-22110-1_47](https://doi.org/10.1007/978-3-642-22110-1_47).
- [LM09] Yu Liu and René Meier. “Resource-Aware Contracts for Addressing Feature Interaction in Dynamic Adaptive Systems”. In: *Proceedings of the 5th International Conference on Autonomic and Autonomous Systems (ICAS 2009)*. USA: IEEE, 2009, pp. 346–350. DOI: [10.1109/ICAS.2009.24](https://doi.org/10.1109/ICAS.2009.24).
- [Loc+16] Malte Lochau, Stephan Mennicke, Hauke Baller, and Lars Ribbeck. “Incremental Model Checking of Delta-Oriented Software Product Lines”. In: *Journal of Logical and Algebraic Methods in Programming* vol. 85, no. 1 (2016), pp. 245–267. DOI: [10.1016/j.jlamp.2015.09.004](https://doi.org/10.1016/j.jlamp.2015.09.004).
- [Luc+19] Matt Luckcuck, Marie Farrell, Louise A. Dennis, Clare Dixon, and Michael Fisher. “Formal Specification and Verification of Autonomous Robotic Systems: A Survey”. In: *ACM Computing Surveys* vol. 52, no. 5 (2019), 100:1–100:41. DOI: [10.1145/3342355](https://doi.org/10.1145/3342355).
- [Mac+22] Steven Macenski, Tully Foote, Brian P. Gerkey, Chris Lalancette, and William Woodall. “Robot Operating System 2: Design, Architecture, and Uses in the Wild”. In: *Science Robotics* vol. 7, no. 66 (2022), eabm6074. DOI: [10.1126/SCIROBOTICS.ABM6074](https://doi.org/10.1126/SCIROBOTICS.ABM6074).

III. Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System

- [PBL05] Klaus Pohl, Günter Böckle, and Frank J. van der Linden. *Software Product Line Engineering: Foundations, Principles, and Techniques*. Germany: Springer, 2005. DOI: [10.1007/3-540-28901-1](https://doi.org/10.1007/3-540-28901-1).
- [Päβ+23a] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, S. Lizeth Tapia Tarifa, and Einar Broch Johnsen. “Formal Modelling and Analysis of a Self-Adaptive Robotic System”. In: *Proceedings of the 18th International Conference on integrated Formal Methods (iFM 2023)*. Ed. by Paula Herber and Anton Wijs. Vol. 14300. Lecture Notes in Computer Science. Springer, 2023, pp. 343–363. DOI: [10.1007/978-3-031-47705-8_18](https://doi.org/10.1007/978-3-031-47705-8_18).
- [Päβ+23b] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, S. Lizeth Tapia Tarifa, and Einar Broch Johnsen. *Formal Modelling and Analysis of a Self-Adaptive Robotic System (Artifact)*. Version 1.0.0. 2023. DOI: [10.5281/zenodo.8275533](https://doi.org/10.5281/zenodo.8275533).
- [Päβ+24] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Clemens Dubslaff, Einar Broch Johnsen, and Silvia Lizeth Tapia Tarifa. *Feature-Oriented Modelling and Analysis of a Self-Adaptive Robotic System (Artifact)*. Zenodo. Version 1.0.0. 2024. DOI: [10.5281/zenodo.14228090](https://doi.org/10.5281/zenodo.14228090).
- [Päβ+25] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, Einar Broch Johnsen, and S. Lizeth Tapia Tarifa. “A Configurable Software Model of a Self-Adaptive Robotic System”. In: *Science of Computer Programming* vol. 240 (2025), p. 103221. DOI: [10.1016/j.scico.2024.103221](https://doi.org/10.1016/j.scico.2024.103221).
- [QJK22] Tim Quatmann, Sebastian Junges, and Joost-Pieter Katoen. “Markov Automata with Multiple Objectives”. In: *Formal Methods in System Design* vol. 60, no. 1 (2022), pp. 33–86. DOI: [10.1007/S10703-021-00364-6](https://doi.org/10.1007/S10703-021-00364-6).
- [Qua+16] Tim Quatmann, Christian Dehnert, Nils Jansen, Sebastian Junges, and Joost-Pieter Katoen. “Parameter Synthesis for Markov Models: Faster Than Ever”. In: *Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA 2016)*. Ed. by Cyrille Artho, Axel Legay, and Doron Peled. Vol. 9938. Lecture Notes in Computer Science. Germany: Springer, 2016, pp. 50–67. DOI: [10.1007/978-3-319-46520-3_4](https://doi.org/10.1007/978-3-319-46520-3_4).
- [Qui+09] Morgan Quigley, Brian Gerkey, Ken Conley, Josh Faust, Tully Foote, Jeremy Leibs, Eric Berger, Rob Wheeler, and Andrew Ng. “ROS: An Open-Source Robot Operating System”. In: *Proceedings of the Open-Source Software Workshop of the International Conference on Robotics and Automation (ICRA 2009)*. USA: IEEE, 2009.

- [QVW24] Sergio Quijano, Mahsa Varshosaz, and Andrzej Wąsowski. “Modeling and Safety Analysis of Autonomous Underwater Vehicles Behaviors”. In: *Proceedings of the 17th International Conference on Software Testing, Verification and Validation Workshops (ICSTW 2024)*. USA: IEEE, 2024, pp. 63–67. DOI: [10.1109/ICSTW60967.2024.00022](https://doi.org/10.1109/ICSTW60967.2024.00022).
- [Rez+23] Gustavo Rezende Silva, Juliane Päßler, Jeroen Zwanepol, Elvin Alberts, S. Lizeth Tapia Tarifa, Ilias Gerostathopoulos, Einar Broch Johnsen, and Carlos Hernández Corbato. “SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles”. In: *Proceedings of the 18th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2023)*. IEEE, 2023, pp. 181–187. DOI: [10.1109/SEAMS59076.2023.00031](https://doi.org/10.1109/SEAMS59076.2023.00031).
- [Rod+15] Genáina N. Rodrigues, Vander Alves, Vinicius Nunes, André Lanna, Maxime Cordy, Pierre-Yves Schobbens, Amir Molzam Sharifloo, and Axel Legay. “Modeling and Verification for Probabilistic Properties in Software Product Lines”. In: *Proceedings of the 16th International Symposium on High-Assurance Systems Engineering (HASE 2015)*. USA: IEEE, 2015, pp. 173–180. DOI: [10.1109/HASE.2015.34](https://doi.org/10.1109/HASE.2015.34).
- [Sch+06] Pierre-Yves Schobbens, Patrick Heymans, Jean-Christophe Trigaux, and Yves Bontemps. “Feature Diagrams: A Survey and a Formal Semantics”. In: *Proceedings of the 14th International Conference on Requirements Engineering (RE 2006)*. USA: IEEE, 2006, pp. 136–145. DOI: [10.1109/RE.2006.23](https://doi.org/10.1109/RE.2006.23).
- [Sch+08] Klaus Schmid, Sooyong Park, Mike Hinchey, and Svein Hallsteinsen. “Dynamic Software Product Lines”. In: *IEEE Computer* vol. 41, no. 4 (2008), pp. 93–95. DOI: [10.1109/MC.2008.123](https://doi.org/10.1109/MC.2008.123).
- [Sch+10] Ina Schaefer, Lorenzo Bettini, Viviana Bono, Ferruccio Damiani, and Nico Tanzarella. “Delta-oriented Programming of Software Product Lines”. In: *Proceedings of the 14th Software Product Lines Conference (SPLC 2010)*. Ed. by Jan Bosch and Jaejoon Lee. Vol. 6287. Lecture Notes in Computer Science. Germany: Springer, 2010, pp. 77–91. DOI: [10.1007/978-3-642-15579-6_6](https://doi.org/10.1007/978-3-642-15579-6_6).
- [SLR13] Karsten Saller, Malte Lochau, and Ingo Reimund. “Context-Aware DSPLs: Model-Based Runtime Adaptation for Resource-Constrained Systems”. In: *Proceedings of the 17th International Software Product Line Conference (SPLC 2013)*. Vol. 2. USA: ACM, 2013, pp. 106–113. DOI: [10.1145/2499777.2500716](https://doi.org/10.1145/2499777.2500716).
- [Thü+12] Thomas Thüm, Ina Schaefer, Sven Apel, and Martin Hentschel. “Family-Based Deductive Verification of Software Product Lines”. In: *Proceedings of the 11th International Conference on Generative Programming and Component Engineering (GPCE 2012)*. USA: ACM, 2012, pp. 11–20. DOI: [10.1145/2371401.2371404](https://doi.org/10.1145/2371401.2371404).

- [Thü+14] Thomas Thüm, Sven Apel, Christian Kästner, Ina Schaefer, and Gunter Saake. “A Classification and Survey of Analysis Strategies for Software Product Lines”. In: *ACM Computing Surveys* vol. 47, no. 1 (2014), 6:1–6:45. DOI: [10.1145/2580950](https://doi.org/10.1145/2580950).
- [Van+18] Andrea Vandin, Maurice H. ter Beek, Axel Legay, and Alberto Lluch Lafuente. “QFLan: A Tool for the Quantitative Analysis of Highly Reconfigurable Systems”. In: *Proceedings of the 22nd International Symposium on Formal Methods (FM 2018)*. Ed. by Klaus Havelund, Jan Peleska, Bill Roscoe, and Erik de Vink. Vol. 10951. Lecture Notes in Computer Science. Germany: Springer, 2018, pp. 329–337. DOI: [10.1007/978-3-319-95582-7_19](https://doi.org/10.1007/978-3-319-95582-7_19).
- [VBM18] Mahsa Varshosaz, Harsh Beohar, and Mohammad Reza Mousavi. “Basic Behavioral Models for Software Product Lines: Revisited”. In: *Science of Computer Programming* vol. 168 (2018), pp. 171–185. DOI: [10.1016/J.SCICO.2018.09.001](https://doi.org/10.1016/J.SCICO.2018.09.001).
- [Wey+12] Danny Weyns, M. Usman Iftikhar, Didac Gil de la Iglesia, and Tanvir Ahmad. “A Survey of Formal Methods in Self-Adaptive Systems”. In: *Proceedings of the 5th International C* Conference on Computer Science and Software Engineering (C3S2E 2012)*. USA: ACM, 2012, pp. 67–79. DOI: [10.1145/2347583.2347592](https://doi.org/10.1145/2347583.2347592).
- [Wey20] Danny Weyns. *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons, 2020.

Paper IV

Behavioral Feature Models

**Juliane Päßler, Sophie Fortz, Maurice H. ter Beek,
Ferruccio Damiani, Einar Broch Johnsen,
Mohammad Reza Mousavi, S. Lizeth Tapia Tarifa**

Submitted for journal publication (short version)

Abstract

Modeling software product lines (SPLs) involves addressing two challenges: representing the structure of the product line, called structural variability, and capturing its behavior, called behavioral variability. For structural variability, several established compositional modeling languages have been proposed, most notably feature models (FMs). For behavioral variability, there are no proposals that offer the same level of compositionality as offered by structural models. Moreover, these two types of models are traditionally developed in isolation. This separation often necessitates the maintenance of different models for structural and behavioral aspects, requiring synchronized updates to prevent discrepancies.

In this paper, we propose a novel formalism called *Behavioral Feature Model* (BFM) that integrates structural and behavioral aspects within a single unified model. A BFM is an FM in which each feature has an associated behavioral model, namely an event structure, that models the behavior of the feature; constraints between behavior of different features can be modeled as constraints between their associated events; and the behavior of each product of the SPL consists of the combined behavior of its features and the constraints between their behavior. Thus, the proposed formalism integrates both structural and behavioral variability in a single compositional model, while preserving the separation of concerns typical of feature modeling. We illustrate the concept of a BFM through a running example of a cleaning robot SPL, and provide both a theoretical and an empirical evaluation which demonstrate that BFMs are as expressive and succinct as a type of featured transition systems, the state-of-the-art formalism for behavioral modeling of SPLs.

IV

IV.1 Introduction

Software product line engineering [CN02; PBL05] is a software development paradigm concerned with developing a family of software products that share a common core and differ in some of their functionalities, called a *Software*

Template Decision Diagrams for Meta Control and Explainability

Clemens Dubsiaff, Verena Klös, Juliane Päßler

Published in *Explainable Artificial Intelligence*. xAI 2024. Communications in Computer and Information Science, vol. 2154, pp. 219–242. Springer, Cham. DOI: 10.1007/978-3-031-63797-1_12.

Abstract

Decision tree classifiers (DTs) provide an effective machine-learning model, well-known for its intuitive interpretability. However, they still miss opportunities well-established in software engineering that could further improve their explainability: separation of concerns, encapsulation, and reuse of behaviors. To enable these concepts, we introduce *templates* in *decision diagrams (DDs)* as an extension of multi-valued DDs. Templates allow to encapsulate and reuse common decision-making patterns. By a case study from the autonomous underwater robotics domain we illustrate the benefits of template DDs for modeling and explaining *meta controllers*, i.e., hierarchical control structures with underspecified entities. Further, we implement a template-generating refactoring method for DTs. Our evaluation on standard controller benchmarks shows that template DDs can improve explainability of controller DTs by reducing their sizes by more than one order of magnitude.

V.1 Introduction

Cyber-physical systems are complex. Their behaviors may depend on different kinds of sensor data, have to deal with concurrent executions, and face heterogeneous components that all can be subject to uncertainty such as noise or partial observability. *Controllers* are usually used to steer their behaviors and ensure correct functioning, e.g., in safety-critical environments. Handcrafted engineering of controllers is error-prone and does not scale to large systems. Differently, provably correct controllers can be constructed from formal specifications by *automated controller synthesis* [Oue+11]. Due to recent demands by legal authorities [19; 20; 21; Uni16] and to increase overall trustworthiness in systems, synthesized controllers should be *explainable*.

Authors in alphabetic order.

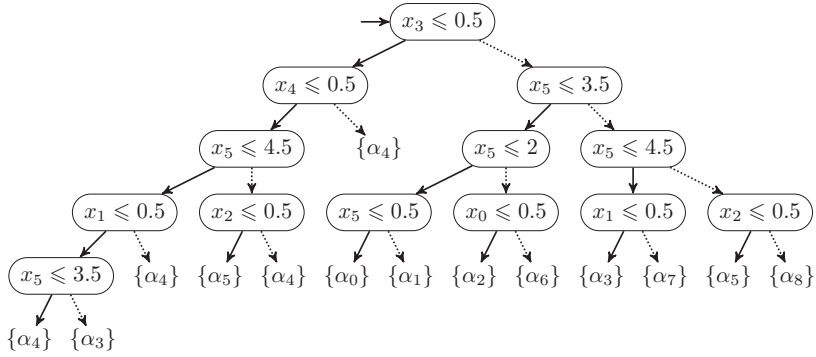


Figure V.1: DT representation of a control strategy for the TRIANGLE benchmark; solid edges are taken if the predicate is fulfilled, dotted edges if it is not.

Decision Trees for Strategy Explanation. *Decision trees (DTs)* [Mit97; Qui86] are an effective machine-learning model that also gained attention as explications of synthesized controllers due to their simplistic and interpretable nature [Ash+21; Gar+16]. Figure V.1 shows an example DT of a controller for the probabilistic planning system TIREWORLD [LT07] that has been synthesized by the tool DTCONTROL [Ash+21]. In its basic form, DTs are binary trees where inner *decision nodes* are labeled by predicates over features such as observable system variables and where leafs are labeled with control actions. Each decision node has two outgoing edges for the predicate being fulfilled or not. By starting in the root of the DT and evaluating predicates according to the current state of the controlled system, DTs eventually lead to possible control actions of the controller. Depending on how the controller is synthesized, these actions all fulfill the controller specification, e.g., following its safety requirements.

Due to the intuitive evaluation of predicates in decision nodes, DTs are well-suited to represent and explain the step-wise reasoning in control policies. *Occam's razor* provides a major principle for explainability for a global perspective, according to which the best explanation for a phenomenon is the smallest one amongst all possible explanations [Goo77]. For this, state-of-the-art learning algorithms minimize the size of DTs by choosing suitable decision predicates by entropy-based splitting and placing important decisions first. However, this minimization is only done within the class of DTs, which structurally hinders further improvements for explainability:

- (1) DTs are trees, duplicating shared decision making as isomorphic subtrees
- (2) DTs are monolithic, barely separating concerns [Dij82]
- (3) DTs are deterministically evaluating predicates, only supporting nondeterminism in control actions of DT leafs

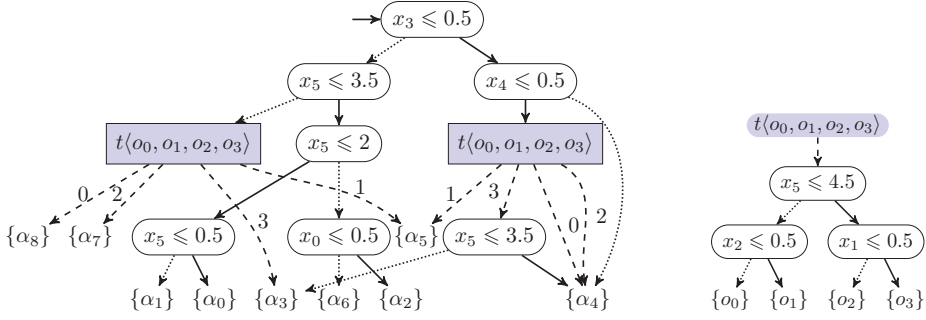


Figure V.3: Template DD for TRIANGLE control strategy

In this paper, we introduce the concept of *templates* in *decision diagrams* (DDs), leading to a new data structure of *template DDs* that remedies the three drawbacks of DTs above. First, template DDs are diagrams instead of trees, allowing for sharing of decision making by multiple incoming edges to decision nodes similar as within *binary decision diagrams* (BDDs) [Bry92]. Therefore, common decision making but also common control actions can directly be deduced from the graphical representation as diagrams. Second, extending the first reasoning by merging isomorphic subtrees, we propose to even further reuse common decision making by introducing *box nodes*. Such nodes are labeled by a template that itself can stand for decision making modeled as template DDs, hence separating concerns similar to procedures in programming languages by encapsulating common behaviors. The hierarchical structure through boxes and assigned templates does not only improve explainability of control policies by allowing for more compact representation, they also enable *underspecification*, enhancing degrees of freedom. Decision making that is, e.g., irrelevant for ensuring safety but important for other requirements such as energy consumption, could hence be left out for explicating safety under *all possible implementations* of templates. To this end, template DDs are also suitable to specify *families of controllers*, enabling their explanation and family-based analysis [Chr+18; Chr+23; Päß+23]. Figure V.3 shows a template DD for the DT controller of Figure V.1, reusing common decision making in a template used twice in the diagram.

Meta Control. Many cyber-physical systems are subject to uncertainty in both their environment and their internal structure. The reasons could be, e.g., partial observability, hardware or software failures, or noisy sensor data. To increase reliability, the systems can be implemented as *self-adaptive systems* (SASs) [Wey20], enabling them to

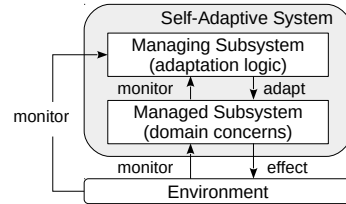


Figure V.2: A two-layered architecture for an SAS [Päß+23]

adapt their structure, task plan, control strategies, etc., mostly according to a predefined strategy. SAs that adapt their control strategy itself include a controller steering the adaptation in the system and can therefore be considered as a *meta control* system. One can distinguish between *internal* and *external* self-adaptation [ST09; Wey20]. An internal self-adaptation mechanism is embedded in the application logic and realized by, e.g., exception handling or fault-tolerance mechanisms. External self-adaptation separates the application logic from the adaptation logic and introduces a two-layered system where the application logic, focussing on the domain concerns, is implemented in a *managed subsystem* (the control system), and the adaptation logic is implemented in a *managing subsystem* [KC03] (the meta control system) – see Figure V.2. The managed subsystem monitors the environment and may affect the environment with its actions. It can also contain internal self-adaptation like exception handling. The managing subsystem receives input from both the managed subsystem and the environment to determine if and which adaptation is needed, and adapts the managed subsystem if necessary.

We focus here on easing the design and comprehension of meta control systems by distinguishing between high-level views for meta control and low-level means of adaptability. For this, we exploit the template DD’s possibility of *underspecification*, realized by (i) leaving the specification of templates open and specifying families of controllers, and by (ii) introducing parameters in box types to steer template instantiations. The templates and parameter values can then be set by the meta controller to adapt the control strategy of the control system.

Contributions. We present three main contributions. First, we formally introduce our new data structure of template DDs as an extension of multi-valued DDs. They have dedicated box nodes which can themselves be identified with other template DDs, implementing box-associated decision making and leading to a hierarchical template DD structure. Second and third, we showcase the potential of template DDs by two applications:

- (1) Controller design with template DDs in the setting of *meta control*, and
- (2) synthesis of concise template DDs from DT controllers.

Here, we contribute to (1) with an example of modeling a control and meta control system with template DDs for the case study of an autonomous underwater vehicle. We show that through the separation of concerns in the control structures, modeling can be eased through template DDs by reusing decision making and control underspecification. Further, we contribute to (2) by refactoring DTs from standard DTCONTROL benchmarks towards template DDs and hierarchical DDs. We implement a simple refactoring heuristics that already shows huge potential in template DDs for explainability by reducing control strategy representations by more than one order of magnitude.

V.2 Related Work

Decision Trees (DTs) and Decision Diagrams (DDs). DT classifiers have a long history in artificial intelligence (AI) and statistical learning [Bre+84; Mit97]. The explanatory power of Boolean DTs recently gains more and more attention in explainable AI [Aud+22]. Plambeck *et al.* [Pla+22] presented an approach to connect environment input to system behaviors and thus foster understanding which input caused what behavior. DTs are also used to generate *why*, *why-not*, *how-to*, and *what-if* explanations for context-aware intelligent systems [LDA09].

While DTs enforce a tree structure, DDs are directed acyclic graphs and thus more general. Most prominently, binary DDs [Bry92] are widely used in hardware design and verification to concisely represent Boolean functions [McM93; Min96]. The latter are in particular powerful when ordered and reduced, enabling a canonical representation and efficient operations. Extensions of binary DDs enable functions on more general co-domains (ADDs or MTBDDs, [FMY97]), multi-valued decisions (multi-valued DDs, [Sri+90]), or nodes labeled by linear predicates instead of Boolean variables (linear DDs, [CGS09]). MTBDDs for control strategy representation in synthesis have been considered in [ZVM18]. Recently, context-free DDs as hierarchical structures for efficient quantum circuit representation have been presented [SCR24]. While this work also follows the idea of subdiagrams as components, advantages for modeling control strategies, explainability, and DT representations were not considered as we do with the use of template DDs.

Explainability of Self-adaptive Systems (SASs). Initially, research on SASs has been mainly focused on achieving self-adaptivity. However, due to increasing complexity of intelligent cyber-physical systems, explainability emerges as an important research direction [Hez+21; HT21]. Comprehensible adaptation rules for context-dependent effect expectations of adaptation options enable expressive, yet explainable, adaptation logics [KGG18]. Li *et al.* proposed to use probabilistic model checking to determine when an explanation improves the overall system utility of an SAS with human-in-the-loop [Li+20]. Camili *et al.* described different levels of explainability with corresponding meta-requirements, and demonstrated human-understandable explanations with robotic examples [CMS22]. Autonomous agents can be also considered as adaptive systems. This research community has already a long tradition of investigating trust and explainability.

DTs have also been used as a specification formalism for SAS adaption rules, e.g., in the managing subsystem or meta controller for adapting control strategies [Cio+10; Jun+08; Rod+18]. Other works directly use DTs for self-adaptation: Chen *et al.* [Che+14] model architectural design decisions available at runtime as DTs and combine them with goal models representing requirements for a new model-based self-adaptation approach. Cheng and Garlan [CG12] propose a language for describing adaptation strategies of architecture-based

SASs where adaptation strategies are represented as DTs. None of these works uses a concept similar to template DDs.

Modularity in Decision Making. Behavior trees [CÖ17] are frequently used for robot controllers [Iov+22]. Their main advantage is a modular design where “individual behaviors can easily be reused in the context of another higher-level behavior, without needing to specify how they relate to the subsequent behaviors” [Bag+12]. There also exist approaches to parameterize behavior trees to improve reuse of subtrees [Sho+11] or reinforcement learning of parameters for complex robot control tasks [May+21]. Similar to these approaches, we introduce (parameterized) templates for DDs towards hierarchical template DDs that also benefit from modularity.

V.3 Foundations

We settle the foundations and notations used throughout the paper. Let us fix a set of *variables* Var on which we define *evaluations* as functions $\epsilon: Var \rightarrow \mathbb{Q}$ where $\mathbb{Q}$ are the rational numbers. The set of evaluations over Var is denoted by $\mathbb{E}$. A linear arithmetic *expression* e is defined by the grammar $e ::= c \mid c \cdot x \mid -e \mid (e + e)$ where x ranges over Var and c ranges over $\mathbb{Q}$. For an evaluation ϵ , the semantics $\llbracket e \rrbracket_\epsilon$ of an expression e is defined recursively as expected, e.g., $\llbracket x \rrbracket_\epsilon = \epsilon(x)$ and $\llbracket (e + e') \rrbracket_\epsilon = (\llbracket e \rrbracket_\epsilon + \llbracket e' \rrbracket_\epsilon)$. We denote by $Var(e)$ the set of variables in the expression e . A linear *predicate* is of the form $(e \sim 0)$ where e is an expression and $\sim \in \{=, \geq, >\}$ is a comparison relation. The set of predicates is denoted by $\mathbb{P}$. A predicate $(e \sim 0) \in \mathbb{P}$ is *satisfied* by an evaluation ϵ , denoted $\epsilon \models (e \sim 0)$, iff $\llbracket e \rrbracket_\epsilon \sim 0$. We mainly consider *axis-aligned predicates*, i.e., predicates of the form $(c - x \geq 0)$ for $c \in \mathbb{Q}$ and $x \in Var$ and denote them by $x \leq c$. We also write x for the predicate $x - 1 \geq 0$, providing a Boolean interpretation of x .

Definition V.3.1. A *strategy* over states $S \subseteq \mathbb{E}$ and outcomes Out is a function $\sigma: S \rightarrow 2^{Out}$. We call σ *deterministic* if $|\sigma(\epsilon)| = 1$ for all $\epsilon \in S$. The strategy σ is *complete* if $S = \mathbb{E}$.

To symbolically represent strategies through a decision diagram (DD) formalism that covers decision tree classifiers (DTs), we take inspiration from *linear DDs* [CGS09] and *multi-terminal binary DDs* [FMY97]. Recall that we fixed a set of variables Var used as domain of evaluations throughout the paper.

Definition V.3.2. A *decision diagram (DD)* is a tuple $\mathcal{D} = (N, Out, \lambda, \text{lo}, \text{hi}, r)$ where $N = D \cup O$ is a finite set of *nodes* comprising *decision nodes* D and *outcome nodes* O to which $\lambda: N \rightarrow \mathbb{P} \cup 2^{Out}$ labels either a predicate or a set of outcomes from Out , respectively, $\text{lo}, \text{hi}: D \rightarrow N$ are decision functions, and $r \in N$ is the root node of $\mathcal{D}$.

A *path* in $\mathcal{D}$ is an alternating sequence $n_0, d_0, n_1, d_1, \dots \in (D \times \{0, 1\})^* \times N$ where for all i with $d_i = 1$ we have $n_{i+1} = \text{hi}(n_i)$ and $n_{i+1} = \text{lo}(n_i)$ if $d_i = 0$. Given a node $n \in N$, we denote by $\text{Reach}(n) \subseteq N$ the set of nodes reachable

from n , i.e., the smallest set R where $n \in R$ and for all $n' \in R \cap D$ we have $\text{lo}(n'), \text{hi}(n') \in R$. We call $\mathcal{D}$ a *decision tree* (DT) if its underlying graph has a tree structure: for all $n, n' \in D$ with $n' \notin \text{Reach}(n)$ and $n \notin \text{Reach}(n')$ we have $\text{Reach}(n) \cap \text{Reach}(n') = \emptyset$. By $\mathcal{D}_n$ we denote the DD rooted at $n \in N$ that arises from $\mathcal{D}$ by replacing r by n . We assume that on all paths through $\mathcal{D}$, any predicate appears only once, i.e., $\lambda(n) = \lambda(n')$ implies $n = n'$ for all $n \in D$ and $n' \in \text{Reach}(n)$. We call $\mathcal{D}$ *deterministic* if $|\lambda(o)| = 1$ for all $o \in O$. Further, $\mathcal{D}$ is *reduced* if for all decision nodes $n, n' \in D$ we have $\text{lo}(n) \neq \text{hi}(n)$ (essentiality) and $\mathcal{D}_n \cong \mathcal{D}_{n'}$ implies $n = n'$ (merged isomorphic subdiagrams).

Figure V.1 exemplifies how we graphically specify decision diagrams: the root node is specified by a sole incoming arrow; **hi**-edges are drawn with solid arrows, **lo**-edges with dotted ones; sets of outcomes are at the leafs of the diagram.

The semantics $\llbracket \mathcal{D} \rrbracket$ of $\mathcal{D} = \mathcal{D}_r$ is a complete strategy over $\mathbb{E}$ and Out , recursively defined for every evaluation $\epsilon \in \mathbb{E}$ as

$$\llbracket \mathcal{D}_n \rrbracket(\epsilon) = \begin{cases} n & \text{if } n \subseteq Out \\ \llbracket \mathcal{D}_{\text{lo}(n)} \rrbracket(\epsilon) & \text{if } n \in D \text{ and } \epsilon \not\models \lambda(n) \\ \llbracket \mathcal{D}_{\text{hi}(n)} \rrbracket(\epsilon) & \text{if } n \in D \text{ and } \epsilon \models \lambda(n) \end{cases}$$

V.4 Template Decision Diagrams

In this section, we extend DDs towards *template DDs* where decisions are not exclusively made through completely specified predicates but can also be encapsulated in *boxes*. Formally, let $\mathbb{F}$ be a set of *function names* and define $\kappa(f)$ to denote the *arity* of the function $f \in \mathbb{F}$. Further, define a set $\mathbb{B}$ of *box types* that have the form $f\langle o_0, \dots, o_{k-1} \rangle$ where $f \in \mathbb{F}$, $k = \kappa(f)$, and o_i for $0 \leq i < k$ are ordered *box outcomes*. The outcomes associated to a box type $\tau \in \mathbb{B}$, are collected in a set $Out(\tau) = \{o_0, \dots, o_{k-1}\}$. Intuitively, a box of type τ as above encapsulates a strategy on $Out(\tau)$ by implementing function f . Enhancing DDs such that they can serve as templates of more complex decision making, we allow for *box nodes* that are non-binary as in *multi-valued DDs* [Sri+90]:

Definition V.4.1. A *template DD* is a tuple $\mathcal{T} = (N, Out, \lambda, \text{succ}, r)$ where $N = P \cup B \cup O$ comprises *predicate nodes* P , *box nodes* B , and *outcome nodes* O to which $\lambda: N \rightarrow \mathbb{P} \cup \mathbb{B} \cup 2^{Out}$ labels either a predicate, a box type, or subsets from the set of outcomes Out , respectively. The decision function **succ** maps every predicate node $p \in P$ to a function $\text{succ}(p): \{0, 1\} \rightarrow N$ and every box node $b \in B$ to a function $\text{succ}(b): Out(\lambda(b)) \rightarrow N$, and $r \in N$ is the root node of $\mathcal{T}$.

Notations and definitions on DDs carry over to template DDs where we consider nodes $D = P \cup B$ as decision nodes. Similar to outcomes in box types, we assume template DD outcomes to be ordered, i.e., $Out = \{o_0, \dots, o_{k-1}\}$ where $k = |Out|$. If a box type is Boolean, we omit the outcome types and assume o_0 standing for “false”, “no”, or “0”, and o_1 standing for “true”, “yes”, or “1”.

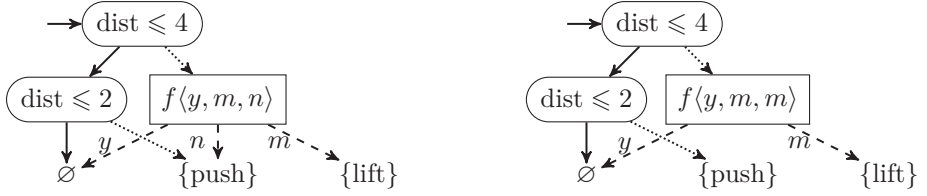


Figure V.4: Two simple example template DDs

Figure V.4 shows example two template DDs to specify a strategy that performs actions in $Out = \{\text{push}, \text{lift}\}$ depending on the distance “dist” to some object. The left template DD contains a box typed as $f\langle y, m, n \rangle$, providing three box outcomes y (“yes”), m (“maybe”), and n (“no”). Already this simple example shows one of the core benefits of template DDs, namely encapsulation and separation of concerns. Here, the decision whether to lift the object is externalized to a strategy given by a function $f\langle y, m, n \rangle$ in case $\text{dist} > 4$. Otherwise, the decision to push iff $\text{dist} > 2$ is made. Similarly, the template DD on the right treats also the “no” outcome of function f as “maybe”, leading to $Out(f\langle y, m, m \rangle) = \{y, m\}$ and hence a binary box.

Semantics of Template DDs. The definition of template DDs hints at decision making that is *underspecified*. For any box node $b \in B$ there is plenty of room which strategy $\sigma: \mathbb{E} \rightarrow 2^{Out(\lambda(b))}$ is actually represented by b . A natural approach to deal with such underspecification is to model outcomes nondeterministically, i.e., every box b represents a strategy σ where $\sigma(\epsilon) = Out(\lambda(b))$ for all evaluations $\epsilon \in \mathbb{E}$. Then, the semantics $\llbracket \mathcal{T} \rrbracket$ of a template DD $\mathcal{T} = \mathcal{T}_r$ as above is the complete strategy that is recursively defined over evaluations $\epsilon \in \mathbb{E}$:

$$\llbracket \mathcal{T}_n \rrbracket(\epsilon) = \begin{cases} n & \text{if } n \subseteq Out \\ \llbracket \mathcal{T}_{\text{succ}(n,0)} \rrbracket(\epsilon) & \text{if } n \in P \text{ and } \epsilon \not\models \lambda(n) \\ \llbracket \mathcal{T}_{\text{succ}(n,1)} \rrbracket(\epsilon) & \text{if } n \in P \text{ and } \epsilon \models \lambda(n) \\ \bigcup_{o \in Out(\lambda(n))} \llbracket \mathcal{T}_{\text{succ}(n,o)} \rrbracket(\epsilon) & \text{if } n \in B \end{cases}$$

V.4.1 Hierarchical Decision Diagrams by Templates

Choosing box outcomes nondeterministically is usually too coarse when specifying strategies that have to fulfill certain requirements. A natural way to refine template DDs is by providing *function evaluations* that map boxes to more specific strategies over box outcomes. While our definitions allow for any representation of strategies, e.g., machine-learning classifiers, state-action lookup tables, etc., we especially have other template DDs as strategy representation for box evaluations in mind. Then, sets of template DDs provide a uniform formalism for strategy specification that follows a hierarchical structure similar to call graphs of procedural programs and context-free BDDs [SCR24]:

Definition V.4.2. A *hierarchical DD* is a tuple $\mathfrak{H} = (\mathcal{T}^0, \dots, \mathcal{T}^{m-1}, \beta)$ where $\mathcal{T}^i = (N^i, Out^i, \lambda^i, \text{succ}^i, r^i)$ are template DDs for $i \in \{0, \dots, m-1\}$ and $\beta: \mathbb{F} \rightarrow \{0, \dots, m-1\}$ is a *function evaluation* that maps function names to template DDs such that for all $f \in \mathbb{F}$:

- $\mathcal{T}^{\beta(f)}$ is compatible with f , i.e., $|Out^{\beta(f)}| = \kappa(f)$, and
- $\mathcal{T}^{\beta(f)}$ is decreasing, i.e., $\beta(\lambda^{\beta(f)}(b)) > \beta(f)$ for all $b \in B^{\beta(f)}$.

The latter two conditions on function evaluations ensure that evaluated boxes provide a decision on outcomes and that no recursion occurs, respectively. This leads to a well-defined semantics $\llbracket \mathfrak{H} \rrbracket = \llbracket \mathcal{T}_{r^0}^0 \rrbracket_\beta$ of a hierarchical DD $\mathfrak{H}$ with function evaluation β , recursively defined over evaluations $\epsilon \in \mathbb{E}$:

$$\llbracket \mathcal{T}_n^i \rrbracket_\beta(\epsilon) = \begin{cases} n & \text{if } n \subseteq Out^i \\ \llbracket \mathcal{T}_{\text{succ}^i(n,0)}^i \rrbracket_\beta(\epsilon) & \text{if } n \in P^i \text{ and } \epsilon \not\models \lambda^i(n) \\ \llbracket \mathcal{T}_{\text{succ}^i(n,1)}^i \rrbracket_\beta(\epsilon) & \text{if } n \in P^i \text{ and } \epsilon \models \lambda^i(n) \\ \bigcup_{k < \kappa(f): o_k^j \in \llbracket \mathcal{T}_{r^j}^j \rrbracket_\beta(\epsilon)} \llbracket \mathcal{T}_{\text{succ}^i(n, o_k)}^i \rrbracket_\beta(\epsilon) & \text{if } n \in B^i \text{ with } j = \beta(f) \text{ and } \lambda^i(n) = f(o_0, \dots, o_{\kappa(f)-1}). \end{cases}$$

The last case of the above case distinction formalizes the evaluation of the j th template DD at its root r^j invoked from a box n in the i th template DD. For each outcome o_k^j the k th outcome o_k of the box type in the i th template DD is then chosen, continuing the evaluation within the i th template DD.

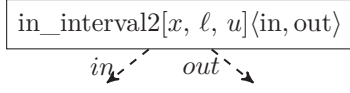
V.4.2 Standard Template Boxes

Templating through boxes can be done to specify common decision patterns that often occur in decision strategies or to encapsulate a sub-decision for reuse in a strategy-representing DD. The corresponding concept of procedures known from software engineering also provides inspiration for common structures how boxes are instantiated in most common applications.

Templates with Parameters. Often, recurring sub-decisions have the same structure but can differ in the values of constants used in the predicates of decision nodes. A common way to deal with that in software engineering is to introduce parameters. To enable the use of *parameterized templates*, we propose to introduce an additional variable in box types that serves as a parameter. For readability, we propose to list parameter names in square brackets behind the function name, e.g., parametrizing a box type $f(o_0, \dots, o_{k-1})$ with parameters p_0 to p_{n-1} yields

$$f[p_0, \dots, p_{n-1}](o_0, \dots, o_{k-1})$$

Standard Templates. In control decisions, we often have the same kind of checks that are performed on observable system variables. To provide explicit modeling solutions for these well-known patterns, we introduce standard



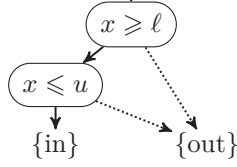
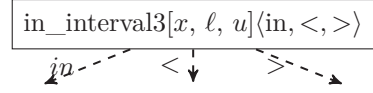
$$\text{in_interval2}[x, \ell, u]\langle \text{in}, \text{out} \rangle$$


Figure V.5: Interval2 template (top) and a feasible implementation (bottom)



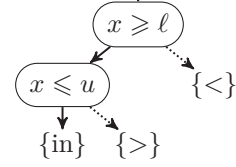
$$\text{in_interval3}[x, \ell, u]\langle \text{in}, <, > \rangle$$


Figure V.6: Interval3 template (top) and a feasible implementation (bottom)

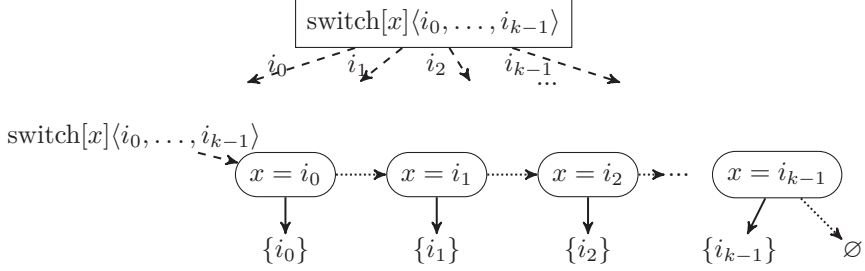


Figure V.7: Switch template (top) and a feasible implementation (bottom)

templates and feasible implementations by (template) DDs. Next to simple checks like *less*, *equal*, or *greater*, variables often have to be within a certain interval and control actions depend on whether the variable is *smaller*, *within*, or *greater* than interval boundaries. We provide two templates for interval containment. First, templates with the box type “ $\text{in_interval2}[x, \ell, u]\langle \text{in}, \text{out} \rangle$ ” decide whether a variable x is evaluated within the interval $[\ell, u]$ or outside. It corresponds to the DD depicted in Figure V.5. This check is often used in control decisions to decide whether an action needs to be performed in order to keep the process variable within a reference interval.

Second, control decision may also depend on whether the value is below or above the desired range. For this, we provide a template with box type “ $\text{in_interval3}[x, \ell, u]\langle \text{in}, <, > \rangle$ ”, which decides whether a variable x is evaluated within the interval $[\ell, u]$, smaller, or greater than the interval. The corresponding DT is depicted in Figure V.6.

Another recurring pattern are *switches*, that compare the variable to several values and execute a corresponding control decision afterwards. The template of box type “ $\text{switch}[x]\langle i_0, \dots, i_{k-1} \rangle$ ” decides whether variable x is evaluated to $i_0, i_1, \dots$, or i_{k-1} . This pattern is used in programming languages to avoid

nested if-statements. The corresponding DT is depicted in Figure V.7. Note that also alternative implementations of this template could be imagined, e.g., using binary search, from which one can abstract by using templates and leaving implementation details to future development steps or when further explanations on the implementation are required.

V.5 Templates for Self-Adaptive Systems and Meta Control

Decision trees (DTs) as an instance of DDs are well-established to specify control policies. Using template DDs instead of DDs further provides two main advantages: (i) it accommodates reuse of decision policies and thereby introduces a separation of concerns by enclosing common behaviors into boxes, similar to procedures in programming, and (ii) it allows underspecification of certain parts of the control strategy, thereby enabling specification of families of controllers.

We illustrate the use of template DDs as modeling formalism for control and meta control policies with template DDs. As explained in Section V.1, an SAS focussed on adapting control strategies can be seen as a special case of a two-layered system with a managed and a managing subsystem (see Figure V.2) where the managed subsystem is a control system and the managing subsystem is a meta control system. The meta control system adapts the control strategies of the control system according to input from the environment and the control system. The control system can be specified as a template DD, making common decision strategies explicit and thereby making it easier to understand the control system. Furthermore, template DDs can be left underspecified and contain parameters which are encoded as variables in the function name.

By instantiating template DDs and parameters of functions instead of adapting the control strategy of the control system in a different way, the adaptation strategy of the meta control system (i) is easier to specify because changing parameters of functions gives a good overview of the choices that can be taken, and the structure of the template DDs can be reused, (ii) has a compact structure which makes it easier to understand. The meta control system can itself be represented as a template DD.

We illustrate how interacting control and meta control systems can be modeled with template DDs on a case study inspired by the exemplar SUAVE [Rez+23]. To do so, the case study is first described in Section V.5.1 and then modeled with template DDs in Section V.5.2. As a first example for how to represent template DDs in a control system, Figure V.8 shows a template DD for an autonomous underwater vehicle (AUV). The template DD makes use of our standard template for interval checking, which can be implemented as shown in Section V.4.2. When considering the AUV to be a two-layered system, the instantiation of the function parameters would be done by the meta control system, setting the values for *min* and *max* to adapt the minimal and maximal diving depth of the AUV.

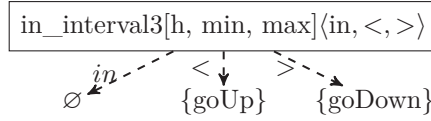


Figure V.8: Example template DD with standard interval template for an AUV

V.5.1 An Overview of the Case Study

In this section, we introduce our case study of an AUV used for searching for a pipeline located on a seabed and then following and inspecting it. The AUV has two kinds of vision sensors, a camera and a sonar, where the sonar consists of a side-scan sonar and a forward-facing sonar. Both kinds of vision sensors can be used for searching for the pipeline and for following it, but the sonar is preferred for searching since it can operate on a higher altitude, giving a bigger field of view, and the camera is preferred for inspecting the pipeline since it is easier to detect faults of the pipeline with the camera.

Depending on the current task of the AUV, i.e., searching for the pipeline or following it, a different camera resolution or sonar is required. When searching for the pipeline with the camera, the resolution does not need to be as high as for inspection since during inspection, faults need to be identified with the camera images. When searching for the pipeline with the sonar, the side-scan sonar is preferred since it provides a wider field of view than the forward-facing sonar. However, when following the pipeline, the forward-facing sonar is preferred. Both the sonar and the camera might fail during runtime, making it necessary to choose the other vision sensor for both tasks. Furthermore, the camera can get blocked temporarily, making a temporary switch to the sonar necessary for inspection. If both sensors fail, the mission should be aborted.

V.5.2 Modeling the Case Study with Template DDs

The AUV can be modeled as an SAS with a control system responsible for searching for and inspecting the pipeline, and a meta control system that adapts the control strategy of the control system depending on the current task of the AUV. Therefore, we model the control system of the AUV as a hierarchical template DD (see Figures V.9 and V.10), and the meta control system as a (template) DD without box nodes that determines the values for the control system's function parameters depending on the state of the control system (see Figure V.11).

Control System. The control system consists of the hierarchical template DD representing how to choose the sensor for searching for the pipeline and following it, see Figure V.9. When searching for the pipeline (the pipeline has not been found, indicated by the Boolean value *pipeline_found*), the sonar is the preferred vision sensor so it is chosen if available. If only the camera is available, then the camera is chosen. Otherwise the mission should be aborted. When the pipeline has been found, the camera is preferred for following it and the

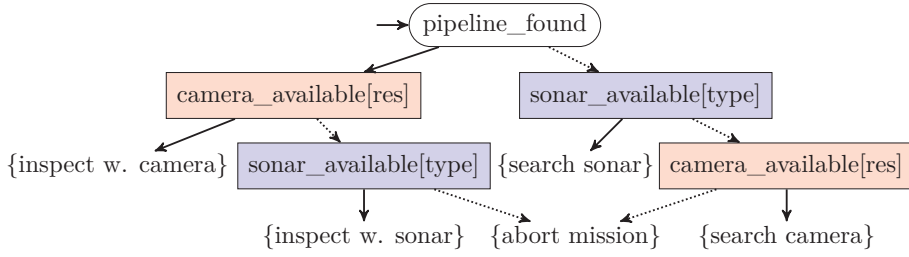


Figure V.9: The control system of the case study as a template DD

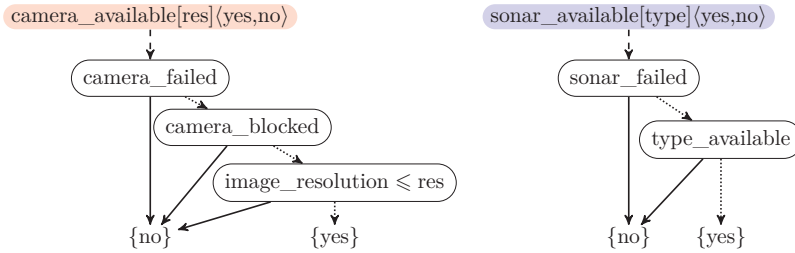


Figure V.10: The template DD instances for the control system

sonar is only chosen if the camera is not available but the sonar is. In both cases, if the pipeline has been found and if it has not, the control system has to determine whether the camera and the sonar are available. These checks depend on the parameters *res*, the “resolution” of the camera, and *type*, the type of sonar used (side-scan and forward-facing), and have been summarized in the box types “camera_available[*res*]” and “sonar_available[*type*]”, respectively. Recall that we simplify box types and outgoing edges of box types if they have only two outcomes, i.e., the box types “camera_available[*res*] $\langle o_0, o_1 \rangle$ ” and “sonar_available[*type*] $\langle o_0, o_1 \rangle$ ” have been simplified by omitting the box outcomes o_0 and o_1 , and the outgoing edges of the box types are drawn as usual “yes/no-edges” for simplification.

Using the box types for checking if the camera and sonar are available makes understanding and adapting the control strategy of the control system much easier because (i) the box types summarize (template) DDs, increasing explainability and clarifying the structure of the DD; without them, this structure would be difficult to spot, (ii) the box nodes can be reordered easily to reflect a different preference for using camera and sonar; with the preferences described in Section V.5.1, the two box types appear in a different order in the sub-template DDs, depending on whether the pipeline has been found.

The camera and sonar have to be available with different resolution and type, respectively, depending on the current task of the AUV, see Section V.5.1. Therefore, the chosen instantiations of these box types depend on the parameters *res* for “resolution” and *type* for the type of sonar to be used (side-scan or forward-facing). The template DDs implementing them are depicted in Figure V.10,

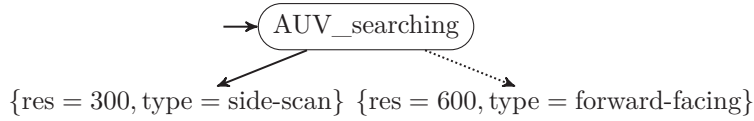


Figure V.11: The meta-control system of the case study

where “camera_failed”, “camera_blocked”, “sonar_failed”, and “type_available” are Boolean variables that are assumed to be set to the correct value by the system depending on sensor input on whether the camera failed, the camera is blocked, the sonar failed and the requested type is available, respectively. The variable “image_resolution” is an integer that is also set by the system depending on the current resolution of the camera images. In conclusion, the control system of the AUV is a hierarchical DD $\mathfrak{H} = (\mathcal{T}^0, \mathcal{T}^1, \mathcal{T}^2, \beta)$ where $\mathcal{T}^0$, $\mathcal{T}^1$, and $\mathcal{T}^2$ are the template DDs for the control system (Figure V.9), the camera and sonar available DDs (Figure V.10), respectively, and

$$\beta: \{\text{control}, \text{camera_available}[\text{res}], \text{sonar_available}[\text{type}]\} \rightarrow \{0, 1, 2\}$$

$$\beta(f) = \begin{cases} 0 & \text{if } f = \text{control}, \\ 1 & \text{if } f = \text{camera_available}[\text{res}], \text{ and} \\ 2 & \text{if } f = \text{sonar_available}[\text{type}]. \end{cases}$$

Meta Control System. The meta control system adapts the control strategy of the control system by instantiating the parameters `res` and `type` of the functions “camera_available[`res`]” and “sonar_available[`type`]” of the control system depending on whether the AUV is searching for the pipeline or following it, see Figure V.11. The variable “AUV_searching” is a Boolean variable that is assumed to be set by the system depending on the state of the control system, i.e., depending on whether the AUV is searching for the pipeline. This variable is different from the variable “pipeline_found” in the control system which captures whether the pipeline has been recognized on the sensor input.

Enabling adaptation of the control system by instantiating parameters of functions enables reuse of box types and makes it easy to spot which parameters can be adapted and where they are used. With box types, the control strategy of the control system can easily be understood on a high level (i.e., without having to look at the implementation of the camera and sonar check) and the adaptation strategy of the meta control system is easy to understand and represent. Without box types, it would both be harder to see which parameters can be adapted and how the parameters in different parts of the DD relate to each other.

V.6 Improving Control Strategy Explanations

In software engineering, well-established concepts help to improve understandability and maintainability of software: separation of concerns, encapsulation,

and reuse of behaviors. Their common intent is to reduce the amount of code that has to be captured and understood during software development. By encapsulating decisions and actions that belong together into subprograms, objects, and procedures, the overall program can be understood on a more abstract level. Encapsulated functionalities then only need to be evaluated once when inspecting software. Further, when changing functionalities or debugging, code modifications are reduced and mitigate negative side-effects of code duplication.

Our definition of *template DDs* and hence *hierarchical DDs* has been inspired by procedural software and thus can benefit from these concepts towards improved explainability of DTs and DDs. As boxes in template DDs usually encapsulate complex behavior including several decisions, a template DD is smaller than the equivalent DD without boxes, and, thus, easier to grasp. Furthermore, boxes allow for separation of concerns and can be examined individually.

V.6.1 Explainability Metrics for Template DDs

While these arguments are purely qualitative, the software engineering community also provides metrics that allow to quantify the understandability of code [Zus91]. In the following, we describe the most important ones and transfer them to our template DD setting. Following Occam's razor, the best explanation for a phenomenon is the smallest one amongst all possible explanations [Goo77]. Metrics to measure the psychological complexity [Zus91] and hence understandability of source code can hence be used as metrics for explainability. Minimizing such metrics corresponds to increasing explainability. Most prominently used are the number of *lines of code (LOC)* and McCabe's *cyclomatic complexity (CC)* [McC76; Wat+96]. The latter measures the number of linearly independent control flows through a program. Formally, CC is defined as the sum of $|E| - |N| + 2$ for each procedure, where $|E|$ and $|N|$ are the number of edges and nodes of the control-flow graph, respectively. Note that with binary decisions and one terminal node only, this measure simplifies to $|N|$, i.e., the LOC measure.

Let us now establish corresponding metrics to measure the complexity of template DDs. For this, let $\mathcal{T} = (N, Out, \lambda, succ, r)$ be a template DD. The LOC measure on source code can directly be transferred to the setting of DDs as the number of decision nodes. This allows for a first impression of the size of information that needs to be understood. Formally,

$$LOC(\mathcal{T}) = |N|.$$

M McCabe's cyclomatic complexity can also be interpreted on template DDs, where the DD is seen as control-flow graph in which each box corresponds to a procedure call and where each terminal node leads to an artificial control-flow sink node. Formally, we define the CC measure as

$$CC(\mathcal{T}) = |P| + \sum_{b \in B} (|Out(\lambda(b))| - 1) + 1.$$

Note that each predicate node has two successors, while boxes might have higher arity. Both measures naturally extend to hierarchical DDs $\mathfrak{H} = (\mathcal{T}^0, \dots, \mathcal{T}^{m-1}, \beta)$

as the sum of local measures, i.e.,

$$LOC(\mathfrak{H}) = \sum_{i=0}^{m-1} LOC(\mathcal{T}^i) \quad \text{and} \quad CC(\mathfrak{H}) = \sum_{i=0}^{m-1} CC(\mathcal{T}^i).$$

In the hierarchical DD of the TRIANGLE control example (see Figure V.3), we obtain $CC(\mathcal{T}^0) = 7 + 2 \cdot 3 + 1 = 14$ and $CC(\mathcal{T}^1) = 3 + 1 = 4$, leading to a cyclomatic complexity of $CC(\mathfrak{H}) = 18$. Note that $CC(\mathcal{D}) = 13 + 0 + 2 = 14$ in the DT representation of the very same control strategy (see Figure V.1). With respect to the LOC measure, we achieve better explainability for this example by $LOC(\mathfrak{H}) = 25$ compared to $LOC(\mathcal{D}) = 27$.

Note that the overall cyclomatic complexity $CC(\mathfrak{H})$ increased with refactoring due to the overhead of a second diagram where profit from reuse is not mitigated by the small size of the extracted subdiagram. However, when considering the meta control example in Figure V.9 and Figure V.10, we get $CC(\mathcal{D}) = 12$ for a non-refactored AUV and $CC(\mathcal{T}^0) = 6$, $CC(\mathcal{T}^1) = 4$ and $CC(\mathcal{T}^2) = 3$. Here, the overall cyclomatic complexity $CC(\mathfrak{H}) = 6 + 4 + 3 = 13$ is also slightly higher than $CC(\mathcal{D}) = 12$ but the individual diagrams that can be investigated in isolation have a much smaller cyclomatic complexity and have the benefit of explicitly showing code reuse.

V.6.2 Decision Diagram Refactoring

In Section V.5, we showcased how template DDs can facilitate modeling of control systems with reuse of decision making. Legacy monolithic control systems, however, do not benefit from the opportunities of concise representation and reuse yet. *Refactoring* [Fow99] is the standard approach in software engineering to improve the design of legacy code by encapsulating functionalities. This concept can also be used in the setting of DDs, factoring out isomorphic subdiagrams into template DDs towards hierarchical DDs that improve explainability measures.

Decision Diagram Refactoring Problem: Given a template DD $\mathcal{T}$ and an explainability measure η , synthesize a hierarchical DD $\mathfrak{H}_{\mathcal{T}} = (\mathcal{T}^0, \dots, \mathcal{T}^{k-1}, \beta)$ such that $\llbracket \mathfrak{H}_{\mathcal{T}} \rrbracket = \llbracket \mathcal{T} \rrbracket$, $\bigcup_{i=0}^{k-1} P^i \subseteq P$, and $\eta(\mathfrak{H}_{\mathcal{T}})$ is minimal.

Note that we required the predicate nodes of the input template DD $\mathcal{T}$ to cover the predicate nodes of all template DDs $\mathcal{T}^i$ in the synthesized hierarchical DD $\mathfrak{H}_{\mathcal{T}}$. Intuitively, this corresponds to reusing nodes of $\mathcal{T}$ in template DDs and removing duplicates. Solving the refactoring problem clearly is intractable, since finding minimal refactoring involves to solve an NP-hard problem. To this end, heuristics for refactoring are favorable towards improving explainability of DD strategies. A naive greedy algorithm on a hierarchical DD $\mathfrak{H} = (\mathcal{T}^0, \dots, \mathcal{T}^{k-1}, \beta)$ over predicate nodes $P = \bigcup_{i=0}^{k-1} P^i$ to factor out one promising template DD $\mathcal{T}^k$ would be to compute the maximal gain of explainability for isomorphic subdiagrams and opt for factoring out the maximal one:

- (1) Initialize the maximal gain $g_{\max} := 0$ and a template candidate $\mathcal{F} := \emptyset$.

- (2) For each pair $n \in P^i$ and $m \in P^j$ with $\lambda^i(n) = \lambda^j(m)$
 - (1) determine the maximal subdiagram $\mathcal{S}$ of $\mathcal{T}_n^i$ such that
 - $\mathcal{S}$ is isomorphic to the top subdiagram of $\mathcal{T}_m^j$, and
 - $\mathcal{S}$ does not contain a node with multiple incoming edges;
 - (2) compute the gain $g = \eta(\mathfrak{H}) - \eta(\mathfrak{H}')$ where $\mathfrak{H}'$ arises from $\mathfrak{H}$ by replacing every isomorphic occurrence of $\mathcal{S}$ in $\mathfrak{H}$ by a box pointing to a tentative fresh template DD $\mathcal{T}^k := \mathcal{S}$;
 - (3) if $g > g_{\max}$, set $g_{\max} := g$ and set $\mathcal{F} := \mathcal{S}$.
- (3) Update $\mathfrak{H}$ with a fresh template DD $\mathcal{T}^k := \mathcal{F}$ and replace every isomorphic occurrence of $\mathcal{F}$ in $\mathfrak{H}$ by a box pointing to $\mathcal{T}^k$.

The above procedure can be initially applied to a hierarchical DD comprising a single DD only, and repeated until no gain in explainability is achieved anymore. Note that factoring out a template DD introduces new outcome nodes in the freshly generated template component, increasing explainability measures if there are only few template occurrences or small template sizes.

V.6.3 Implementation and Evaluation

We implemented template DDs and support for hierarchical DDs building on the PYTHON-based DT learning framework DTCONTROL [Ash+21]. For this, we first transfer DTs learned with DTCONTROL into reduced DDs by merging outcome nodes and isomorphic subtrees. Then, we apply our refactoring heuristics from the last section and evaluate the LOC explainability measure, i.e., improving explainability through reducing DD sizes. Template DDs can be exported graphically towards explainable control strategies – Figures V.1 and V.3 are generated by our implementation. Our implementation and experiments are publicly available¹.

Experiments. To investigate the explanatory potential of hierarchical DDs, we applied DD generation and refactoring through our heuristics to the standard benchmark set of DTCONTROL, comprising different control strategies learned from cyber-physical systems, classical planning tasks, machine learning, as well as synthesized by probabilistic model checkers from examples of the PRISM benchmark suite [KNP11]. Note that for DD generation, we only applied reduction rules to the DT output, not changing the order of predicates. As explainability metrics we chose the LOC measure, since CC does not account well for reuse in templates (see Section V.6.1). Table V.1 shows the stepwise node-count reductions, where in the first half standard PRISM benchmarks and in the second half cyber-physical systems benchmarks are separated through a horizontal line. The “all” node counts are reflecting the standard LOC measure, while “inner” refers to the number of inner nodes, i.e., predicate and box nodes.

¹<https://anonymous.4open.science/r/tdd-97EC>

V. Template Decision Diagrams for Meta Control and Explainability

Table V.1: Results for DTCONTROL experiments

Model	DT		DD		factor(DD)		inner reduction
	all	inner	all	inner	all	inner	
beb	65	32	61	32	62	30	6%
blocksworld	1,235	617	800	616	855	572	7%
cdrive	2,401	1,200	2,152	1,200	2,049	422	65%
consensus	67	33	45	32	51	29	12%
csma	103	51	83	50	86	49	4%
eajs	167	83	112	79	115	78	6%
echoring	1,869	934	1,267	866	1,329	790	15%
elevators	16,327	8,163	6,325	6,260	4,696	4,348	47%
exploding-blocksworld	4,981	2,490	2,405	2,330	2,344	1,907	23%
firewire	307	153	232	153	237	132	14%
ij	1,291	645	510	500	525	418	35%
leader4	125	62	67	55	75	47	24%
mer30	137	68	89	68	91	61	10%
pacman	43	21	40	21	40	21	0%
philosophers-mdp	391	195	222	192	228	170	13%
pnueli-zuck	171,371	85,685	69,637	69,550	55,605	49,050	43%
rabin	111	55	67	55	70	47	15%
rectangle-tireworld	481	240	481	240	435	155	35%
triangle-tireworld	27	13	22	13	25	12	8%
wlan_dl	3,369	1,684	1,613	1,525	1,680	1,400	17%
zeroconf	381	190	187	178	197	165	13%
10rooms	17,297	8,648	748	723	534	313	96%
aircraft	915,877	457,938	332,424	332,408	307,395	264,419	42%
cartpole	253	126	211	126	218	116	8%
dcdc	271	135	138	135	138	135	0%
helicopter	6,339	3,169	3,079	2,841	3,049	2,312	27%
traffic_30m	12,573	6,286	2,888	2,876	2,950	2,281	64%
truck_trailer	338,283	169,141	142,915	141,995	138,245	111,975	34%
vehicle	13,235	6,617	5,878	5,860	6,261	5,256	21%

The latter is more meaningful to mitigate the clear drawback of DTs in multiple outcome node duplications. Thus, we also provide reductions only w.r.t. inner nodes, leading to an inner reduction of $1 - \frac{\text{factor(DD)}}{\text{DT}}$. Here, **DT** is the number of inner DT nodes and **factor(DD)** stands for applying our refactoring heuristic to the DD obtained from the original DT input from DTCONTROL. We can observe clear reductions in size, even more than one order of magnitude in the case of the “10rooms” benchmark from the cyber-physical systems domain. There are diverse reasons for reductions, which can be explained by the diverse benchmark set. Reductions are both, due to switching from DTs to DDs as representation formalism, and refactoring to hierarchical DDs. While refactoring has great impact in almost all cases, switching to DD formalism sometimes does not provide any reduction of inner nodes. For instance, in the case of the “cdrive” benchmark, refactoring provides the only source of more than 60% reduction.

V.7 Conclusion

We introduced templates in decision diagrams (DDs) as an extension of multi-valued DDs, which allow to encapsulate and reuse common decision-making patterns. Therefore, template DDs can yield more explainable representations than standard decision trees (DTs) due to their smaller size and by introducing different levels of abstraction in a hierarchical template DD. Furthermore, they enable underspecification which can be exploited for modeling meta control. We illustrated the benefits of template DDs for modeling and explaining meta controllers with a case study from the autonomous underwater robotics domain. To evaluate the potential benefits of template DDs for existing controllers given as DTs, we implemented a template-generating refactoring method for DTs. Our results on standard controller benchmarks showed template DDs to improve explainability of by reducing DT sizes by more than one order of magnitude.

Our new data structure for explainable AI and control strategy representation can be seen as a starting point for several future investigations. First, we did not opt for *ordered* template DDs, a common requirement usually present in binary DDs [Bry92]. While this could lead to performance advantages for operations on DDs and allows for integration in state-of-the-art DD frameworks [Hus+24], this restriction would limit the modeling power: permuting templates as we illustrated in Figure V.9 would be not contained in the class of ordered template DDs. However, integration of template DDs and the impact of the ordering condition are an interesting future direction. Second, template DDs could be used as runtime models in self-explainable systems that autonomously decide when and how to explain themselves. Explanations could here be supported by what-if analyses for possible template instantiations or meta control decisions, importance of decisions, and causal reasoning [Bai+21; Dub+24; Har+23]. Self-explanation can be also achieved by adding a meta-layer, similar to SASs, that monitors the system and constructs explanations from an abstract model of the system [Blu+19; Dre+18; FFD22]. In [SK22], we proposed a high-level process to extract tree-like explanation models from system models with timing information. Here, template DDs could ease extraction and enable explanations at different levels of abstraction. Also on-the-fly adaptations of parameters and template instantiations could improve control performance and explainability, where in-depth considerations are future work. Third, learning algorithms that directly provide template DDs instead of DTs for control strategies could even further increase sharing and reuse of decision making, directly impacting existing methods in explainable AI. Our refactoring experiments conclusively demonstrated the potential of possible improvements and provide a lower bound on how explainability could be increased with dedicated learning algorithms. In this vein, also the detection of appearances of standard templates as presented in Section V.4.2 could be investigated, e.g., factoring out switch chains common in outputs from DTCONTROL strategies. Fourth, the concept of templates in DD could be further investigated with applications in control theory [ANS23] and reinforcement learning by templating and bootstrapping parts of the learning process [Bai+23; Koh+21; Pad+23; SYH21]. Last but not least, further

explainability measures accompanied with user studies to investigate the human understanding of control strategy explanations by template DDs would be important to achieve a profound integration of template DDs into engineering processes.

Acknowledgements. This work was partially supported by the DFG under the projects EXC 2050/1 (CeTI, project ID 390696704, as part of Germany’s Excellence Strategy) and TRR 248 (see <https://perspicuous-computing.science>, project ID 389792660), by the NWO through Veni grant VI.Veni.222.431, and by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200).

References

- [19] *Ethics Guidelines for Trustworthy AI - European Commission, Directorate-General for Communications Networks, Content and Technology*. <https://data.europa.eu/doi/10.2759/177365>. 2019.
- [20] *Four Principles of Explainable Artificial Intelligence - (U.S.) National Institute of Standards and Technology (NIST)*. <https://doi.org/10.6028/NIST.IR.8312-draft>. 2020.
- [21] *Proposal for a Regulation laying down harmonised rules on artificial intelligence (Artificial Intelligence Act) and amending certain Union legislative acts, COM(2021) 206 final - European Commission*. <https://ec.europa.eu/transparency/regdoc/rep/1/2021/EN/COM-2021-206-F1-EN-MAIN-PART-1.PDF>. 2021.
- [ANS23] Ashwani Anand, Satya Prakash Nayak, and Anne-Kathrin Schmuck. “Synthesizing Permissive Winning Strategy Templates for Parity Games”. In: *Computer Aided Verification*. Ed. by Constantin Enea and Akash Lal. Cham: Springer Nature Switzerland, 2023, pp. 436–458.
- [Ash+21] Pranav Ashok, Mathias Jackermeier, Jan Kretínský, Christoph Weinhuber, Maximilian Weininger, and Mayank Yadav. “dtControl 2.0: Explainable Strategy Representation via Decision Tree Learning Steered by Experts”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12652. Lecture Notes in Computer Science. Springer, 2021, pp. 326–345. DOI: 10.1007/978-3-030-72013-1_17.
- [Aud+22] Gilles Audemard, Steve Bellart, Louenas Bounia, Frédéric Koriche, Jean-Marie Lagniez, and Pierre Marquis. “On the explanatory power of Boolean decision trees”. In: *Data & Knowledge Engineering* vol. 142 (2022), p. 102088. DOI: <https://doi.org/10.1016/j.datak.2022.102088>.

-
- [Bag+12] J. Andrew Bagnell, Felipe Cavalcanti, Lei Cui, Thomas Galluzzo, Martial Hebert, Moslem Kazemi, Matthew Klingensmith, Jacqueline Libby, Tian Yu Liu, Nancy S. Pollard, Mihail Pivtoraiko, Jean-Sebastien Valois, and Ranqi Zhu. “An integrated system for autonomous robotics manipulation”. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2012, Vilamoura, Algarve, Portugal, October 7-12, 2012*. IEEE, 2012, pp. 2955–2962. DOI: [10.1109/IROS.2012.6385888](https://doi.org/10.1109/IROS.2012.6385888).
 - [Bai+21] Christel Baier, Clemens Dubslaff, Florian Funke, Simon Jantsch, Rupak Majumdar, Jakob Piribauer, and Robin Ziemek. “From Verification to Causality-based Explanations”. In: *Proceedings of the 48th International Colloquium on Automata, Languages, and Programming (ICALP)*. 2021.
 - [Bai+23] Christel Baier, Clemens Dubslaff, Patrick Wienhöft, and Stefan J. Kiebel. “Strategy Synthesis in Markov Decision Processes Under Limited Sampling Access”. In: *NASA Formal Methods*. Ed. by Kristin Yvonne Rozier and Swarat Chaudhuri. Cham: Springer Nature Switzerland, 2023, pp. 86–103.
 - [Blu+19] Mathias Blumreiter, Joel Greenyer, Francisco Javier Chiyah Garcia, Verena Klös, Maike Schwammberger, Christoph Sommer, Andreas Vogelsang, and Andreas Wortmann. “Towards self-explainable cyber-physical systems”. In: *22nd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. 2019, pp. 543–548.
 - [Bre+84] Leo Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.
 - [Bry92] Randal E. Bryant. “Symbolic Boolean Manipulation with Ordered Binary-Decision Diagrams”. In: *ACM Computing Surveys* vol. 24, no. 3 (1992), pp. 293–318. DOI: [10.1145/136035.136043](https://doi.org/10.1145/136035.136043).
 - [CG12] Shang-Wen Cheng and David Garlan. “Stitch: A language for architecture-based self-adaptation”. In: *J. Syst. Softw.* vol. 85, no. 12 (2012), pp. 2860–2875. DOI: [10.1016/J.JSS.2012.02.060](https://doi.org/10.1016/J.JSS.2012.02.060).
 - [CGS09] Sagar Chaki, Arie Gurfinkel, and Ofer Strichman. “Decision diagrams for linear arithmetic”. In: *2009 Formal Methods in Computer-Aided Design*. 2009, pp. 53–60. DOI: [10.1109/FMCAD.2009.5351143](https://doi.org/10.1109/FMCAD.2009.5351143).
 - [Che+14] Bihuan Chen, Xin Peng, Yijun Yu, Bashar Nuseibeh, and Wenyun Zhao. “Self-adaptation through incremental generative model transformations at runtime”. In: *36th International Conference on Software Engineering, ICSE ’14, Hyderabad, India - May 31 - June 07, 2014*. Ed. by Pankaj Jalote, Lionel C. Briand, and André van der Hoek. ACM, 2014, pp. 676–687. DOI: [10.1145/2568225.2568310](https://doi.org/10.1145/2568225.2568310).

- [Chr+18] Philipp Chrszon, Clemens Dubslaff, Sascha Klüppelholz, and Christel Baier. “ProFeat: feature-oriented engineering for family-based probabilistic model checking”. In: *Formal Aspects of Computing* vol. 30, no. 1 (2018), pp. 45–75.
- [Chr+23] Philipp Chrszon, Christel Baier, Clemens Dubslaff, and Sascha Klüppelholz. “Interaction detection in configurable systems – A formal approach featuring roles”. In: *Journal of Systems and Software* vol. 196 (2023), p. 111556.
- [Cio+10] Tudor Cioara, Ionut Anghel, Ioan Salomie, Mihaela Dinsoreanu, Georgiana Copil, and Daniel Moldovan. “A self-adapting algorithm for context aware systems”. In: *9th RoEduNet IEEE International Conference*. 2010, pp. 374–379.
- [CMS22] Matteo Camilli, Raffaella Mirandola, and Patrizia Scandurra. “XSA: explainable self-adaptation”. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 2022, pp. 1–5.
- [CÖ17] Michele Colledanchise and Petter Ögren. “Behavior Trees in Robotics and AI: An Introduction”. In: *CoRR* vol. abs/1709.00084 (2017). arXiv: [1709.00084](https://arxiv.org/abs/1709.00084).
- [Dij82] Edsger W. Dijkstra. “On the role of scientific thought”. In: *Selected Writings on Computing: A Personal Perspective*. Transcribed 1974. Berlin: Springer, 1982, pp. 60–66.
- [Dre+18] Rolf Drechsler, Christoph Lüth, Goerschwin Fey, and Tim Güneysu. “Towards Self-Explaining Digital Systems: A Design Methodology for the Next Generation”. In: *3rd International Verification and Security Workshop (IVSW)*. IEEE, 2018, pp. 1–6. DOI: [10.1109/IVSW.2018.8494900](https://doi.org/10.1109/IVSW.2018.8494900).
- [Dub+24] Clemens Dubslaff, Kallistos Weis, Christel Baier, and Sven Apel. “Feature causality”. In: *Journal of Systems and Software* vol. 209 (2024), p. 111915. DOI: <https://doi.org/10.1016/j.jss.2023.111915>.
- [FFD22] Goerschwin Fey, Martin Fränzle, and Rolf Drechsler. “Self-explanation in systems of systems”. In: *2022 IEEE 30th International Requirements Engineering Conference Workshops (REW)*. IEEE. 2022, pp. 85–91.
- [FMY97] Masahiro Fujita, Patrick C. McGeer, and Jerry Chih-Yuan Yang. “Multi-Terminal Binary Decision Diagrams: An Efficient Data Structure for Matrix Representation”. In: *Formal Methods Syst. Des.* vol. 10, no. 2/3 (1997), pp. 149–169.
- [Fow99] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Boston, MA, USA: Addison-Wesley, 1999.
- [Gar+16] Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. “Learning invariants using decision trees and implication counterexamples”. In: *POPL*. ACM, 2016, pp. 499–512.

-
- [Goo77] I. J. Good. “Explicativity: a mathematical theory of explanation with statistical applications”. In: *Proc. R. Soc. Lond.* vol. A354 (1977), pp. 303–330.
 - [Har+23] Hans Harder, Simon Jantsch, Christel Baier, and Clemens Dubslaff. “A Unifying Formal Approach to Importance Values in Boolean Functions”. In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*. Ed. by Edith Elkind. Main Track. International Joint Conferences on Artificial Intelligence Organization, 2023, pp. 2728–2737. DOI: [10.24963/ijcai.2023/304](https://doi.org/10.24963/ijcai.2023/304).
 - [Hez+21] Sara M. Hezavehi, Danny Weyns, Paris Avgeriou, Radu Calinescu, Raffaella Mirandola, and Diego Perez-Palacin. “Uncertainty in Self-Adaptive Systems: A Research Community Perspective”. In: *ACM Trans. Auton. Adapt. Syst.* vol. 15, no. 4 (2021). DOI: [10.1145/3487921](https://doi.org/10.1145/3487921).
 - [HT21] Imre Horváth and Jože Tavčar. “Designing Cyber-Physical Systems for Runtime Self-Adaptation: Knowing More about What We Miss...” In: *Journal of Integrated Design and Process Science* vol. 25, no. 2 (2021), pp. 1–26. DOI: [DOI 10.3233/JID210030](https://doi.org/10.3233/JID210030).
 - [Hus+24] Nils Husing, Clemens Dubslaff, Holger Hermanns, and Maximilian A. Köhl. “OxiDD: A Safe, Concurrent, Modular, and Performant Decision Diagram Framework in Rust”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Bernd Finkbeiner and Laura Kovács. Cham: Springer Nature Switzerland, 2024, pp. 255–275.
 - [Iov+22] Matteo Iovino, Edvards Scukins, Jonathan Styrud, Petter Ögren, and Christian Smith. “A survey of Behavior Trees in robotics and AI”. In: *Robotics Auton. Syst.* vol. 154 (2022), p. 104096. DOI: [10.1016/J.ROBOT.2022.104096](https://doi.org/10.1016/J.ROBOT.2022.104096).
 - [Jun+08] Gueyoung Jung, Kaustubh R. Joshi, Matti A. Hiltunen, Richard D. Schlichting, and Calton Pu. “Generating Adaptation Policies for Multi-tier Applications in Consolidated Server Environments”. In: *2008 International Conference on Autonomic Computing*. 2008, pp. 23–32. DOI: [10.1109/ICAC.2008.21](https://doi.org/10.1109/ICAC.2008.21).
 - [KC03] Jeffrey O. Kephart and David M. Chess. “The Vision of Autonomic Computing”. In: *IEEE Computer* vol. 36, no. 1 (2003), pp. 41–50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
 - [KGG18] Verena Klös, Thomas Göthel, and Sabine Glesner. “Comprehensible and dependable self-learning self-adaptive systems”. In: *Journal of Systems Architecture* vol. 85 (2018), pp. 28–42.

- [KNP11] Marta Z. Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of Probabilistic Real-Time Systems”. In: *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV)*. Vol. Lecture Notes in Computer Science. 2011, pp. 585–591.
- [Koh+21] Ryosuke Kohita, Akifumi Wachi, Daiki Kimura, Subhajit Chaudhury, Michiaki Tatsubori, and Asim Munawar. “Language-based General Action Template for Reinforcement Learning Agents”. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Ed. by Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli. Online: Association for Computational Linguistics, 2021, pp. 2125–2139. DOI: [10.18653/v1/2021.findings-acl.187](https://doi.org/10.18653/v1/2021.findings-acl.187).
- [LDA09] Brian Y Lim, Anind K Dey, and Daniel Avrahami. “Why and why not explanations improve the intelligibility of context-aware intelligent systems”. In: *SIGCHI conference on human factors in computing systems*. 2009, pp. 2119–2128. DOI: [10.1145/1518701.1519023](https://doi.org/10.1145/1518701.1519023).
- [Li+20] Nianyu Li, Sridhar Adepu, Eunsuk Kang, and David Garlan. “Explanations for human-on-the-loop: A probabilistic model checking approach”. In: *Proceedings of the IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 2020, pp. 181–187.
- [LT07] I. Little and S. Thiébaux. “Probabilistic planning vs replanning”. In: *Proc. of ICAPS Workshop on IPC: Past, Present and Future*. 2007.
- [May+21] Matthias Mayr, Konstantinos Chatzilygeroudis, Faseeh Ahmad, Luigi Nardi, and Volker Krueger. “Learning of Parameters in Behavior Trees for Movement Skills”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021, pp. 7572–7579. DOI: [10.1109/IROS51168.2021.9636292](https://doi.org/10.1109/IROS51168.2021.9636292).
- [McC76] Thomas J McCabe. “A complexity measure”. In: *IEEE Transactions on software Engineering*, no. 4 (1976), pp. 308–320.
- [McM93] Kenneth L. McMillan. *Symbolic Model Checking*. Boston, MA: Springer US, 1993. DOI: [10.1007/978-1-4615-3190-6_3](https://doi.org/10.1007/978-1-4615-3190-6_3).
- [Min96] Shin-ichi Minato. *Binary decision diagrams and applications for VLSI CAD*. USA: Kluwer Academic Publishers, 1996.
- [Mit97] T. M. Mitchell. *Machine learning*. McGraw Hill series in computer science. McGraw-Hill, 1997.
- [Oue+11] Lucien Ouedraogo, Ratnesh Kumar, Robi Malik, and Knut Akesson. “Nonblocking and Safe Control of Discrete-Event Systems Modeled as Extended Finite Automata”. In: *IEEE Transactions on Automation Science and Engineering* vol. 8, no. 3 (2011), pp. 560–569. DOI: [10.1109/TASE.2011.2124457](https://doi.org/10.1109/TASE.2011.2124457).

-
- [Pad+23] Abhishek Padalkar, Gabriel Quere, Franz Steinmetz, Antonin Raffin, Matthias Nieuwenhuisen, João Silvério, and Freek Stulp. “Guiding Reinforcement Learning with Shared Control Templates”. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 2023, pp. 11531–11537. DOI: [10.1109/ICRA48891.2023.10161058](https://doi.org/10.1109/ICRA48891.2023.10161058).
- [Pla+22] Swantje Plambeck, Görschwin Fey, Jakob Schyga, Johannes Hinckeldeyn, and Jochen Kreutzfeldt. “Explaining Cyber-Physical Systems Using Decision Trees”. In: *2022 2nd International Workshop on Computation-Aware Algorithmic Design for Cyber-Physical Systems (CAADCPS)*. 2022, pp. 3–8. DOI: [10.1109/CAADCPS56132.2022.00006](https://doi.org/10.1109/CAADCPS56132.2022.00006).
- [Päß+23] Juliane Päßler, Maurice H. ter Beek, Ferruccio Damiani, S. Lizeth Tapia Tarifa, and Einar Broch Johnsen. “Formal Modelling and Analysis of a Self-Adaptive Robotic System”. In: *Proceedings of the 18th International Conference on integrated Formal Methods (iFM 2023)*. Ed. by Paula Herber and Anton Wijs. Vol. 14300. Lecture Notes in Computer Science. Springer, 2023, pp. 343–363. DOI: [10.1007/978-3-031-47705-8_18](https://doi.org/10.1007/978-3-031-47705-8_18).
- [Qui86] J. Ross Quinlan. “Induction of Decision Trees”. In: *Mach. Learn.* vol. 1, no. 1 (1986), pp. 81–106.
- [Rez+23] Gustavo Rezende Silva, Juliane Päßler, Jeroen Zwanepol, Elvin Alberts, S. Lizeth Tapia Tarifa, Ilias Gerostathopoulos, Einar Broch Johnsen, and Carlos Hernández Corbato. “SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles”. In: *Proceedings of the 18th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2023)*. IEEE, 2023, pp. 181–187. DOI: [10.1109/SEAMS59076.2023.00031](https://doi.org/10.1109/SEAMS59076.2023.00031).
- [Rod+18] Arthur Rodrigues, Ricardo Diniz Caldas, Genáina Nunes Rodrigues, Thomas Vogel, and Patrizio Pelliccione. “A learning approach to enhance assurances for real-time self-adaptive systems”. In: *Proceedings of the 13th International Conference on Software Engineering for Adaptive and Self-Managing Systems, SEAMS@ICSE 2018, Gothenburg, Sweden, May 28-29, 2018*. Ed. by Jesper Andersson and Danny Weyns. ACM, 2018, pp. 206–216. DOI: [10.1145/3194133.3194147](https://doi.org/10.1145/3194133.3194147).
- [SCR24] Meghana Aparna Sistla, Swarat Chaudhuri, and Thomas Reps. “CFLOBDDs: Context-Free-Language Ordered Binary Decision Diagrams”. In: *ACM Trans. Program. Lang. Syst.* (2024). Just Accepted. DOI: [10.1145/3651157](https://doi.org/10.1145/3651157).
- [Sho+11] Alexander Shoulson, Francisco M Garcia, Matthew Jones, Robert Mead, and Norman I Badler. “Parameterizing behavior trees”. In: *Motion in Games: 4th International Conference, MIG 2011*,

- Edinburgh, UK, November 13-15, 2011. Proceedings 4.* Springer. 2011, pp. 144–155.
- [SK22] Maike Schwammberger and Verena Klös. “From Specification Models to Explanation Models: An Extraction and Refinement Process for Timed Automata”. In: *Electronic Proceedings in Theoretical Computer Science* vol. 371 (2022), pp. 20–37. DOI: [10.4204/eptcs.371.2](https://doi.org/10.4204/eptcs.371.2).
- [Sri+90] A. Srinivasan, T. Ham, S. Malik, and R.K. Brayton. “Algorithms for discrete function manipulation”. In: *1990 IEEE International Conference on Computer-Aided Design. Digest of Technical Papers.* 1990, pp. 92–95. DOI: [10.1109/ICCAD.1990.129849](https://doi.org/10.1109/ICCAD.1990.129849).
- [ST09] Mazeiar Salehie and Ladan Tahvildari. “Self-Adaptive Software: Landscape and Research Challenges”. In: *ACM Transactions on Autonomous and Adaptive Systems* vol. 4, no. 2 (2009), 14:1–14:42. DOI: [10.1145/1516533.1516538](https://doi.org/10.1145/1516533.1516538).
- [SYH21] Yanchao Sun, Xiangyu Yin, and Furong Huang. “TempLe: Learning Template of Transitions for Sample Efficient Multi-task RL”. In: *Proceedings of the AAAI Conference on Artificial Intelligence.* Vol. 35. 11. 2021, pp. 9765–9773.
- [Uni16] European Union. “On the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)”. In: *Regulation (EU) 2016/679 of the European Parliament And of the Council.* Vol. Article 13(2)(f). 2016.
- [Wat+96] A.H. Watson, D.R. Wallace, T.J. McCabe, McCabe & Associates, National Institute of Standards, and Technology (U.S.) *Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric.* NIST special publication v. 13. U.S. Department of Commerce, Technology Administration, National Institute of Standards and Technology, 1996.
- [Wey20] Danny Weyns. *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective.* John Wiley & Sons, 2020.
- [Zus91] H. Zuse. *Software Complexity: Measures and Methods.* Programming complex systems. W. de Gruyter, 1991.
- [ZVM18] Ivan S. Zapreev, Cees Verdier, and Manuel Mazo. “Optimal Symbolic Controllers Determinization for BDD storage”. In: *IFAC-PapersOnLine* vol. 51, no. 16 (2018). 6th IFAC Conference on Analysis and Design of Hybrid Systems ADHS 2018, pp. 1–6. DOI: <https://doi.org/10.1016/j.ifacol.2018.08.001>.